

—— // ZSHRS // ——

zshrs

v0.12.10

THE COMPANION

the first compiled Unix shell, and the machine inside it

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The First Compiler	4
Chapter 2 — No Fork, No Problems	9
Chapter 3 — The Line Editor and Building Widgets	14
Chapter 4 — Modules and zmodload	23
Chapter 5 — zstyle and the Style System	33
Chapter 6 — Diagnostics, Profiling, and Tracing	40
Chapter 7 — Hooks, Traps, and Startup	48
Chapter 8 — The Parameter and Option Zoo	58
Chapter 9 — The Endgame Premise	65
Chapter 10 — From Source to Bytecode	69
Chapter 11 — The Virtual Machine and the JIT	75
Chapter 12 — The Tree-Walker That Isn't There	82
Chapter 13 — The Cache	87
Chapter 14 — Compiling Scripts to Native (the AOT Frontier)	95
Chapter 15 — The Discipline of the Port	100
Chapter 16 — Parameter Expansion and the Glob	104
Chapter 17 — The Line Editor	110
Chapter 18 — Completion	114
Chapter 19 — The Parameter Zoo and the Grammar	118
Chapter 20 — Proving It's Zsh	123
Chapter 21 — The Worker Pool	128
Chapter 22 — Parallel and Async Verbs	133
Chapter 23 — Fish Inside	139
Chapter 24 — Aspect-Oriented Shell	145

Chapter 25 — The Daemon	150
Chapter 26 — The Recorder	157
Chapter 27 — Stryke Inside	162
Chapter 28 — The Single Directory	165
Chapter 29 — Using zshrs	169
Chapter 30 — Regular Expressions and the PCRE Module	176
Chapter 31 — The Completion System in Depth	178
Chapter 32 — Job Control and Backgrounding	182
Chapter 33 — The Ledger: History as a Database	185
Chapter 34 — The Prompt	188
Chapter 35 — Arithmetic and the Two-Tier JIT	191
Chapter 36 — The Federation: Many Shells, One Catalog	193
Chapter 37 — The Endgame	195
Appendix A — The Command-Line Surface	199
Appendix B — The Daemon Operation Catalog	205
Appendix C — The ZLE Widget Catalog	212
Appendix D — Glob Qualifiers and Parameter Flags	215
Appendix E — A Glossary of zshrs Terms	219

Before You Begin

This is not the reference manual. The reference enumerates every builtin, every option, every one of the 193 ZLE widgets, and you open it when you need to know precisely what `zstyle -L` prints. This book is the other thing — the one you read straight through, to understand a single slightly outrageous claim and everything that had to be true to make it real: zshrs is the first Unix shell to be a compiler.

zshrs compiles to bytecodes and executes on a purpose-built virtual machine with fused superinstructions. Since the Bourne shell at Bell Labs in 1970, every Unix shell has been an interpreter. That is fifty-five years of shells — sh, csh, ksh, bash, zsh, fish, nu — every one of them walking a tree or a wordcode array at runtime, re-deriving on each command what the command means. zshrs is the break in that line: it compiles your `.zshrc` to bytecode and runs it on a virtual machine with a Cranelift JIT underneath. The distinction is not a marketing flourish you have to take on faith. It is a structural fact about where the work happens — once, at compile time, captured in a cached artifact, rather than repeatedly, at run time, reconstructed from raw bytes on every line — and the whole book is an unpacking of why that single change is the one that fifty-five years of shell evolution never made.

zshrs is a drop-in zsh replacement written in Rust, on the order of 658,000 lines across 614 source files. It is large because it carries two things at once. One is a strict, line-for-line port of zsh's C, so that the shell you already have keeps working — proven, not promised, against a corpus of more than forty-three thousand tests. The other is an architecture zsh could never have: a bytecode VM, a persistent worker pool that abolishes `fork`, a memory-mapped bytecode cache, an aspect-oriented intercept system, and a background daemon that does the heavy thinking once instead of in every shell you open.

The book is in five movements. The first — Chapters 1 through 3 — is the case for a compiled shell, the anti-fork architecture, and the philosophy that drives both. The second — Chapters 4 through 9 — is the machine: the compiler, the VM and its JIT, the tree-walker that was deliberately deleted, the cache, the ahead-of-time frontier, and the porting discipline that holds the compatibility floor. The third — Chapters 10 through 15 — is the zsh surface in detail: parameter expansion and globbing, the line editor, completion, the parameter zoo, the parity corpus, and the worker pool. The fourth — Chapters 16 through 21 — is everything beyond zsh: the parallel verbs, the fish-style interactivity, the aspect-oriented intercepts, the daemon, the recorder, and the language that runs inside the shell. The fifth closes the argument and is followed by reference appendices — the command line, the daemon operations, the widget catalog, the glob qualifiers, a glossary.

Everything here is grounded in the actual repository. Every command is real, every architecture claim traces to a specific source file, and where a count or a phase is still provisional — the ahead-of-time compiler is designed but not yet built; a test-suite name predates its current size — this book says so plainly rather than rounding up. The numbers that recur throughout — 717 milliseconds cold and 7 milliseconds warm, a worker pool clamped between 2 and 18 threads, 23 coreutils built into the binary, 658,000 lines across 614 source files — are the measured and committed figures, not estimates. zshrs is the work of MenkeTechnologies, and this book is grounded in his code.

Let's start.

Chapter 1 – The First Compiler

Every Unix shell ever written has been an interpreter. That sentence is the whole pitch, and it is worth slowing down to feel its weight. The Bourne shell arrived at Bell Labs in 1970. In the fifty-five years since, the C shell, the Korn shell, bash, zsh, and fish have each added syntax, line editing, completion, history, and modules — but not one of them changed the thing underneath. They all read source text, build a tree, and walk that tree to run your commands. They parse, they traverse, they fork. The architecture that made sense when memory was measured in kilobytes is the same architecture running on a machine with a hundred gigabytes of RAM and a dozen cores.

zshrs is the first to break the pattern. It compiles to bytecodes and executes on a purpose-built virtual machine with fused superinstructions. Since the Bourne shell at Bell Labs in 1970, every Unix shell has been an interpreter; zshrs is the first to be a compiler.

That is the central claim of this book, and the rest of this chapter is about why it is real rather than rhetorical — what “compiler” means here, what it changes, and why the optimization tricks that exist to make zsh feel fast are evidence that the problem was never solvable by trickery in the first place. The word “compiler” is doing precise work. It does not mean zshrs translates your shell scripts into a standalone executable, the way `gcc` turns C into a binary. It means the shell’s own execution engine compiles each command into bytecode for a virtual machine and runs the bytecode instead of re-traversing the parse tree — the same architectural move that separates a tree-walking language interpreter from a bytecode VM like CPython’s, Lua’s, or the JVM, except that no Unix shell had ever made it. The difference between walking a tree and running bytecode is the difference between re-reading and re-understanding a recipe every time you cook it versus following a checklist you wrote down once. The recipe is the source; the checklist is the bytecode; and zshrs is the first shell to keep the checklist.

What “interpreter” has always meant

To understand why compiling matters, you have to understand what the interpreters were actually doing on every command. The pattern is identical across the family: take the raw bytes of a command, lex them into tokens, parse the tokens into an abstract syntax tree, and then — for each execution — walk that tree, dispatching on node type, re-deriving what every piece means.

The cost is not in the parse. The parse is cheap and happens once for a given line. The cost is in the walk, and the walk happens every single time. Worse, the walk re-discovers structure that was already known. The diagnosis is blunt:

Tree-walking interpretation — Parse and traverse AST on every execution

To see why this is expensive, follow what a tree walker actually does to run one word. The node it reaches says “this is a command argument.” It does not yet know whether that argument contains a variable to expand, a tilde to resolve to a home directory, a glob to match against the filesystem, or a backtick to execute. So it scans the raw bytes of the word looking for those sigils, classifies what kind of expansion each one demands, and dispatches accordingly — and it does this not once but on every single execution of that line, because the tree node carries no memory of what it concluded last time. A loop that runs its body ten thousand times re-classifies the same words ten thousand times. The structure was fully knowable after the first pass, and the interpreter throws that knowledge away and rebuilds it on each iteration. That is the dispatch tax, and it is paid per node, per execution, forever.

A tree-walking interpreter is an architecture, not a bug. It is flexible, it is simple to extend, and it is what every

shell author reached for because it was the obvious tool for a program that has to handle arbitrary text the user might type next. But it leaves performance on the floor in two places at once: the dispatch overhead of re-walking and re-classifying nodes, and the fork overhead of shipping the actual work out to external processes. The shell, structurally, was a coordinator that pushed real work elsewhere and re-derived its own plan on every run — never accumulating any of the understanding it built, because the architecture gave it nowhere to put it.

zsh's `.zwc` files cache the wrong phase

The obvious objection is that zsh already has a compiled form. You run `zcompile` on a function file, you get a `.zwc`, and zsh loads “wordcode” instead of re-reading source. Doesn't that make zsh a compiler too?

No — and the reason is the most important technical point in this chapter. zsh's `.zwc` caches the cheap phase and leaves the expensive phase uncached by design:

Cache layering inverted — `.zwc` and `.zcompdump` cache the *cheap* parse phase; every exec still re-walks the wordcode array and re-scans raw word bytes for `$/~/*/'` sigils to classify expansion. The expensive layer (`prefork/singsub/multsub/filesub`) is *uncached by design*

Read that carefully. The `.zwc` saves you the lexing and parsing. But every execution still walks the wordcode array, and — this is the part that bites — it re-scans the raw bytes of each word looking for the sigils (`$`, `~`, `*`, backtick) that tell it how to expand. Then it runs the genuinely expensive machinery: `prefork`, the pass that prepares each word for expansion; `singsub`, single-word substitution; `multsub`, multiple-word substitution that can split one word into many; and `filesub`, filename substitution. That four-stage pipeline — `prefork`, `singsub`, `multsub`, `filesub` — is the heavy lifting of shell semantics, the part that turns `${(j,:.)files[2,-1]}` into an actual string, and `.zwc` does nothing for it. It cannot, because it was never designed to. The wordcode in a `.zwc` is a serialized parse tree, not a serialized expansion plan; loading it spares zsh the lexer and the parser, and then hands the interpreter right back to the same per-execution scan-classify-expand loop it would have run on raw source.

This is the inversion the phrase “cache layering inverted” names. There are two phases in running a shell line: a cheap one (lex, parse) and an expensive one (the expansion pipeline). zsh's compiled form caches the cheap phase and re-runs the expensive one. The right place to cache is the opposite — the expensive phase, the plan, the thing that costs real time on every execution — and that is precisely the phase `.zwc` cannot reach. The audit's section heading for this is “`.zwc` Files: Fake Compilation,” and the point of the phrase is precise: there is bytecode in a `.zwc`, but it is bytecode for re-interpretation, not bytecode that captures the expansion plan. The interpreter still has to figure out, on every run, what every word means. A `.zwc` is a cache of the answer to the wrong question.

Lipstick on a pig

If the architecture is the problem, then the community's accumulated workarounds are the symptom. This is the argument that animates the whole project:

“I can't have instant shell with all features in zsh. Period. No matter how many hacks you layer onto it. `zinit turbo` and `p10k instant prompt` do NOT SOLVE ZSH INHERENT LIMITS. Lipstick on pig == pig.”

Look at what a heavily-tuned zsh setup actually stacks up to feel fast: `zinit turbo mode`, `Powerlevel10k instant prompt`, `.zwc` precompilation, `zcompile`, `BG_NICE`, lazy autoload, `.zcompdump`. Each of these is a workaround, and each works by deferring or hiding rather than eliminating. Turbo mode defers plugin loading so the prompt appears before the plugins are actually ready — your aliases, completions, and key bindings arrive a beat after you can see the cursor. Instant prompt paints a fake prompt immediately and fills in the real state afterward, so the first prompt you look at is a rendered placeholder, not a shell that can yet do anything. Lazy autoload postpones reading function bodies until first use, trading a slow startup for a stutter the first time

you call each function. `.zwc` precompilation, as the previous section showed, caches the cheap phase. Every one of them is the same move: take the latency that the tree-walking, fork-per-command architecture imposes and rearrange when the user notices it, rather than removing it. None of them touches the architecture. They cannot, because they live in userspace, on top of the shell, and the cost they are fighting lives in the shell's execution model.

The argument that follows is structural. When a tool accumulates N+1 well-known optimization layers all aimed at the same problem — startup latency — each layer is evidence that the problem cannot be fixed in userspace. A healthy architecture has one canonical fast path, not a half-dozen competing tricks that each shave a slice off the same wound. Communities do not stack workarounds on top of problems that have real fixes; they stack them precisely because there is no real fix available in the architecture they are stuck with, and so each contributor adds another layer aimed at a different symptom of the same root cause. The very existence of an instant-prompt plugin is a confession that the shell cannot present a working prompt fast enough on its own. This connects directly to the lived pain that started the project:

“zshrs because zsh is crashing on me when I stick 10k completions in a zwc and the shell still lags on startup with p10k instant prompt and zinit turbo. zsh menucompletion is laggy at extreme scale. zsh is not designed for scale. Who loads 10k completions in a zwc? I guess just me.”

That is the test case that broke the camel's back: a `.zwc` with ten thousand completions, the supposedly-compiled fast path, still lagging the shell on startup even with both flagship optimization layers installed. The `.zwc` was lipstick. The pig was the tree-walking, sigil-rescanning, fork-per-command architecture underneath it.

What compiling actually changes

So what does zshrs do instead? It compiles every command to bytecode for a purpose-built virtual machine and runs that bytecode. The pipeline for an interactive command runs:

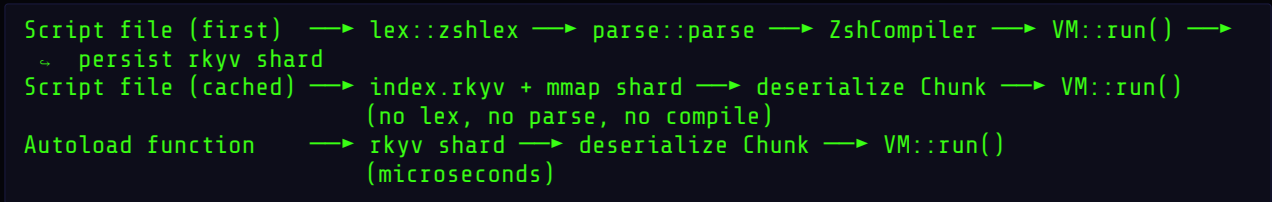
```
Interactive command → lex::zshlex → parse::parse → ZshCompiler → fusevm::Op →  
    ~ VM::run()
```

The lexer and parser are faithful ports of zsh's own `Src/lex.c` and `Src/parse.c` — the four-tier `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand` AST is preserved verbatim so that per-construct behavior matches zsh exactly. This is not incidental. Porting the front end line-for-line is the only way to guarantee that the new back end inherits zsh's exact parsing quirks — the precedence of `&&` and `||`, the boundary between a `for` head and its body, the dozens of edge cases that decades of zsh users have come to depend on. What is new is the back end: a bytecode compiler, original Rust, targeting the `Op` enum of a shared VM called fusevm:

The lexer and parser are direct ports from zsh's C source (`Src/lex.c`, `Src/parse.c`); only the bytecode compiler is original Rust.

That split — ported front end, original back end — is the whole design in one sentence. The front end exists to be bug-compatible with zsh; the back end exists to be everything zsh's back end is not. Where zsh walks the AST it just built, the `ZshCompiler` lowers that AST into a flat sequence of `fusevm::Op` instructions, and from that point on the tree is gone. The expansion classification that a tree walker redoes on every execution — is this word a glob, a variable, a tilde — is resolved once, at compile time, and baked into the opcodes. The VM that runs those opcodes never re-scans a raw word for sigils, because the compiler already did and emitted the right instruction.

The crucial difference is what happens to the result. In zsh, the parse is thrown away and re-walked. In zshrs, the compiled bytecode is the artifact, and for scripts and autoloaded functions it persists. The pipeline branches for a file that has been seen before:



This is the inversion of zsh's caching, and it is worth holding the two pictures side by side. zsh cached the cheap parse and re-ran the expensive semantics: load `.zwc`, then walk and expand on every execution. zshrs caches the expensive semantics and skips everything cheap: it persists the compiled `Chunk` — the actual VM program, expansions already classified into opcodes — as an rkyv shard, and on a warm run it memory-maps that shard and runs it. rkyv is the mechanism that makes this cheap: it is a zero-copy serialization format, so “deserialize the chunk” does not mean parsing a file format into live objects, it means mapping the shard's bytes into memory and reading the program structure directly out of them, with no allocation and no copy. The shard on disk and the program in memory have the same layout. That is why a warm start can skip lexing, parsing, and compiling entirely and still be ready to execute in single-digit milliseconds — there is almost no work between the file and the running program.

This is “the only shell bytecode cache.” Not the only shell with a cache — zsh's `.zwc` and `.zcompdump`, fish's universal-variable store, and others all cache something. The only shell whose cache is the compiled execution form rather than a re-interpretation seed. Every other shell's cache shortens the road to the interpreter; zshrs's cache *is* the destination.

Once a command is bytecode, more becomes possible. The VM does not stop at interpreting the bytecode; it tiers up. The execution model has four tiers:

Tier	What	When
rkyv image hit	Skip lex/parse/compile	Warm script runs
Block JIT	Native x86-64 via Cranelift	Loops, conditionals
Linear JIT	Native x86-64 via Cranelift	Straight-line arithmetic
Interpreter	Jump table + superinstructions	Builtins, I/O, strings

A loop or a conditional can be handed to Cranelift and compiled to native x86-64 machine code. Straight-line arithmetic gets a linear JIT. Everything else runs on a jump-table interpreter with fused superinstructions — multiple common operations collapsed into one dispatch, so that a sequence the interpreter would otherwise step through opcode by opcode becomes a single fused handler. None of this is reachable from a tree walker, and the reason is mechanical, not a matter of effort. A JIT compiles an intermediate form to native code; a tree walker has no intermediate form. You cannot hand an AST node to a native code generator and expect throughput, because an AST node is a branching data structure full of pointers to other nodes, not a linear instruction stream a code generator can lower. You hand a code generator a bytecode chunk — a flat array of opcodes with known operands — and it can emit registers and jumps for it. The same is true of the cache: you can memory-map a flat bytecode chunk and run it, but you cannot meaningfully memory-map a pointer-chasing tree and expect it to be a runnable program on the other side. The bytecode is the common prerequisite. It is the foundational decision from which JIT, persistence, and superinstructions all follow, and it is the decision no prior shell made — which is why no prior shell could JIT, persist a runnable program, or fuse opcodes, no matter how much engineering was poured into the layers above.

The number: 717 cold, 7 warm

Claims about architecture are cheap. The number is pinned:

Cold (cache miss):	717ms	- lex + parse + compile + cache write + execute
Warm (cache hit):	7ms	- deserialize + execute

A hundredfold speedup between the cold path and the warm path. The cold path does everything: lex, parse, compile, write the cache, and execute. The warm path does two things: deserialize the chunk and execute. The difference between 717 milliseconds and 7 milliseconds is exactly the work that zsh's `.zwc` could never eliminate — except here it is eliminated, because the cached form is the compiled program, not a re-interpretation seed. Trace the 710-millisecond gap and every component of it is a phase the warm path skips: the lex, the parse, the compile, and the cache write. The warm path inherits the result of all four for free, because they were done once and serialized into the shard. This is the entire payoff of caching the expensive phase instead of the cheap one made concrete in a single benchmark — the gap between the two lines is the size of the expensive phase, and on the warm path it is zero.

It is worth being honest about what these numbers are and are not. The 717ms cold figure is a one-time cost paid on a cache miss, and it is not fast in absolute terms — it is the full pipeline running from scratch, lexer and parser and compiler and a write to disk, the price of building the artifact for the first time. The 7ms warm figure is the one that matters for daily use, because in daily use almost everything is warm: your `.zshrc`, your plugins, your autoloaded functions are compiled once, on the first run after they change, and memory-mapped on every run thereafter. You pay 717ms the day you edit a file and 7ms every other day. The hundredfold figure describes the gap between those two states — the cost of a cache miss versus a cache hit on zshrs itself — not a steady-state throughput claim against another shell. It is a warm-start speedup, and that is what it is.

The pitch

Set the pieces side by side and the shape of the argument is clear. For fifty-five years, shells were interpreters that re-derived their own execution plan on every command and forked real work out to external processes. The startup models of the shells most people use today all share the same shape and differ only in which corner they cut. bash reads and interprets your `.bashrc` line by line on every launch, with no compiled cache at all. zsh adds `.zwc` and `.zcompdump` to cache the parse, then leans on the community's turbo-mode and instant-prompt layers to hide what the cache cannot remove. fish precompiles nothing comparable and instead keeps startup tolerable by being smaller and shipping fewer features to load. Three different shells, three different compromises, and not one of them caches the compiled execution form, because not one of them has a compiled execution form to cache. zsh tried to soften the cost with `.zwc` and the community piled turbo mode and instant prompt on top, but every one of those layers caches or hides the wrong thing — the cheap parse, the visible prompt — while the expensive expansion machinery and the per-command forks stay exactly as costly as they were in 1970. The accumulation of those layers is not a sign of a well-optimized shell. It is a sign of an architecture that cannot be optimized further from outside.

zshrs changes the architecture instead of decorating it. It compiles every command to fusevm bytecode through a faithful port of zsh's lexer and parser, persists the compiled chunk as an mmap-ready rkyv shard, JITs the hot tiers to native machine code via Cranelift, and falls back to a superinstruction interpreter for the rest. The cache holds the compiled program, not a re-interpretation seed, which is why the warm path is 7ms instead of 717. That is what “the first compiler” means. Not a faster interpreter. Not a better cache on the same old layer. A different machine underneath the same zsh syntax — one that does the expensive work once, remembers the answer in the form that runs, and never re-walks a tree it has already understood.

The chapters that follow take the two other halves of the same idea. The next one is about forks: how a shell that stopped re-walking trees also stopped shipping work out to other processes, running command substitution, coreutils, and parallel verbs in-process on the same VM. The one after that is about the constraint that holds the whole thing together — the endgame premise, the rule that a shell meant to last thirty years cannot reach for the very tricks it was built to make unnecessary.

Chapter 2 – No Fork, No Problems

The first chapter was about the half of the problem that lives inside the shell: the tree walk, the re-derivation, the cache on the wrong layer. This chapter is about the half that lives outside it. A traditional shell does not do most of its work itself. It forks. When you type `cat file`, the shell forks a copy of itself, replaces that copy with `/bin/cat` via `exec`, waits for it to finish, and reaps it. When you write `$(date)`, the shell forks a child, pipes its output back, and reads it. The shell, in the old model, is a coordinator that spawns processes to do the actual work and stitches their outputs together.

The tagline is three words: “No fork, no problems.” It is a joke and a thesis at once. The thesis is that the shell stopped forking work out and started running it:

Every operation that zsh forks for runs in-process. **Zero forks for builtins.**

To feel why this is a different shell rather than a faster one, it helps to remember what a fork is. `fork()` is the Unix system call that duplicates the calling process — same memory, same open files, same everything — into a parent and a child, and `exec()` is the call that throws away the child's program and loads a different one in its place. The fork/exec pair is how every traditional shell runs an external command, and it is a heavyweight operation: the kernel has to set up a new process, and then the new binary has to be found, loaded, dynamically linked, and initialized before it can do a thing. A shell that runs thousands of commands an hour is doing this thousands of times an hour, and most of the time the command it goes to all that trouble to launch does microseconds of actual work. The no-fork thesis is that this trade — milliseconds of process setup for microseconds of work — is one the shell should refuse to make whenever it can do the work itself.

This chapter is about what that refusal costs to build and what it buys — the worker pool that absorbs the work a fork used to carry, the `dup2` trick that captures command substitution without a child, the coreutils built straight into the binary, and the parallel verbs that run on the VM instead of spawning processes.

What a fork actually costs

It helps to be concrete about why forking is expensive enough to design a whole shell around avoiding it. A fork is not free creation of a process. It is, at minimum, a copy of the parent's address space metadata, an `exec` that loads a new binary from disk, the dynamic linker (`ld.so`) resolving and mapping every shared library the binary depends on, and the C library performing its startup initialization — before a single byte of `cat`'s actual work happens:

`cat file` | fork + exec + ld.so + libc init | 2-5ms

Walk that chain one stage at a time. The fork itself asks the kernel to clone the process; even with copy-on-write memory it is a syscall into the scheduler with page-table bookkeeping. The `exec` discards the clone's program and loads `/bin/cat` from disk — a file open, a read, a parse of the executable format. The dynamic linker then resolves `cat`'s shared-library dependencies, maps them into the new address space, and fixes up its symbol references. Finally `libc` runs its constructors and sets up the C runtime. Only after all of that does `cat` begin to copy bytes. Two to five milliseconds, per invocation, for the privilege of running a program whose actual job takes microseconds. The overhead dwarfs the work by three orders of magnitude.

And it compounds. A pipeline of `cat | grep | sort | uniq | wc` is five separate fork/exec chains, five trips through the linker, five `libc` startups — and the cost is measured rather than assumed: `bash` runs a hundred such pipelines in 2.3 seconds with five forks each, `zsh` in 2.1 seconds, and `zshrs`'s builtins run the same hundred in 0.09 seconds with zero forks, 23 times faster. The gap is not `bash` being poorly written or `zsh` being slow; `bash` and `zsh` are within ten percent of each other precisely because they are doing the same

thing — paying the same fork tax. zshrs is faster by a different category of margin because it stopped paying the tax at all. Now scale that to a shell that runs millions of commands a day, in CI pipelines, in container health checks, in interactive loops. The implication is direct: “Builtins cost 0 forks; externals cost 2-5ms each. Millions of commands per day; every fork avoided is real wall time.”

The worker pool

The fork did do something useful, though. It gave the shell concurrency. A forked child runs independently of the parent; a `&` background job, a process substitution `<(cmd)`, an async history write — all of these used a separate process precisely so the shell could keep going. If you stop forking, you need somewhere else for that concurrent work to live.

That somewhere is a persistent worker thread pool — a “Persistent pool of [2-18] threads.” The sizing rule is the bracket: the pool is sized to the machine’s available parallelism and clamped to a range, never fewer than 2 and never more than 18, so that it is neither too small to overlap any work nor large enough to thrash a busy machine with context switches. A laptop with four cores and a server with sixty-four both land inside the same sane band. It is configurable through `~/.zshrs/zshrs.toml`:

```
[worker_pool]
size = 8
```

The word “persistent” is the load-bearing part. The pool is created once and lives for the whole session, rather than being spun up per task. This matters because the alternative — spawning a thread or a process each time you need concurrency — would reintroduce exactly the startup cost the no-fork principle exists to eliminate, just for threads instead of processes. A persistent pool pays its creation cost once and then hands out already-warm workers for the rest of the session. A unit of work dispatched to the pool does not wait for a thread to be born; it waits for a thread to be free, which on an idle pool is no wait at all.

What the pool absorbs is the work that forks used to carry. Process substitution `<(cmd) / >(cmd)`, which zsh implements with a fork plus a FIFO, becomes a worker pool thread plus a FIFO — same plumbing, no new process. A synchronous `fsync` on history write, which would otherwise block your prompt until the disk acknowledges, becomes a fire-and-forget dispatch to the pool so the prompt returns immediately and the write lands in the background. A serial `readdir` scan of every `PATH` directory during `rehash` becomes a parallel scan across the pool, several directories read at once instead of one after another. The pool is the structural answer to the obvious objection — “if you do not fork, where does the parallel work go?” It goes to threads that already exist, sharing the parent’s address space, with no `exec`, no `ld.so`, and no libc startup. The concurrency a fork used to provide is preserved; the process-creation cost it used to charge for that concurrency is not.

There is one careful exception, and it is worth noting for honesty about the model. A genuine background job — `sleep 30 &` — still creates a real process; the README’s own example notes it is “fork + `setsid`; parent gets `Status(0)`.” The no-fork principle is about the work the shell used to push out for no reason — builtins, substitution, `coreutils` — not about genuine job control where a separate process is the correct semantics. The thesis is “stop forking work that did not need a process,” not “never create a process again.”

dup2: command substitution without a child

The sharpest single example of the no-fork principle is command substitution. `$(cmd)` is everywhere in shell code, and in every traditional shell it forks. The shell creates a child, connects a pipe to the child’s stdout, runs the command in the child, and reads the captured output back through the pipe. One fork per substitution, and substitutions are dense — a single prompt can fire several.

zshrs captures the output in-process. The mechanism is one line:

```
$(cmd) | fork + pipe | In-process stdout capture via dup2 |
```

`dup2` is the system call that duplicates one file descriptor onto another, so that writes meant for the second descriptor go to wherever the first one points. File descriptor 1 is stdout; that is the number every program writes to when it prints. The trick exploits the fact that “stdout” is just a number in a table the process controls. The technique is: open a pipe or a buffer, use `dup2` to point file descriptor 1 at it so that anything written to stdout now lands in the buffer, run the builtin command in-process so its output flows into that buffer, then use `dup2` again to restore the original stdout, and read the captured bytes back out of the buffer. No child process is created at any point. The shell ran the command itself, temporarily rewired its own output stream to catch the result, and then put the stream back exactly as it was. The whole maneuver is four cheap syscalls and no fork.

This is one of the four anti-fork mechanisms: “Command substitution — `$(builtin)` captures stdout via `dup2`, no fork.” The reason it works is that a builtin runs *inside* the shell process, so its writes to stdout are the shell’s own writes, which the shell can intercept by adjusting its own descriptor table. An external command runs in a child, whose descriptor table the parent cannot reach into after `exec` — which is exactly why the traditional implementation needs a fork and a pipe between two processes. For a builtin command, the entire `$(...)` round trip happens without leaving the process. This is why command substitution, one of the most fork-heavy constructs in any shell — a single prompt theme can fire a dozen `$(...)` per redraw — becomes one of the cheapest in zshrs.

The 23 coreutils, built in

The reason command substitution can avoid a fork so often is that the commands inside it are frequently coreutils — `$(date)` in a prompt, `$(basename "$x")` in a function, `$(wc -l < file)` in a loop — and zshrs ships those coreutils as builtins. The count and the list are exact:

23 coreutils commands run in-process with zero fork overhead:

```
cat head tail wc sort find uniq cut tr seq rev tee
basename dirname touch realpath sleep whoami id hostname
uname date mktemp
```

When you run `cat` in zshrs, nothing forks and nothing execs `/bin/cat`. The shell calls a function — a Rust function compiled into the binary, reachable by a direct call rather than a process launch. There is a gradient of how thin the implementation gets. Some builtins do the coreutil’s work in Rust; others skip even that and go straight to the kernel. `date`, `hostname`, and `uname` are marked “**Builtin** — direct syscall,” and the `xattr` operations are “**Direct syscall** — zero fork,” reading and writing extended attributes through `getxattr/setxattr` rather than forking the `xattr` tool. The implementations are exactly what those names imply: `hostname` is a direct `gethostname()`, `id/whoami` are direct `getuid/getgid`, `sleep` is `std::thread::sleep`, `date` is a direct `strftime`. Consider what running `hostname` costs in each model. The traditional way: fork, exec `/bin/hostname`, link it, init libc, have it call `gethostname()`, print the result, exit, and be reaped. zshrs’s way: call `gethostname()`. The external binary’s entire job, once you strip away its startup, was that one syscall — so zshrs does the one syscall and skips the rest.

The speedup is the headline number of the chapter:

Speedup: 2000-5000x per invocation (2-5ms fork overhead → 0.001ms builtin call).

The ratio is exactly the fork chain divided away. The 2-5ms was never `cat`’s work; it was the cost of standing up a process to do `cat`’s work. Delete the process and you delete the 2-5ms, leaving the microsecond of actual copying — 0.001ms against 2-5ms is where the 2000-5000x comes from, it is simply the one number divided by the other. Some commands go further: `date`, replaced by a direct `strftime`, is listed at 3000-8000x because the external `date` binary’s startup is even heavier relative to how cheap its output is. The more trivial the command’s real work, the more the fork overhead dominates its total cost, and the larger the speedup from removing the fork. The commands that benefit most are exactly the ones a shell runs most — the tiny, frequent ones in prompts and loops.

These are still preferences, not mandates. You can always reach the external binary explicitly with `command cat` or `/bin/cat`; the builtins are the default because the default should be fast, not because the external is

forbidden. And the behavior is identical — `cat file` “produces the same output whether it forks to `/bin/cat` or runs the builtin. The difference is 2000x faster execution.” This is the only kind of speed a compatibility-floor shell is allowed to ship: speed with no change in what the command does. A builtin that ran 2000x faster but printed different bytes than `/bin/cat` would be a regression, not an optimization, because the entire premise of a drop-in zsh replacement is that the user’s existing scripts keep producing the same results. The builtins are faster *and* observationally identical, or they are not builtins.

Parallel verbs as first-class shell ops

There is one more place the no-fork principle reaches, and it is the one that flips the old model entirely. Parallelism in a traditional shell means forking. If you want to run a command across many files in parallel, you spawn many processes — or you reach for `xargs -P`, or GNU `parallel`, both of which fork N times. The shell had no native notion of parallel work that was not “more processes.”

zshrs makes parallelism a set of builtins rather than a pattern you assemble out of `&` and `wait` or reach to an external tool for. Where a traditional shell offers no native vocabulary for “run this across these inputs in parallel and collect the results in order,” zshrs offers verbs that name exactly that intent. The concurrent-primitives section shows them with their real examples:

```
# Async/await
id=$(async 'sleep 5; curl https://api.example.com')
result=$(await $id)

# Parallel map - ordered output
pmap 'gzip {}' *.log

# Parallel filter
pgrep 'grep -q TODO {}' **/*.rs

# Parallel for-each - unordered, fire as completed
peach 'convert {} {}.png' *.svg

# Barrier - run all, wait for all
barrier 'npm test' ::: 'cargo test' ::: 'pytest'
```

`async` ships a unit of work to the pool and hands back an id; `await` collects the result. `pmap` maps a body over inputs in parallel and preserves output order; `pgrep` filters; `peach` runs unordered, firing results as they complete; `barrier` runs a set of commands in parallel and waits for all of them. The README’s no-fork table contrasts these against the old way directly: where the old model is “fork N times,” zshrs’s is “**VM execution — zero fork.**”

The implementation detail is where the chapter’s no-re-walk theme reappears. These verbs compile their body once and then reuse the compiled form across every input, rather than re-parsing the body for each item. The mechanism:

The body is parsed and compiled to a fusevm `Chunk` ONCE before the input is iterated; the per-iteration loop sets the param via `setparam`, acquires a VM from `fusevm:VMPool` (preserves Vec capacities across iterations), runs the cached chunk, and releases the VM back to the pool. `unsetparam` runs once at the end. No fork, no per-iteration parse/compile, no per-iteration VM allocation.

Read the sequence carefully, because it is the no-re-walk principle from Chapter 1 applied to a loop. The body — say `gzip {}` — is parsed and compiled to a fusevm `Chunk` exactly once, before the first input is touched. Then the per-iteration loop does only the cheap part: it sets the parameter that `{}` expands to via `setparam`, borrows a VM from `fusevm:VMPool`, runs the already-compiled chunk, and releases the VM back to the pool. Borrowing from the pool rather than allocating a fresh VM is itself an optimization — the pooled VM “preserves Vec capacities across iterations,” so the growable buffers a run needs are already sized from the last run and do not have to reallocate. At the end, `unsetparam` cleans up the parameter once. A thousand-file

run is therefore one parse, one compile, and a thousand chunk executions on recycled VMs, not a thousand parse-compile-run cycles. The `{ }` substitution is faithful to zsh semantics: it expands as `${=__zshrs_p_arg__}`, matching zsh's `SH_WORD_SPLIT` behavior on literal substitution, so a filename with a space in it splits exactly the way a zsh user would expect. The point that carries the chapter is the parse-once, run-many shape: this is the same refusal to re-derive known structure that defined the compiled shell in Chapter 1, now governing how a parallel verb treats its body across a large input.

The thesis, assembled

Put the pieces together and the chapter's claim is whole. The old shell forked for everything it did not do itself, and it did very little itself — coreutils were external, substitution was external, parallelism was more external. Every one of those forks cost 2-5ms of process setup to do microseconds of real work, and the cost scaled with how heavily you used the shell, which for a power user is constantly.

zshrs absorbs that work into the process. Command substitution captures its own stdout with `dup2` instead of forking a child, four syscalls where there used to be a process. Twenty-three coreutils run as in-process builtins, several straight through syscalls, at 2000-5000x the speed of the fork-and-exec they replace, with output identical to the binaries they stand in for. The concurrent work that a fork used to carry — background substitution, async history writes, parallel PATH scans — moves to a persistent worker pool of 2 to 18 threads that share the address space and need no `exec`, no linker, no libc startup. And parallelism itself becomes a set of verbs whose body is compiled once and reused across the whole input rather than re-parsed per item. The shell stopped being a coordinator that pushes work out to other processes and re-derives its plan each time. It became a runtime that does the work itself, on its own VM, on its own threads, keeping the concurrency a fork provided while shedding the process-creation cost a fork charged for it. No fork, no problems — and the next chapter is about the discipline that keeps it that way for thirty years.

Chapter 3 – The Line Editor and Building Widgets

Every interactive shell is, at bottom, two programs glued together: a parser that turns text into commands, and an editor that lets a human assemble that text one keystroke at a time. The second program is the one you spend your life inside. In zsh it is called ZLE — the Zsh Line Editor — and it is the largest single subsystem in the codebase. zshrs reproduces it from the C source under `src/ported/zle/`, where `zle_refresh.rs` alone runs to roughly 8,500 lines and the whole directory exceeds 87,000. The pieces that matter for understanding how a keystroke becomes an edit are smaller and more legible: `zle_main.rs` owns the read-dispatch loop, `zle_keymap.rs` owns the keymaps and `bindkey`, `zle_thinky.rs` owns widgets and the `zle` builtin, `zle_params.rs` owns the `$BUFFER` family of special parameters, and `zle_refresh.rs` repaints the screen. This chapter walks the path a byte takes from the terminal driver, through the keymap trie, into a widget, out through the special parameters a widget can read and write, and finally back onto the glass through redisplay.

How ZLE reads a key and dispatches a widget

The editor's heart is `zlecore()` in `zle_main.rs` (port of zsh's `zlecore()` at `zle_main.c:1110`). It is a single loop that runs until a global `DONE` flag is set — by `accept-line`, by an abort, or by end-of-file on an empty line. Each iteration does one thing: resolve the next bound widget and execute it. The resolution happens in `get_key_cmd()`, which is the Rust analog of zsh's `getkeycmd()`. The reason this is a function and not a single `read()` is that key bindings are not one byte each. A binding can be a multi-byte escape sequence — the up-arrow sends `ESC [ A`, a function key sends a longer run — and the editor cannot know how many bytes a binding consumes until it has read enough to either match a leaf or fall off the edge of the trie. So `get_key_cmd()` reads bytes one at a time and walks the active keymap as it goes:

```
loop {
    // Read one byte. Use timed read once we have a partial match
    // (a prefix that already hit a binding); otherwise block.
    let do_keytmout = last_match.is_some();
    let b = getbyte(do_keytmout)?;
    buf.push(b);

    // Look up the current buffer.
    let (current_match, is_prefix) = if buf.len() == 1 {
        let m = km.first[b as usize].clone();
        let pfx = km.multi.keys().any(|k| k.len() > 1 && k[0] == b);
        (m, pfx)
    } else {
        let entry = km.multi.get(&buf[..]);
        let m = entry.and_then(|e| e.bind.clone());
        let pfx = entry.map(|e| e.prefixct > 0).unwrap_or(false);
        (m, pfx)
    };
};
```

Several mechanisms are visible in this fragment. First, the keymap is split into two tables: a flat 256-entry `first` array for single-byte bindings, indexed directly by the byte value, and a `multi` hash map keyed by the full byte sequence for everything longer. A single-byte lookup is therefore an array index, not a hash probe;

only multi-byte sequences pay the hashing cost. Second, every multi-byte key carries a `prefixct` — a count of how many longer bindings have this sequence as a prefix — and the loop keeps reading only while `is_prefix` is true. The moment the accumulated bytes are no longer a prefix of any binding, it stops. Third, the read is blocking until the first partial match arrives, then switches to a timed read (`do_keytmout = last_match.is_some()`). This is the `$KEYTIMEOUT` behavior: after you press `ESC`, the editor waits a bounded interval for the rest of an escape sequence to arrive; if nothing comes within the timeout, the bare `ESC` binding fires instead of hanging forever. `KEYTIMEOUT` defaults to 10 (one tenth of a second), matching zsh 5.9.1's observed startup default.

When the loop settles on a match it may have over-read — it might have consumed bytes belonging to the *next* keystroke while probing for a longer binding that did not exist. Those surplus bytes are pushed back onto the input queue so the next loop iteration sees them:

```
if last_match.is_some() && buf.len() > last_match.len {
    let extra = buf[last_match.len..].to_vec();
    ungetbytes(&extra);
}
```

`ungetbytes()` walks the slice in reverse and pushes each byte onto the front of a `VecDeque` called `KUNGETBUF`, so a subsequent forward read returns `extra[0]` first — the same stack discipline as zsh's `while(len--)` `ungetbyte(s[len])`. This unget buffer is the substrate behind the `zle -U` builtin and the `$KEYS_QUEUED_COUNT` parameter, which simply reports `KUNGETBUF.len()`.

Back in `zlecore()`, once a `Thingy` (zsh's name for a named binding target) is in hand, the loop performs the bookkeeping that every widget invocation depends on. It rotates the binding history — `LBINDK` takes the previous value of `BINDK`, and `BINDK` becomes the current thingy — then executes the widget. The `LBINDK/BINDK` pair is what makes `$WIDGET` and `$LASTWIDGET` work: they are read straight off these two slots. After the widget returns, the loop runs `handleprefixes()` (to either promote or reset a pending numeric argument), applies a vi-mode cursor adjustment so the cursor never sits on a trailing newline in command mode, queues the `zle-line-pre-redraw` hook via `redrawhook()`, and repaints if any widget set `ZLE_RESET_NEEDED`:

```
if ZLE_RESET_NEEDED.load(SeqCst) != 0 {
    zrefresh();
    ZLE_RESET_NEEDED.store(0, SeqCst);
}
```

That flag is the connective tissue between editing and display. A widget does not paint the screen; it mutates the line and sets `ZLE_RESET_NEEDED`, and the core loop decides when to refresh. We return to that below.

Builtin widgets versus user widgets

A widget in zshrs is the struct `widget` (in `zle_h.rs`), carrying a `flags: i32` field and a tagged union `u: WidgetImpl`. The union has three variants: `Internal(ZleIntFunc)` for a built-in implemented as a Rust function pointer, `UserFunc(String)` for a user widget that names a shell function, and `Comp { fn_, wid, func }` for a completion widget registered with `zle -C`. This mirrors zsh's C `widget` union exactly — `WIDGET_INT` widgets store a function pointer, the rest store a shell-function name. The flag bits drive editor behavior: `WIDGET_INT` (`1 << 0`) marks a built-in, `WIDGET_NCOMP` (`1 << 1`) a new-style completion widget, `ZLE_MENUCMP` (`1 << 2`) a widget that must not invalidate the completion list, and so on through `ZLE_YANKAFTER`, `ZLE_KILL`, `ZLE_LASTCOL`, and `ZLE_NOTCOMMAND`.

The built-in widget table lives in `zle_bindings.rs` and contains on the order of a hundred entries, every name from `accept-line` and `self-insert` through `backward-kill-word`, `bracketed-paste`, `capitalize-word`, `complete-word`, `copy-prev-shell-word`, `digit-argument`, `down-line-or-history`, and the rest of the canonical zsh widget set. A built-in is registered with `addzlefunction()` (port of `zle_thingy.c:279`), which allocates a `widget` with `WidgetImpl::Internal(ifunc)` and `WIDGET_INT` in its flags, then binds it to a thingy of the given name.

A user widget is created with `zle -N name [func]`. That routes to `bin_zle_new()` (port of `zle_thingy.c:583`). The implementing shell function defaults to the widget name when the second argument is omitted — exactly zsh’s `args[1] ? args[1] : args[0]`:

```
let fname = if args.len() >= 2 {
    args[1].clone()
} else {
    args[0].clone()
};
let w = Arc::new(widget {
    flags: 0i32, // c:588
    first: None,
    u: WidgetImpl::UserFunc(fname), // c:590 fnnam
});
rthingy(&args[0]); // c:591 rthingy(args[0])
if bindwidget(w.clone(), &args[0]) == 0 {
```

`bindwidget()` (port of `zle_thingy.c:199`) is where protection is enforced. A thingy flagged `TH_IMMORTAL` — the canonical dot-prefixed names like `.self-insert` and `.accept-line` — cannot be rebound; `bindwidget` returns `-1` and `bin_zle_new` frees the new widget and warns `widget name is protected`. This is why you can shadow `accept-line` with `zle -N accept-line my-fn` but never `.accept-line`: the dotted form is the immortal escape hatch the editor itself dispatches through (via `execimmortal()`), guaranteeing that even a user who rebinds every public widget still has a working line acceptor underneath.

The dispatch difference between the two widget kinds is in `execzlefunc()` (port of `zle_main.c:1420`). For an internal or completion widget it calls the function pointer directly — completion widgets route through `completetecall()`, internal widgets invoke `w.u.fn(args)`. For a user widget it resolves the named shell function and runs its body:

```
let rc = crate::ported::exec::doshfunc(&mut shf, largs, true, body_runner);
// c:1530 - capture LASTVAL after the call.
LASTVAL.store(rc, Ordering::Relaxed);
```

The `argv` passed to the function is the widget name followed by any extra arguments, matching zsh’s convention that `$0` inside a widget is the widget name. The function’s exit status becomes `LASTVAL`, so a widget can fail and the editor will beep. The Rust port runs the shell function inside the same key-loop call frame through the fusevm bridge, which has a consequence the source comment is explicit about: a user widget’s mutations to `$BUFFER`, `$CURSOR`, and the other special parameters land on the live editor state synchronously, rather than being deferred to a later drain pass. There is no asynchronous boundary between “the widget set `BUFFER`” and “the editor sees the new line.”

Keymaps and bindkey

A keymap is the `Keymap` struct in `zle_keymap.rs`: the same two-level structure the dispatch loop reads — a `first: [Option<Thingy>; 256]` array and a `multi: HashMap<Vec<u8>, KeyBinding>` — plus a `flags` field and a `primary` name. Keymaps are reference-counted via `Arc` and named through a separate `keymapnametab`, so the name `main` and the name `viins` can resolve to the same underlying keymap when the user links them. The standard keymaps are built once by `default_bindings()` (port of `zle_keymap.c:1309`), which constructs `viins`, `emacs`, `vicmd`, `viopp`, `visual`, and the immutable `.safe` fallback. The `.safe` keymap is the floor: it binds all 256 bytes to `.self-insert` except newline and carriage return, which go to `.accept-line`. Because those targets are the dotted immortal widgets, `.safe` keeps working no matter how thoroughly the user has mangled the other maps — it is the keymap the editor falls back to when a requested keymap does not exist.

`default_bindings()` populates the maps from static byte-indexed tables. Control characters 0–31 come from `VIINSBIND` and `EMACSBIND`; bytes 32–255 are `self-insert` in both insert maps; byte 127 (DEL) is aliased to the same binding as byte 8 (^H); the `vicmd` map draws 0–127 from `VICMDBIND` and leaves 128–255

as `undefined-key`. Arrow and keypad keys are installed through `add_cursor_key()`, which emits the terminal's reported cursor-key sequence so that up-arrow maps to `up-line-or-history`, left-arrow to `vi-backward-char` in the vi maps, and so on. Crucially the loop installs the arrow bindings into both the insert map and the command map, so vi users get working arrows in both modes.

The user-facing entry point is the `bindkey` builtin, `bin_bindkey()` in `zle_keymap.rs` (port of `zle_keymap.c`). It dispatches on its option letter to a family of handlers — `bin_bindkey_bind` for the common `bindkey SEQ` widget form, `bin_bindkey_list` for `-L`, `bin_bindkey_new` for `-N` (new keymap), `bin_bindkey_link` for `-A` (link/alias a keymap), `bin_bindkey_meta` for `-m`, `bin_bindkey_del` for `-r`, and `bin_bindkey_delall` for `-D`. All the real work bottoms out in `bindkey()` (port of `zle_keymap.c:566`), which is the single function that mutates a keymap. Its body is a four-way match over (bind, send-string, sequence-length):

```
(Some(t), None, 1) => {
    // c:600 - `km->first[f] = bind; return 0;`
    km.first[seq[0] as usize] = Some(t);
    0
}
(Some(t), None, _) => {
    // c:631-641 - multi-char Thingy binding. Mark prefixes
    // first (so getkeymapcmd's trie walk knows to keep
    // reading bytes), then insert the full-seq binding.
    for i in 1..seq.len() {
        km.multi
            .entry(seq[..i].to_vec())
            .and_modify(|kb| kb.prefixct += 1)
            .or_insert(KeyBinding {
                bind: None,
                str: None,
                prefixct: 1,
            });
    }
}
```

A single-byte binding is a direct array store. A multi-byte binding does two things: it inserts the full sequence as a leaf, and it walks every proper prefix of the sequence, incrementing each one's `prefixct`. That `prefixct` increment is exactly the bookkeeping the dispatch loop reads when it decides whether to keep accumulating bytes. The `bindkey -s` send-string form (`(None, Some(s), _)`) stores a string instead of a thingy and is otherwise identical, and `bindkey -r` (`(None, None, _)`) binds the sequence to `undefined-key` so the bound key beeps instead of doing something stale. Immutable keymaps reject all mutation up front (`if (km.flags & KM_IMMUTABLE) != 0 { return 1; }`), which is what protects `.safe`.

The active keymap is chosen by `selectkeymap()` (port of `zle_keymap.c:495`). It opens the named keymap, and if that fails and a fallback is permitted it falls back to `.safe`; on success it stores the resolved name in `curkeymapname` and the keymap in `curkeymap`. The `$KEYMAP` parameter reads `curkeymapname` directly. There is also a `LOCALKEYMAP` slot, set by `selectlocalmap()`, that the dispatch loop checks *before* the current keymap — `local.or(cur)` at the top of `get_key_cmd()` — which is how a widget can install a temporary keymap for the duration of one operation and have it transparently removed afterward. Switching between vi insert and vi command mode is just `selectkeymap("viins", ...)` versus `selectkeymap("vicmd", ...)`; the helpers `is_viins()` and `is_vicmd()` test `*curkeymapname()` against the two names.

The parameters a widget sees

A widget is a shell function, and the only way it can read or change the line being edited is through a set of special parameters. `zshrs` registers them in `makezleparams()` (port of `zle_params.c:194`) and backs each one with an accessor that touches the live editor state rather than a stored copy. The line itself lives in `ZLELINE`, a `Vec<char>` guarded by a mutex; the cursor is `ZLECS` (an index into that vector); the line length is `ZLELL`; the mark is `MARK`. The parameters are thin views over these.

`$BUFFER` is the whole line. Its getter collects `ZLELINE` into a `String`; its setter (`set_buffer()`, port of the `BUFFER` setfn) replaces the entire vector, updates the length, and clamps the cursor so it cannot point past the new end:

```
pub fn set_buffer(s: &str) {
    // c:245
    *ZLELINE.lock().unwrap() = s.chars().collect();
    ZLELL.store(ZLELINE.lock().unwrap().len(), Ordering::SeqCst);
    ZLECS.store(
        ZLECS
        .load(Ordering::SeqCst)
        .min(ZLELL.load(Ordering::SeqCst)),
        Ordering::SeqCst,
    );
    ZLE_RESET_NEEDED.store(1, Ordering::SeqCst);
}
```

Every setter ends with `ZLE_RESET_NEEDED.store(1, ...)`. This is the single most important detail about the parameter layer: assigning to `$BUFFER` from a widget does not paint anything, it raises the same redraw flag that built-in widgets raise, and the core loop repaints after the widget returns. A widget that does `BUFFER="echo hi"; CURSOR=4` produces exactly one repaint, not two.

`$LBUFFER` and `$RBUFFER` are the line split at the cursor. `get_lbuffer()` returns `ZLELINE[..ZLECS]`; `get_rbuffer()` returns `ZLELINE[ZLECS..]`. Their setters are written so the cursor follows the split intuitively. Assigning to `$LBUFFER` replaces the text before the cursor and leaves the cursor at the end of the new prefix; assigning to `$RBUFFER` replaces the text after the cursor and leaves the cursor where it was:

```
pub fn set_lbuffer(s: &str) {
    // c:332
    let rbuf: String = ZLELINE.lock().unwrap()[ZLECS.load(Ordering::SeqCst)..]
        .iter()
        .collect();
    *ZLELINE.lock().unwrap() = s.chars().chain(rbuf.chars()).collect();
    ZLELL.store(ZLELINE.lock().unwrap().len(), Ordering::SeqCst);
    ZLECS.store(s.chars().count(), Ordering::SeqCst);
    ZLE_RESET_NEEDED.store(1, Ordering::SeqCst);
}
```

`$CURSOR` reads and writes `ZLECS`, with the setter clamping to `ZLELL` so a widget cannot place the cursor off the line. `$MARK` reads and writes `MARK` with the same clamp. `$REGION_ACTIVE` reads and writes a flag and coerces its assigned value to 0 or 1 with the C `!!x` idiom. `$NUMERIC` reflects the pending numeric argument (`zmod.mult`), `$BUFFERLINES` is the count of newlines in the buffer plus one, and `$KEYMAP` is the current keymap name.

The reflective parameters that tell a widget *how it was invoked* read the binding slots the dispatch loop maintains. `$WIDGET` returns the name of the thingy currently bound to the active key — `get_widget()` is literally `bindk ? bindk->nam : ""`. `$LASTWIDGET` returns `lbindk->nam`, the previously executed widget, which is how `yank-pop` knows whether the last command was a yank. `$WIDGETFUNC` returns the implementing function name (`.internal` for built-ins, the shell-function name for user widgets), and `$WIDGETSTYLE` carries the completion widget's underlying style. `$KEYS` returns the raw bytes of the key sequence that triggered the widget — `get_keys()` returns a clone of the keymap-walk byte buffer `keybuf` — so a widget bound to several sequences can branch on which one fired. `$KEYS_QUEUED_COUNT` reports the length of the unget buffer.

`makezleparams()` itself pushes the current line, lbuffer, rbuffer, cursor, numeric, and bufferlines into the parameter table at setup time, and the setfns plumb writes back into the editor. Because the user widget runs synchronously in the dispatch frame, the round trip is direct: the editor seeds the parameters, the function reads and assigns them, the assignments mutate `ZLELINE/ZLECS`, and the dispatch loop repaints. When ZLE is not active, the same parameters are made read-only and `zleunsetfn()` swaps in a null setter so out-of-editor writes are silently dropped — which is why `BUFFER=foo` at a normal prompt does nothing.

vared

vared — “variable edit” — is the editor offered as a builtin so a script can hand the user a pre-filled, fully editable line. It is `bin_vared()` in `zle_main.rs` (port of `zle_main.c:1678`). The early body is all guards: it refuses to run if `$TERM` is `emacs` (`ZLE not enabled`), refuses to run recursively while another ZLE session is active (`ZLE cannot be used recursively (yet)`), and parses the option set — `-a/-A` choose array or hash editing, `-p/-r` give left and right prompts, `-M/-m` choose insert and command keymaps, `-i/-f` give line-init and line-finish widgets, `-c` permits creating a nonexistent variable, and `-g` suppresses the create warning. The variable to edit is the first non-option argument, and **vared** looks it up in the parameter table — not the OS environment — so scalars, arrays, and hashes all count as existing:

```
let exists = {
    let tab = crate::ported::params::paramtab().read().unwrap();
    tab.contains_key(varname)
};
if !exists && !OPT_ISSET(ops, b'c') {
    // c:1728
    unqueue_signals(); // c:1729
    zwarnnam(name, &format!("no such variable: {}", varname)); // c:1730
    return 1; // c:1731
}
```

If the editor lacks a controlling terminal, **vared** opens `/dev/tty` (or the path given to `-t`) and verifies it is a real terminal, erroring with `can't access terminal` otherwise rather than silently degrading. The canonical zsh path then calls `zle_read()` with the `ZLCON_VARED` context — the same core read loop the interactive prompt uses, which is the whole point of **vared**: it is not a reimplementaion of editing, it is the editor pointed at a variable’s value. The Rust port writes the prompt and the variable’s current value to the tty and, on the static-link path where the live editor entypoint is not reachable from this lib-side builtin, falls back to a line read; in either case the accepted text is committed back to the named variable with `assignsparam()`, honoring the `-g` warning flag. The same `zle_read()` that drives **vared** is the one driven by the prompt and by `read -e`, so anything you can do at the prompt — bindkey customizations, user widgets, vi mode — works inside **vared** unchanged.

Numeric arguments, prefixes, and undo

The dispatch loop carries two pieces of state that survive across widget invocations and are not exposed through `$BUFFER`: the numeric argument and the undo chain. Both are managed entirely by `zlecore()` and `execute_widget()`, and both are visible to widgets only indirectly.

The numeric argument lives in a `modifier` struct, `ZMOD`, with fields `mult`, `tmult`, `vibuf`, `base`, and a flag word. `initmodifier()` resets it to the editor’s neutral state — `mult = 1`, `tmult = 1`, `base = 10`:

```
pub fn initmodifier() {
    *ZMOD.lock().unwrap() = modifier {
        flags: 0,
        mult: 1,
        tmult: 1,
        vibuf: 0,
        base: 10,
    };
}
```

The distinction between `mult` and `tmult` is what makes a repeated count accumulate. When you type `universal-argument` (Ctrl-U in emacs mode) followed by `digit-argument` presses, each prefix widget builds the count into `tmult` — the *temporary* multiplier — and sets `PREFIXFLAG`. After every widget the loop calls `handleprefixes()` (port of `zle_main.c:1618`), which promotes the temporary multiplier into the live one only when a prefix widget just ran, and otherwise wipes the whole modifier back to neutral:

```
pub fn handleprefixes() {
  if (PREFIXFLAG.load(SeqCst) != 0) {
    PREFIXFLAG.store(0, SeqCst);
    if ZMOD.lock().unwrap().flags & MOD_TMULT != 0 {
      ZMOD.lock().unwrap().flags &= !MOD_TMULT;
      ZMOD.lock().unwrap().flags |= MOD_MULT;
      let mut __g_zmod = ZMOD.lock().unwrap();
      __g_zmod.mult = __g_zmod.tmult;
    }
  } else {
    initmodifier();
  }
}
```

This is the mechanism that lets `C-u C-u C-u` build a count of 64 ($4 \times 4 \times 4$) while a plain command after the count consumes the multiplier and resets it to 1. The live multiplier is surfaced to widgets as `$NUMERIC`; `set_numeric()` writes it straight back into `ZMOD.mult`, so a widget that wants to behave as if invoked with a count can set the parameter and call another widget. The `prefixflag` itself is the sticky bit that distinguishes “the previous widget was a prefix command” from “the previous widget was the real command,” and it is consulted by `vi-set-buffer` and the digit-argument family as well as the universal argument.

Undo is handled with a one-change-per-widget model. `execute_widget()` brackets every widget call with `handleundo()` before and `mkundoent()` after:

```
fn execute_widget(widget: &widget) {
  // Reset sticky column unless the widget keeps it.
  if (widget.flags & ZLE_LASTCOL) == 0 {
    LASTCOL.store(-1, SeqCst);
  }

  // Snapshot the line so mkundoent can diff it post-widget.
  handleundo();
}
```

`handleundo()` snapshots the current line into `LASTLINE/LASTLL` (collapsing zsh’s multi-key `nextchanges` accumulation into a single snapshot, since each widget produces exactly one undo entry). After the widget runs, `mkundoent()` compares the post-widget line against the snapshot; if nothing changed it just records the new cursor position and returns, and otherwise it computes the common prefix and suffix of the old and new lines and stores only the differing middle as a `Change` record. Storing the minimal diff rather than the whole line is what keeps undo cheap for a long command line edited one character at a time. The same fragment shows the sticky-column reset: a widget not flagged `ZLE_LASTCOL` clears `LASTCOL` to `-1`, and the vertical-motion widgets (`up-line`, `down-line`) carry that flag so they can preserve the target column across a run of line moves — the column you started a vertical motion from is remembered until you do something horizontal.

Two more per-widget effects round out the bookkeeping. After the widget returns, `LASTCMD` is updated with the widget’s flag bits unless the widget is flagged `ZLE_NOTCOMMAND` — digit-argument and the prefix widgets carry that flag precisely so they do not overwrite the record of the last *real* command, which is what `yank-pop` reads to decide whether the previous edit was a yank. And in vi command mode the loop applies a cursor correction so that after a command the cursor never sits on a trailing newline: if it is past the beginning of the line and on (or just before) a `\n`, it backs up one. That single adjustment is why vi command mode in zshrs feels like vi — the cursor rests *on* the last character of a line, not in the empty column after it.

When a resolved key has no widget at all, or a widget’s function pointer is missing, the loop falls through to `handle_feep()`, which ultimately calls `handlefeep()` — the body of which is one line, `zbeep()`. The “feep” is the editor’s only way of telling you a key did nothing, and it is deliberately the same path whether the key was unbound, bound to `undefined-key`, or bound to a user function that failed.

The rest of the zle builtin

`zle -N` and `zle -C` create widgets, but the `zle` builtin (`bin_zle()`, port of `zle_thingy.c:343`) is a dispatcher over a table of single-letter subcommands, each routing to a `bin_zle_*` handler. The full set the port wires up includes `-l` (list widgets), `-D` (delete), `-A` (link/alias one widget name to another), `-N` (new user widget), `-C` (new completion widget), `-R` (refresh), `-M` (message), `-U` (unget), `-K` (keymap), `-I` (invalidate), `-f` (set flags), `-F` (file-descriptor handler), and `-T` (transform), with a sentinel no-flag entry that means “call a widget.”

The no-flag form, `bin_zle_call()` (port of `zle_thingy.c:703`), is how one widget invokes another. It refuses to run unless ZLE is active (`widgets can only be called when ZLE is active`), saves the current modifier, parses a small option set — `-n NUM` to set the numeric multiplier for the called widget, `-N` to reset the count to 1, `-K keymap` to run the widget under a different keymap, `-w` to make `$WIDGET` report the called widget rather than the caller, and `-f nohist` to suppress updating `$LASTWIDGET` — and then dispatches the named widget through the same `execzlefunc()` path the key loop uses. Because the modifier is saved and restored around the call, a widget can invoke `zle forward-word -n 3` to move three words without disturbing its own pending count.

The display-related subcommands are the ones a user widget reaches for most. `zle -R` (`bin_zle_refresh`) forces a redraw and can also display a status line and a completion-style listing; `zle -M msg` (`bin_zle_msg`) shows a transient message below the line and, when not called from inside a widget body, repaints immediately so the message is visible before the next event:

```
if crate::ported::builtin::SFCONTEXT.load(std::sync::atomic::Ordering::Relaxed) !=
    SFC_WIDGET {
    crate::ported::zle::zle_refresh::zrefresh(); // c:467
}
```

The `SFC_WIDGET` check is the same logic `zsh` uses: a message raised from within a widget will be picked up by the redraw the core loop is about to do anyway, so forcing one here would be redundant; a message raised from outside that path (a hook, say) has no such guarantee and must paint now. `zle -U str` (`bin_zle_unget`) pushes a string’s bytes back onto the input queue in reverse order so they read back in the order written — this is how a widget can synthesize keystrokes, feeding them to the dispatch loop as though the user had typed them. `zle -K keymap` (`bin_zle_keymap`) switches the active keymap and, like `-M` and `-U`, refuses to run outside a widget (`can only be called from widget function`); it is the builtin behind vi-mode switching, calling `selectkeymap(name, 0)` with no fallback so a typo’d keymap name fails loudly rather than silently dropping to `.safe`. `zle -f` (`bin_zle_flags`) lets a widget declare itself a yank or kill or vi-change command after the fact, so `$LASTWIDGET`-driven behavior treats it correctly. `zle -A old new` aliases one widget to another, and `zle -D` removes a widget — both reaching through `bindwidget/unbindwidget` and both refusing to touch the immortal dotted widgets.

There is also `savekeymap()/restorekeymap()` (ports of `zle_main.c:1632` and `:1656`), the pair a widget uses to temporarily swap a keymap and put the original back. `savekeymap` links the requested keymap under the slot name and returns the displaced keymap for safekeeping; `restorekeymap` links the saved keymap back. Together with `selectlocalmap()`, this is the machinery behind widgets like incremental search that need their own keymap for the duration of one interaction and must leave the user’s bindings untouched afterward.

How redisplay works

Nothing in the edit path draws to the screen. Widgets and parameter setters mutate `ZLELINE/ZLECS` and raise `ZLE_RESET_NEEDED`; the core loop checks that flag after each widget and calls `zrefresh()` (port of `zle_refresh.c:975`) when it is set. `zrefresh()` is the painter. It first syncs the editor’s idea of the terminal geometry to reality each frame — `WINW` from the column count, `WINH` clamped (a single physical line when the terminal reports `TERM_SHORT`, 24 as a floor otherwise) — because the cursor-movement primitives read those globals and would otherwise drift on a resized window. It then expands the left and right prompts, measures their display widths with `countprompt()`, and computes where the cursor falls in screen columns.

For the wide line the painter scrolls horizontally. It reserves a margin near the right edge and, when the cursor column approaches it, shifts the visible window so the cursor sits roughly mid-line:

```
let scroll_margin = 8;
let effective_cols = cols.saturating_sub(1);
let scroll_offset = if cursor_col >= effective_cols.saturating_sub(scroll_margin) {
    cursor_col.saturating_sub(effective_cols / 2)
} else {
    0
};
```

It then composes a per-character attribute overlay once, up front (`compute_render_attrs()`), so it does not re-walk the highlight list for every cell, emits `\r\x1b[K` to return to column zero and clear the line, draws as much of the prompt as the scroll offset leaves visible, and walks the buffer characters from the scroll start applying the precomputed attributes within a width budget that leaves room for the right prompt. The whole frame is built into a single `String` and written to the controlling terminal `SHTTY` in one write, rather than zsh's character-at-a-time `putshout`. The full diff-based engine — `refreshline()`, the `nbuf/obuf` old-versus-new line comparison, `singlerefresh()` for the single-line `read -e/vared` path, and the cursor-motion primitives `moveto()`, `tc_rightcurs()`, `tc_downcurs()` — is ported and present in `zle_refresh.rs`; the prompt-relative `redisplay()` widget (port of `zle_refresh.c:2435`) moves the cursor home, sets `RESETNEEDED`, and forces a full `zrefresh()`. The structure to take away is the separation of concerns: editing changes data, the loop notices, and the painter is the only code that knows about escape sequences and terminal width. A widget author never writes a control code; they assign `$BUFFER` and `$CURSOR` and trust the redraw flag to do the rest.

Chapter 4 – Modules and zmodload

Every shell of consequence eventually grows a way to bolt on features that not every user wants compiled into the core. In zsh that mechanism is the module: a unit of functionality, identified by a path-like name such as `zsh/pcre` or `zsh/net/tcp`, that contributes some mixture of builtins, condition operators (the infix tests usable inside `[[ ... ]]`), arithmetic functions, special parameters, and hook functions. Historically, on the C zsh, a module is a shared object — a `.so` on Linux, a `.bundle` on macOS — that the shell loads at runtime with `dlopen(3)` and probes with `dlsym(3)` to find its well-known entry points. The `zmodload` builtin is the user-facing handle on that machinery: `zmodload zsh/pcre` pulls in the PCRE matching builtins, `zmodload -u zsh/pcre` removes them again, and bare `zmodload` lists what is currently loaded. zshrs reimplements this entire surface, but the underlying loading model is fundamentally different, and that difference is the right place to begin.

Compiled in, not dynamically loaded

The header of `src/ported/module.rs`, the port of zsh's 3,646-line `Src/module.c`, states the architecture in plain terms: “In C, module.c provides dynamic loading of `.so` modules at runtime via `dlopen/dlsym`. In Rust, all modules are statically compiled into the binary — there's no dynamic loading. This module provides the registration, lookup, and management API that the rest of the shell uses to interact with module features (builtins, conditions, parameters, hooks, and math functions).” This is a deliberate design choice rather than an unfinished port. zshrs is a single self-contained binary; every module that exists is a Rust module under `src/ported/modules/`, linked into the executable at compile time. There is no plugin path to scan, no `dlopen` failure mode, no version-skew between a shared object built against one shell version and a binary built from another. The cost — that you cannot drop a third-party `.so` next to the binary and load it — is one the project accepts, because the entire C module ecosystem it cares about is the set of modules shipped with zsh itself, all of which it has ported in-tree.

The consequence for `zmodload` is that “loading” a module never touches the filesystem; it flips state bits on an in-memory record. The central data structure is a hash of `module` records seeded at startup by `register_builtin_modules`, which inserts one entry per statically-compiled module name. A faithful trace of the C control flow survives in `load_module`, the port of `Src/module.c:2200`. It still walks the same decision tree the C does — check `MOD_SETUP` and return early if a load is already in progress, clear `MOD_UNLOAD` if the module was previously marked for removal, detect circular dependencies with the `MOD_BUSY` flag and emit `circular dependencies for module ;NAME` if found, recurse into the module's declared dependency list, run the per-module `setup_` entry point, then `do_boot_module` — but the branch the C reserves for “module not yet present, must `dlopen` it” can never fire, because, in the header's words, “every linked module is pre-registered by `register_builtin_modules`.” The `load_module` body comments this explicitly: “The `!find_module` branch (c:2219-2247) requires DSO loading and never fires in zshrs's static-link path.”

What `register_builtin_modules` records is worth reading directly, because it is the authoritative inventory of which module names `zmodload` will recognize. Each entry pairs a module name with the builtins it contributes:

```

let builtin_modules = [
  ("zsh/complete", &[ "compctl", "compcall", "comparguments", ... ][..]),
  ("zsh/complists", &[ "complists" ][..]),
  ("zsh/computil", &[ "compadd", "compset" ][..]),
  ("zsh/datetime", &[ "output_strftime" ][..]),
  ("zsh/files", &[ "mkdir", "rmdir", "ln", "mv", "cp", "rm", "chmod", "chown", "sync"
    ][..]),
  ...
  ("zsh/pcre", &[ "pcre_compile", "pcre_match", "pcre_study" ][..]),
  ("zsh/stat", &[ "zstat" ][..]),
  ("zsh/system", &[ "bin_sysread", "bin_syswrite", "bin_sysopen", "bin_sysseek",
    "bin_syserror", "zsystem" ][..]),
  ("zsh/zutil", &[ "zstyle", "zformat", "zparseopts", "zregexparse" ][..]),
  ...
];

```

A second design decision lives just below that table. Not every registered module is loaded at boot. zshrs goes to some length to match the precise default-load set of stock zsh 5.9.1 as shipped by Homebrew, which the comment in `register_builtin_modules` documents as verified against `${(@k)modules}` under `zsh -fc: zsh/compctl, zsh/complete, zsh/computil, zsh/main, zsh/param/private, zsh/parameter, zsh/rlimits, zsh/sched, zsh/termcap, zsh/terminfo, zsh/watch, zsh/zle, zsh/zleparameter, and zsh/zutil` — fourteen modules present from the start. Everything else (`zsh/system`, `zsh/stat`, `zsh/zftp`, `zsh/zpty`, `zsh/zselect`, `zsh/files`, `zsh/mapfile`, and the rest) is registered into the table so that `zmodload NAME` can resolve it, but is marked at init with the `MOD_UNLOAD` bit set. The effect is that `${modules[zsh/system]}` reads as unset until an explicit `zmodload zsh/system` clears the flag, exactly as the upstream shell behaves. The loop that implements this is short and direct:

```

for (name, _builtins) in &builtin_modules {
  let mut module = module::new(name);
  if !zsh_default_loaded.contains(name) {
    // Mark as registered-but-not-loaded so
    // ${modules[NAME]} reads as unset until zmodload.
    module.node.flags |= crate::ported::zsh_h::MOD_UNLOAD;
  }
  self.modules.insert(name.to_string(), module);
}

```

This fidelity extends to a subtle distinction the C source maintains between two notions of “loaded.” The `register_builtin_modules` body explains that `zmodload -a` — the autoload listing — reports a curated set of twenty-seven builtin-to-module bindings (`bindkey` → `zsh/zle`, `compadd` → `zsh/complete`, `zstyle` → `zsh/zutil`, `sched` → `zsh/sched`, and so on), and that this is “NOT the same as the full module → builtin index”: the `zsh/files` builtins (`mkdir`, `rm`, etc.) are statically linked but deliberately excluded from the autoload registry, because stock zsh requires an explicit `zmodload zsh/files` before those names shadow the external commands. zshrs preserves that asymmetry rather than smoothing it over, because the goal is byte-for-byte parity with what a user’s existing scripts already observe.

The port even reproduces what stock zsh refuses to load. The comment block at the tail of `builtin_modules` lists names that are deliberately not registered — `zsh/calendar`, `zsh/deltochar`, `zsh/compwid` — because probing with `zsh -fc` on the reference system confirmed that those dynamic bundles fail to load there, so letting zshrs succeed on them “would diverge from `zsh -fc` which reports the `dlopen` error.” Negative compatibility is still compatibility.

The zmodload command surface

`zmodload` is not a single operation but a multiplexer whose behaviour is decided entirely by its option flags, and zshrs ports the dispatch table verbatim from `Src/module.c:2440`. The entry point, `bin_zmodload`, first validates that flags are not combined in illegal ways — the same diagnostics the C emits, carried with their

line citations — and then routes to one of five sub-handlers. The validation block is worth reading because it shows just how literal the port is:

```
let ops_bcpf = OPT_ISSET(ops, b'b') || OPT_ISSET(ops, b'c')
               || OPT_ISSET(ops, b'p') || OPT_ISSET(ops, b'f');
let ops_au = OPT_ISSET(ops, b'a') || OPT_ISSET(ops, b'u');
...
if ops_bcpf && !ops_au {
    zwarnnam(nam, "-b, -c, -f, and -p must be combined with -a or -u");
    return 1;
}
if OPT_ISSET(ops, b'F') && (ops_bcpf || OPT_ISSET(ops, b'u')) {
    zwarnnam(nam, "-b, -c, -f, -p and -u cannot be combined with -F");
    return 1;
}
```

Each error string, and even the precise order in which the checks fire, matches the C so that a script that deliberately triggers a usage error sees the identical message. Once validation passes, the dispatch is a cascade: `-F` routes to `bin_zmodload_features`, `-e` to `bin_zmodload_exist`, `-d` to `bin_zmodload_dep`, the `-a/-b/-c/-p/-f` autoload family to `bin_zmodload_auto`, and the bare-name case — plain `zmodload zsh/pcre` — to `bin_zmodload_load`, which is the path that calls `load_module` and flips the state bits described above. The `-A/-R` forms are intercepted earlier and sent to `bin_zmodload_alias`, which implements module aliases: `zmodload -A myname=zsh/zutil` installs an alias, `zmodload -R myname` removes one, and bare `zmodload -A` lists them, with the long form `zmodload -AL` emitting re-parseable `zmodload -A name=target` lines. The alias handler detects self-cycles before committing, refusing to let a name alias to itself.

The dependency subcommand, `bin_zmodload_dep`, lets a user declare that loading one module should first load another, so that `zmodload -d mymod zsh/zutil` records `zsh/zutil` as a prerequisite of `mymod`. Those declared dependencies are precisely what `load_module` walks in its recursion step, loading each dependency before booting the dependent module and aborting the whole chain if any link fails. The feature subcommand, `bin_zmodload_features` (the `-F` form), exposes the per-module feature lists — which builtins, conditions, parameters, and math functions a module advertises — and lets them be individually enabled or disabled with `+name/-name` arguments, with `zmodload -Fl` producing a re-parseable listing and `zmodload -F` including features that are currently disabled. The mechanism behind it is the `features_module/enables_module` pair, the ports of the C `features/enables` entry points, which report and toggle the advertised set without unloading the module.

Unloading runs through `unload_module`, the port of the C `Src/module.c` unload path. Because `zshrs` cannot `dlclose` anything, “unloading” means running the module’s `cleanup_` and `finish_` entry points, tearing down the parameters and builtins it installed, and re-setting the `MOD_UNLOAD` bit so that `${modules[NAME]}` reads as unset again — the inverse of the boot the load performed. The dependency walk runs in reverse here too: a module that other live modules still depend on is protected, and the unload scan only considers modules that are actually loaded (`MOD_LINKED` set, `MOD_UNLOAD` clear). The net result is that `zmodload zsh/system`; `zmodload -u zsh/system` returns the shell to the state it was in before, including making `$errnos` and `$sysparams` disappear, with no resident shared object left behind because there never was one.

The module-loader contract

Each ported module carries the six well-known entry points the C module interface defines, named `setup_`, `features_`, `enables_`, `boot_`, `cleanup_`, and `finish_`. In C these are the symbols `dlsym` looks up; in `zshrs` they are ordinary Rust functions dispatched by name. The header of `src/ported/modules/clone.rs` lays the contract out, listing the declaration order it preserves from `Src/Modules/clone.c`: `bin_clone` at c:43, the `static struct builtin bintab[]` at c:109, the `static struct features module_features` at c:113, and then the loader sextet at c:122-162. `setup_` runs once when the module is first brought in and may initialize module-private state; `features_` and `enables_` report and toggle the module’s advertised features;

`boot_` is the point at which builtins, parameters, and conditions actually become live; `cleanup_` and `finish_` reverse the process on unload. The `zsh/watch` module is the one place where `zshrs` runs a `boot_` at startup outside the normal `zmodload` path — `register_builtin_modules` calls `watch::boot_` directly so that `$watch` and `$WATCH` report the correct `array-special` type from the first prompt, matching how upstream treats `zsh/watch` as effectively part of the shell proper.

The dispatch among these named entry points is what replaces C's `dlsym`. In the dynamic-loading shell, the shell finds a module's `setup_` by looking up the symbol `setup_` in the freshly-opened shared object; in `zshrs`, where there is no shared object, the loader dispatches by module name to the right Rust function. The port's comments record that this name-based dispatch was a deliberate refactor (the callbacks “dispatch by name”), so that the `linkedmod` record carries the module's name the way C's `linkedmod` struct does, and `setup_module/boot_module/features_module/enables_module` each switch on that name to reach the per-module function. The single concession to the static-link model is that the `linkedmod` record's function-pointer fields (`setup`, `features`, `boot`, and the rest) are filled with `None` and the dispatch happens by name instead — the fields exist for structural parity with the C union but are not the dispatch mechanism. This keeps the porting rule the project enforces everywhere — faithful 1:1 ports, no adhoc replacements — intact even at the one place where the runtime model genuinely diverges.

With the loading model established, the rest of this chapter tours the modules that actually exist as files under `src/ported/modules/`, grouped by what they add to the shell.

Regular expressions: `zsh/pcre` and `zsh/regex`

Two separate modules give `zsh` two separate regex flavours, and `zshrs` ports both. `zsh/regex` (`regex.rs`) provides the POSIX-regex condition operator `-regex-match`, usable as `[[ "$str" -regex-match "$pattern" ]]`. Its header describes the side effects precisely: “On match, the cond op writes `$MATCH` / `$MBEGIN` / `$MEND` plus `$match[1..N]` / `$mbegin[1..N]` / `$mend[1..N]` (or `$BASH_REMATCH` if `BASHREMATCH` is set), exactly as the C source does at `regex.c:97-185`.” The C uses libc's `regex_t/regexmatch_t` directly and defines no structs of its own, and the Rust port matches that with zero types.

`zsh/pcre` (`pcre.rs`) is the heavier of the two, contributing three builtins — `pcre_compile`, `pcre_match`, `pcre_study` — and the `-pcre-match` infix condition. Where C links `libpcre2`, the Rust port uses the `regex` crate, which provides Perl-compatible syntax over UTF-8. The header is candid about that substitution: the `PCRE2_CODE_UNIT_WIDTH` constant is ported “as a search anchor for the C source” even though “Rust uses the `regex` crate (UTF-8 by default).” The infix dispatcher is `cond_pcre_match`, the port of `Src/Modules/pcre.c:422`:

```
/// Port of `cond_pcre_match(char **a, int id)` from `Src/Modules/pcre.c:422`. The
/// `-pcre-match` operator dispatch hook the lexer wires for
/// `[ s -pcre-match pat ]`. Compiles `a[1]` and matches `a[0]`.
pub fn cond_pcre_match(a: &[String], _id: i32) -> i32 {
```

One detail of the C that the port carefully preserves is per-evaluator compile state. C `zsh` keeps a single file-static `static pcre2_code *pcre_pattern;` so that `pcre_compile` stashes a compiled pattern for a later `pcre_match` to use. Under `zshrs`'s threaded execution model, a single global would be wrong; the port mirrors it as a `thread_local!`, so each worker thread owns its own compiled regex, preserving the file-static semantics without sharing across threads.

Arithmetic: `zsh/mathfunc`

`zsh/mathfunc` (`mathfunc.rs`) adds the mathematical functions usable inside `${( ( ... ) )}`. The header notes that all dispatch lives in a single `math_func()` switch “matching the C structure 1:1,” and that the C source has three anonymous `enum {}` blocks generating integer constants, mirrored in Rust as plain `pub const` definitions. The function table registered through `features_` is a direct port of the C `NUMMATHFUNC/STRMATHFUNC` macro expansions, each carrying its C line citation:

```

num("abs", 1, 1, MF_ABS | tflag(TF_NOCONV | TF_NOASS)), // c:115
num("float", 1, 1, MF_FLOAT), // c:133
num("int", 1, 1, MF_INT | tflag(TF_NOASS)), // c:139
num("sin", 1, 1, MF_SIN), // c:161
num("sqrt", 1, 1, MF_SQRT), // c:163

```

Some functions zsh exposes are not in Rust's standard library — the Bessel functions `j0/j1/jn/y0/y1/yn`, plus `erf`, `erfc`, `lgamma`, `tgamma`, and others — so `mathfunc.rs` declares the C ABI bindings against `<math.h>` directly, calling into `libm` exactly where `math_func()` does in the C at `Src/Modules/mathfunc.c:172-436`. The string-valued `rand48` function (the `STRMATHFUNC` form) is also present, routed through `random_real`.

Time: zsh/datetime

`zsh/datetime` (`datetime.rs`) supplies the `strftime` builtin and the `$EPOCHSECONDS` and `$EPOCHREALTIME` special parameters. `getcurrentsecs`, the port of `datetime.c:206`, backs `$EPOCHSECONDS`; a companion provides the floating-point `$EPOCHREALTIME`. The builtin's interesting feature is bidirectional conversion: `strftime -r` parses a formatted time string back into epoch seconds via `reverse_strftime` (`c:42`), the inverse of `output_strftime` (`c:99`). The C extends POSIX `strftime` with a `%.N` nanosecond syntax through its own `zstrftime()`; the Rust port calls `crate::ported::utils::zstrftime()` for the base format and layers the `%.N` extensions on top, while leaning on the `chrono` crate for the reverse parse.

System calls: zsh/system

`zsh/system` (`system.rs`) is one of the larger feature modules, exposing the low-level system-call builtins that let scripts do raw I/O without forking: `sysread`, `syswrite`, `sysopen`, `sysseek`, `syserror`, and the multiplexer `zsystem` with its `flock` and `supports` subcommands. It also contributes the `syseval` arithmetic function and two special parameters, `$errno` (the symbolic `errno` names) and `$sysparams` (a small assoc exposing things like the current process's pid and ppid). The header enumerates the twenty-one ported entry points and notes the file mirrors C source order verbatim, down to keeping the anonymous `static struct { const char *name; int oflag; } openopts[]` table from `system.c:283` as a Rust `&[(&str, i32)]` slice rather than promoting it to a public type.

Filesystem operations: zsh/files, zsh/stat, zsh/mapfile

Three modules touch the filesystem from different angles. `zsh/files` (`files.rs`) provides builtin implementations of the standard file-manipulation commands — `chgrp`, `chmod`, `chown`, `ln`, `mkdir`, `mv`, `rm`, `rmdir`, `sync`, plus the `zf_`-prefixed safe-named aliases — so a script can run these without spawning `/bin/rm`. The header notes one structural divergence from the C: the `dirsav`-based `chdir`-back stack that C's `recursivecmd` uses to safely descend directories is omitted, because Rust's `std::fs` operations take absolute paths and don't need the `chdir` dance.

`zsh/stat` (`stat.rs`) provides the `zstat` builtin, a richly-flagged `stat(2)` front end. Its rendering helpers honour a set of `STF_*` format flags — `STF_RAW`, `STF_OCTAL`, `STF_STRING` — so `zstat` can emit a mode word as raw octal, as an `ls -l`-style permission string, or both with the raw form parenthesised, and can populate an associative array under `zstat -H`. The header lists the helper functions one to one with their C line numbers: `statmodeprint` at `c:47`, `statuidprint` at `c:132`, `stattimeprint` at `c:191`, through `bin_stat` at `c:368`.

`zsh/mapfile` (`mapfile.rs`) is the most unusual of the three: it exposes the filesystem as an associative array named `$mapfile`. The header documents the four operations directly: `"$mapfile[fname] →` reads the file's contents; `mapfile[fname]=... →` writes the value to that file; `unset 'mapfile[fname]' →`

unlinks the file; `{{(k)mapfile}}` → enumerates files in the cwd.” It is implemented as a magic assoc whose get/set/unset/scan callbacks (`getpmmmapfile`, `setpmmmapfile`, `unsetpmmmapfile`, `scanpmmmapfile`) translate hash operations into file I/O.

Shell introspection: `zsh/parameter`, `zsh/zutil`, `zsh/param/private`, `zsh/zprof`

`zsh/parameter` (`parameter.rs`) is the introspection module, exposing the shell’s own internal tables as special parameters: `$commands`, `$functions`, `$aliases`, `$builtins`, `$modules`, `$dirstack`, `$history`, `$historywords`, `$options`, `$nameddirs`, `$userdirs`. Each is backed by get/scan callbacks reading the live hashtables. The function whose correctness the module test in `modules/mod.rs` pins is `paramtypestr`, the helper behind every `{{(t)foo}}` type query: the test asserts that a scalar parameter reports the string “scalar”, because that encoding is consulted by every type query and by the `scanpmpparameters` callback.

`zsh/zutil` (`zutil.rs`) is the workhorse behind zsh’s completion and configuration systems, and at nearly a quarter of a megabyte of source it is the most consequential introspection module for daily use. It contributes `zstyle` (the styled-configuration database that the completion system reads on every completion attempt), `zformat` (the %-directive string formatter behind much of the prompt-theming ecosystem), `zparseopts` (the option parser that completion functions and zsh scripts use to digest their own arguments), and `zregexparse` (a regex-driven argument-grammar matcher). The header maps its internal sections to the C: the style hashtable at c:104, the format machinery at c:800, and the zregexparse engine at c:1091. `savematch`, ported from `zutil.c:40`, snapshots `$match/$mbegin/$mend` so a `zregexparse` run can restore them, and its port comment records a fix where the earlier Rust version hardcoded the captures to empty because the function signature couldn’t take a name string — the kind of subtle behavioural bug that only surfaces when a completion grammar actually depends on the captured groups surviving a nested match. Because the whole completion system zshrs inherits from zsh reads styles through `zstyle`, this module is load-bearing for any user who has a populated `.zshrc`: it is in the default-loaded set precisely so that the first completion attempt after startup finds the style database already present.

`zsh/param/private` (`param_private.rs`) provides the `private` builtin — a stricter `local` that scopes a parameter to the immediately enclosing function and refuses to let nested scopes see it. The header reproduces the C design comment: `bin_private` opens a new parameter scope, calls `bin_typeset`, then `makeprivate` walks the new scope and either promotes each new param into the surrounding scope with its get/set/unset struct swapped to per-type private callbacks, or rejects it. The port lists nineteen functions, including the four GSU triples (`pps_*` for scalars, `ppi_*` for integers, `ppf_*` for floats, `ppa_*` for arrays, `pph_*` for hashes) that enforce the private access rules.

`zsh/zprof` (`zprof.rs`) is the function profiler. It wraps every shell-function call through `zprof_wrapper` to record entry and exit times, building a per-function table and a per-arc caller → callee table, then emits a sorted report from `bin_zprof`. The port carries the three C structs (`pfunc`, `sfunc`, `parc`) and the six file-statics tracking the call stack and accumulated counts.

Networking: `zsh/net/tcp`, `zsh/net/socket`, `zsh/zftp`

`zsh/net/tcp` (`tcp.rs`, with its header constants in `tcp_h.rs`) provides the `ztcp` builtin for opening, listening on, accepting, and closing TCP connections. The port mirrors the C `tcp.h` layout faithfully — a `Tcp_session` struct matching the C field set (`fd`, `sock`, `peer`, `flags`), a `TcpSockaddr` union wrapping `sockaddr/sockaddr_in/sockaddr_in6`, and the `ZTCP_LISTEN/ZTCP_INBOUND/ZTCP_ZFTP` flag constants — replacing C’s file-static linked list of live sessions with a `thread_local!` Vec. `zsh/net/socket` (`socket.rs`) is the Unix-domain analogue, providing the `zsocket` builtin and defining, like its C source, zero types of its own.

`zsh/zftp` (`zftp.rs`) is a full builtin FTP client, the largest module file in the tree; its header preserves the C author’s own wry comment that FTP “is a TELNET based protocol, but don’t think I like doing this.” Beyond

the protocol machinery, what makes the module observable from a script is the family of `ZFTP_*` special parameters it maintains through `zfsetparam`, the port of `zftp.c:494`: after a connection opens, `$ZFTP_HOST`, `$ZFTP_PWD`, and others reflect live session state, and read-only parameters such as `$ZFTP_REPLY` and `$ZFTP_CODE` carry the most recent server response and its numeric code. The port handles a real edge case the C must also handle: if a user has pre-declared one of these names with an incompatible type — `typeset -i ZFTP_HOST` before connecting — `zfsetparam` reconciles the existing `PM_INTEGER` parameter against the scalar value it wants to install, and teardown flushes the `ZFTP_*` names even when they are marked read-only, because the read-only flag exists to protect them from the user, not from the module itself.

Terminals and PTYs: `zsh/termcap`, `zsh/terminfo`, `zsh/zpty`, `zsh/zselect`, `zsh/curses`

`zsh/terminfo` (`terminfo.rs`) exposes the live terminfo database as the `${terminfo[capname]}` assoc, binding ncurses' `setupterm/tigetstr/tigetnum/tigetflag` through Rust FFI, with the same lookup precedence as C's `gettermInfo()`: string capability first, then numeric, then boolean rendered as `"yes"/"no"`. Unknown capabilities return `None` so callers emit the empty string, matching zsh's `PM_UNSET` fallback at `terminfo.c:165-168`. It also provides the `echoti` builtin for emitting a capability by name. `zsh/termcap` (`termcap.rs`) provides `echotc` and `termcap` access, but takes a deliberate architectural divergence the header flags clearly: rather than drag `libtermcap` into the build, it “computes a minimal capability set inline based on `$TERM`,” while keeping function signatures and observable outputs matching C one to one. `zsh/zpty` (`zpty.rs`) provides the `zpty` builtin for running subprocesses under a pseudo-terminal — useful for driving programs that demand a tty — with a 1 MB master-read cap ported from the C `READ_MAX`. `zsh/zselect` (`zselect.rs`) wraps `select(2)` for fd-readiness testing, letting a script block until one of several file descriptors becomes readable or writable; its helper `handle_digits`, ported from `zselect.c:40`, validates each fd argument and grows the `fd_set` in place, operating directly on a `libc::fd_set` so the underlying `FD_SET` runs on the caller's set rather than a copy. `zsh/curses` (`curses.rs`) implements the `zcurses` windowing builtin for building terminal user interfaces with multiple windows; the curses port currently emits ANSI escape sequences directly instead of linking `libncurses`, but mirrors the C's function names, signatures, control flow, validation pipeline, error-numbering, and the module-static window-table layout line by line, with the only deviations — the points where C would call into ncurses — flagged inline by `WARNING` comments so the substitution is never silent.

Locale, color, and the remainder

Login monitoring: `zsh/watch`

`zsh/watch` (`watch.rs`) is one of the fourteen modules loaded by default, and it is the reason zshrs runs a `boot_` directly at startup rather than waiting for an explicit `zmodload`. It monitors the system's login/logout records to print a notice when a user named in the `$watch` array logs in or out, formatting each event through `$WATCHFMT`. The port defines `DEFAULT_WATCHFMT` as `"%n has %a %l from %m."`, the same default the C ships, and types `WATCH_STRUCT_UTMP` as an alias for `libc::utmpx`, reading the system's `wtmp` file to recover login times via `getlogtime`, the port of `watch.c:161`. The module also contributes the `log` builtin and the `$LOGCHECK` interval parameter. The reason the early `boot_` matters is type-reporting fidelity: without it, `${(t)watch}` would return empty instead of `array-special`, because the parameter table entries for `watch/WATCH` would not yet exist — a divergence from `zsh -fc` that the port closes by registering the parameters at init while honouring emulation mode, so the default `WATCHFMT/LOGCHECK` values stay empty until an explicit `zmodload zsh/watch` when running in strict-zsh mode.

ksh93 compatibility: `zsh/ksh93`

`zsh/ksh93` (`ksh93.rs`) supplies the glue that lets `zsh` emulate corners of `ksh93`. The header notes its C author's comment that "implementing 'namespace' requires creating a new keyword," and the module's surface centres on the `${.sh.*}` family of special parameters plus the `nameref` builtin (a reference-to-another-parameter declaration). The port routes `nameref` through `bin_typeset` rather than reimplementing it, and backs the `${.sh.*}` specials with file-static scalars — `sh_name`, `sh_subscript`, `sh_edchar`, `sh_edmode` — each ported with its get/set callback (`edcharsetfn` at c:46, `matchgetfn` at c:60) and each initialized to a shared sentinel `sh_unsetval` so an unread `.sh.*` parameter reports as unset exactly as `ksh93` would.

Locale, color, and the remainder

A cluster of smaller modules round out the set. `zsh/langinfo` (`langinfo.rs`) exposes `${langinfo[CODESET]}` and similar via `nl_langinfo(3)`, backed by a static name-array (`nl_names[]`, ported from c:65) paired with the matching `nl_item` codes. `zsh/nearcolor` (`nearcolor.rs`) does RGB-to-256/88-colour quantisation by converting to CIELAB and minimizing the deltaE colour distance — a real algorithm, ported with its `cielab` struct and `RGBtoLAB/mapRGBto256/mapRGBto88` helpers exactly in C declaration order. `zsh/hlgroup` (`hlgroup.rs`) bridges ZLE highlight groups to rendered ANSI, exposing `${.zle.esc[name]}` and `${.zle.sgr[name]}` read-only specials — the former the full `\033[...m` escape stream, the latter the bare `;-joined` SGR parameter list — by converting the user-defined `$.zle.hlgroups` hash through `convertattr`. `zsh/random` (`random.rs`) and `zsh/random_real` (`random_real.rs`) supply `$SRANDOM` and the 64-bit/high-precision RNG behind `$RANDOM`, reading kernel random sources; `random_real.rs` ports the C's `clz64` count-leading-zeros macro and the `random_64bit/random_real` generators in declaration order. `zsh/clone` (`clone.rs`) provides the `clone` builtin that forks the shell onto another terminal, returning the child pid in the parent and zero in the child, with `$TTY` reset via `setsparm`. `zsh/attr` (`attr.rs`) gives the extended-attribute builtins `zgetattr/zsetattr/zdelattr/zlistattr`, wrapping the platform `getxattr/setxattr/listxattr/removexattr` calls with symlink-aware variants. `zsh/cap` (`cap.rs`) provides POSIX.1e capability builtins `cap/getcap/setcap`, Linux-only by nature — on macOS and BSD the header notes the stubs return `Unsupported`, because those systems have no POSIX.1e capability sets. `zsh/db/gdbm` (`db_gdbm.rs`) ties parameters to a GDBM database with `ztie/zuntie/zgdbmpath`, exposing the database as a standard hash. `zsh/newuser` (`newuser.rs`) is the boot-time dotfile probe that kicks off the first-run setup wizard when a user has none of `.zshenv/.zprofile/.zshrc/.zlogin` under `$ZDOTDIR` (or `$HOME`); its `setup_` is the C's `return 0;` and the real work is the dotfile-presence check at boot. And `zsh/example` (`example.rs`) is the documentation/template module that prints "The example module has now been set up." on boot — registered so that `zmodload zsh/example` succeeds, exactly as the canonical example expects.

Taken together, these modules reproduce `zsh`'s loadable-feature surface as a statically-linked whole. The user types the same `zmodload` commands, observes the same load/unload semantics down to which modules are present by default and which are autoload-only, and gets the same builtins, conditions, math functions, and special parameters — with the entire mechanism collapsed from a runtime `dlopen` dance into a compile-time link and a set of in-memory state bits.

The trade implicit in that collapse is worth restating plainly, because it is the kind of decision a 30-year-horizon project makes deliberately. By giving up dynamic loading, `zshrs` gives up the ability to load a foreign `.so` it was not built with — but it also gives up every failure mode that came with that ability: version skew between a module compiled against one shell ABI and a binary built from another, missing-symbol crashes discovered only at `zmodload` time, dependency on the host's dynamic linker behaving identically across macOS and the two Linux architectures the project targets, and the security surface of loading arbitrary shared objects out of `$module_path`. Everything the module system can do is decided at link time and proven by the build; a `zmodload` of any name the binary knows about cannot fail for reasons outside the binary itself. For a shell whose entire reason to exist is being trustworthy enough to replace `zsh` permanently, moving the module system from a runtime contract to a compile-time one is the conservative choice, and the parity work documented throughout `module.rs` — matching the default-load set, the autoload registry, the negative cases that stock `zsh` refuses, the exact usage-error strings — is what keeps that conservative choice invisible to the scripts that depend on it.

Native Rust plugins: dynamic loading on the shell's own terms

The trade the previous section described — giving up dynamic loading to gain a compile-time-proven module system — is the right default, but it is not the whole story, because there is a version of dynamic loading that carries none of the failure modes that made it worth giving up. zshrs adds it as a separate, additive path: native plugins written in a compiled language (Rust), loaded at runtime with `zmodload -R <path>`, with no recompile of the shell.

The distinction that makes this safe is the one zsh's own module system never had: a stable, versioned, independently-published ABI. A zsh C module compiles against the shell's *internal* private headers (the `.mdd` build against `Src/`), so it is welded to one exact build and can crash a mismatched one — which is exactly why the ecosystem has no meaningful third-party native modules; almost every one that exists is bundled by the shell itself. zshrs instead publishes its plugin interface as the `znative` crate on crates.io — the same crate that provides the `znative` package manager, published from the zshrs repo. A plugin author runs `cargo add znative`, writes a handler, and ships a `cdylib`. Everything crossing the boundary is `#[repr(C)]`, and the host checks the plugin's `ABI_VERSION` against its own before trusting a single pointer the plugin returns; a mismatch is refused at load, not left to corrupt the process. That one guarantee is what turns “load a foreign .so” from the liability this chapter rejected into a safe, first-class extension path — and makes zshrs the first shell to expose its native-plugin interface as an independently-published, versioned ABI package rather than its own build-tree internals.

Mechanically it is small. `zmodload -R libfoo.dylib` dlopens the library (via `libloading`), resolves the single symbol `znative_plugin_init`, and calls it with a host handle. The plugin registers its builtins into an in-process registry and calls back through a curated host API — `print`, `eval`, `getvar`, `setvar` — rather than reaching into shell internals the way a C module does. Command resolution then slots that registry into `execute_external_bg` ahead of `PATH` lookup, the same place zsh resolves `zmodload -ab` autoloaded builtins, so a plugin command shadows an external of the same name but never a real builtin or a shell function. `zmodload -uR` unloads it, purging the registry before the library's `dlclose` so no dangling function pointer survives. Script plugins — oh-my-zsh, zinit bundles, any `.zsh` you `source` — keep working exactly as before; the Rust path sits alongside them, for the cases where a plugin wants native machine-code speed and real type checking instead of re-interpreted shell text on every call.

Installing native plugins with znative

A published ABI makes a native plugin buildable by a stranger; a package manager makes it installable by one, and that is `znative` — the zshrs package manager, ported from stryke's own dependency manager, which compiles a whole dependency graph to native. Every shell already has “plugin managers” — oh-my-zsh, zinit, antigen, sheldon, zplug — but each of them does one thing: clone a git repository and `source` its `.zsh` files. None installs a *compiled* plugin, because until the stable ABI above there was nothing safe to install. `znative` does, and it does it for script plugins and native Rust plugins alike, which makes it the first shell package manager whose unit of installation can be native code rather than shell text. `forgit`, `git-fuzzy`, `revolver`, `kubectl`, and `zsh-z` ship as worked plugin ports to prove the path end to end.

The command surface is deliberately small. `znative load` is the one a `.zshrc` calls: given a name or a source already in the store it loads it with zero network; given a source not yet present it installs it first, then loads — so there is no separate install step for the common case. `znative add` (aliases `install`, `i`) resolves, installs, records, and loads without needing a `.zshrc` line; `znative remove` unloads a native plugin, deletes its store copy, and drops its index row; `znative list` prints one line per installed plugin — name, version, kind, source; `znative info` shows the full record including store path, integrity hash, and the lib or source files; and `znative update` re-resolves and reinstalls from the recorded source, one plugin or all of them. Errors print as `znative: <reason>` on stderr and the command returns non-zero, the same terse contract the rest of the shell uses.

A source spec is auto-classified rather than flagged. A bare `owner/repo` clones from GitHub; `github:owner/repo` says so explicitly; a `git+URL` or any URL ending in `.git` clones that URL; and `path:DIR` or an absolute `/ ./ / ~` path installs a local directory with no network at all. Any remote form may carry an `@ref` suffix — `owner/repo@v1.2.0`, `git+https://host/x.git@main` — to pin a tag, branch, or commit, and clones are shallow (`--depth 1 --branch REF`) with a full-clone fallback only when a shallow branch clone cannot reach a requested commit.

Everything installed lives under one content-addressed store at `$ZSHRS_HOME/pkg/` (default `~/.zshrs/pkg/`): `store/<name>@<version>/` holds the plugin with `.git/` and `target/` stripped out, `installed.toml` is the global index and the source of truth, and `git/`, `cache/`, and `bin/` are internal scratch. Each install is SHA-256 pinned as `sha256-<hex>` in the index, so a record carries its own integrity check. There is no per-project manifest and no lockfile — `znative` is global only, which is the right model for a shell whose plugins are part of the user's environment rather than a project's dependency set.

The kind of a plugin is auto-detected when the repo does not declare one. A prebuilt `lib*.{dylib,so}` at the root, or a `Cargo.toml` whose `[lib] crate-type` includes `cdylib`, marks it **native** — loaded through the `zmodload -R` host of the previous section, built with `cargo build --release` when no prebuilt library is present. Otherwise a `*.plugin.zsh`, a `functions/` directory, or loose `*.zsh` files mark it **script** — added to `$fpath` and sourced as-is, exactly as an oh-my-zsh or zinit plugin would be, so the entire existing ecosystem installs with no metadata at all. A repo that wants to be explicit ships a `znative.toml` at its root: a `[plugin]` table for name, version, and description, and either a `[native]` table naming the lib stem (with `build = true` the default when a `Cargo.toml` is present) or a `[script]` table listing the files to `source` and the dirs to prepend to `$fpath`. The manifest overrides auto-detection; without one, detection covers the common cases.

In practice all of this collapses to a few lines in a `.zshrc`, one `znative load owner/repo` per plugin in load order — `history-search-multi-word`, `zsh-autosuggestions`, a native port like `zshrs-forgit`, then `zsh-syntax-highlighting` kept last. The first start installs each; every start after loads from the store with no network. A bare `znative load` with no argument loads everything already in the store, for the user who prefers to `znative add` interactively and keep a single line in the startup file. It is the script-plugin workflow every zsh user already knows, extended — without a second tool or a second mental model — to compiled native plugins.

Chapter 5 – zstyle and the Style System

Of all the subsystems zsh accreted over its lifetime, the style system is the one that most resembles a small in-process database. It is not a feature you reach for directly very often, yet almost everything sophisticated in zsh’s interactive layer — the new completion system, the VCS info prompt machinery, the colorized listings, the directory-stack widgets — sits on top of it. The contract is deceptively small: a global table maps a *style* name to an ordered list of *pattern* → *values* entries, and at runtime a consumer asks “given this context string, what is the value of this style?” The answer is the value list of the most-specific pattern that matches the context. That single lookup, repeated thousands of times across a completion or a prompt render, is what `zsh/zutil` exists to make fast and predictable. The Rust reimplementation ports the C module `Src/Modules/zutil.c` close to line-for-line, and because the port carries the original C line numbers as comments throughout, it is unusually easy to see exactly which behaviors are faithful and which are deliberate Rust-shape adaptations. This chapter walks the data model, the weighting algorithm that decides “most specific,” the full set of `zstyle` subcommands, how lookups feed back into completion and other consumers, and the three companion builtins — `zparseopts`, `zformat`, and `zregexparse` — that the same module provides.

The context:style:value model

A style is addressed by two coordinates. The first is the *context*, a colon-separated string that the asking subsystem constructs to describe where it is and what it is doing — for completion this looks like `:completion:complete:cd:argument-rest:directories`, a hierarchy that narrows from the broad subsystem down to a specific completion tag. The second is the *style name* itself, a flat string like `list-colors`, `menu`, `verbose`, or `format`. The user never registers (*context*, *style*) pairs directly; instead they register (*pattern*, *style*) pairs, where the pattern is a glob that will be matched against future context strings. So `zstyle ':completion:*' menu select` says “for any context matching `:completion:*`, the `menu` style has the value `select`.” The value is always stored as an array of strings — even a single scalar value is held as a one-element list — because many styles are genuinely multi-valued (a list of completers, a list of colors, a list of formats).

In the port, the storage shape is a `style_table` wrapping a `HashMap<String, Vec<stypat>>` keyed by style name (`zutil.rs:3434-3440`). The `stypat` struct is the per-pattern entry and mirrors the C `struct stypat` field-for-field:

```
pub struct stypat {
    pub next: Option<Box<stypat>>, // c:98 Stypat next
    pub pat: String,                // c:99 char *pat
    pub prog: Option<Patprog>,      // c:100 Patprog prog (compiled)
    pub weight: u64,                // c:101 zulong weight
    pub eval: Option<Eprog>,        // c:102 Eprog eval
    pub vals: Vec<String>,          // c:103 char **vals
}
```

The C original splits the model across two structs — a `struct style` holding the style name plus a `Stypat pats` linked list, and the `stypat` nodes themselves — and threads them through zsh’s generic `HashTable`. The Rust port flattens the *style*-node layer into the `HashMap` key while preserving the `stypat` list as a `Vec<stypat>` per key. The `style` struct still exists (`zutil.rs:130-137`) for fidelity with the C node-freeing paths, but the live table is the `HashMap`. Crucially, the per-style `Vec<stypat>` is kept sorted by weight in

descending order at insertion time, which is what makes lookup a simple linear “first match wins” scan rather than a search for a maximum. The global table is a `LazyLock<Mutex<style_table>>` named `zstyletab`, the direct analog of C’s `static HashTable zstyletab`:

```
pub static zstyletab: std::sync::LazyLock<Mutex<style_table>> =
    std::sync::LazyLock::new(|| Mutex::new(style_table::new())); // c:209
```

A `Mutex` is used because the table is process-global and several entry points — `bin_zstyle`, `lookupstyle`, `testforstyle`, and the listing helpers — all need shared mutable access. This is a faithful translation of the C reality where the table is a static singleton mutated from anywhere; the mutex makes the sharing explicit and the lock-ordering visible. One consequence the port comments call out repeatedly is that `setstypat` locks `zstyletab` itself, so callers must not hold the lock when invoking it, on pain of a re-entrant deadlock on the non-reentrant mutex.

The allocation and teardown paths are carried across too, even though in Rust most of them collapse into `Drop`. C creates the table with `newzstyletable` (`zutil.rs:728-801`) and allocates a fresh style node with `addstyle`, which `zshcalloc`s a `struct style`, registers it in the table under a duplicated name, and returns it for the caller to populate. The freeing side is more interesting because it is where C’s manual linked-list surgery becomes Rust ownership. `freestylepatnode` (`zutil.rs:149-160`) in C frees a node’s pattern string, compiled program, value array, and eval program; in Rust it is a single `drop(p)` because the `stypat` owns all of those fields and dropping the box reclaims them in one move. `freestypatnode` (`zutil.rs:166-178`) similarly collapses C’s `while (p) { pn = p->next; freestylepatnode(p); p = pn; }` walk into dropping the node, since the `Option<Box<stypat>>` chain unwinds recursively on drop. The one path that still does pointer surgery is `freestypat` (`zutil.rs:185-207`), which unlinks a single entry from the middle of a style’s pattern list — it `take()`s the `next` pointer out of the doomed node and splices it into the predecessor (or the list head), matching C’s `prev->next = p->next / s->pats = p->next` relink exactly, because here the move semantics genuinely matter and a naive drop would orphan the tail. These paths are exercised by `zstyle -d` and by the `scanpatstyles` removal walk (`zutil.rs:511-535`), which iterates a style’s patterns dispatching on a `spatflags` selector for the name-collect, pattern-collect, and remove operations that the introspective `-g` and the targeted `-d` forms drive.

How patterns are matched and weighted

The heart of the style system is the rule that decides which pattern “wins” when several patterns all match a context. `zsh`’s answer is *most-specific-wins*, and “specific” is given a precise numerical definition. The weighting lives in `style_table::set` (`zutil.rs:235-303`) and is a direct port of the C scoring loop. The weight is a 64-bit value split into two halves: the high 32 bits count the number of colon-separated components in the pattern, and the low 32 bits accumulate a per-component specificity score. More components beats fewer components (a deeper, more-targeted context pattern outranks a shallow one), and within the same depth, more literal components beat more wildcard components.

The per-component score has exactly three values. A component that is a bare `*` — the wildcard standing alone, terminated by either the end of the string or a `:` — scores 0. A component that contains any pattern metacharacter from the set `( | * [ < ? # ^` scores 1. A plain literal component, with no metacharacters at all, scores 2. The loop walks the pattern byte by byte, resetting the per-component accumulator at each `:`, and folds the component scores into the weight:

```

if first
    && ch == '*'
    && (next.is_none() || next == Some(b':'))
{
    tmp = 0;
    i += 1;
    continue;
}
first = false; // c:370
if matches!(ch, '(' | '|' | '*' | '[' | '<' | '?' | '#' | '^') {
    // c:372
    tmp = 1;
}
if ch == ':' {
    // c:377
    weight += 1u64 << 32; // c:379
    first = true; // c:381
    weight += tmp; // c:382
    tmp = 2; // c:383
}

```

The subtlety the port’s comments flag is the lookahead on the bare-star case. The check is not “the component starts with `*`” but “the first character is `*` and the very next character ends the component” — that is, `(!str[1] || str[1] == ':')` in C. Without that lookahead, a pattern like `*foo:bar` would score its first component as a bare wildcard (0) instead of recognizing that `*foo` contains a metacharacter and so should score 1. The comment notes that an earlier Rust port omitted exactly this lookahead and the result was incorrect ordering — “more-specific patterns lost out to less-specific siblings.” This is the kind of bug that is invisible until a user’s completion config has two overlapping patterns and the wrong one silently wins; it is worth understanding that the byte-level fidelity here is load-bearing, not pedantry.

Once the weight is computed, insertion finds the position where the new pattern’s weight is no longer less than the existing entries and splices it in, keeping the `Vec` sorted highest-first:

```

let pos = style_patterns
    .iter()
    .position(|p| p.weight < weight)
    .unwrap_or(style_patterns.len());
style_patterns.insert(pos, sp);

```

Two further details of the algorithm are worth pinning down because they govern edge cases that users hit in practice. First, the high half of the weight — the colon-component count — is incremented at every `:` plus the implicit final component handled after the loop (`weight += tmp` at [zutil.rs:282](#)), so a pattern with `N` colons contributes `N+1` to the depth count and the low-half score of its trailing component. This means a pattern like `:completion*:cd` outranks `:completion:*` not only because it has a more-specific final literal but also because it is one level deeper, and both effects push in the same direction. Second, the per-component accumulator `tmp` starts each component at 2 (the literal score) and is only ratcheted *down* to 1 when a metacharacter is seen or *down* to 0 in the bare-star case; it never moves back up within a component. So a component that mixes a literal prefix with a trailing wildcard, like `cd*`, is scored as a metacharacter component (1), never as a literal (2) — partial-literal patterns are treated as wildcards for ranking purposes, which is why a pure-literal context segment always beats a glob segment at the same depth. The earlier-port regression the comments document is exactly the boundary between the bare-star case (0) and the metacharacter case (1): `*foo` must score 1, not 0, and the lookahead is what enforces that the 0 score is reserved for `*` standing genuinely alone in its component.

If a pattern is already present for the style, `set` does not re-insert; it replaces the existing entry’s values and eval program in place ([zutil.rs:230-234](#)), preserving its list position. Lookup then becomes trivial — `style_table::get` walks the weight-sorted list and returns the values of the first entry whose pattern matches the context ([zutil.rs:310-324](#)). The pattern match itself goes through the real glob engine: the special-cased `*` short-circuits to always-true, and everything else is tokenized and compiled with `patcompile` then tested with

`pattry` against the context string. The public `lookupstyle(ctxt, style)` wrapper ([zutil.rs:845-858](#)) is the thin entry point the rest of zsh calls; it locks `zstyletab`, delegates to `get`, and returns the value vector (or empty on no match). Because the weight order is enforced at insertion, `lookupstyle` never has to reason about specificity — first match is best match by construction.

The zstyle subcommands

`bin_zstyle` ([zutil.rs:895-1360](#)) is the single builtin that drives every mutation and query of the table. The C builtin has no option string in its `BUILTIN` spec — it parses `args[0]` directly — so the port reconstructs a local `options` struct from the first argument to keep the downstream `OPT_ISSET`-driven arms unchanged while matching zsh's exact dispatch order and diagnostics. The flag character is extracted from a leading `-X` argument, and a few error shapes are reproduced precisely: an option with extra characters (`-Lx`) produces `invalid argument: -Lx`; an unknown option letter produces `invalid option: -X`; a lone `-` or `--` with nothing following is `not enough arguments`.

The default form, with no flag, is the `set` form: `zstyle PATTERN STYLE VALUES...`. This validates the pattern by tokenizing and compiling it (rejecting malformed globs with `invalid pattern: PATTERN`), then routes through `setstypat` to insert or replace the entry ([zutil.rs:1326-1355](#)). The `-e` flag turns this into an `eval style`: `zstyle -e PATTERN STYLE BODY` stores the value as a shell program that is executed at lookup time rather than a static list. In `setstypat` ([zutil.rs:750-778](#)), the `-e` path joins the value words, parses them into an `Eprog`, and stores that program in the `stypat.eval` field; a parse failure returns 1 without modifying the table. The evaluation happens in `evalstyle` ([zutil.rs:806-839](#)), which unsets `reply`, runs the program, and then reads back `$reply` — as an array if it was set, falling back to a one-element list from the scalar form — exactly mirroring the C `getaparam("reply") / getsparam("reply")` cascade, and restoring `errflag` while preserving the user-interrupt bit.

The `-d` flag is `delete`, and it has three forms the port handles explicitly ([zutil.rs:1051-1081](#)). `zstyle -d PATTERN STYLE...` deletes one pattern from each of the named styles; the port loops over `args[1..]` for the style names, fixing an earlier bug where `zstyle -d pat sty1 sty2 sty3` silently dropped everything past `sty1`. `zstyle -d PATTERN` removes that pattern from every style. Bare `zstyle -d` empties the entire table. These map onto the `style_table::delete` method's `(pattern, style)` match arms ([zutil.rs:330-351](#)), which also prune now-empty style entries from the `HashMap` so the table does not accumulate dead keys.

The `-g` flag is `get` in its inverse, introspective form: `zstyle -g ARRAY [PATTERN [STYLE]]` collects results into the named array rather than testing a context ([zutil.rs:1282-1324](#)). Its argument layout is different from every other lookup flag — `args[0]` is the output array name, not a context — so it has its own dispatch block. With a pattern and style it fetches that exact entry's values; with a pattern alone it collects the style names attached to that pattern; with neither it collects the distinct context patterns across the whole table, de-duplicated through a `HashSet`. The result is assigned with `setaparam`.

The typed-lookup flags are where the style system meets shell variables. `zstyle -s CONTEXT STYLE NAME [SEP]` looks up the value, joins it with `SEP` (default a single space), and assigns the result to the scalar `NAME`, returning 0 if found and 1 (with an empty assignment) if not ([zutil.rs:1210-1224](#)). `zstyle -a CONTEXT STYLE NAME` assigns the value list to the array `NAME` ([zutil.rs:1258-1270](#)); its exit code keys on whether the pattern matched *at all*, so a pattern that matched but carried zero values still exits 0 with an empty array assigned — the port probes the table separately for the defined-versus-undefined distinction because the Rust `lookupstyle` collapses “no match” and “empty match” into one empty `Vec`. `zstyle -b CONTEXT STYLE NAME` coerces a single boolean-ish value to the literal string `yes` or `no` and assigns it ([zutil.rs:1227-1239](#)), treating `yes/true/on/1` as truthy.

The `style_table` wrapper exposes a small set of convenience methods that factor out the common shapes these flags need, even though C inlines them at each call site. `test` ([zutil.rs:405-418](#)) implements the `-t/-T` core: given an optional set of test values it returns true if any style value is in that set, and given no test values it returns true when the first value is one of `true/yes/on/1`. `test_bool` ([zutil.rs:424-436](#)) is the stricter single-value interrogation behind `-b`: it returns `Some(true)` for `yes/true/on/1`, `Some(false)` for

`no/false/off/0`, and `None` for anything that is not a clean single boolean token, which lets the `-b` arm fall back to the “no” assignment. These methods are flagged in the source as Rust-only conveniences (not present in `zutil.c`) precisely so a reader auditing fidelity knows they are refactoring sugar over the C logic rather than ported functions in their own right; the actual exit-code semantics still live in the `bin_zstyle` arms so the tri-state and membership behaviors are not lost.

The boolean test flags are `-t` and `-T`. Both share an arm (`zutil.rs:1117-1177`) that, when extra arguments are supplied, performs a value-membership test — exit 0 if any style value string-equals any of the extra arguments — and otherwise tests whether the first value is one of `true/yes/on/1`. The two differ in their handling of the undefined case, and `-t` in particular has a tri-state exit code that the port reproduces faithfully: exit 2 when the style is undefined for the context, exit 1 when it is defined but empty or its first value is non-boolean, and exit 0 when truthy. The comment records that an earlier port collapsed the undefined and defined-but-empty cases both to 1, losing the distinction scripts rely on. `-T` (the “true by default” variant) returns 0 in the undefined case instead. The `-m` flag is a *match* test: `zstyle -m CONTEXT STYLE PATTERN` compiles `PATTERN` and returns 0 if any of the style's values matches it (`zutil.rs:1180-1206`), guarding the static pattern buffer with `queue_signals`. The `-q` flag, which an earlier port omitted entirely, is a quiet existence check delegating to `testforstyle` (`zutil.rs:1107-1112`).

Finally, `-L` (and its alias `-l`) is the *list* form (`zutil.rs:1029-1049`), and bare `zstyle` with no arguments is the basic list (`zutil.rs:999-1028`). Both route through `printstylenode` (`zutil.rs:439-504`), which has two output modes. The basic mode prints the style name on its own line followed by indented per-pattern lines, prefixing eval styles with `(eval)`. The syntax mode emits re-feedable `zstyle` commands — `zstyle [-e] 'PATTERN' 'STYLE' 'VALUE'...` — so that `zstyle -L` output can be pasted back to reconstruct the configuration. The port's comments note that getting the `-e` flag onto the re-feedable line matters: without it an eval style would round-trip into a plain literal style, silently changing its meaning.

How completion and other subsystems consume styles

The style table is write-rare and read-hot. A user configures it once at shell startup with a block of `zstyle` commands in `.zshrc`, and from then on it is read continuously. The new completion system is the dominant reader: every time you press TAB, the completion machinery constructs context strings describing the command, the argument position, and the candidate tags, and then queries dozens of styles against those contexts to decide how to behave. Should it use a menu (`menu`)? Which completers should run, and in what order (`completers`)? How should matches be grouped and labeled (`group-name`, `format`)? Should listings be colorized, and with which colors (`list-colors`)? Each of these is a `lookupstyle` call internally, and because the per-style pattern list is pre-sorted by weight, each call is a short linear scan that stops at the first matching pattern. This is why a user can write a broad `:completion:*` default and then override it with a narrower `:completion*:cd:*` rule and trust the narrower one to win for `cd` contexts — the weighting algorithm in `set` guarantees the deeper, more-literal pattern carries a higher weight and sorts ahead of the broad default.

The same mechanism backs other subsystems that ship style-driven configuration: the VCS-info prompt module reads styles under the `:vcs_info:*` context to format branch and dirty-state indicators, and various contributed widgets read their own context namespaces. From the table's point of view there is nothing special about any of these consumers — they are all just `lookupstyle(ctxt, style)` callers with different context conventions. The contract that makes this composable is precisely the most-specific-wins rule: every consumer can publish a documented context hierarchy, ship sensible `*` defaults, and let users override at any depth without the consumer needing to know about the override. The eval-style mechanism extends this further by letting a value be computed per-lookup; a user can make `list-colors` depend on the current directory or the time of day by registering it with `-e` and a body that sets `$reply`. The cost is one shell-program execution per matching lookup, which is why eval styles are reserved for genuinely dynamic values rather than the common static case.

The listing path closes the loop on configurability. Because `zstyle -L` emits its output through `printstylenode` in the re-feedable mode (`zutil.rs:489-497`), the table is self-describing: a user can dump the entire configuration as a list of `zstyle` commands, edit it, and re-source it, with the `(eval)`-bearing entries

correctly round-tripping back through the `-e` form rather than degrading into literals. This matters for debugging completion behavior, which is otherwise opaque — when a completion does something unexpected, `zstyle -L` against the relevant context is the canonical way to see which patterns are registered and at what weight order they will be consulted. The two output formats `printstyle` switches on (`zutil.rs:474-497`) are the basic indented dump for human reading and the syntax dump for machine round-tripping, and both walk the same weight-sorted pattern list, so the listing order itself reveals the specificity ranking the lookup will follow. That transparency is part of why the style system, for all its indirection, remains debuggable: every layer — the table, the weights, the listing, the lookup — is observable through the one builtin, and the port preserves each of those observable behaviors byte-for-byte against the C source.

The companion `zutil` builtins: `zparseopts`, `zformat`, `zreg-exparse`

`zsh/zutil` bundles three more builtins that have nothing to do with styles but share the module. The most widely used is `zparseopts`, the option parser that lets shell functions accept getopt-style flags without hand-rolling a `while/case` loop. `bin_zparseopts` (`zutil.rs:2712` onward) runs in phases: first it parses its own control flags — `-D` to delete consumed options from the positional parameters, `-E` to extract (stop at the first non-option but keep scanning), `-F` to fail on an unrecognized option, `-K` to keep existing array values, `-M` to honor `=`-mappings between option names and target arrays, `-G` for GNU-style long options, and `-a/-A/-v` to name the default array, associative array, and argv array respectively. Then it parses the option *descriptions*, each a small grammar: a name, an optional `+` for multiple-occurrence, `:` for a required argument, `::` for an optional argument, and `=` to redirect into a named array. These suffixes become the `ZOF_*` flag bits (`zutil.rs:2384-2407`): `ZOF_ARG` for a required argument, `ZOF_OPT` for optional, `ZOF_MULT` for repeatable, `ZOF_GNUL/ZOF_GNUS` for the GNU long/short variants, and so on. Matching an input word against the description list happens in `lookup_opt` (`zutil.rs:2433-2462`), which honors those flags with three distinct comparison rules: a `ZOF_GNUL` option (GNU long, takes an argument) matches either exactly or as a prefix terminated by `=`; a plain `ZOF_ARG` option (cuddled, takes an argument) matches by prefix so that `-xvalue` binds `value` to `-x`; and a no-argument option matches only by exact string equality. That last rule matters — the comment records that an earlier port used a prefix match for the no-argument case too, with the result that `--foobar` mis-resolved to a `--foo` spec and silently swallowed the trailing `bar` instead of falling through to short-option dispatch. The value-collection logic in `push_val` (`zutil.rs:2757-2793`) carries two further fidelity details: the stored option name is always the dashed form (`-name`, never the raw argv word, which for cuddled args would include the value), and non-`ZOF_MULT` options reuse their existing slot so a repeat overwrites — last occurrence wins while keeping the original list position. The `str_` field decides single-versus-double element emission: a plain mandatory-argument option emits the name and its argument as two separate array elements, while optional-argument, same-slot, and GNU-long options pre-combine into one `-name[=]arg` element. The `=`-mapping between option names is resolved by `map_opt_desc` (`zutil.rs:2487` onward), which chases the alias chain from one description to another and detects cycles, so that several option spellings can deposit into a shared target array. Because all the collected values are tagged with a monotonic sequence number at push time, the final emit phase sorts by that sequence to reproduce argv order across every spec that points at the same array — the Rust analog of C's single linked list appended to in input order.

`zformat` (`bin_zformat`, `zutil.rs:1378-1548`) is a printf-like templating builtin with two modes. The `-f/-F` mode substitutes `%`-specs in a format string: each extra argument is a `c:replacement` pair registering that `%c` expands to `replacement`, with `%%` and `%)` always meaning literal `%` and `)`. The substitution supports field widths (an optional `-` for right alignment, a minimum width, and a `.MAX` truncation) and, most interestingly, a ternary form `%(c.true-text.false-text)` that selects one of two sub-strings based on whether the spec's value satisfies a numeric test. The ternary is implemented by the recursive `ZFormat::substring` walker (`zutil.rs:537-725`), which descends into each branch with a `skip` flag so that only the selected branch's text is emitted:

```

ZFormat::substring(bytes, idx, out, endcharl, specs, presence, skip || actval)?;
// Skip the delimiter
if *idx < bytes.len() && bytes[*idx] == endcharl {
    *idx += 1;
}
ZFormat::substring(bytes, idx, out, ' ', specs, presence, skip || !actval)?;

```

The `-F` variant differs from `-f` only in setting a *presence* flag that changes the ternary's test semantics from numeric comparison to “is this spec present and non-empty.” The `-a` mode is a column formatter: given a separator and a list of `left:right` pairs, it computes the maximum left-hand width across all pairs and pads every left side to that width before inserting the middle string, producing aligned two-column output (`zutil.rs:1452-1540`). The result of `-f/-F` goes to a scalar via `setsparam`; the result of `-a` goes to an array via `setaparam`.

`zregexparse` (`bin_zregexparse`, `zutil.rs:2246-2328`) is the most specialized of the three — a state-machine matcher used internally by completion to drive multi-token argument parsing. It takes two variable names (which receive match positions) and a subject string, followed by a regex-like specification of states and transitions. The builtin forces `EXTENDEDGLOB` on for the duration, parses the specification into a graph of `RParseState` nodes and `RParseBranch` transitions (`zutil.rs:3465-3503`), and then runs `rmatch` (`zutil.rs:1957` onward) to walk the subject against the state machine. The match loop is a worklist over branch sets: it starts from the result's `in_` branches and, at each iteration, scans every candidate branch looking for one whose target state's pattern matches the current cursor position. A branch is taken only if three conditions hold in sequence — the state's pattern compiles and `pattr` matches the remaining subject; the state's optional *guard* (a shell snippet run via `execstring`) leaves `LASTVAL` zero; and the match has a length read back from the `mend` parameter. When a branch is taken, the cursor advances by that length, stepping carefully over zsh's metaified bytes (each “character” may be a `Meta` byte plus the byte it escapes, `zutil.rs:2089-2103`), the branch's actions run as shell code with the match data saved and restored around them so the actions see the right `$match/$mbegin/$mend`, and the second position variable is bumped. The state's *cutoff* marker then governs backtracking: a `-` cutoff (hard) or a `/` cutoff with a non-zero match length discards the accumulated worklist and resets the first position variable to the second, committing to the path taken so far. The empty-subject case short-circuits to the result's `nullacts`, running those actions and returning success without entering the loop. The port's comments are candid that the alternation parser `iparsealt` is still incomplete, so the full regex grammar is open work; what is wired is the state-machine execution and the position-variable contract, plus the exact diagnostic split between `invalid regex : TOKEN` (a malformed token remains) and `not enough regex arguments` (the parse bailed mid-stream with nothing left over). Together these three builtins are the unglamorous plumbing that makes zsh's interactive layer expressible — option parsing, output formatting, and tokenized matching — all sharing the `zsh/zutil` module with the style table they have no direct relationship to, bound together only by the accident of where the C source happened to put them.

Chapter 6 – Diagnostics, Profiling, and Tracing

When a shell startup balloons from forty milliseconds to four hundred, when a completion stalls, when an autoloaded function silently fails to find its definition file, the question is always the same: *where did the time and the control flow actually go?* Conventional shells answer this question badly. They have `set -x`, which floods the terminal with a transcript, and they have a handful of profiling modules that count function calls, and beyond that the user is left attaching `strace`, `dtruss`, or `perf` to a forked process and reconstructing intent from syscall noise. `zshrs` treats diagnosis as a first-class architectural concern rather than a bolt-on, and it does so along two separate axes that are worth keeping distinct from the outset. The first axis is `zsh-compatible` introspection — `setopt XTRACE`, the `$functrace` / `$funcfiletrace` / `$funcsourcetrace` call-stack arrays, the `zsh/zprof` profiling module — all of which behave exactly as a thirty-year `zsh` user expects because they are faithful ports of the original C, line numbers and all. The second axis is `zshrs-original` infrastructure with no upstream counterpart: a structured `tracing`-based log that captures everything the shell does without ever touching the terminal, a `doctor` builtin that dumps the live state of every cache and pool, a `dbview` browser over the SQLite mirrors, a nanosecond-accurate `profile` block builtin, and feature-gated chrome-tracing and flamegraph emitters. This chapter walks both axes, and the line that divides them is sharp: anything that prints to your terminal is either something you asked for or something `zsh` would also have printed; everything else — every byte of init progress, every cache decision, every span — goes to a log file you will never see unless you go looking.

Execution tracing: `setopt XTRACE` and `$PS4`

The oldest diagnostic in the shell is execution tracing, reached through `setopt XTRACE`, its short form `set -x`, or the per-function flags `functions -t` and `functions -T`. When `XTRACE` is on, the shell prints each command immediately before running it, prefixed by an expansion of the `PS4` prompt parameter. In `zshrs` this is not a reimplementation but a port of the relevant pieces of `src/exec.c` and `src/utils.c`, and the comments in the Rust carry the original C line numbers as a permanent audit trail. The prefix itself is emitted by `printprompt4`, ported in `src/ported/utils.rs`, and the very first thing it does mirrors the C exactly — if tracing is off, it returns and emits nothing:

```
pub(crate) fn printprompt4() {
    // c:utils.c:1720 - `if (!isset(XTRACE)) return;`.
    if !isset(XTRACE) {
        return;
    }
}
```

The function then reads the trace prefix from the `PS4` parameter, falling back to `PROMPT4`, and if neither is set it uses the same emulation-dependent default the C source uses at `init.c:1192`: the terse `"+"` under `ksh/sh` emulation, and the informative `"+%N:%i> "` under native `zsh`, where `%N` expands to the current function or script name and `%i` to the current line number. There is one subtlety the port preserves with care. Expanding `PS4` can itself involve prompt-escape machinery that might recursively want to trace; the C code disables `XTRACE` around the expansion and restores it afterward, and the Rust does precisely the same dance:

```

let saved = isset(XTRACE);
opt_state_set(&opt_name(XTRACE), false);
let prefix = crate::prompt::expand_prompt(&prefix_template);
opt_state_set(&opt_name(XTRACE), saved);
eprint!("{}", prefix);

```

Note the destination: `eprint!`. Trace output is a diagnostic stream, and like `zsh` it goes to standard error, never standard output, so that `2>/dev/null` suppresses it without disturbing a pipeline's data. The actual command line that follows the prefix is assembled in `makecline` in `src/ported/exec.rs`, the port of `Src/exec.c:2046`. Here the machinery that prevents double-printing comes into play. A static flag `doneps4`, ported one-for-one from `Src/exec.c:262`, records whether the `PS4` prefix has already been emitted for the current logical line, so that an inner sub-evaluation does not stamp a second `+` in the middle of an argument list:

```

if isset(XTRACE) {
    if doneps4.load(Ordering::Relaxed) == 0 {
        printprompt4();
    }
    let mut first = true;
    let mut err = std::io::stderr().lock();
    use std::io::Write;
    for s in list.iter() {
        if !first {
            let _ = err.write_all(b" ");
        }
        first = false;
        let _ = err.write_all(quotedzputs(s).as_bytes());
    }
    let _ = err.write_all(b"\n");
    let _ = err.flush();
}

```

Each argument is passed through `quotedzputs` before being written, which is what makes `xtrace` output safe to read and, in principle, re-executable: arguments containing spaces, globs, or control characters come out quoted exactly as `zsh` quotes them. The `doneps4` flag is set to `1` after a `typeset/assignment` trace (`c:2520`) and reset to `0` at the start of each timed command region (`c:2939`), so the prefix appears once per command, not once per word. Tracing is not confined to external commands. Function invocation is also traced — when the shell is about to call a shell function it emits the `PS4` prefix and the function's argument vector, a port of `Src/exec.c:5559-5570`:

```

if isset(XTRACE) {
    printprompt4();
    for (i, a) in args.iter().enumerate() {
        if i > 0 {
            eprint!(" ");
        }
        eprint!("{}", quotedzputs(a));
    }
    eprintln!();
}

```

Assignment tracing is handled separately in the parameter-setting path (around `src/ported/exec.rs:5126`), where the variable `xtr` is latched from the option once at the top of the assignment loop and then drives the printing of `name=value` and `name=( v1 v2 )` forms to `stderr`. This is a port of `Src/exec.c:2516` onward, and the structure is instructive because it shows where `doneps4` is set rather than merely read. Before the loop over the assignment opcodes, if tracing is on, the prefix is emitted exactly once and the suppress-flag is raised so nothing downstream re-stamps it:

```

xtr = isset(XTRACE);          // c:2517 `xtr = isset(XTRACE);`
if xtr {
    printprompt4();           // c:2519
    doneps4.store(1, Ordering::Relaxed); // c:2520 `doneps4 = 1;`
}

```

Then for each `name=value` pair the loop writes `name=` or, for an augmenting `+=` assignment, `name+=`, followed by the quoted value, matching the C `fprintf(xtrerr, ... "%s+= " : "%s=", name)` at `c:2531-2532` and the trailing space at `c:2582`. The effect is that a line like `foo=bar baz=( a b )` traces as a single `+` prefix followed by both assignments, not two prefixes, because `doneps4` was raised by the first one. The per-function controls documented in the reference behave as upstream: `functions -t name` (equivalently `typeset -ft name`) marks a single function for tracing, and `functions -T name` does the same but turns tracing off on entry to any named function it calls that is not itself flagged, so you can scope a trace to one function's own body without drowning in the trace of everything it dispatches. The reference notes the interaction with `emulate`: if `emulate` runs inside a function marked with `functions -t`, `XTRACE` is forced on regardless of the emulation's own option set — though code run via `.`, `source`, or `eval` from that function is not considered to be running directly from it and so does not inherit the forced trace.

The call-stack arrays: `$funcstack`, `$functrace`, `$funcsourcetrace`, `$funcfiletrace`

`XTRACE` shows you commands as they run; the four call-stack special arrays let a running function ask *where am I and how did I get here?* without any tracing option set at all. These are read-only arrays populated on demand from the live function-call stack, and `zshrs` ports all four from `Src/Modules/parameter.c`, registering them in the same parameter table the C module registers (`c:2257–c:2267`). They are computed by getter functions that walk a shared `FUNCSTACK`, the Rust analogue of `zsh`'s `funcstack` linked list, and the four differ only in what they project out of each frame. `$funcstack` is the simplest: the names of the functions, sourced files, and (when `EVAL_LINENO` is set) `eval` commands currently executing, most-recent first. The reference describes it as carrying “the names of the functions, sourced files, and (if `EVAL_LINENO` is set) `eval` commands currently being executed,” with the first element being the name of the function reading the parameter.

`$functrace` reports, for each active frame, the *caller* and the line number within the caller, formatted as `caller:lineno`. Its getter, `functracegetfn`, is a port of `Src/Modules/parameter.c:648`, and the port keeps the original C body inline in the doc comment so the mapping is verifiable; the C builds each element with `sprintf(colonpair, "%s:%lld", f->caller, f->lineno)`, and the Rust does the same with the linked-list walk reversed to match C's head-first ordering:

```

for f in f_stack.iter().rev() {
    let caller = f.caller.as_deref().unwrap_or(""); // c:661
    let colonpair = format!("{}", caller, f.lineno); // c:663
    ret.push(colonpair); // c:668 *p = colonpair
}

```

`$funcsourcetrace` has the same shape but answers a different question: instead of where each frame was *called from*, it reports where each frame was *defined* — the file name and the line where the `function name` or `name ()` definition began, formatted as `filename:lineno`. For an autoloaded function loaded from a native-`zsh-format` file (body only, no enclosing `function` keyword) or for a file run via `source` or `.`, the reference specifies the line is reported as `0` because the entire file constitutes the definition, and the source file name is resolved to an absolute path when the function is loaded. The Rust getter `funcsourcetracegetfn` (port of `c:679`) reads `f.filename` and `f.flineno` rather than the caller fields, but otherwise mirrors `functracegetfn` exactly.

`$funcfiletrace` is the one the reference explicitly recommends to most users, because it gives the point of *call* expressed as absolute line numbers in real files rather than relative offsets within a function body. As the reference puts it, the array “differs from `funcsourcetrace` in that the line and file are the point of call,

not the point of definition, and differs from `functrace` in that all values are absolute line numbers in files, rather than relative to the start of a function.” This is the array that lets a logging or error-reporting helper print a meaningful `file:line` for its own call site even when buried several function frames deep. Its getter `funcfiletracegetfn` (port of `c:711`) is the most intricate of the four because it has to distinguish frames whose parent is a sourced file from frames whose parent is itself a function or an `eval`. When the parent frame is a source file (`!f->prev || f->prev->tp == FS_SOURCE`), the element is simply the caller and its line number; otherwise the absolute line is reconstructed by adding the parent’s `flineno` to the frame’s own `lineno`, with a one-line adjustment when the parent is an `FS_EVAL` frame:

```
let mut flineno = prev.flineno + f.linenno; // c:747
if prev.tp == FS_EVAL {
    flineno -= 1; // c:752-753
}
let fname = prev.filename.as_deref().unwrap_or(""); // c:754
ret.push(format!("{}", fname, flineno)); // c:756-761
```

The reason these getters all iterate the cloned stack in reverse deserves a note, because it is the kind of detail a careless port gets wrong and a faithful one documents. zsh’s `funcstack` is a singly linked list walked from the head outward via `f->prev`, so the head is the most recent frame and the C loops naturally produce most-recent-first arrays. The Rust `FUNCSTACK` is a `Vec` that pushes the newest frame at the back, so to reproduce C’s ordering the getters lock the stack, clone it to release the lock quickly, and then iterate `.rev()`. The `funcfiletracegetfn` port cannot use a simple `.rev()` iterator because it needs to look at the *parent* frame (the C `f->prev`) to decide whether to emit a caller-relative or an absolute file line, so it indexes explicitly with `n - 1 - i` for the current frame and `n - 1 - i - 1` for the parent, preserving the exact relationship the C pointer walk encodes. This is why the four arrays are guaranteed mutually consistent: they all derive from one snapshot of one stack, taken under one lock, projected four ways.

All four arrays are kept the same length and the same ordering, so element *N* of `$funcstack`, `$functrace`, `$funcsourcetrace`, and `$funcfiletrace` describe the same stack frame from four angles. Because they read from the live `FUNCSTACK` under a lock and synthesize the array fresh on each access, there is no staleness window: the values reflect the stack at the instant of the parameter read. The reference also points users at the standard `zsh_eval_context` array for the complementary view — it carries the *kind* of construct at each depth (including a `toplevel` entry absent from `$funcstack`) but in the opposite, oldest-first order.

Function profiling: the `zsh/zprof` module

For aggregate timing of where the shell spends its time across many calls, zshrs ships the `zsh/zprof` module, a faithful port of `Src/Modules/zprof.c` in `src/ported/modules/zprof.rs`. Its interface is exactly upstream’s: load it with `zmodload zsh/zprof`, run your workload, then invoke the `zprof` builtin to print accumulated statistics, or `zprof -c` to reset the internal counters without printing. As the reference states, “As long as the `zsh/zprof` module is loaded, profiling will be done and multiple invocations of the `zprof` builtin command will show the times and numbers of calls since the module was loaded,” and the format is “comparable to that of commands like `gprof`.” The module installs a wrapper, `zprof_wrapper`, around every shell-function call; on entry it pushes a stack frame recording the start time, and on exit it computes elapsed and self time and folds the result into two tables — a per-function table (`Pfunc`, port of the C `struct pfunc` at `c:38`) keyed by name, and a per-arc table (`Parc`, port of `struct parc` at `c:57`) keyed by caller → callee edge. The Rust faithfully preserves the three C structs and the six file-statics (`calls`, `ncalls`, `arcs`, `narcs`, `stack`, and the module handle) that hold the accumulating state, replacing the C linked lists with `Vecs` but keeping the same fields, including the `self` field renamed `self_time` only because `self` is a Rust keyword.

The report sort order is the heart of `zprof`’s value — functions are ranked by descending *self* time so the dominant time consumers float to the top — and the comparator `cmpsfncs` (port of `c:148`) implements exactly the C predicate. The header and per-row format are reproduced byte-for-byte from the C `printf` chain at `c:170`:

```
println!("num  calls           time           self           name");
println!(
    "-----"
);
```

Each row then reports, for one function, its display number, call count, total time, average time per call, percentage of total time, self time, average self time per call, self-time percentage, and finally the name, all from the original printf format string:

```
println!(
    "{:2}) {:4}           {:8.2} {:8.2} {:6.2}%  {:8.2} {:8.2}  {:6.2}%  {}",
    f.num, f.calls, f.time, avg_t, pct_t, f.self_time, avg_s, pct_s, f.name
);
```

After the primary listing, zprof emits per-function caller and callee breakdowns from the arc table — and the port fixes a real defect in a prior version of itself, worth mentioning because it illustrates the verification discipline. The arc blocks are sorted descending by time via `cmpparcs`; an earlier port skipped that sort and listed arcs in chronological insertion order, which “buried the dominant time consumers under low-cost arcs, defeating zprof’s ‘find the hot edges’ purpose.” Anonymous functions, which have no real name, are labeled in the report by `name_for_anonymous_function` (port of `c:217`), which synthesizes a `name [filename:lineno]` label from the current `funcstack[0]` frame so an anonymous block is still locatable in the output. Timing comes from `zgettime_monotonic_if_available`, a monotonic clock so that wall-clock adjustments cannot corrupt the deltas.

The wrapper’s bookkeeping is what distinguishes zprof’s two time columns, and the distinction is the reason zprof is worth reaching for over a simple call counter. *Total* time for a function is the entire span from entry to exit, including everything it called; *self* time is total minus the time attributable to the functions it in turn invoked. The wrapper maintains the active-call stack of `Sfunc` frames (port of `struct sfunc` at `c:49`), each holding an index into the `CALLS` table and a `beg` start timestamp; on exit it computes the elapsed for the popped frame, adds it to that function’s total, and subtracts the child’s elapsed from the parent’s self time by walking up to the parent frame on the stack. The arc table accumulates the same split per caller → callee edge, which is what feeds the per-function caller and callee breakdowns printed after the main listing. Because the report ranks by self time, a function that is called ten thousand times but does almost nothing per call will still surface near the top if its aggregate self time dominates — exactly the signal you want when hunting the real cost in a slow startup. The `zprof -c` reset clears all four tables (`CALLS`, `NCALLS`, `ARCS`, `NARCS`, per `c:141-147`) and prints nothing, which is the idiom for delimiting a measurement window: load the module, run a warmup, `zprof -c`, run the real workload, then `zprof` to read only the workload’s numbers.

The profile builtin: nanosecond block profiling

`zprof` profiles function calls and only function calls; it cannot tell you that the time went into a glob expansion or a parameter substitution rather than a function body. For that zshrs adds a builtin with no zsh counterpart, `profile`, defined in `src/extensions/ext_builtins.rs`. Its own doc comment draws the distinction precisely: “Unlike `time` (which measures one command) or `zprof` (which only profiles function calls), `profile` traces every `execute_command`, `expansion`, `glob`, and builtin dispatch inside the block.” The reference summarizes it as “in-process profiling with nanosecond accuracy. No fork, no `strace`.” It accepts a block, a script string via `-s`, or a function name via `-f`, plus management flags `--clear` and `--dump`. The implementation does not duplicate zprof’s accounting — it composes with it. Both `--clear` and the start of a fresh profiling run route through the canonical builtin dispatch to `zprof -c` so the function-level tables start clean, and `--dump` and the post-run breakdown route to `zprof` with no arguments to print them, gating on the `NCALLS` atomic so an empty run prints nothing misleading:

```

if args[0] == "--clear" {
    crate::fusevm_bridge::dispatch_builtin("zprof", vec!["-c".to_string()]);
    println!("profile data cleared");
    return 0;
}

```

Around the profiled code, `profile` records wall time with `std::time::Instant`, runs the body through `execute_script`, then prints a results block: total elapsed time formatted adaptively as seconds, milliseconds, or microseconds depending on magnitude, the exit status, and — only when functions were actually profiled — the `zprof` function breakdown. It closes by pointing the user at the two finer-grained instruments it cannot show inline: setting `ZSHRS_LOG=trace` for per-command tracing into the log, and the log file path itself. This is the deliberate layering: `profile` gives you the wall-clock total and the function rollup synchronously on your terminal, and defers the per-command firehose to the log so it does not scroll your session away.

The tracing log: `~/.zshrs/zshrs.log`

Everything the shell does that is not output your script asked for and not an error on `stderr` goes to a structured log, and this is the single most important diagnostic facility in `zshrs` because it is the mechanism that lets the shell have *zero startup chatter* while still being fully observable after the fact. The module is `src/extensions/log.rs`, and it is explicit that this has “no C source counterpart”: where C `zsh` emits diagnostics through `zwarn/zerr` directly to `stderr`, `zshrs` routes structured records through a `tracing` subscriber that writes to a file. The default file is `zshrs.log`, resolved by `log_path` to live under `$ZSHRS_HOME` or, failing that, `$HOME/.zshrs`. The module deliberately uses five separate files so that concurrent processes never interleave their output — `zshrs.log` for the interactive or scripted shell, `zshrs-lsp.log` for the language server, `zshrs-dap.log` for the debug adapter, `zshrs-daemon.log` for the background daemon, and `zshrs-recorder.log` for single-shot recorder runs.

Initialization happens once, guarded by a `OnceLock`, in `init/init_named`. The file layer is always on, opened append-mode, and writes through a blocking `Mutex<File>` rather than a non-blocking background writer — a deliberate choice the comment explains: log writes are microseconds, and a blocking writer “guarantees data reaches disk even when `std::process::exit()` skips destructors,” which matters because a shell exits hard. The verbosity is controlled by a three-level precedence: the `ZSHRS_LOG` environment variable for ad-hoc one-shot debugging takes priority, then the `[log]` level persisted in `$ZSHRS_HOME/zshrs.toml` for a set-and-forget knob, and finally a hardwired `info` default. The filter precedence is documented inline at the resolution site:

```

let env_filter = std::env::var("ZSHRS_LOG")
    .unwrap_or_else(|_| crate::daemon_presence::read_log_directive());

```

The file layer is configured with `with_ansi(false)` so the log is plain text, `with_target(true)` so each record carries its module path, `with_thread_names(true)` so the worker-pool thread that emitted a record is identifiable, and `compact()` formatting. Records carry structured key=value fields and timestamps. Because the level filter is an `EnvFilter`, you can scope it to a module — `ZSHRS_LOG=zshrs::compsys=trace` will turn on completion-subsystem tracing without flooding the file with everything else. Two feature-gated layers stack onto the same subscriber registry, both compiled out entirely when their feature is off (the comment promises “zero cost when off”). With `--features profiling`, a `tracing_chrome` layer emits `chrome://tracing` JSON to `trace-{PID}.json`, viewable in a browser’s trace viewer, capturing span timings with arguments included. With `--features flamegraph`, a `tracing_flame` layer writes folded stacks to `flame-{PID}.folded`, the input format for `inferno/flamegraph.pl` to render an SVG flame graph of where span time accumulates. A third feature, `--features prometheus`, spawns a metrics HTTP endpoint on `127.0.0.1:9090/metrics`. These are not runtime toggles or builtins — there is no `flamegraph` command — they are compile-time feature flags, and the `doctor` builtin reports which of the three were compiled into the running binary. The flag accessors `profiling_enabled`, `flamegraph_enabled`, and `prometheus_enabled` simply return the corresponding `cfg!(feature = ...)`, and a test pins each accessor to its `cfg!` so nobody can special-case a helper to lie about what was built.

doctor: live shell-state report

The `doctor` builtin, in `src/extensions/ext_builtins.rs`, is the one-shot health report — a single command that dumps the live state of the running shell across every subsystem. Its own description calls it a “diagnostic report of shell health, caches, and performance.” It prints sectioned output: an **Environment** block (zshrs version, pid, cwd, `$SHELL`, and crucially the worker pool’s size, completed-task count, and pending queue depth — the persistent worker pool being a zshrs-specific resource with no zsh analogue); a **Config** block showing whether `zshrs.toml` exists or defaults are in use; a **PATH** block counting total, valid, and missing directories plus the command-hash-table size; an **FPATH** block counting fpath directories, loaded functions, and — derived live, never hardcoded — the number of pending autoloads, computed by scanning the canonical `shfunctab` for entries carrying the `PM_UNDEFINED` flag:

```
let autoload_count = crate::ported::hashtable::shfunctab_lock()
    .read()
    .map(|t| {
        t.iter()
            .filter(|(_, shf)| (shf.node.flags as u32 & PM_UNDEFINED) != 0)
            .count()
    })
    .unwrap_or(0);
```

It then reports the **Caches (rkyv-mmapped)** block — and the comments here are load-bearing for understanding the architecture. The doctor is explicit that the only shell cache layer is rkyv-mmapped bytecode under `~/.zshrs/images/` with a top-level `index.rkyv`, and that “Hot lookups never hit SQLite — clients mmap rkyv exclusively.” The following **SQLite (read-only mirrors)** block is labeled with the same warning the architecture demands: “daemon-maintained; not read on cache lookup / hot path.” This distinction is the whole point — the rkyv images are the authoritative store the running shell reads via zero-copy mmap; the SQLite databases are *inspection-only mirrors* maintained by the daemon, for humans and tools to query, and they play no part in deciding a Tab-completion cache hit or in running compiled code. The doctor goes on through **History** (entry count), **Shell State** (counts of aliases, global and suffix aliases, scalar variables, arrays, associative arrays, set options, active traps, and registered hooks — every number derived live from the parameter and option tables), a **Log** block reporting the log file path and size, and a **Profiling** block reporting whether chrome tracing, flamegraph, and prometheus were compiled in. Every count in the report is computed at call time from the live tables; nothing is cached or hardcoded, so the report is always an honest snapshot of the instant you ran it.

dbview: browsing the SQLite mirrors

The final instrument is `dbview`, also in `src/extensions/ext_builtins.rs`, a structured browser over the read-only SQLite mirrors that does not require you to know SQL or open the database files by hand. Its purpose follows directly from the cache architecture the doctor describes: the rkyv images are not human-readable, but the daemon maintains parallel SQLite tables precisely so their contents can be inspected, and `dbview` is the front door to that inspection. Invoked with no arguments it lists every table and its row count, grouped by database — `compsys.db` with its `autoloads` (and how many of those carry compiled bytecode), `comps`, `services`, `patcomps`, `executables`, and `zstyles` tables; `history.db` with its entry count; and `plugins.db` with plugin and function counts:

```
if let OK(n) = cache.count_table("autoloads") {
    let bc_count = cache
        .count_table_where("autoloads", "bytecode IS NOT NULL")
        .unwrap_or(0);
    println!("    autoloads:    {:>6} rows    ({} compiled)", n, bc_count);
}
```

Given a table name it dumps that table; given a table and a filter it narrows — `dbview autoloads _git`

shows the single autoload row for `_git` (its source path and body/AST sizes), `dbview comps git` searches completions, `dbview history docker` searches history. A `--count` (or `-c`) flag on any table reduces the output to a bare number for scripting. Errors follow the house format — `zshrs:dbview:1: ...` on stderr — and missing subsystems are reported plainly (`no compsys cache, no history engine`) rather than crashing. The chapter-level documentation reiterates the boundary one more time so it cannot be missed: compsys is “backed by rkyv-mmap’d shards (zero-copy hot path),” and the “Read-only SQLite mirrors sit beside them for `dbview` / SQL inspection only — no role in Tab cache hit/miss.” That sentence is the single most important thing to internalize about diagnosing zshrs: when you query with `dbview`, you are reading the daemon’s mirror, not the live cache the shell actually executes from, and the two are kept in sync by the daemon rather than by the read path. Diagnosis observes; it never sits in the hot path it observes.

Choosing the right instrument

The instruments in this chapter overlap deliberately, and knowing which to reach for is most of the skill. `setopt XTRACE` answers *what ran, in what order, with what arguments* — it is the instrument for control-flow bugs, for “why did this branch fire,” for a script that does the wrong thing rather than the slow thing. Its output is a chronological transcript and it scales badly: turn it on for a complex function and you will measure your terminal’s scrollbar, not your shell. The four call-stack arrays answer *where am I in the call graph right now* from inside running code, with no option toggled and no output stream — they are the instrument a logging or error helper uses to attribute a message to its true call site, and `$funcfiletrace` specifically is the one that yields a clickable `file:line`. `zprof` answers *across the whole run, which functions cost the most self time* — it is the aggregate profiler, the right tool when startup is slow and you do not yet know which of two hundred autoloaded functions is the culprit, and its self-versus-total split is what separates a function that is itself expensive from one that merely calls expensive things. `profile` answers *for this one block, how long did it take and what did the per-command and per-function breakdown look like* — it is the scoped, synchronous probe, narrower than `zprof` in span but wider in what it accounts for, since it sees expansions, globs, and builtin dispatch that `zprof`’s function-only wrapper never observes. The `tracing` log is the substrate beneath all of them: it is always on at `info`, costs nothing you will notice, and turns into a per-command firehose the moment you set `ZSHRS_LOG=trace`, with optional chrome-tracing and flamegraph layers compiled in for the cases where a flat log is not enough and you want a timeline or a flame graph instead. `doctor` and `dbview` are orthogonal to all of this — they report state rather than behavior, the live configuration and cache contents at an instant rather than the flow of execution over time. Together they cover the full diagnostic surface without any of them forking a process, attaching a tracer, or — for everything but XTRACE and the explicit report builtins — writing a single byte to your terminal. The shell is silent by default and fully observable on demand, which is the entire design intent restated: nothing on screen you did not ask for, nothing about the shell’s behavior you cannot recover from the log after the fact.

Chapter 7 – Hooks, Traps, and Startup

A shell is not a single program that runs once; it is a host process that repeatedly hands control to user-supplied code at well-defined moments. Three distinct mechanisms govern those handoffs in zshrs, and although they are easy to conflate in everyday use, the source keeps them rigorously separate. The first is the family of *special hook functions* — `precmd`, `preexec`, `chpwd`, `periodic`, `zshaddhistory`, and `zshexit` — which fire on shell *events* rather than on operating-system signals. The second is the *trap* mechanism, which binds user code to actual POSIX signals (plus the pseudo-signals `EXIT`, `ERR`, `DEBUG`, and `ZERR`) and dispatches it from a signal-aware runtime. The third is the *startup-file sequence*, the ordered cascade of `/etc/zshenv`, `~/.zshenv`, `zprofile`, `zshrc`, and `zlogin` files that runs exactly once when the shell is born, gated by whether the shell is a login shell, an interactive shell, and whether resource-file sourcing has been disabled. These three systems share plumbing — both hooks and traps ultimately call into the same function dispatcher, and the startup files are themselves sourced through the same `source()` entry point the `.` builtin uses — but they answer different questions and fire on different schedules. This chapter walks each of them through the zshrs source, which throughout carries `// c:NNNN` annotations pinning every Rust statement to the line of the original C in `Src/init.c`, `Src/utils.c`, `Src/signals.c`, and `Src/builtin.c` it was ported from.

The Hook Dispatcher: `callhookfunc`

Every special hook in zshrs converges on a single function, `callhookfunc`, ported from C's `callhookfunc` and living in `src/ported/utils.rs`. Understanding it first makes the individual hooks fall out as trivial call sites. The function takes a hook `name`, an optional argument list, an `arrayp` flag that decides whether to also fire the parallel `*_functions` array, and a raw `retval` out-pointer through which the dispatched function's return value is reported. Its contract is subtle: the *return value* of `callhookfunc` is a “did any hook run?” status (`stat`, which is `1` when nothing fired and `0` when at least one body ran), whereas the *user code's* return value is written through the `retval` pointer. Conflating these two has real consequences — there is a load-bearing comment in `hist.rs` explaining that passing `NULL` for `retval` and reading the `stat` return instead caused every history line to be dropped under `zsh -f`, because `zshaddhistory's` caller needs the hook's actual return, not the “no hook present” sentinel.

The dispatcher has two halves. First, it looks up a shell function whose name is exactly the hook name and, if found, runs it via `doshfunc`:

```
// c:1495 - `if ((shfunc = getshfunc(name))) { doshfunc(...); stat = 0; }`
let shf_clone: Option<crate::ported::zsh_h::shfunc> = shfunctab_lock()
    .read()
    .ok()
    .and_then(|t| t.get(name).cloned());
if let Some(mut shf) = shf_clone {
    let args = mk_args(name);
    ...
    ret = crate::ported::exec::doshfunc(&mut shf, args, true, body_runner);
    stat = 0; // c:1504
}
```

Second, when `arrayp` is nonzero, it reads the array parameter named `${name}_functions` and runs every function named in it, in order:

```

if arrayp != 0 {
  // c:1507-1525 - fire every `${name}_functions` hook in order.
  let arr_name = format!("{}",_functions", name); // c:1511
  let arr = crate::ported::params::paramtab()
    .read()
    .ok()
    .and_then(|t| t.get(&arr_name).and_then(|p| p.u_arr.clone()))
    .unwrap_or_default(); // c:1512 getaparam
  for fn_name in arr {
    ...
  }
}

```

This is the entire architectural basis of the `add-zsh-hook` idiom. There is nothing magic about `precmd_functions`; it is an ordinary array parameter whose every element is invoked, by name, after the singular `precmd` function. The `mk_args` closure handles a fine point that bit an earlier port: the argument vector's element `[0]` is `$0` for the function (its own name), and the *real* positional arguments start at `[1]`. The naive version prepended the hook name to the whole list, so `preexec`'s `$1` came out as the literal string `"preexec"` instead of the command line — the comment at the closure's definition spells this out explicitly. Because each `*_functions` member is dispatched with its own name in slot `[0]`, every function in the array sees correct positional parameters.

precmd, periodic, and the Pre-Prompt Path

The `precmd` and `periodic` hooks fire from `preprompt`, ported from C's `preprompt` in `src/ported/utils.rs`, which runs the pre-prompt machinery before each interactive prompt is painted. The function first lets a pending `SIGWINCH` through and re-blocks it (so the prompt picks up the current terminal width without being interrupted mid-paint), then synchronizes job status if `NOTIFY` is unset, then bails early if `errflag` already carries an error. With those preliminaries done it fires `precmd` unconditionally on every prompt:

```

// c:1576-1580 - `callhookfunc("precmd", NULL, 1, NULL);` + errflag bail.
callhookfunc("precmd", None, 1, std::ptr::null_mut());
if (errflag.load(Ordering::Relaxed) & ERRFLAG_ERROR) != 0 {
  return;
}

```

Note the `arrayp` argument of `1`: both the `precmd` function and every member of `precmd_functions` fire here, and no argument list is passed, so these hooks receive no positional parameters. The `periodic` hook is conditional. It runs only when the `PERIOD` parameter is a positive integer and at least `PERIOD` seconds have elapsed since the last firing. The cadence is tracked in a function-local `static LAST_PERIODIC` atomic, mirroring the C file-static `lastperiodic`:

```

if period > 0 {
  let now = std::time::SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .map(|d| d.as_secs() as i64)
    .unwrap_or(0);
  if now > LAST_PERIODIC.load(Ordering::Relaxed) + period
    && callhookfunc("periodic", None, 1, std::ptr::null_mut()) == 0
  {
    LAST_PERIODIC.store(now, Ordering::Relaxed);
  }
}

```

There is a deliberate ordering subtlety here that the C source also has: the timestamp is updated only when `callhookfunc` returns `0`, i.e. only when a `periodic` hook actually existed and ran. If no `periodic` function

is defined the timestamp is left untouched, so the very moment one is defined it will fire at the next prompt rather than waiting for a stale interval to expire. After `periodic`, `preprompt` performs the mail check on the `MAILCHECK` cadence and then walks the C-level `prepromptfns` registry — a list of function pointers (not shell functions) that modules register to run pre-prompt work; the `addprepromptfn` / `delprepromptfn` pair in the same file manage that registry. Two parts of the C `preprompt` are intentionally not ported: the `PROMPT_SP` dangling-output heuristic is left as a no-op because a faithful port would need `countprompt` to measure visible width against zshrs's escape-emitting prompt expander, and the comment marks it explicitly as deferred rather than dropped.

preexec

Whereas `precmd` fires *before* the prompt, `preexec` fires *after* a command line has been read and *before* it executes, and it is the one common hook that receives meaningful arguments. Its call site is in `src/ported/init.rs`, inside the input-to-execution path. The runtime builds an argument vector that carries the job text and the permanent text of the command about to run, then dispatches:

```
let job_text = crate::ported::text::getjobtext(placeholder, None); // c:199
args.push(crate::ported::mem::dupstring(&job_text));
// c:200 - getpermtext(prog, NULL, 0)
let cmdstr = getpermtext(placeholder2, None, 0); // c:200
args.push(cmdstr.clone());
callhookfunc(
    // c:202
    "preexec",
    Some(&args),
    1,
    std::ptr::null_mut(),
);
```

Because an argument list is supplied, `preexec` and each `preexec_functions` member see the command line as `$1` (and the unexpanded/expanded variants as `$2/$3` in zsh's full contract). Immediately after the hook returns, the code clears the error flag (`errflag &= ~ERRFLAG_ERROR`, c:214) so that a hook that happened to set the error bit does not abort the user's actual command, and then, if at top level and ZLE is loaded, it notifies ZLE via `zleentry(ZLE_CMD_PREEEXEC)`.

chpwd

The `chpwd` hook fires after a successful directory change. Its dispatch is not inlined in the `cd` builtin body itself; the builtin (`bin_cd` / `cd_new`) in `src/ported/builtin.rs` carries only a comment marking the spot, while the executor wrapper in `fusevm_bridge.rs` performs the actual `callhookfunc("chpwd", ...)` after a completed `cd`:

```
// c:1258 - `callhookfunc("chpwd", NULL, 1, NULL)` fires the chpwd
// hook. Not surfaced here: the executor wrapper (fusevm_bridge.rs
// cd builtin) calls `callhookfunc("chpwd", ...)` after a successful
// cd, which dispatches both the `chpwd` shfunc and the
// `chpwd_functions` array (utils.rs:1532).
```

Like `precmd`, `chpwd` is dispatched with `arrayp = 1` and no positional arguments, so both the singular function and every member of `chpwd_functions` run. The directory-stack trim to `$DIRSTACKSIZE` happens in the same region of the `cd` machinery.

zshaddhistory

The `zshaddhistory` hook is the most behaviorally consequential of the array hooks, because its *return value* — not merely its existence — controls whether a command line is saved to history. It fires from the history-end path in `src/ported/hist.rs`. The caller carefully saves and restores `errflag` around the dispatch and, critically, passes a real out-pointer for the return value:

```
let args = vec!["zshaddhistory".to_string(), chline_text.clone()]; // c:1520-1521
// c:1522 - `callhookfunc("zshaddhistory", hookargs, 1, &hookret);`.
crate::ported::utils::callhookfunc(
    "zshaddhistory",
    Some(&args),
    1,
    &mut hookret as *mut i32,
);
```

The accompanying comment documents the bug that motivated reading `hookret` rather than the `stat` return: under `zsh -f` no `zshaddhistory` hook exists, so `callhookfunc` returns `stat = 1`; the prior port passed that `1` to the history-store decision, which interpreted it as “the hook asked to drop this line,” causing every command to be purged. With the out-pointer wired correctly, `hookret` is `0` when no hook ran, and the line is stored. A nonzero return from a real `zshaddhistory` function suppresses the save, which is exactly how filters like “don’t record commands beginning with a space” are implemented.

zshexit

The `zshexit` hook fires once, at shell teardown, from `zexit` in `src/ported/builtin.rs`. It is worth tracing the surrounding code because it shows precisely how `zshexit` (a shell-event hook) coexists with the `EXIT` trap (a signal-style trap) — they are two distinct mechanisms that both fire at exit, in a fixed order. `zexit` first runs any raw-text `EXIT` trap body stored in `traps_table`, then calls `dotrap(SIGEXIT)` to fire a function-form `TRAPEXIT`, and only then fires the `zshexit` hook:

```
// c:Src/init.c::zexit - `callhookfunc("zshexit", NULL, 1, NULL)`.
// The hook fires both a `zshexit` function (if defined) AND
// walks the `zshexit_functions` array. Distinct from SIGEXIT
// trap dispatch above - zshexit is a zsh-specific shell-event
// hook that operates parallel to trap.
let _ = crate::ported::utils::callhookfunc("zshexit", None, 1, std::ptr::null_mut());
crate::ported::signals::in_exit_trap.store(0, Relaxed);
realexit(); // c:6082
```

The same `zexit` function also encodes zsh’s “you have running jobs” behavior: when `MONITOR` is set and the exit did not come from a signal, it runs `checkjobs`, and if that warns it sets `stopmsg = 2` and defers the exit rather than terminating. There is also a Rust-only gate for in-process subshells: because `zshrs` runs (...) subshells in-process rather than as forked children, an `exit` inside a subshell must not call `process::exit` (which would kill the whole shell), so it sets `EXIT_PENDING/EXIT_VAL` and unwinds to the subshell boundary instead. This is one of the places where the in-process execution model forces a structural divergence from C’s fork-per-subshell assumption while preserving the observable semantics.

add-zsh-hook

The user-facing way to register array hooks is `add-zsh-hook`, which is not a builtin but an autoloadable shell function shipped with zsh and bundled in `zshrs`’s function set. Its header enumerates exactly which hooks it manages, and the list matches the array-hook call sites above:

```
# HOOK is one of chpwd, precmd, preexec, periodic, zshaddhistory,
# zshexit, zsh_directory_name (the _functions subscript is not required).
```

Its mechanism is mundane and reveals that hooks are just arrays. To add a function it computes `hook="${1}_functions"`, declares that array global with `typeset -ga`, appends the function name if not already present, and marks the function for autoload; to remove it, the `-d` form rewrites the array with the function filtered out and unsets the array entirely when empty:

```
local hook="${1}_functions"
local fn="$2"
...
typeset -ga $hook
set -A $hook ${P}hook $fn
autoload $autoopts -- $fn
```

Because `add-zsh-hook` does nothing but maintain the `*_functions` array that `callhookfunc` walks, supporting it required no special hook-registration machinery in the runtime — only correct array-parameter semantics and `callhookfunc`'s array arm.

Signal Traps: `settrap`, the `trap` Builtin, and `dotrap`

Traps are a separate system from hooks: they bind code to operating-system signals and the pseudo-signals `EXIT`, `ERR`, `DEBUG`, and `ZERR`. There are two storage paths for a trap body, and the `zshrs` source is explicit that it keeps them in different places than C does. A trap installed with the string form, `trap 'commands' SIG`, stores its raw body text in a `traps_table` `HashMap` keyed by canonical signal name. A trap installed as a function, `TRAPSIG() { ... }`, goes through `settrap(sig, None, ZSIG_FUNC)` and lives in the shell-function table with its `sigtrapped[sig]` slot flagged. C `zsh`'s `dotrap` handles both forms through one array; `zshrs` splits the storage and re-unifies at dispatch.

The `trap` builtin, `bin_trap` in `src/ported/builtin.rs`, parses the three forms. With no arguments it lists current traps, sorting by signal index to reproduce C's array-walk ordering (the comment notes a bug where the `HashMap`'s nondeterministic iteration order produced random listing order). With a first argument that is a signal name or a bare `-`, it clears: `trap - SIG` removes that signal's entry, and `trap -` with no signals clears all. Otherwise the first argument is the body and the rest are signals on which to install it. Installation normalizes the signal key to a canonical name — `0`, `EXIT`, and `SIGEXIT` all collapse to the `"EXIT"` key, and numeric names like `2` resolve to `INT` — and then, crucially, calls `settrap` so that `sigtrapped[idx]` is flagged:

```
// c:Src/signals.c settrap - register both the libc signal
// handler AND the sigtrapped[idx] flag. Without setting
// sigtrapped, handletrap() early-returns 0 (sees the slot
// as "not trapped") and the dotrap dispatch never fires.
```

This is a real dependency: `handletrap` — the function the OS signal path calls — early-returns when `sigtrapped[idx]` is zero, so storing a body in `traps_table` without also calling `settrap` would leave the trap dormant. The comment records that an earlier port passed `ZSIG_FUNC` for string-form traps, which sent dispatch down the function-lookup arm and silently no-opped when no `TRAPSIG` function existed (bug #80).

Function-form traps acquire their `ZSIG_FUNC` flag at function-definition time, not at `trap`-command time. In `exec.rs`, when a function whose name begins with `TRAP` is defined, the runtime resolves the suffix to a signal and calls `settrap` with `ZSIG_FUNC`:

```

if nm.len() > 4 && nm.starts_with("TRAP") {
  if let Some(sn) = getsigidx(&nm[4..]) {
    signum = sn;
    // c:5462 - `if (settrap(signum, NULL, ZSIG_FUNC))`
    if settrap(signum, None, ZSIG_FUNC) != 0 {
      ...
    }
    // c:5474 - `removetrapnode(signum);`
    removetrapnode(signum);
  }
}

```

Defining `TRAPINT` is therefore equivalent to installing an `INT` trap; the function table entry *is* the handler. The `removetrapnode` call and the surrounding logic also drop any pre-existing string-form `traps_table` entry for that signal, enforcing zsh's last-definition-wins rule so a `TRAPxxx` function that replaces an earlier `trap '...' SIG` does not cause both to fire.

dotrap: Unified Dispatch

Both storage paths are reconciled in `dotrap`, ported from C's `dotrap` in `src/ported/signals.rs`. It is reached from `handletrap` (the OS-signal path) and directly from `zexit` (for `SIGEXIT`). It first reads `sigtrapped[sig]`, returns immediately if the signal is `ZSIG_IGNORED`, and then looks up any string-form body — handling the alias sets `ZERR/ERR`, `DEBUG`, and `EXIT` so the canonical and shorthand names resolve to the same body:

```

let table_body: Option<String> = {
  let aliases: &[&str] = match sig {
    x if x == SIGZERR => &["ZERR", "ERR"],
    x if x == SIGDEBUG => &["DEBUG"],
    x if x == SIGEXIT => &["EXIT"],
    _ => &[],
  };
  ...
};
if trapped & (ZSIG_TRAPPED | ZSIG_FUNC) == 0 && table_body.is_none() {
  return 0;
}

```

A re-entry guard (if `intrap` returns early) prevents a trap body that itself fails — `trap "false" ERR; false` — from re-entering `dotrap` on its own failed status and recursing into a stack overflow. The function then dispatches in priority order: if `ZSIG_FUNC` is set (or a `TRAP<signame>` function exists in the function table even when the flag was not propagated), it calls the function with the signal number as `$1`; otherwise it runs the string-form body. A boolean `fn_dispatched` ensures the two arms never both fire for one signal — the comment ties this to zsh's last-defined-replaces semantics (bug #541). For `SIGEXIT` it maintains an `in_exit_trap` depth counter rather than a boolean, so a nested exit trap does not prematurely clear the flag. Throughout, signal queueing is disabled for the duration of dispatch and the queue level is restored to the value captured at entry, not unconditionally zeroed — a fix the comment notes was needed so a nested `dotrap` inside a `queue_signals/unqueue_signals` block does not corrupt the outer level.

Trap Bodies and Control Flow: `dotrapargs`, `trap_state`, `trap_return`

A trap body is not run in the calling context's control-flow scope; it runs in a fresh one, and the rules for how `break`, `return`, and the exit status flow across that boundary are encoded in two globals, `trap_state` and `trap_return`, both ported into `src/ported/exec.rs`. `trap_state` takes one of three values — `TRAP_STATE_INACTIVE = 0`, `TRAP_STATE_PRIMED = 1`, `TRAP_STATE_FORCE_RETURN = 2` — and tracks

whether a trap is currently being processed and whether a `return` issued inside it should promote to forcing a return out of the trap's caller:

```
/// Port of `int trap_state;` from `Src/exec.c:134`. Tracks whether
/// a trap handler is currently being processed and, paired with
/// `TRAP_RETURN` below, whether a `return` inside the trap should
/// promote to `TRAP_STATE_FORCE_RETURN` to unwind the trap caller.
pub static TRAP_STATE: std::sync::atomic::AtomicI32 = ...
```

`trap_return` carries the pending exit status from inside the trap, with a sentinel `-2` meaning “running an EXIT/DEBUG-style trap at the current level,” and is promoted to the user's `return N` value by the `return` builtin when POSIX-trap semantics apply. The function that actually drives a single trap callback with all of this scoping in place is `dotrapargs`, a direct port of C's `dotrapargs`. Its embedded C reference, reproduced verbatim in the Rust doc-comment, shows the save/restore discipline: it captures `obreaks = breaks`, `oretflag = retflag`, and `olastval = lastval` on entry, marks the signal `ZSIG_IGNORED` for the duration so a recurrence of the same signal cannot re-enter, saves the lexer and execution context with `zcontext_save/execsav`, zeroes `breaks` and `retflag` so the body starts with a clean control-flow slate, runs the `BEFORETRAPHOOK` runtime hook, dispatches either the `ZSIG_FUNC` shell function or the non-FUNC eprog body (priming `trap_return = -2` and `trap_state = TRAP_STATE_PRIMED` for the latter), runs `AFTERTRAPHOOK`, and then restores `breaks/retflag/lastval` according to whether the body forced a return and whether it set the error flag. This is why a `return` inside a `TRAPxxx` function does not silently leak into the enclosing function's control flow except through the explicit `FORCE_RETURN` promotion path, and why the exit status visible after a trap fires is governed by `trap_return` rather than the body's last command's status alone.

Local Traps and Save/Restore Scoping

Traps in zsh have a scoping rule that is easy to overlook: by default a `trap` installed inside a function persists after the function returns, but with the `LOCALTRAPS` option set, the previous trap state is saved on entry and restored on exit, so a function's trap changes are local to that function. The `EXIT` trap is special-cased — it is local unless `POSIXTRAPS` is set. The mechanism lives in `removetrap` in `src/ported/signals.rs`, which decides whether to snapshot the current state before clearing it:

```
let cond_local_or_exit = if sig == SIGEXIT {
    !isset(POSIXTRAPS) // c:771 sig==SIGEXIT branch
} else {
    isset(LOCALTRAPS) // c:771 else branch
};
if DONTSAVETRAP.load(Ordering::Relaxed) == 0 // c:769
    && cond_local_or_exit
    && locallevel != 0 // c:772
    && `locallevel &&`
    && (trapped == 0 // c:773
        && `!trapped` (logical NOT)
        || locallevel > (trapped >> ZSIG_SHIFT))
{
    dosavetrap(sig, locallevel); // c:774
}
```

The encoding here is worth unpacking. The `sigtrapped[sig]` slot stores not only flag bits but the *local level* at which the trap was installed, shifted up by `ZSIG_SHIFT`. The condition `locallevel > (trapped >> ZSIG_SHIFT)` therefore reads “the current function nesting is deeper than the level where this trap was last saved,” which is the test that prevents double-saving within the same scope while still capturing the outer trap the first time a deeper function modifies it. The comment flags a subtle porting hazard: `!trapped` in the C source is a *logical* not (`trapped == 0`), not Rust's bitwise `!` on an `i32`, which would invert all 32 bits and silently break the predicate. When the save fires, `dosavetrap` pushes a snapshot onto a save stack; the

matching `endtrapscope`, called when the function scope unwinds, pops and restores it. After deciding whether to save, `removetrapp` clears the slot, drops the eprog from `siglists`, and resets the OS-level disposition for the signal — with special branches that re-install `SIGHUP` so the shell keeps catching it, re-enable `SIGINT` delivery for interactive shells, and consult `forklevel` so `SIGPIPE` is only re-installed in the top-level shell and never inside a forked subshell. There is a guard against losing the body in the wrong order in `bin_trap`: `settrap` is deliberately called *before* the new body is inserted into `traps_table`, because `settrap`'s internal `unsettrap/dosavetrapp` chain snapshots the current body, and snapshotting after the insert would capture the new body instead of the outer scope's, causing the just-installed trap to be restored back over itself at function exit.

handletrap, Queueing, and TMOUT

The bridge between the OS signal-delivery path and `dotrap` is `handletrap` in `src/ported/signals.rs`. It looks up the signal's `sigtrapped` entry and returns immediately if the signal is not trapped, then either queues the signal or dispatches it. Queueing matters because signals can arrive at moments when running arbitrary trap code would corrupt shell state; when `trap_queueing_enabled` is set, the signal number is pushed onto a fixed-size ring buffer for later processing rather than dispatched inline:

```
if trap_queueing_enabled.load(Ordering::SeqCst) != 0 {
    // c:980-986 - push onto `trap_queue` ring buffer.
    let r = trap_queue_rear.load(Ordering::SeqCst);
    let new_rear = (r + 1) % MAX_QUEUE_SIZE;
    if new_rear != trap_queue_front.load(Ordering::SeqCst) {
        trap_queue[new_rear].store(sig, Ordering::SeqCst);
        trap_queue_rear.store(new_rear, Ordering::SeqCst);
    }
    return 1;
}
dotrap(idx); // c:990
```

Whether queueing is enabled is itself decided by `queue_traps`, which only turns it on when neither `TRAPSASYNC` is set nor the caller is the `wait` builtin — `wait` wants traps to fire immediately so a blocking wait can be interrupted, and `TRAPSASYNC` is the user's explicit request for asynchronous trap delivery. After `dotrap` runs, `handletrap` re-arms one special case: if the signal was `SIGALRM` and `TMOUT` is a positive integer, it calls `alarm(TMOUT)` again so the shell's idle-timeout timer keeps firing on its cadence:

```
if sig == libc::SIGALRM {
    let tmout = getiparam("TMOUT");
    if tmout > 0 {
        libc::alarm(tmout as u32); // c:996
    }
}
```

This is the substrate for the `TMOUT` auto-logout feature: the timer fires `SIGALRM`, `handletrap` dispatches whatever `TRAPALRM` the user defined (or zsh's default timeout behavior), and then re-arms the alarm for the next interval.

The Startup-File Sequence

When zshrs starts, `main` (in `src/ported/init.rs`) builds the option table, parses arguments, sets up I/O and values, initializes signals and builtin modules, and then calls `run_init_scripts()` exactly once, before entering the read-eval loop:

```

init_signals(); // c:1911
init_bltinmods(); // c:1912
crate::ported::builtin::init_builtins(); // c:1913
run_init_scripts(); // c:1914
setupshin(runscript.as_deref()); // c:1915
init_misc(cmd.as_deref(), &zsh_name); // c:1916

```

`run_init_scripts`, ported from C's function of the same name, captures three predicates up front — whether the shell is a login shell (`islogin()`, which is simply `isset(LOGINSHELL)`), whether it is interactive (`isset(INTERACTIVE)`), and whether it is privileged (`isset(PRIVILEGED)`) — and then branches on emulation mode. Under `ksh/sh` emulation it sources `/etc/profile`, `~/.profile`, and the `$ENV` file in the POSIX order. Under native `zsh` emulation it runs the canonical `zsh` cascade:

```

// c:1473 - source(GLOBAL_ZSHENV);
let _ = source(crate::ported::config_h::GLOBAL_ZSHENV);

// c:1476-1490 - if (isset(RCS) && unset(PRIVILEGED))
//               { newuser-probe; sourcehome(".zshenv"); }
if isset(RCS) && !privileged {
    sourcehome(".zshenv");
}
// c:1491-1498 - if (islogin) { GLOBAL_ZPROFILE? + .zprofile }
if is_login {
    if isset(RCS) && isset(GLOBALRCS) {
        let _ = source(crate::ported::config_h::GLOBAL_ZPROFILE);
    }
    if isset(RCS) && !privileged {
        sourcehome(".zprofile");
    }
}
// c:1499-1506 - if (interact) { GLOBAL_ZSHRC? + .zshrc }
if interact {
    if isset(RCS) && isset(GLOBALRCS) {
        let _ = source(crate::ported::config_h::GLOBAL_ZSHRC);
    }
    if isset(RCS) && !privileged {
        sourcehome(".zshrc");
    }
}
// c:1507-1514 - if (islogin) { GLOBAL_ZLOGIN? + .zlogin }
if is_login {
    if isset(RCS) && isset(GLOBALRCS) {
        let _ = source(crate::ported::config_h::GLOBAL_ZLOGIN);
    }
    if isset(RCS) && !privileged {
        sourcehome(".zlogin");
    }
}
}

```

The order is fixed and total: global `zshenv` always runs first, then the user `~/.zshenv`; then, for login shells only, global `zprofile` and `~/.zprofile`; then, for interactive shells only, global `zshrc` and `~/.zshrc`; then, for login shells only, global `zlogin` and `~/.zlogin`. The gating is governed by two options. `RCS` is the master switch — when it is unset, no startup files of any kind are sourced (this is what `zsh -f --no-rcs` does), which is why every clause tests `isset(RCS)`. `GLOBALRCS` independently gates only the `/etc/*` system files; unsetting it (`--no-global-rcs`) skips the global files while still allowing the per-user dotfiles, which is why the global clauses test `isset(RCS) && isset(GLOBALRCS)` while the home clauses test `isset(RCS) && !privileged`. The `!privileged` test on the user files reproduces `zsh`'s safety rule that a `setuid/setgid` invocation does not read user-writable dotfiles. The global-file paths themselves are compile-time constants in `config_h.rs` — `GLOBAL_ZSHENV = "/etc/zshenv"`, `GLOBAL_ZPROFILE = "/etc/zprofile"`, `GLOBAL_ZSHRC`

= `"/etc/zshrc"`, `GLOBAL_ZLOGIN = "/etc/zlogin"`, and `GLOBAL_ZLOGOUT = "/etc/zlogout"`.

The home-file sourcing goes through `sourcehome`, which resolves the directory differently per emulation: native zsh prefers `$ZDOTDIR` and falls back to `$HOME`, while POSIX emulation uses `$HOME` directly. It reads those from the parameter table (not the OS environment) under a `queue_signals/unqueue_signals` guard, builds `"$dir/$file"`, and calls `source`:

```
let h = if is_posix {
    getsparm("HOME")
} else {
    getsparm("ZDOTDIR").or_else(|| getsparm("HOME"))
};
...
let buf = format!("{}", h, s); // c:1713
unqueue_signals(); // c:1713
source($buf); // c:1713
```

That `source` is the same function the `.` and `source` builtins use. It first tries `try_source_file` to pick up a compiled `<file>.zwc` wordcode sibling — a missing or unreadable plain file is rescued by a loadable `.zwc`, and only both failing returns `SOURCE_NOT_FOUND`. It saves and restores `scriptname/scriptfilename/sourcelevel`, pushes an `FS_SOURCE` frame onto `FUNCSTACK` so `$funcstack`, `$functrace`, and `$funcfiletrace` correctly attribute lines to the sourced file, and routes the body through the fusevm bytecode pipeline. The reason `~/.zshenv` is the right place for `$PATH` and the reason `~/.zshrc` exists only for interactive shells both fall directly out of this cascade: `zshenv` is the only file with no gate beyond `RCS` and `privileged`, so it runs for every non-`-f` invocation including non-interactive scripts, whereas `zshrc` is wrapped in `if interact`. There is no separate “logout” sourcing in `run_init_scripts`; `~/.zlogout` and `/etc/zlogout` are consumed at exit time, on the login-shell teardown path, parallel to where `zexit` fires the `zshexit` hook and the `EXIT` trap — the same teardown machinery covered earlier in this chapter, not part of the one-shot startup cascade.

The login and interactive predicates that drive all of this are set in `parseargs`. The login flag comes from how the shell was invoked: `opts[LOGINSHELL]` is set when the `PARSEARGS_LOGIN` flag is present, which corresponds to the `-l` flag or an `argv[0]` beginning with `-`. The interactive flag defaults to whether file descriptor 0 is a terminal (`isatty(0)`), and the privileged flag is set when the real and effective uid/gid differ. These three booleans, decided once at startup, fully determine which of the five startup files run — and, together with `RCS` and `GLOBALRCS`, they make the entire boot sequence a pure function of invocation mode and option state, with no hidden order and no conditional sourcing beyond what the source above makes explicit.

Chapter 8 – The Parameter and Option Zoo

Every shell is, at bottom, a small database with a syntax bolted on top. The database is the set of named variables — parameters, in zsh’s vocabulary — and the rows that matter most are not the ones the user creates with `name=value` but the ones the shell itself owns: `?`, `$$`, `$RANDOM`, `$PATH`, `$funcstack`. These are the special parameters, and they are where the shell’s internal state leaks out into a form that scripts can read. Alongside them lives a second, parallel control surface: the option system, the roughly two hundred named flags toggled with `setopt` and `unsetopt` that change how the parser globs, how history is stored, how completion fires, whether an unset variable is an error. Together these two tables — special parameters and options — are the configuration substrate of the entire shell. zshrs reimplements both from the zsh C source rather than inventing its own, because the compatibility contract is absolute: a `.zshrc` that read `$ZSH_VERSION` or ran `setopt extendedglob` yesterday must behave identically today. This chapter is a tour of both tables as they actually exist in the zshrs source — `src/ported/params.rs` for the parameters and `src/ported/options.rs` for the options — with attention to the structures that make them work, the cases that are subtle, and the zsh idioms that exercise them.

The Shape of a Special Parameter

In the C source, special parameters are declared through a family of preprocessor macros — `IPDEF1`, `IPDEF2`, `IPDEF4` through `IPDEF10`, and `LCIPDEF` — each of which expands to an `initparam` struct row carrying a name, a type, a flag set, and a pointer to a get/set/unset function triple (the “GSU”). zshrs preserves these macros as functions. `IPDEF1` in `params.rs` builds a `paramdef` with flags `(PM_INTEGER | PM_SPECIAL) | C`, mirroring the C expansion `{ {NULL,A,PM_INTEGER|PM_SPECIAL|C}, BR(NULL),GSU(B),10,0,...}` from `Src/params.c:296`; `IPDEF2` does the same with `PM_SCALAR | PM_SPECIAL` for string-valued specials. `LCIPDEF` is a thin convenience that calls `IPDEF2(name, lc_blah_gsu, PM_UNSET)` — it builds the lowercase array half of a tied colon/array pair, starting life unset. The point of preserving these as the building blocks is that the type and flag combinations are not arbitrary: the difference between, say, a settable integer and a read-only one, or between an importable-from-environment scalar and one marked `PM_DONTIMPORT`, is entirely a matter of which flag bits the macro sets. Reimplementing them as ad-hoc constructors would invite drift; keeping the macro shape keeps the flag arithmetic honest.

The runtime table itself is `special_params`, a `6[special_paramdef]` in `params.rs`. Each row is a four-field record: a `name`, a `pm_type` (one of `PM_INTEGER`, `PM_SCALAR`, `PM_ARRAY`), a `pm_flags` bitset, and an optional `tied_name` that, when present, links the row to its counterpart in a tied pair. The table is deliberately laid out in the same order as the C `special_params[]` array, because the order is observable: it determines the bucket-chain layout of the parameter hash table and therefore the iteration order of constructs like `${(k)parameters}`. The comment in the source is blunt about the boundary in the middle of the table — the C source uses an empty `NULL` row to mark “the end of parameters available in all emulations,” and everything below that point is added only under zsh emulation (the `else` branch of the `EMULATE_SH | EMULATE_KSH` test in `createparamtable`). zshrs tracks that boundary with a `SPECIAL_PARAMS_ZSH_START` index, and ships a second, smaller table, `special_params_sh`, holding the alternative sh-emulation versions of the colon-separated path parameters — scalars with no array partner, because sh has no `$path` array.

Process, Status, and Identity

The first cluster of special parameters reports who the shell is and what just happened. `$$` is the process ID, set once when the shell initializes and — critically — *not* changed in subshells that fork without `exec`, so a command substitution sees the same `$$` as its parent; the reference index spells this out under the `$` entry. `$PPID` is the parent's PID, read-only. `$_` holds the PID of the most recent background job, the thing you capture right after `cmd &` to later `wait` on it. `$?` is the exit status of the last foreground command, and it is the parameter that scripts branch on more than any other. In `special_params` all four — `!`, `$`, `?`, `PPID` — are `PM_INTEGER` with `PM_READONLY`, alongside `HISTCMD`, `LINENO`, and `ZSH_SUBSHELL`. The identity integers `UID`, `EUID`, `GID`, `EGID` are `PM_INTEGER` but carry `PM_DONTIMPORT` instead of `PM_READONLY`: they are settable (a privileged shell can drop to another uid by assigning `UID=`) but must never be seeded from the inherited environment, since trusting an environment-supplied uid would be a security hole. `USERNAME` and `IFS` are scalars with the same `PM_DONTIMPORT` treatment for related reasons.

Two members of this group are quietly interesting in the port. `TRY_BLOCK_ERROR` and `TRY_BLOCK_INTERRUPT` are integers that default to `-1`, surfacing the state of an `always { }` block from zsh's exception mechanism. The source comment records a real bug: an earlier zshrs port set `PM_UNSET` on the table row as a “no value yet” sentinel, but that broke `{{(k)parameters}}` parity, because the C scanner emits the name of a special even when it has no stored value, and `PM_UNSET` here made the scan skip it. The fix was to drop `PM_UNSET` from the row and instead let the `-1` default emerge through the getter function. This is the recurring shape of compatibility work: the visible behavior — does `{{(k)parameters}}` list this name? — is downstream of a single flag bit, and getting the bit wrong produces a subtle, script-visible divergence that no amount of “looks right” testing would catch.

`$ARGC` and the `status` parameter are zsh-only aliases living below the emulation boundary: `ARGC` is read-only integer same as `#` (the positional-parameter count), and `status` is a read-only integer alias for `?`. The reference index confirms `$ARGC` is documented as “Same as `#`.” The `argv` array is the lowercase array form of the positional parameters, and `$0` is the scalar name of the running script or shell.

Time and Randomness

zsh exposes several views of time, and zshrs backs them with a single shared clock. `$SECONDS` is the number of seconds since the shell started; reading it computes `now - shtimer` against a process-wide `shtimer` static, and the getter `intsecondsgetfn` in `params.rs` clamps a negative difference to zero with `diff.max(0)` to guard against a lazy-init race where `now` could momentarily read earlier than the timer. Assigning to `$SECONDS` does not set a counter — it back-dates the timer: `intsecondssetfn` computes `now_sec - x` and stores that as the new origin, so `SECONDS=0` resets the stopwatch. The source preserves a precise C-fidelity detail here: the C body uses `zwarn` (informational), not `zerr` (fatal), when an assignment truncates, and it stores the value anyway; an earlier Rust port had used `zerr` and early-returned, skipping the store, which was a behavioral divergence now corrected. `$SECONDS` can be promoted to a floating-point value with `typeset -F SECONDS`, at which point the float getter `floatsecondsgetfn` returns `(now - timer).as_secs_f64()`; the promotion itself flows through `setsecondstype`, which in C swaps the GSU function pointer.

`$EPOCHSECONDS` and `$EPOCHREALTIME` (and the `epochtime` array) are not core specials but live in the `zsh/datetime` module. The port gates them on module load: the read arm for `EPOCHSECONDS` checks that `zsh/datetime` has been loaded before binding, because in stock zsh you must `zmodload zsh/datetime` before the parameter is bound — and if it isn't loaded, `{{EPOCHSECONDS:-x}}` correctly falls through to `x`. `epochtime` is a two-element read-only array of `[tv_sec, tv_nsec]` produced by `getcurrenttime()`. `$RANDOM` is the classic pseudo-random integer: `randomgetfn` returns `libc::rand() & 0x7fff`, a 15-bit value, and assigning to `$RANDOM` calls `srand` to seed the generator — so `RANDOM=42` makes the subsequent sequence deterministic, which a test in the module verifies. `$TTYIDLE` is a read-only integer reporting terminal idle time.

Tied Pairs: PATH and path

The most architecturally distinctive special parameters are the tied pairs — the scalar/array duos `PATH/path`, `CDPATH/cdpath`, `FPATH/fpath`, `MANPATH/manpath`, `MODULE_PATH/module_path`, `MAILPATH/mailpath`, `IGNORE/ignore`, `PSVAR/psvar`, and the read-only `ZSH_EVAL_CONTEXT/zsh_eval_context`. Each pair is two parameters that are two views of one underlying list. `$PATH` is a colon-joined scalar (`/usr/bin:/bin:/usr/sbin`); `$path` is the same data as an array (`(/usr/bin /bin /usr/sbin)`). Writing either one updates both, and the synchronization is what makes the abstraction useful: a script can `path+=(/opt/bin)` to append cleanly without string surgery, or `PATH=$PATH:/opt/bin` in the old style, and both reach the same place. In `special_params` the uppercase scalar rows carry `PM_TIED` with a `tied_name` pointing at the lowercase array (`PATH → path`), and a separate cluster of lowercase array rows — added below the emulation boundary — carry `PM_TIED` pointing back the other way (`path → PATH`). The bidirectional `tied_name` linkage is what `${(t)path}` needs to report the `-tied-special` type suffix; the source comment notes that without the lowercase rows in the table, `${(t)path}` and `${(t)cdpath}` would miss that suffix entirely.

The sync mechanism is `tiedarrsetfn`. When the array half is assigned, the function splits or joins through the partner's storage. It first determines the separator: in C this comes from a `joinchar` field on the tied data riding the parameter's union; the port reads `pm.u_tied` and uses the byte, defaulting to `:` for the standard colon-tied path parameters when there is no explicit join character. The C source writes through a raw pointer *into the partner array's storage* — `*dptr->arrptr` — which a safe Rust port cannot hold across hash-table entries, so instead, when `pm.ename` names the partner, the split result is written to the partner's `u_arr` via the parameter table directly. After the array side is updated, `arrfixenv` is called to push the joined string into the process environment, so a child process inherits the new `PATH`. There is a measured-against-the-binary subtlety in the empty-separator case: with `typeset -T 5 s ''` (an explicitly empty join character) zsh 5.9.1 keeps the whole assigned string as a single array element rather than splitting it character by character, and the port matches that observed release behavior rather than a literal reading of `sepsplit("")`. This is the “parity floor” discipline — when the C source and the shipped binary disagree, the binary wins, because that is what users' scripts were written against.

Introspection: funcstack, functrace, and Friends

A second category of array specials lets a function see its own call context. `$funcstack` is the call stack as an array of function names, innermost first — `funcstack[1]` is the most recently entered function. In the port (the read path in `subst.rs`) it walks the canonical `FUNCSTACK` Vec, which is stored in push order (oldest first), and reverses on read so that index 1 is the innermost frame, matching zsh. `$functrace`, `$funcsourcetrace`, and `$funcfiletrace` are the parallel arrays that report *where* each frame was called from rather than its name. They route through the ported getter functions `functracegetfn`, `funcsourcetracegetfn`, and `funcfiletracegetfn` rather than an inlined frame walk — and the source records why that matters: an earlier inline copy emitted plain `filename:lineno` for `funcfiletrace`, but the correct C behavior joins `f->prev->flineno + f->lineno` for frames whose parent is a function or an `eval`, so a naive inline version showed a caller-relative line of 0 instead of the absolute file line. These four parameters are what make a zsh error handler or a profiler possible — a `TRAPDEBUG` or an `always` block can reconstruct exactly how control arrived where it is.

`$pipestatus` belongs to the same introspective family but reports status rather than location: after a pipeline, it is an array holding the exit status of *each* command in the pipeline, so `a | b | c` leaves `pipestatus=(rc_a rc_b rc_c)` while `$?` holds only `rc_c` (unless `pipefail` is set, of which more below). In `special_params` `pipestatus` is a plain `PM_ARRAY` with no flags and no tie. `$LINENO` rounds out this group on the integer side: read-only, it reports the current line number being executed, and combined with `$funcfiletrace` it gives a script complete coordinates on itself.

Prompts and Editor Scalars

A large block of `special_params` is scalar configuration for the prompt and the line editor. The prompt scalars come in two naming schemes that zsh keeps as aliases: the short `PS1/PS2/PS3/PS4` and the long `PROMPT/PROMPT2/PROMPT3/PROMPT4`, plus the all-lowercase `prompt`. `PS1` (the main prompt) and `PS2` (the continuation prompt) are plain scalars with no flags, while the right-side prompts `RP51/RPROMPT` and `RP52/RPROMPT2` carry `PM_UNSET` because they start life with no value — an empty right prompt is the default. `PS3` is the `select`-loop prompt, `PS4` is the execution-trace prefix used by `xtrace`, and `SPROMPT` is the spelling-correction prompt. The table gives `PS4` the flag `PM_DONTIMPORT_SUID`, a hardening detail: the trace prefix is evaluated, so importing it from the environment under a `setuid` shell would be an injection vector, and the flag blocks that specific import. The line-editor scalars include `WORDCHARS` (the set of non-alphanumerics treated as part of a word for movement and deletion), `POSTEDIT` (a terminal string emitted after editing), and `ZLE_RPROMPT_INDENT` (an integer controlling right-prompt spacing, also `PM_UNSET` by default). `NULLCMD` and `READNULLCMD` configure what runs for a bare redirection like `> file` with no command — defaulting to `cat` and a pager respectively — and `OPTARG/OPTIND` back the `getopts` builtin, with `OPTIND` flagged `PM_DONTIMPORT` so option-parsing state never leaks in from a parent.

Several terminal and locale scalars round out the block. `TERM`, `TERMINFO`, `TERMINFO_DIRS`, `HOME`, `LANG`, `LC_ALL`, and the per-category `LC_COLLATE`, `LC_CTYPE`, `LC_MESSAGES`, `LC_NUMERIC`, `LC_TIME` are all `PM_UNSET`-by-default scalars: the shell binds the name so it is special — assignments to `$TERM` can re-init the terminal, assignments to `$LANG` re-init the locale — but the parameter holds no value until something sets it or it is imported from the environment. `HISTSIZE`, `SAVEHIST`, and the scalar `histchars/HISTCHARS` configure the history mechanism's sizing and its trigger characters (the `!` of history expansion, the `^` of quick substitution, the `#` of comments). `KEYBOARD_HACK` and `sunkeyboardhack` exist for an old terminal quirk. `COLUMNS` and `LINES` are settable integers reporting and controlling the terminal geometry the shell believes it has, used for `select` list layout and by the line editor; the reference index notes `$COLUMNS` is “the number of columns for this terminal session.” `SHLVL` is the shell-nesting depth, incremented on each nested interactive shell, and `FUNCNEST` caps recursion depth to prevent runaway function calls from exhausting the stack.

The Magic Hashes

Beyond the fixed-name specials, zsh exposes a set of associative-array “magic” parameters implemented by the `zsh/parameter` module: `$functions`, `$aliases`, `$options`, `$commands`, `$parameters`, `$builtins`, `$modules`, `$jobstates`, `$jobdirs`, `$jobtexts`, `$nameddirs`, `$history`, `$widgets`, and more. Reading `${functions[foo]}` returns the body of function `foo`; reading `${commands[ls]}` returns the resolved path of `ls`. Some are read-only and some are writable, and the port enforces this carefully: in `subst.rs` a `is_readonly_magic` match lists the genuinely read-only ones (`builtins`, `modules`, `parameters`, `history`, `jobstates`, `widgets`, and others), while `options`, `commands`, `functions`, and `aliases` are deliberately *excluded* from that list because they accept writes that have side effects. Assigning `options[extendedglob]=on` is translated into a `setopt` call through `setpmoption`; assigning `commands[x]=/y` installs a hashed command node exactly as `hash x=/y` would, a behavior verified against zsh 5.9 down to `whence x` returning the new path. The `$options` hash is the bridge between the parameter world and the option world — it is the option table presented as a readable, writable associative array, with each value the string `on` or `off`.

The Option Table

Options are the shell's other configuration surface. The canonical list lives in `OPTNS` in `options.rs`, a `&[&str]` of roughly two hundred names in the exact source order of the C `optns[]` table. The order is, again, load-bearing: it is the insertion sequence into the option hash table, and therefore the iteration order of `${(k)options}` and the first element of a bare `$options` under `ksharrays`. The port goes so far as to reconstruct the bucket layout — `OPTIONTAB` is a `LazyLock` that hashes every name into a 101-bucket table

with front-prepend chaining, exactly as `createoptiontable()` does, so that `print -rl ${k}options` reproduces zsh 5.9's 197-key order key-for-key. The first key it yields is `posixargzero`, observable as the first `off` printed by `setopt ksharrays; print $options` — a detail no one would notice unless it were wrong, and exactly the kind of thing the compatibility floor demands be right.

Each option has a default-on-or-off state that depends on the emulation. The C source encodes this with the insight that `#define OPT_X EMULATE_X` — the option-default bits are the emulation bits, so a flag field of `OPT_ZSH` on an option row means “on by default under zsh emulation.” `options.rs` mirrors these constants directly: `OPT_CSH`, `OPT_KSH`, `OPT_SH`, `OPT_ZSH`, the union `OPT_ALL`, and the convenience masks `OPT_BOURNE` (ksh|sh), `OPT_BSHELL`, `OPT_NONBOURNE`, `OPT_NONZSH`. Three further bits — `OPT_EMULATE`, `OPT_SPECIAL`, `OPT_ALIAS` — are layered above the emulation field and control whether an option participates in emulation switching, whether it is a special pseudo-option, and whether it is an alias for another. The function `defset(name, emulation)` answers “is this option on by default in this emulation,” and `default_on_options()` walks every option, testing `(flags & emu) != 0 && (flags & OPT_SPECIAL) == 0` to build the live default set when the shell starts or switches emulation via `emulate/install emulation`.

NO_ Negation and Aliases

zsh's most characteristic option idiom is that every option can be addressed in negated form by prefixing `no`. `setopt noclobber` is identical to `unsetopt clobber; setopt no_case_glob` (underscores are ignored) turns `caseglob` off. The mechanism is in `optlookup`, the port of `Src/options.c:684`. The function first strips underscores and ASCII-lowercases the name — and the source is careful to fold *only* the ASCII A–Z range manually rather than calling Rust's Unicode-aware `to_lowercase`, because option names are ASCII by definition and a locale like `tr_TR.UTF-8` has non-standard case mappings that would corrupt the fold. After canonicalizing, it tries the name directly; failing that, it checks for a `no` prefix and looks up the remainder, returning the *negated* option number (`-optno`) so that the caller flips the sense. `do_setopt` then negates the value when the option number is negative. So `optlookup("noglob")` resolves to `-GLOB`, and `setopt noglob` flows through as “set GLOB to off.”

The negative-optno encoding does double duty for the alias options. The tail of `OPTNS` — `braceexpand`, `dotglob`, `hashall`, `histappend`, `histexpand`, `log`, `mailwarn`, `onecmd`, `physical`, `promptvars`, `stdin`, `trackall`, `dvorak` — are `OPT_ALIAS` rows that provide bash/ksh-compatible names for native zsh options. Most map straight across (`dotglob` → `GLOBDOTS`, `hashall` → `HASHCMDS`, `physical` → `CHASELINKS`), but some are *inverting* aliases stored with a negative target: `braceexpand` → `-IGNOREBRACES` (turning brace expansion on means turning ignore-braces off) and `log` → `-HISTNOFUNCTIONS`. The port resolves these in `optlookup` both on the bare name and, crucially, after stripping a `no` prefix — the source comment records the bug where `setopt no_dotglob` (canonicalized to `nodotglob`, stripped to `dotglob`) fell through because the alias table was only consulted on the head lookup, so the alias resolution had to be duplicated on the post-`no` path so that `setopt no_dotglob` correctly flips `GLOBDOTS` off. These alias names are also filtered out of the no-argument `setopt/unsetopt` enumeration via the `ZSH_OPTION_ALIASES` set, matching the C behavior of passing `OPT_ALIAS` as an exclusion mask to the table scan — they still accept `setopt`, they just don't clutter the listing.

Single-Letter Options

Long before `setopt name`, the Bourne shell let you toggle behavior with single-letter flags — `set -e`, `set -x`, `sh -f` — and zsh keeps that surface alive. `options.rs` carries `zshletters`, a table mapping each letter to a canonical option name and a default-sense flag: 'e' is `errexit`, 'x' is `xtrace`, 'f' is `rcs`, 'm' is `monitor`, 'n' is `exec`, 'i' is `interactive`, 'l' is `login`, 'C' is `clobber`, 'F' is `glob`, 'u' is `unset`, and so on through the alphabet plus the digit aliases '0'–'9'. The third field of each row records whether the letter's natural sense is inverted relative to the option it names. The clearest example is 'n' mapped to `exec` with the flag set: `sh -n` means “no execute,” but the option it controls is `exec`, so the letter must flip the value — turning the `-n` flag on means turning `exec` off. The lookup function `optlookupc` walks this table

and returns a negative option number for the inverted letters, so `-n` and `noexec` arrive at the same place by the same negative-optno convention the long names use. The digit letters are a zsh convenience: '1' is `printexitvalue`, '2' is `badpattern`, '6' is `bgnice`, '7' is `ignoreeof`, giving terse forms for options that have no traditional single letter. The 'y' letter maps to `shwordsplit`, the option that restores Bourne word-splitting, which is fitting since the whole single-letter surface exists for Bourne compatibility. Because `optlookupc` and `optlookup` both funnel into the same `dosetopt`, every behavior reachable by `setopt name` is reachable by its letter and vice versa — there is one option store, addressed three ways (long name, negated long name, letter).

Listing Option State

A bare `setopt` with no arguments lists the options that differ from their emulation default; `unsetopt` with no arguments lists the complementary set; and `setopt kshoptionprint` switches both to an explicit two-column on/off dump. All three behaviors come out of one function, `printoptionnode`, ported from `Src/options.c:450`. The logic is compact and worth following because it shows how the default-sense bit drives output. The function reads three facts about an option: its current state `on`, whether it is on by default in the active emulation (`default_on`, from `default_on_options()`), and whether `kshoptionprint` is set. In ksh-print mode it always prints a name and a value, but it chooses the *name's polarity* from the default: an option that is on by default is printed with a `no` prefix and its value reported as the negation, so a defaulted-on option that is still on prints `nofoo off` — the column is phrased relative to the deviation. In the normal (non-ksh) mode the function prints a name only when the requested set membership matches the exclusive-or of current state and default: `set == (on ^ default_on)`. That XOR is the heart of “show me what changed” — an option appears in the `setopt` listing exactly when its current state differs from its default, and the leading `no` is emitted when the deviation is in the off direction (`set ^ on`). The result is that `setopt` with no arguments produces precisely the minimal set of toggles you would have to re-issue to recreate the current configuration from a fresh shell, which is the property that makes the output useful for saving and restoring state.

Categories of Options

The two hundred options sort, loosely, into functional families. The emulation options — `emacs`, `vi`, `sh`, `ksh`, and the per-emulation defaults — change the shell's whole personality; `posixaliases`, `posixbuiltins`, `posixcd`, `posixjobs`, `posixstrings`, `posixtraps`, `posixargzero`, and `posixidentifiers` selectively dial in POSIX behavior without a full emulation switch. The globbing family is large and is where zsh's power is most concentrated: `extendedglob` enables the `^`, `#`, `~` operators; `globdots` makes `*` match leading dots; `nullglob` and `cshnullglob` control what happens when a pattern matches nothing; `nomatch` decides whether a non-matching glob is an error; `numericglob`, `globstarshort`, `bareglobqual`, `caseglob`, `casepaths`, `globsubst`, `globassign`, and `rcexpandparam` each tune a corner of expansion. The history family governs the fact files: `appendhistory`, `incappendhistory`, `incappendhistorytime`, `sharehistory`, `extendedhistory`, `histignoredups`, `histignorealldups`, `histignoreospace`, `histsavenodups`, `histreduceblanks`, `histverify`, `histfindnodups`, `histexpiredupsfirst`, `histnostore`, `histnofunctions`, `banghist` — a dozen and a half options for what is conceptually one feature, which is itself evidence of how much weight users put on history.

Completion options (`automenu`, `autolist`, `menucomplete`, `listambiguous`, `listpacked`, `listrowsfirst`, `listtypes`, `alwayslastprompt`, `completealiases`, `completeinword`, `recexact`, `autoparamslash`, `autoremoveslash`, `globcomplete`) shape how the line editor offers and displays matches. Job-control options (`monitor`, `notify`, `bgnice`, `checkjobs`, `checkrunningjobs`, `hup`, `longlistjobs`, `autocontinue`, `autoresume`, `posixjobs`) control background process behavior — `notify` makes the shell report job completion immediately rather than at the next prompt, `hup` sends SIGHUP to jobs on shell exit. The I/O and execution family includes `clobber` (whether `>` overwrites), `multios` (zsh's multiple-redirection feature), `pipefail` (which makes a pipeline's status reflect the rightmost *failing*

command rather than only the last, changing what `$?` reports after `a | b | c`), `errexit`, `errreturn`, `unset` (whether referencing an unset variable is an error), `noexec`, `xtrace`, and `verbose`. The prompt family — `promptsubst`, `promptbang`, `promptpercent`, `promptcr`, `promptsf`, `transientprompt` — governs how `$PS1` and friends are expanded and redrawn. And a long tail of behavioral options (`autocd`, `cdablevars`, `pushdthome`, `autopushd`, `interactivecomments`, `rcquotes`, `shwordsplit`, `warncreateglobal`, `warnnestedvar`, `typesetsilent`, `localoptions`, `localtraps`) each adjust one specific corner of day-to-day use. `shwordsplit` deserves a mention as a zsh-vs-everyone-else marker: zsh, alone among major shells, does *not* word-split unquoted parameter expansions by default, and `setopt shwordsplit` is how a script asks for the Bourne behavior.

A handful of options are worth calling out as zsh-specific inventions with no analog elsewhere. `extendedglob` and its operators are zsh's signature globbing extension. `warncreateglobal` and `warnnestedvar` are debugging aids that warn when a function creates a global or shadows an outer-scope variable — invaluable for catching scope bugs in large `.zshrc` setups. `rcquotes` lets `'` inside single quotes stand for a literal apostrophe, a small ergonomic win. `combiningchars` and `multibyte` control Unicode handling. And `localoptions`, `localtraps`, `localloops`, `localpatterns` make a function's option, trap, loop, and pattern changes automatically revert on return — the mechanism that lets a well-behaved function `setopt extendedglob` locally without poisoning the caller's environment.

Locked Options and the Write Path

Setting an option is not always allowed. `dosetopt` — the port of `Src/options.c:735` and the single choke point through which every `setopt`, `unsetopt`, and single-letter toggle flows — enforces a set of locks before it writes `opts[optno] = value`. The negation convention is handled first: a negative option number (from a `no` prefix or an inverting letter) flips the value and takes the absolute index. Then the locked-option gates run, but only on the unforced path (`force == 0`); internal callers that set `force` bypass them. The first gate refuses `setopt noexec` in an interactive shell — turning off command execution interactively would wedge the terminal, so the `EXECOPT-off-while-interactive` case is rejected. The second gate covers `interactive`, `shinstdn`, and `singlecommand`: these describe how the shell was *launched* and cannot be changed afterward, so `dosetopt` treats a request that matches the current value as a silent no-op success and any request to change it as an error. The third gate guards `zle`: turning the line editor on requires both an interactive shell and a real terminal, since a `zle` with no tty has nothing to edit. The source is explicit that one further C branch — the `PRIVILEGED-off` case at `c:756-850`, which in stock zsh drops `setuid` privileges by resetting the real and effective uid/gid — is an unported, security-critical gap flagged in the code rather than silently glossed, exactly the kind of honesty the compatibility floor requires when a port is incomplete. Past the gates, the write goes to the process-wide `OPTS_LIVE` store, the same store that backs `[[ -o name ]]`, `$options[name]`, and the `setopt` listing — there is one authoritative option state, and every reader and writer agrees on it because they all resolve through `optlookup` to the same option number.

Why Two Tables, Faithfully Ported

The discipline running through both the parameter table and the option table is the same: the C source is the specification, the shipped binary is the parity floor when they disagree, and every flag bit is potentially script-visible. A read-only flag that should be a don't-import flag, a `PM_UNSET` sentinel that suppresses a name from `${(k)parameters}`, an alias resolution missing on the post-`no` path, an empty-separator split that diverges from the 5.9.1 binary — each of these is the kind of one-bit error that no casual test surfaces but that breaks a real `.zshrc` somewhere. The two tables are small in code but enormous in contract: they are the surface every script, every plugin, and every prompt theme reads and writes, and reimplementing them means reproducing not just the names but the exact flag arithmetic, ordering, and edge-case behavior that thirty years of zsh scripts have come to depend on.

Chapter 9 – The Endgame Premise

The first two chapters described what zshrs is: a compiler instead of an interpreter, a runtime that does its own work instead of forking it out. This chapter is about why those choices are not free to revisit. zshrs is an “endgame shell,” and the word is not marketing. It is an engineering constraint — a set of rules about what the project is allowed to do, derived from the premise that this shell has to last one person’s entire working lifetime:

zshrs is the **endgame shell for its maintainer’s lifetime** — the substrate that hosts the most powerful single-author CLI environment ever assembled.

“Endgame” is a word about finality, and finality changes the math. A project that expects to be replaced can defer hard problems — break a format and write a migration later, drop a feature and bring it back if anyone complains, ship a workaround now and fix the architecture in the rewrite. An endgame project has no later and no rewrite. There is no zshrs 2 that will clean up zshrs 1’s compromises, because the premise is that this is the shell, the last one its maintainer adopts, the one that has to still be running in thirty years. That premise forbids certain designs outright. Several things that would be reasonable shortcuts in an ordinary shell are disqualified here precisely because they would not survive the horizon, and the rest of this chapter is a catalog of those prohibitions and the single premise they all descend from.

A thirty-year horizon as a design input

Most software plans for the next version. zshrs plans for the next three decades. The section that says so is titled “Plan in decades”:

zshrs is the maintainer’s life work and endgame shell. 30+ year horizon. Every architectural decision must survive that.

That single input cascades into concrete rules: bytecode formats are versioned; SQLite schemas are migration-safe; dependencies are vendorable, audited, and durable; the minimum supported Rust version is pinned; **Cargo.lock** is committed; there is “no fashionable churn”; and the code is “readable over clever — future-maintainer-at-65 has to navigate this.” Each of these is a constraint that only makes sense under a long horizon, and each is a habit that an ordinary project can skip without consequence. Pinning a minimum Rust version and committing the lockfile are how you guarantee the build is reproducible years from now instead of dependent on whatever the toolchain happened to be the day it last compiled. Auditing every dependency for durability is how you avoid waking up in 2032 to find a crate you depended on was abandoned, yanked, or rewritten with a breaking API. “No fashionable churn” is a refusal to adopt a dependency because it is currently popular rather than because it is foundational — the kind of choice that ages a codebase fast when the fashion passes. You do not take on any of this overhead if you are shipping a tool you might abandon in two years. You take it on when a **cargo build** on a fresh machine has to still work in 2035 with whatever toolchain exists then, with no human around to babysit a broken dependency graph.

The “readable over clever” rule deserves its own mention because it inverts a common instinct. A performance-obsessed shell might be expected to reach for every micro-optimization. The charter explicitly does not, on the grounds that the person maintaining the code in three decades — the same person — has to be able to navigate it. Cleverness that obscures intent is a thirty-year liability even when it is a nanosecond win. The horizon turns readability from a nicety into a requirement.

The compatibility floor is sacred

The most concrete consequence of the endgame premise is that the project cannot break what already works. This is its own principle, “Compat with the existing world is sacred”:

zpw, zsh-more-completions, the .zshrc, zinit plugins — these continue working as zshrs evolves. Compat-floor regressions are catastrophic; ship none.

This is not the usual “we value backward compatibility” boilerplate. It is a hard gate. The maintainer’s existing world is enormous: zpw at 172k lines of code with 506+ subcommands, a zsh-more-completions corpus counted at 27,387 files, and a .zshrc spanning decades. A shell that aspires to replace zsh permanently for this person cannot regress any of it, because there is no fallback — once zsh is gone, a feature zshrs gets wrong is a feature that no longer works anywhere in the maintainer’s environment. v1.0 is gated on exactly this: “Phase G is the dedicated compat-floor work; v1.0 isn’t shipped until the maintainer’s full config loads cleanly.” The release criterion is not a feature list or a date; it is the maintainer’s actual configuration loading without error.

The endgame premise turns compatibility from a goal into a constraint with teeth, and the teeth come from the time horizon. An ordinary shell can survey its users, find that some zsh feature is rarely invoked, and decide it is not worth supporting — the cost of dropping it is a few complaints. An endgame shell cannot make that trade, because “rarely used” measured against a thirty-year horizon means “used eventually.” A glob qualifier touched once a year is touched thirty times over the life of the shell, and the one time it breaks is a real failure in a config that took decades to accumulate and cannot be reconstructed from memory. A single such regression is, in the charter’s word, catastrophic. This is why Chapter 1’s lexer and parser are faithful ports of zsh’s C source rather than fresh reimplementations: a reimplementation is a new program that happens to resemble zsh and will diverge in the corners no test thought to cover, while a line-for-line port inherits zsh’s behavior including the corners. Per-construct behavior parity is the only way to guarantee the floor holds, and the only way to get per-construct parity is to port the construct, not to rebuild it from a description of what it does.

Versioned, migration-safe formats

Compatibility over decades is not only about syntax; it is about the data the shell persists. zshrs writes bytecode shards and SQLite mirrors to disk, and those formats will change as the project evolves — opcodes get added, the chunk layout gets refined, a schema grows a column. An endgame shell cannot let any of those changes break yesterday’s caches. The commitments are two and specific: “Bytecode formats versioned” and “SQLite schemas migration-safe.” The reasoning is the thirty-year horizon applied to on-disk state, where the failure mode is concrete and unacceptable: you upgrade zshrs, the bytecode format changed, and every cached .zshrc and autoloaded function on disk is now garbage that the new VM cannot read. Versioning is what prevents that — the new shell recognizes the old shard as old, and either upgrades it or rebuilds it, rather than misreading it.

The architectural rule that follows is that old caches must either upgrade cleanly or rebuild silently, and they must never break a configuration that worked the day before. This is why the authoritative store is kept separate from the inspection mirror. The compiled bytecode lives in rkyv shards under ~/.zshrs/images/; those are the cache. The SQLite files beside them are “read-only mirrors” that are “not read when deciding cache hit/miss or when running compiled code.” Keeping the authoritative format (rkyv) separate from the queryable format (SQLite) is itself a migration-safety decision, and a subtle one. The shard format has the hardest durability requirement, because the shell’s correctness depends on reading it back exactly; the SQLite mirror has the easiest, because it can be regenerated from the shards at any time and nothing depends on it for execution. By making sure nothing but the shell itself depends on the rkyv format, the design keeps the hard-to-evolve thing isolated, so that the easy-to-query thing can change freely without touching the part that has to survive thirty years. A format that has to last that long is far easier to evolve when its only consumer is the program that wrote it.

Cross-architecture from day one

A thirty-year shell cannot be tied to one chip. The JIT tier described in Chapter 1 reflects this directly: the execution table lists “Native x86-64 via Cranelift” for the JIT tiers, and Cranelift’s role here is precisely that it is a code generator built to target multiple instruction sets from one intermediate form. The supported targets are named explicitly: “x86-64, aarch64 (Linux, macOS, *BSD).” That pairing is not arbitrary — x86-64 covers the Linux servers, aarch64 covers the Apple-silicon daily driver, and the same bytecode compiles to native code on both because Cranelift, not zshrs, owns the architecture-specific lowering.

The reason this is an endgame constraint and not just a portability nicety is the horizon again. The maintainer’s daily machine is one architecture; the servers are another; the future holds others still. A shell that bet on a single architecture would be making a thirty-year wager on the chip landscape staying put — and the chip landscape did not stay put even over the last decade, as the industry’s move to aarch64 on the desktop demonstrated. Betting on one instruction set is exactly the kind of fashion-following the charter’s “no fashionable churn” rule is designed to refuse, just inverted: instead of chasing a trendy dependency, it would be clinging to a current hardware assumption. Building cross-architecture from day one, on a code generator that already speaks multiple targets, is how the project avoids a forced rewrite the first time the hardware moves again. The portability is bought once, in the choice of Cranelift, rather than paid for repeatedly in per-architecture porting work.

No instant-prompt fakery

The sharpest thing the endgame premise forbids is the very category of trick that Chapter 1 diagnosed in zsh. Having argued that zinit turbo and p10k instant prompt are lipstick on a pig, the project would undermine its entire reason for existing if it shipped its own version of the same fakery. So the prohibition is stated in the strongest terms the charter uses anywhere:

“Fully featured” means: 27,387+ completions registered, full .zshrc loaded, full zpwr loaded (506+ subcommands), all hooks installed, prompt rendered. **NEVER instant-prompt fakery.** First paint = full functionality, period.

And the list of architectural constraints ends with the same prohibition in capitals: “**NEVER instant-prompt fakery, NEVER lazy-load deferral, NEVER eventual-feature-loading.**” The distinction it draws is exact. An instant-prompt trick paints a prompt before the shell is actually ready and fills in the real state afterward — the prompt you see is a promise, not the functioning shell, and the gap between the two is where the unready time hides. The endgame premise rejects this outright. The first thing painted must be the fully functional shell: every one of the 27,387 completions registered, every hook installed, the whole .zshrc and the whole 506-subcommand zpwr loaded, the prompt rendered against real state. Not a visible prompt while real work happens in the background. The reason this is a matter of principle and not taste is consistency with the diagnosis: if the only way zshrs can appear fast is to defer work the way zsh’s plugins do, then zshrs has not solved the problem it claimed zsh could not solve — it has merely re-implemented the workaround inside the shell instead of bolting it on top. The acceptance test follows directly: a feature passes only if it removes the need for an optimization trick, never if it implements one. The consequence is explicit: “If a feature can’t fit the budget while loading everything synchronously, the cache strategy needs improvement, not a fakery layer.” The pressure of a missed budget is directed back at the architecture, where the real fix lives, rather than relieved by a deferral that would hide the miss.

This is the endgame premise functioning as a forbidding constraint in its purest form. The fast path in zshrs is not allowed to be a deferral. It is allowed to be a better cache, a faster compile, a leaner load — but the moment the answer becomes “show something fake while the real thing loads,” the charter says the architecture has failed by its own standard. The whole project exists to prove that the trick was never necessary if the architecture were right.

The target the prohibitions serve

All of these prohibitions exist to make one number reachable honestly. The performance target is hard and framed as a commitment rather than an aspiration:

Cold start, fully featured | 10-20ms (one 60fps frame + slack) | **<30ms** (below human reaction threshold) |

Ten to twenty milliseconds, fully featured — the full config, all completions, all hooks — with a hard ceiling below thirty, the threshold of human reaction. The framing is deliberate: 10-20ms is roughly one frame at 60 frames per second, so the shell is ready in about the time it takes the screen to refresh once, and 30ms is the point below which a human cannot perceive the delay at all. Hit the ceiling and the shell is, for practical purposes, instant. The phrase “fully featured” is what ties the target to the prohibitions. Any other shell could hit a low cold-start number by deferring features, which is exactly what the instant-prompt and turbo-mode tricks do — they report a fast prompt by not having loaded the slow parts yet. zshrs forbids itself that escape, which makes the target far harder, because the 10-20ms has to cover everything loading synchronously, not just a prompt appearing. That is why the target comes paired with ten architectural constraints — cache at install time rather than startup time, mmap the shards rather than cold-open SQLite, no DNS lookups at startup, no directory scans at startup, lazy worker-pool init — each of which is a way to shrink real startup work rather than a way to hide it. Every constraint moves a cost either earlier (to install time) or to a cheaper mechanism (mmap over open-and-parse), and none of them defers a feature past first paint.

The honest status of that number is worth stating plainly. The 10-20ms fully-featured cold start is a target, distinct from the warm-start figure of 7ms from Chapter 1, and the two measure different things: warm start is re-running an already-cached single program, while the fully-featured cold start is bringing the entire configured shell up to first paint. There is also a separate warm-start goal of “<5ms,” and the measured warm figure of 7ms is not yet inside it. The fully-featured cold-start target of 10-20ms is the commitment the architecture is being held to, not a benchmark already banked. These are “load-bearing numerical commitments, not aspirations” — which is a statement about how seriously the targets bind the design, not a claim that they are already met. The distinction matters for an honest reading of the project: the architecture is built to make these numbers reachable, and where the current measurement sits outside the goal, the book says so rather than rounding the goal down to the measurement.

Why the premise is the point

The endgame premise is what makes zshrs a different kind of project from the shell it replaces. zsh evolved by accretion — features added, optimization layers stacked, compatibility maintained loosely — because no one designed it under a thirty-year personal-lifetime constraint. zshrs is designed under exactly that constraint, and the constraint forbids the moves that accretion permits. It forbids breaking the compatibility floor, because there is no fallback over thirty years. It forbids unversioned formats, because yesterday’s cache cannot break tomorrow. It forbids betting on one architecture, because the hardware will move. And it forbids instant-prompt fakery, because the entire diagnosis that justified the project was that fakery is the symptom of an architecture that cannot be fixed honestly.

The two earlier chapters described capabilities — compilation, no forking. This chapter described the discipline that keeps those capabilities from decaying into the same pile of workarounds they were built to replace. A compiled, fork-free shell is an achievement; an endgame shell is a commitment to keep it compiled and fork-free, with formats that migrate and a floor that holds and a startup that is fast because it is fast, not because it pretends to be, for as long as the person who built it is alive to use it.

Chapter 10 — From Source to Bytecode

Every Unix shell since the Bourne shell in 1970 reads the command line, builds a tree out of it, and then walks that tree to do the work. Reading, building, and walking happen together, blended into one motion. `zshrs` breaks that motion apart. It reads and builds the way `zsh` does — the same lexer, the same grammar, the same abstract syntax tree, ported function-for-function from the C source — and then it stops. Instead of walking the tree, it compiles it. The output is bytecode for a stack machine called `fusevm`, and that bytecode is what runs.

The split is not cosmetic. A tree-walker conflates two jobs that have different cost profiles: understanding a command (parsing it correctly, which is hard but happens once per source line) and executing it (which happens once per pass through a loop, once per function call, potentially billions of times). By fusing them, a tree-walking shell pays the understanding cost on every execution. `zshrs` pays it once, at compile time, and bakes the conclusion into an instruction stream that the rest of the run consumes without re-deriving anything.

This chapter follows a line of shell source from the moment it enters the lexer to the moment it becomes a `fusevm::Chunk` ready to execute. There are three stages, and each one corresponds to a real file you can open:

- `src/ported/lex.rs` — the lexer, a direct port of `zsh's Src/lex.c`.
- `src/ported/parse.rs` — the parser, a direct port of `zsh's Src/parse.c`, which builds the AST.
- `src/extensions/compile_zsh.rs` — the `ZshCompiler`, original Rust with no C counterpart, which lowers the AST to `fusevm` bytecode.

The first two are faithful translations. The third is what makes `zshrs` a compiler rather than an interpreter. Understanding why the front end is split this way, and what the compiler does that `zsh's` wordcode does not, is the point of this chapter.

The lexer is `zsh's` lexer

Open `src/ported/lex.rs` and the first line of the doc comment states the contract: “Zsh lexical analyzer - Direct port from `zsh/Src/lex.c`.” This is a translation, not a reimplemention inspired by `zsh`. The token constants it imports — `AMPER`, `DAMPER`, `DINANG`, `OUTANGAMP`, `SEMIBAR`, `BAR_TOK`, and dozens more — are the same token names `zsh` uses internally, lifted out of `super::zsh_h`, the port of `zsh's` `zsh.h` header. The character markers that the lexer emits for quoted regions — `Snull`, `Bnull`, `Qstring`, `Tick`, `Inpar`, `Outpar` — are `zsh's` own internal sentinel bytes, the high-bit “tokenized” characters `zsh` uses to distinguish a literal `$` from an active one inside a partly-expanded word.

These markers are the reason the lexer cannot be a clean-room rewrite. They are not a public format; they are an internal contract between `zsh's` lexer and everything downstream of it. A word like `$'a\tb'$x[2,4]` does not lex into “the string `$'a\tb'$x[2,4]`.” It lexes into a precise interleaving of literal bytes and sentinel bytes that encodes, byte by byte, which characters are quoted, which are active metacharacters, and where one expansion candidate ends and the next begins. Every later stage — parameter expansion, glob matching, subscript parsing — reads those sentinels back out. Get one wrong and the divergence does not surface as a lexer error; it surfaces as the wrong file being globbed three stages later.

The functions carry their provenance in their bodies. `lex_context_save`, for instance, is annotated `zsh/Src/lex.c:216 lex_context_save` and copies the live lexer state field-by-field into a `lex_stack`, exactly as the C does, “so a nested parse starts from a clean slate.” The same care extends to the long-tail details that make `zsh's` lexer notoriously hard to clone — here-documents, the `RCQUOTES` option, `SHGLOB` versus `KSHGLOB` brace handling, interactive-comment behavior, the alias and reserved-word and suffix-alias

tables. The lexer locks the real `aliastab`, `reswdtab`, and `sufaliastab` hash tables (`aliastab_lock`, `reswdtab_lock`, `sufaliastab_lock`) so that alias expansion happens at exactly the point in tokenization where zsh does it. None of this is approximated.

The reason for the faithful port is the compatibility floor. zsh's tokenization rules are not documented anywhere except the C source; they are the spec. If the lexer guessed instead of porting, the guess would diverge on some obscure interaction — and that divergence would surface as a `.zshrc` that worked yesterday and does not today. The port exists so the bytes are bit-identical to what zsh produces, on the long tail of cases nobody writes down: a here-document terminated mid-pipeline, a `RCQUOTES` doubled quote, a `KSHGLOB` pattern adjacent to a suffix alias. A shell that aims to be a daily-driver replacement cannot afford to be 99% compatible, because the 1% is exactly the hand-tuned config the user has accumulated over years, and there is no warning when it breaks.

The parser builds zsh's AST

The lexer hands tokens to the parser, `src/ported/parse.rs`, described in its own header as a “direct port from zsh/Src/parse.c.” The parser's productions follow zsh's grammar function-for-function — the `par_*` family in the C becomes the `par_list`, `par_sublist`, `par_pipeline`, `par_cmd` shapes here. What comes out is an abstract syntax tree.

That tree has a specific four-tier shape, and the shape matters because every downstream stage is built around it:

```
ZshProgram → ZshList → ZshSublist → ZshPipe → ZshCommand
```

A `ZshProgram` holds a vector of `ZshLists`. A list is a sublist plus a list operator (`;`, `&`, newline). A sublist is a pipe plus a sublist operator (`&&`, `|`). A pipe is a chain of commands joined by `|`. A command is the leaf — a simple command (words and redirections), a compound command (`if`, `while`, `for`, `case`, a subshell, a function definition), or a nested list. These types live in `src/extensions/zsh_ast.rs` as a Rust-only IR, but the structure is zsh's, preserved verbatim. The README states the constraint: “The 4-tier `ZshProgram → ZshList → ZshSublist → ZshPipe → ZshCommand` AST is preserved verbatim from zsh, ensuring per-construct behavior parity.”

The four tiers are not arbitrary nesting; they are the four levels at which zsh's semantics actually differ. Operator precedence between `&&`, `;`, and `|` is encoded by which tier an operator lives at — `|` binds tightest because it is inside `ZshPipe`, `&&/|` next because they are at the sublist boundary, `;/&` loosest at the list boundary. Job control, `$?` propagation, and the short-circuit rules of `&&/|` all key off these tier boundaries. Flatten the AST into a single node type and that precedence has to be reconstructed by the compiler, which is exactly the kind of re-derivation the port exists to avoid. By keeping the tiers, the compiler reads precedence off the tree's shape rather than recomputing it.

There is one subtlety worth pulling out, because it shows how literally the port follows the original. zsh's `parse.c` does two things at once: it builds a tree *and* it serializes that tree into “wordcode,” the flat integer encoding zsh writes into `.zwc` files and then walks at runtime. The zshrs parser keeps both halves. Its header notes that it “builds an AST tree ... plus emits wordcode into ECBUF via the P9b/P9c pipeline.” The wordcode path is retained for fidelity to the C — `.zwc` compatibility, here-doc bookkeeping, the `bld_eprog` serialization — but it is *not* the execution path. zshrs never walks that wordcode. The AST is what flows forward into the compiler. The wordcode is a faithful side-effect kept alive so that the port stays honest and so `.zwc` files still load.

How literal the port has to be: `par_if`

It is tempting to read “function-for-function port” as a slogan and assume it means “roughly the same shape.” A pair of fixes to the `if/elif` parser in the latest source shows it means something far more exacting: the

control-flow structure of the C function is itself part of the spec, and reshaping it into something that looks more idiomatic introduces bugs that only surface on real-world configs.

zsh's `par_if` (`Src/parse.c:1411`) is a single `for(;;)` loop. The `if` arm and every `elif` arm flow through the same path inside that one loop — evaluate the condition, parse the then-body, and then `if (tok == SEPER) break (parse.c:1471)`. An earlier zshrs port had rewritten this as a pre-loop `if`-body followed by a separate `elif/else` loop — tidier-looking, structurally divergent. The divergence had a cost the refactor commit spells out: C's single `break` had to be duplicated per arm, and a special “fi-gate” workaround was needed to stop a brace-form arm from stealing an enclosing `fi`. Restoring the C's one-loop structure — `par_if as C's single for(;;) loop (faithful to parse.c:1411)` — deleted seventy lines, the duplicated break, and the fi-gate all at once, because the single break now covers `if`- and `elif`-bodies uniformly and a brace arm can never reach the loop-top `fi`-consume. The lesson is blunt: the C author's loop structure was load-bearing, and the “cleaner” two-loop rewrite was the source of the very special-cases it seemed to avoid.

The companion fix is the kind of bug that the strict-port discipline exists to catch, and it is worth following because it shows what one structural gap does downstream. zsh's brace `if, if { cond } { body } elif { cond } { body }`, when the `elif` had no trailing `else` and was followed by a separator and another command, aborted the *entire* top-level parse: the `elif` loop's top-of-iteration `skip_separators()` ate the `SEPER` that the outer `par_list` needed, so the next command was misread as a `STRING after compound` error. The then-body had a pre-loop early-exit for exactly this case; the `elif` arm lacked the equivalent. The real-world detonation was precise — `zinit-install.zsh:91, a } elif { ! .zinit-download-file-stdout ... } { ... }` line, aborted that whole file's parse, so its line-5 `builtin source` never ran and the recorder captured zero events from it. The fix adds the same early break after a brace-form `elif` when a separator (not another `elif/else`) follows, and was verified against real zsh on six brace-`elif` shapes. A one-arm asymmetry in a parser loop, invisible on every toy `if`, took out a line of one of the most widely used zsh plugin managers. That is why the port copies the C's structure and not merely its behavior on the cases someone thought to test.

The compiler is the new thing

Here the lineage from zsh ends and zshrs begins. `src/extensions/compile_zsh.rs` opens with an unambiguous statement of where it stands:

zshrs-original infrastructure — no C source counterpart. C zsh has `Src/parse.c::bld_eprog()` (line 547) which serializes a parsed program into wordcode + strings for `.zwc` caches; `Src/exec.c::exectree()` / `execfuncs[]` (around line 268) runs that wordcode on zsh's native **wordcode VM** (*Estate* over the buffer). zshrs compiles the same AST to **fusevm** bytecode instead (typed ops, compile-time word decomposition, tilde/glob/param classification); the fusevm Cranelift JIT can specialize hot paths.

The `ZshCompiler` struct itself is a bytecode builder wrapped around a `fusevm::ChunkBuilder`, plus the bookkeeping a compiler needs: a `slots` map from variable name to slot index, a `next_slot` counter, and vectors of jump-patch sites for `break`, `continue`, and `return/exit`. The entry points are deliberately few. `ZshCompiler::new()` constructs one; `compile(program)` consumes the whole `ZshProgram` and returns a finished `fusevm::Chunk`:

```

pub fn compile(mut self, program: &ZshProgram) -> fusevm::Chunk {
    self.compile_program(program);

    // Patch return/exit jumps to past chunk end.
    let end_pos = self.builder.current_pos();
    for patch in std::mem::take(&mut self.return_patches) {
        self.builder.patch_jump(patch, end_pos);
    }

    self.builder.build()
}

```

From there `compile_program` walks the lists, `compile_list` walks each sublist, and so on down the four tiers — a recursive descent over the AST, but a descent that emits ops rather than performs work. This is the line that separates a compiler from an interpreter, and it is worth being precise about it. A tree-walker’s recursion and a compiler’s recursion look identical on the page: both are a function per node kind, each calling itself on sub-nodes. The difference is what happens at the leaf. The tree-walker, at the leaf, *does the thing* — runs the command, opens the redirection, sets the variable. The compiler, at the leaf, *writes down how to do the thing* and returns. The recursion bottoms out in `builder.emit(...)`, not in a syscall. When the recursion finishes, nothing has executed; a `fusevm::Chunk` exists.

When `compile_list` reaches a statement, then, it does not run that statement. It emits `Op::LoadInt(rel_line)` followed by an op that records the line number, so that `$LINENO` and `xtrace` read the right value at runtime. The compiler is computing, once, at compile time, things zsh recomputes on every single execution.

Decomposition happens once, not every time

The clearest example of what the compiler buys is word handling. In the AST, a `ZshSimple`’s words are raw `Vec<String>` — strings still carrying the lexer’s marker bytes. Something has to turn `~/src/*.rs` into “expand the tilde, then glob the result,” and `${foo:-bar}` into a parameter expansion with a default. In zsh, that “something” runs every time the line executes: the wordcode VM re-examines the word, re-classifies which parts are tildes and which are globs and which are parameter references, and dispatches accordingly. The classification is correct, but it is repeated on every pass through a loop body, every call of a function.

zshrs does the classification once, at compile time, and bakes the result into typed ops. The compiler’s header spells it out: “We decompose at compile time into typed expansion ops (`Op::ExpandParam`, `Op::Glob`, `Op::TildeExpand`, `Op::CmdSubst`, etc.).” The driver for this is `compile_word_str`, which inspects a word’s marker bytes and decides its fate then and there. A word that is wholly single-quoted — `'hello'`, arriving from the lexer as `<Snull>hello<Snull>` — takes a literal shortcut: the compiler decodes it to the constant string `hello`, calls `self.builder.add_constant(...)`, and emits a single `Op::LoadConst`. No expansion machinery will ever touch it again, because the compiler proved at compile time that none is needed:

```

let decoded = decode_ansi_c(&body);
let idx = self.builder.add_constant(Value::str(decoded.as_str()));
self.builder.emit(Op::LoadConst(idx), 0);
return;

```

A word that needs a glob gets `Op::Glob`; one that needs tilde expansion gets `Op::TildeExpand`; one with a parameter reference gets `Op::ExpandParam`; a `${...}` gets `Op::CmdSubst`. The runtime never asks “what kind of word is this?” — the compiler already answered. The op stream is the answer. The texture of the emitted code reflects this: heavy use of `Op::LoadConst` and `Op::LoadInt` (constants resolved at compile time), `Op::CallBuiltin` (the dispatch target chosen at compile time), `Op::Concat` for joining word segments, `Op::GetSlot/Op::SetSlot` for variables that the compiler assigned to numbered slots rather than hash lookups.

The win compounds with the size of the word. A bare literal like `echo hi` collapses to two `Op::LoadConsts` — the compiler proved no expansion applies and emitted the finished strings. A word with three segments

— `prefix${var}suffix` — emits a `LoadConst`, an `ExpandParam`, another `LoadConst`, and a `Concat`, and that op sequence is the same whether the line runs once or a million times. The classification cost — “is this segment a parameter? a glob? a literal?” — is paid exactly once, by the compiler, no matter how hot the line becomes.

That last point — slots — is its own win. `zsh` stores every parameter in a hash table and looks it up by name on every reference. The compiler, when it can see a variable’s whole lifetime inside a chunk (a loop counter, say), assigns it a `u16` slot index and emits `Op::GetSlot(n)/Op::SetSlot(n)`. A loop that increments `i` a million times does a million array-index operations into the VM’s slot array, not a million string-keyed hash lookups. The name-to-slot mapping is computed once, while compiling, and stored in the `slots` field.

Control flow becomes jumps

Compound commands lower the same way a real compiler lowers them: into conditional and unconditional jumps with back-patching. An `if` compiles its test, emits `Op::JumpIfFalse` to a not-yet-known target, compiles the then-branch, and later patches the jump once the target address is known. Loops emit a backward jump to the top and register their `break/continue` exit points in the `break_patches` and `continue_patches` stacks so nested `break 2` resolves to the right loop. `return` and `exit` push their jump sites into `return_patches`, which `compile()` resolves to “past the end of the chunk” at the very end, as shown above.

The for-loop body is where the slot machinery and the jump machinery meet. The compiler emits `Op::SlotArrayGet` to pull the iteration array, `Op::PreIncSlotVoid` to bump the index in place without producing a value on the stack, and `Op::GetSlot/Op::SetSlot` to walk the list — all referring to slots it allocated at compile time. The shape of a counted loop in the bytecode is recognizably a counted loop, which is precisely what lets the JIT (Chapter 5) recognize and specialize it.

Why this beats re-walking wordcode

The contrast with `zsh` is not that `zsh` is slow because it is old. `zsh`’s wordcode VM is a well-engineered piece of work — a compact integer encoding walked by a tight C loop, and faster than a naive AST walk for exactly the reason a flat encoding beats pointer-chasing. The contrast is structural, and it survives no matter how tight the C loop is. `zsh`’s wordcode preserves the *tree*, flattened into integers; executing it means re-discovering, every time, what each node means. Is this word a glob? Re-check the marker bytes. Is this variable set? Re-look-it-up by name in the hash table. Which branch does this `if` take? Re-evaluate the condition and re-dispatch. The information the parser computed about *what each node is* is discarded after parsing and recomputed on every execution. Wordcode optimizes the *walk*; it does not eliminate the *re-derivation*, because the encoding still represents the tree’s structure rather than the decisions that structure implies.

`zshrs`’s bytecode preserves the *decision*, not the tree. The compiler runs the classification once and emits ops that encode the conclusion. By the time the bytecode exists, “this is a glob,” “this variable lives in slot 4,” “this branch jumps to address 87” are settled facts baked into the instruction stream. A hot loop pays the decomposition cost on the single compile pass and then never again. This is the same move every modern language VM makes — the JVM, V8, LuaJIT all lower a parse tree to a flat bytecode before execution precisely so the interpreter loop dispatches on opcodes rather than re-traversing structure. `zshrs` brings that move to the shell, which is the one corner of the toolchain that never made the transition; every production shell shipping today still walks a tree or a tree-equivalent.

And there is a second-order payoff that the wordcode design forecloses entirely: because the output is a flat, typed `fusevm::Chunk`, it can be serialized and cached. The README’s pipeline shows it — a script’s first run goes `lex → parse → ZshCompiler → fusevm::Chunk` and then `persist rkyv shard`; a warm run skips straight to `index.rkyv + mmap shard → deserialize Chunk → VM::run()` with “no lex, no parse, no compile.” The measured difference is a 717ms cold start against a 7ms warm start. You cannot mmap a tree-walk. You can mmap a chunk of bytecode. The compiler is what makes the whole bytecode-cache architecture possible, and the cache is what makes a fully-loaded shell start in single-digit milliseconds.

The front end, then, is two faithful ports and one original compiler. The ports guarantee that zshrs reads and understands shell exactly as zsh does — the compatibility floor. The compiler guarantees that what it understood is turned into a form that runs fast and caches cold. The next chapter picks up the [fusevm](#): [Chunk](#) where this one drops it and follows it into the machine that runs it.

Chapter 11 – The Virtual Machine and the JIT

The previous chapter ended with a `fusevm::Chunk` — a flat stream of typed opcodes, a constant pool, a slot count, and a little metadata. This chapter is about what happens when that chunk runs. The runner is `fusevm`, a stack-based bytecode virtual machine declared in `Cargo.toml` as a single dependency:

```
fusevm = { version = "0.14.5", features = ["jit-disk-cache", "aot"] }
```

That one line carries more than it looks — and it has grown a feature since the early builds. The `jit-disk-cache` flag is the runtime JIT's persistence; the `aot` flag is newer, and it pulls in the Cranelift object-file backend that the ahead-of-time path in Chapter 14 links against. The `0.14.4` and `0.14.5` bumps that landed it were specifically “native AOT through heap values” and “wider native AOT op coverage” — the VM's object-emitter learning to lower more of the opcode set, including heap-allocated values, to native code rather than only the arithmetic core. `fusevm` is not specific to `zshrs`. It is a shared VM — the same crate, the same `Op` enum, the same execution engine — used by both `zshrs` and `strykelang`. One virtual machine, two languages. The README states it directly: “The bytecode compiler targets the same `Op` enum that `strykelang` uses. Both frontends share fused superinstructions, extension dispatch, and the Cranelift JIT path.” When `zshrs`'s `ZshCompiler` emits `Op::Glob` and `strykelang`'s compiler emits an arithmetic op, they are emitting into the same instruction set, and the same VM executes both. The shell and the scripting language meet at the bytecode layer.

The crate documents this as a deliberate target. Its module header sketches three frontends — `stryke source`, `awkrs source`, `zshrs source` — all flowing through their own compilers into a single `fusevm::Op` stream consumed by `VM::run()` (`../fusevm/src/lib.rs:28-32`). A shared VM forces a discipline that a bespoke shell VM would not: the instruction set has to be language-agnostic, which means it cannot encode shell-specific assumptions into the opcodes themselves. Shell behavior lives in the bridge and the builtins, not in the VM core — and that separation is what keeps the VM general enough to also run a scripting language, while keeping the JIT and the superinstruction set shared rather than duplicated. The cost of building one VM well is amortized across two languages.

How `zshrs` hands a chunk to the VM

The seam between `zshrs` and `fusevm` is a single function in `src/vm_helper.rs`, `run_chunk`, which every execution path funnels through. It is short enough to read whole:

```

fn run_chunk(&mut self, chunk: fusevm::Chunk, label: &str) -> Result<i32, String> {
    if chunk.ops.is_empty() {
        return Ok(self.last_status());
    }
    crate::fusevm_disasm::maybe_print_stdout(label, &chunk);
    let mut vm = fusevm::VM::new(chunk);
    register_builtins(&mut vm);
    vm.last_status = self.last_status();
    let _ctx = ExecutorContext::enter(self);
    match vm.run() {
        fusevm::VMResult::Ok(_) | fusevm::VMResult::Halted => {
            self.set_last_status(vm.last_status());
        }
        fusevm::VMResult::Error(e) => return Err(format!("VM error: {}", e)),
    }
    Ok(self.last_status())
}

```

Four things happen. `fusevm::VM::new(chunk)` constructs a VM around the bytecode. `register_builtins(&mut vm)` wires the shell's builtins onto the VM's `CallBuiltin` opcodes — this is the bridge in `src/fusevm_bridge.rs`, the table that connects a `BUILTIN_SET` or `BUILTIN_EXEC_DYNAMIC` opcode to the actual ported zsh builtin behind it. `ExecutorContext::enter(self)` installs a thread-local pointer back to the shell state so those builtins can reach `paramtab`, options, jobs, the cwd. Then `vm.run()` executes the bytecode to completion and returns a status. The status flows back into `self.last_status()`, which is `$?` .

Notice what `run_chunk` does *not* contain: any `match` over AST nodes. There is no `if cmd is a pipeline` branch, no recursion over command structure. The structure was consumed by the compiler. By the time `run_chunk` runs, “this is a pipeline” is no longer a runtime question — it is a sequence of ops the VM executes in order. This single funnel is shared by `execute_script_zsh_pipeline`, the bytecode-cache hit path in `execute_script_file`, and the function-dispatch body runner, so VM setup stays identical everywhere.

The bridge: where opcodes meet real shell behavior

A VM that only did arithmetic would be simple. A shell VM has to *do shell things* — set a variable in the real parameter table, run a builtin, open a redirection, fork a pipeline stage. `fusevm` itself knows none of that; it is a general-purpose bytecode machine shared with a scripting language that has no concept of `$PATH`. The connection is made in `src/fusevm_bridge.rs`, and its header is emphatic about what it is and is not:

```

!!! LAST-RESORT FILE — NOT FOR NEW LOGIC !!! This file is a bridge, not a port. It exists
ONLY because zshrs uses a fusevm bytecode VM where C zsh uses its own wordcode walker
(Src/exec.c execlist). Every line here is plumbing that hooks fusevm opcodes onto the canonical
ports in src/ported/.

```

The bridge holds the `register_builtins` table — the “giant ... handler table that wires zsh builtins onto fusevm `CallBuiltin` opcodes” — plus the `BUILTIN_*` opcode constants and the thread-local executor pointer that `ExecutorContext::enter` installs. When the VM hits an `Op::CallBuiltin(BUILTIN_SET, argc)`, it looks up the handler `register_builtins` registered for that opcode, and that handler reaches through the thread-local executor pointer to call the *real* ported `set` builtin in `src/ported/builtin.rs`. The bridge does not reimplement `set`; it routes to it.

The header's discipline is deliberate and load-bearing in its own right. It forbids putting expansion, glob, typeset, or redirect logic in the bridge — those have canonical ports in `src/ported/subst.rs`, `src/ported/glob.rs`, and so on — and it states the design goal directly: “The bridge should be SHRINKING over time, not growing.” The reason maps onto the same compatibility floor that drove the lexer and parser ports. The bridge is the one place where zshrs's architecture (a bytecode VM) and zsh's behavior (the ported C) have to be stitched together, and stitching is exactly where divergence creeps in. By keeping the bridge to pure plumbing — opcode decode, value conversion, stack manipulation, the executor pointer — and forcing all *behavior* to live

in the ports, the project keeps the behavior bit-faithful to zsh and the VM general enough to also run strykelang. The bridge is the seam, and a thin seam is a strong one.

The interpreter core: jump-table dispatch and fused superinstructions

At its base, fusevm is an interpreter. It holds a stack, a slot array, a constant pool, and an instruction pointer, and it executes ops one at a time. The README names the dispatch strategy: “Interpreter: jump table dispatch + fused superinstructions.” A jump-table dispatch means the VM does not run a long `if/else` chain to decide what each opcode does; it indexes directly into a table of handlers by opcode number. For a VM running hundreds of millions of ops in a tight loop, the difference between a chain of comparisons and a single indexed jump is the difference between a fast inner loop and a slow one.

The more interesting half is “fused superinstructions.” A superinstruction is a single opcode that does the work of several. The compiler, when it sees a common multi-op pattern, emits one fused op instead of the sequence. The for-loop counter from Chapter 4 is the clearest case: incrementing a loop variable is, naively, “load the slot, push one, add, store back to the slot, and discard the result.” That is four or five ops and four or five trips around the dispatch loop. The compiler instead emits a single `Op::PreIncSlotVoid(i_slot)` — pre-increment the value in slot `i_slot`, in place, producing nothing on the stack. One opcode, one dispatch, no stack traffic. The compiler emits exactly this in its loop lowering:

```
self.builder.emit(Op::PreIncSlotVoid(i_slot), 0);
```

`Op::SlotArrayGet` is another — fetch a slot’s array value in one op rather than load-then-index. Fusing collapses dispatch overhead by collapsing the number of ops, and because the fusing happens in the compiler, the VM’s interpreter loop stays simple: it gains a few more handlers in its jump table, not a new layer of logic. The README’s banner counts 224 opcodes in the instruction set (README.md:532; the `Op` enum in `./fusevm/src/op.rs` has 224 variants), a fraction of which exist to fuse common patterns rather than to express a primitive operation.

The fused family the JIT specifically recognizes is named in the `jit.rs` eligibility list: alongside the plain `PreIncSlot/PreIncSlotVoid`, it lists `SlotLtIntJumpIfFalse`, `SlotIncltIntJumpBack`, `AccumSumLoop`, and `AddAssignSlotVoid` (`./fusevm/src/jit.rs:45`). Read those names and the design intent is legible: `SlotLtIntJumpIfFalse` is “compare a slot against an integer constant and conditionally branch” — a whole loop-condition test in one op. `SlotIncltIntJumpBack` is “increment a slot, test it against a bound, and jump back to the loop top if still in range” — an entire counted-loop back-edge fused into a single instruction. `AccumSumLoop` reduces a running sum without per-iteration stack traffic. These are not micro-optimizations bolted on after the fact; they are the shapes the compiler is taught to emit and the JIT is taught to compile, chosen because they are what shell loops and arithmetic actually do. The interpreter wins by doing fewer dispatches; the JIT wins because a fused op maps cleanly onto a handful of native instructions with no intervening stack model to model.

Above the interpreter: the three-tier Cranelift JIT

For hot code, interpretation — even fast interpretation — is not the ceiling. fusevm has a Cranelift just-in-time compiler that turns bytecode into native machine code, and it is three tiers, not one. The crate’s module documentation in `./fusevm/src/lib.rs` names them (`./fusevm/src/lib.rs:7-10`): “Three-tier Cranelift JIT: Linear (straight-line, compile on first call); Block (whole-chunk CFG, threshold 10); Tracing (hot loop body, threshold 50, full side-exit machinery).”

Three tiers rather than one is a response to the fact that hot code does not come in a single shape. A branchless arithmetic sequence, a chunk full of loops and conditionals, and a single scorching loop inside an otherwise cold function each have a different optimal compilation strategy, and a single JIT design has to compromise

across all three. fusevm declines the compromise and ships a tier matched to each shape, with the VM choosing among them at runtime by how the code behaves. The tiers target different code shapes:

- **Linear JIT** handles straight-line bytecode — a run of arithmetic and slot operations with no branches. The `jit.rs` header describes it as compiling “straight-line (no branches) sequences in a single Cranelift basic block” (`../fusevm/src/jit.rs:27-29`), with the stack ending on exactly one value. The slot loads become register loads, the adds become native add instructions, and the sequence runs without re-entering the dispatch loop. This is the cheapest tier to produce — no control-flow analysis, no stack-merge reasoning — so it compiles on the first call. It is the right tool for the common case of a few arithmetic ops with no branch to reason about.
- **Block JIT** handles control flow — “bytecodes with control flow (loops, conditionals, short-circuit &&/||) via a basic-block CFG with abstract stack merges at block joins” (`../fusevm/src/jit.rs:31-33`). It compiles the *whole chunk* to native code with the jumps wired up as native conditional branches, reasoning about how the abstract value stack lines up where two control-flow paths rejoin. This is the tier that matters most for shell scripts, because shell scripts spend their time in loops: a `for` over a glob expansion, a `while read` over a file, a `repeat` counting iterations. The compiler emitted that loop as a recognizable shape — counter slot, `PreIncSlotVoid`, backward jump — and the Block JIT turns the whole thing into a native loop. Its advantage over the Tracing tier is that when an entire chunk is block-eligible there is no interpreter in the picture at all: the chunk runs as one native function with zero VM-side dispatch.
- **Tracing JIT** is the deepest tier. Rather than compiling a static block, it records the *actual hot path* through the bytecode, anchored at backward branches (loop headers). The recorder mechanism lives in the VM’s `TraceRecorder` (`../fusevm/src/vm.rs:44-73`): the recorder is armed when a loop header crosses the backedge threshold, and while armed “every dispatched op is appended to `ops` before the op’s effect is applied,” closing “when the interpreter takes a backward jump that lands at `close_anchor_ip`” — producing a linear trace of the real path taken on the hot iteration. Because that path may, on a later iteration, take a branch the trace did not record, the Tracing tier carries full side-exit and deoptimization machinery: guards that fall out of native code, reconstruct the value stack, and resume in the interpreter when a trace’s assumptions no longer hold. This is the tier for the case Block JIT cannot take — a small hot loop nested inside a chunk that is otherwise ineligible because it calls builtins, forks, or expands globs. Block JIT needs the *whole chunk* to be eligible; Tracing needs only the hot *path* to be, so it reaches code Block JIT structurally cannot. `lib.rs` documents this tier as built out across nine phases (`../fusevm/src/lib.rs:12-24`), from “Loop bodies, int slots, no calls” (Phase 1) up through “Side-trace stitching from hot deopt sites” (Phase 9), with intermediate phases covering cross-call inlining of branchless callees, caller- and callee-frame branches with side-exits, value-stack reconstruction on deopt, a side-exit deopt counter with auto-blacklist, persistent trace-metadata export/import, and bounded recursion inlining to depth 4.

At runtime the tiers are tried in priority order, documented in `VM::run()` (`../fusevm/src/vm.rs:735-755`): **Block JIT first** — “if the entire chunk is block-eligible ... the whole chunk runs in native code with zero interpreter dispatch”; then **Tracing JIT** — “when block JIT doesn’t apply, the dispatch loop runs with the recorder armed at backward branches; hot loops compile to traces that take over subsequent iterations”; then the **Interpreter** as “fallback for cold code and chunks neither tier handles.” Block JIT goes first because, when it applies, it has zero VM-side overhead — “direct fn-ptr through the slot pointer” (`../fusevm/src/vm.rs:749`); the two JIT tiers “don’t compete on the same chunk: block-eligible chunks short-circuit before tracing JIT records anything” (`../fusevm/src/vm.rs:751-753`). The eligibility test is cached per-VM (`block_eligible_cached`, `../fusevm/src/vm.rs:773-778`) so the dispatch decision is made once rather than on every invocation. Linear JIT is the simplest case of native compilation — a branchless sequence compiled on first call.

The trace threshold is 50 backedges before the recorder arms (`../fusevm/src/vm.rs:38`, `../fusevm/src/jit.rs:359`). The companion knobs in `TraceJitConfig` are documented at the same default-block (`../fusevm/src/jit.rs:357-366`): `max_side_exits = 50` (side-exits before a main trace is auto-blacklisted), `max_inline_recursion = 4`, `max_trace_chain = 4`, and `max_trace_len = 256` recorded ops before recording aborts. All are tunable per-thread via `TraceJitConfig` and the trace and block thresholds are overridable per-process with the `FUSEVM_JIT_TRACE_THRESHOLD` / `FUSEVM_JIT_`

`BLOCK_THRESHOLD` environment variables (`./fusevm/src/jit.rs:5094-5111`).

The `jit-disk-cache` feature in the `Cargo.toml` line above means the JIT's native-code output can itself be cached to disk, so the cost of JIT-compiling a hot block is paid once across runs, not once per process.

The anatomy of a chunk

To understand what the VM is actually chewing on, it helps to see a chunk's shape — and `zshrs` lets you see it. Passing `--disasm` on the command line flips a flag (`fusevm_disasm::set_enabled(true)`), and `run_chunk` then calls `maybe_print_stdout(label, &chunk)` to print a listing before `VM::run`. The disassembler in `src/fusevm_disasm.rs` walks the chunk's fields in order, and those fields are the chunk's anatomy:

```
fn append_chunk(out: &mut String, chunk: &Chunk, indent: &str) {
    if !chunk.source.is_empty() {
        let _ = writeln!(out, "{indent}; source: {}", chunk.source);
    }
    for (i, n) in chunk.names.iter().enumerate() {
        let _ = writeln!(out, "{indent}; name[{i}] = {n}");
    }
    // ... sub_entries ...
    for (i, op) in chunk.ops.iter().enumerate() {
        let line = chunk.lines.get(i).copied().unwrap_or(0);
        let _ = writeln!(out, "{indent}{i:04} {line:>5}      {op:?}");
    }
    for (si, sub) in chunk.sub_chunks.iter().enumerate() {
        let _ = writeln!(out, "{indent}; --- sub_chunk[{si}] ---");
        let sub_indent = format!("{indent} ");
        append_chunk(out, sub, &sub_indent);
    }
}
```

A `Chunk` is `source` (a label), `names` (string identifiers), `sub_entries` (named entry points into the bytecode, used for things like function bodies), `ops` (the flat instruction vector, each paired with a source line in `lines`), and `sub_chunks` (nested chunks — a function definition compiles to a sub-chunk so it can be dispatched independently). The disassembler recurses into `sub_chunks`, which is why a function-containing script prints as an outer listing with `--- sub_chunk[0] ---` blocks indented underneath. That nesting is not cosmetic — a sub-chunk is a self-contained `fusevm::Chunk` that the VM can run on its own, which is exactly what makes an autoloaded function dispatchable as a standalone unit of bytecode.

This anatomy is also what makes the bytecode *serializable*. Because a chunk is just these flat fields — vectors of ops, constants, names, and nested chunks — it can be written to disk and read back without any of it pointing into live memory. That is the foundation of the cache.

Skipping the front end entirely: the cache-hit path

Chapter 4 ended on the observation that you can `mmap` a chunk but not a tree-walk. `execute_script_file` is where that pays off. Before touching the lexer, it asks the bytecode cache whether it has already compiled this exact file:

```

if let Some(bc_blob) = crate::script_cache::try_load_bytes(path) {
    if let Ok(chunk) = bincode::deserialize::<fusevm::Chunk>(&bc_blob) {
        if !chunk.ops.is_empty() {
            return self.run_chunk(chunk,
                &format!("execute_script_file:cache:{abs_path}"));
        }
    }
}
}

```

On a hit, the path is `deserialize` → `run_chunk`. No lexing, no parsing, no compiling — the chunk that the front end produced on a previous run is reconstituted and handed straight to the VM. The cache validates path, mtime, and the zshrs binary's own mtime, so a changed script or a rebuilt shell invalidates the entry and falls through to a fresh compile. On a miss, the function compiles via `execute_script_zsh_pipeline`, then re-parses and re-compiles once more to serialize the resulting chunk back into the cache (`bincode::serialize(&chunk)`), paying one extra compile on the cold path to make every subsequent run a hit. The README's measured numbers — 717ms cold, 7ms warm — are the difference between running the whole `lex` → `parse` → `ZshCompiler` → `VM` pipeline and running only the last stage of it. The VM is the same in both cases; the cache just deletes everything in front of it.

Two architectures, from day one

All three JIT tiers target both of the architectures zshrs runs on natively: x86-64 and aarch64. Cranelift is a multi-target backend, and the JIT emits for whichever architecture the host is. That matters for this codebase specifically because the daily-driver machine is Apple Silicon aarch64 while the servers are Linux x86-64. A JIT that only emitted x86-64 would mean the JIT path never fires on the primary development machine — a server-only optimization, dead weight in daily use. Because Cranelift covers both, the same hot loop compiles to native ARM64 on the laptop and native x86-64 on the server, from the same bytecode, with no source-level difference. The bytecode is the portable artifact; the native code is generated per-host. This is the same division of labor that lets the bytecode cache be machine-independent while the `jit-disk-cache` native output is keyed to the host.

How a hot block becomes native code

Trace one loop through all three layers to make it concrete. Suppose a script counts to a million:

```

local i=0
while (( i < 1000000 )); do
    (( i++ ))
done

```

The compiler, in `compile_zsh.rs`, assigns `i` a slot. It emits the loop as: a label at the top, a test that loads slot `i`, compares against the constant `1000000`, and `Op::JumpIfFalse`s past the loop body if the test fails; then the body, which is the fused `Op::PreIncSlotVoid(i_slot)`; then a backward `Op::Jump` to the top. That is the entire loop in bytecode — a handful of ops referencing a single slot, with two jumps.

On the first few iterations, `VM::run()` interprets these ops via the jump table. As the loop proves itself hot, the dispatch routes the chunk to native code — to the Block JIT if the whole chunk is block-eligible, or to a recorded loop trace once the backedge threshold is crossed. Cranelift compiles it: slot `i` becomes a machine register, the comparison becomes a native compare, the `PreIncSlotVoid` becomes a native increment, the backward jump becomes a native branch. The million iterations now run as native code with the counter in a register, never touching the interpreter's dispatch loop, never allocating, never doing a hash lookup. On the laptop that is ARM64 machine code; on the server it is x86-64 machine code; both came from the identical `fusevm::Chunk`.

This is the payoff of the whole pipeline laid end to end. The lexer and parser established *what the script means*

(Chapter 4), with bit-exact fidelity to zsh. The compiler turned that meaning into a flat, typed, cacheable op stream and computed all the per-execution decisions once. And the VM runs that stream four ways depending on heat — interpreted with fused superinstructions for cold and branchy code, linear-JIT'd for straight-line arithmetic, block-JIT'd for whole eligible chunks, and tracing-JIT'd down the hot path through loops where shells spend their time, with side-exits back to the interpreter when a trace's assumptions break. The four ways are not alternatives the user picks; they are tiers the VM escalates through as a piece of code proves itself hot, paying compilation cost only where it returns. No interpreter walks a tree at any point. There is no tree left to walk by the time the VM starts. That absence is not an accident or an optimization that happens to hold; it is enforced, and the next chapter is about how.

Chapter 12 — The Tree-Walker That Isn't There

The deepest fact about `zshrs` is a negative one. There is no tree-walker. The recursive interpreter that every other shell has at its heart — the function that takes a parsed command, looks at its kind, and calls itself on the sub-parts — does not exist in `zshrs`. It was deleted. And because a deletion is the easiest thing in the world to silently undo, its absence is pinned by tests that fail the build if it ever comes back.

This is not a stylistic claim. It is the load-bearing architectural commitment of the entire project, and the one most likely to erode under maintenance pressure if left unguarded. A future contributor, faced with some construct that is awkward to compile, will be tempted to add a small special case: “just handle this one thing by matching on the AST and doing the work directly.” That small special case is a tree-walker. One is enough to make `zshrs` an interpreter again, because once one construct is handled by walking the tree, the property that *zshrs compiles rather than interprets* is false — not 99% true, false, because the claim is categorical. A compiler with one interpreted construct is an interpreter with a bytecode fast path. The tests in `tests/tree_walker_absent.rs` and `tests/no_tree_walker_dispatch.rs` exist to make that erosion impossible to commit by accident.

The five deleted functions

A real tree-walking shell interpreter is a family of mutually-recursive functions, one per node kind, each calling the others on sub-nodes. `zshrs` used to have exactly that family. Five functions named the pieces:

- `execute_simple` — run a simple command (words + redirections).
- `execute_pipeline` — run a `|`-chain by recursing on each stage.
- `execute_list` — run a `;/&/newline` list by recursing on each item.
- `execute_compound` — run an `if/while/for/case/subshell` by recursing on its parts.
- `execute_command_bg` — run a command in the background.

These were the recursive AST-walking functions that “pattern-matched the legacy AST and called themselves on sub-commands,” in the words of the test that now guards their absence. Every one is gone from `src/vm_helper.rs`. Their work was absorbed by the compiler (which lowers each construct to ops) and the VM (which runs the ops). A pipeline is no longer “recurse on each stage”; it is a sequence of ops the compiler emitted. A `for` loop is no longer “recurse on the body N times”; it is bytecode with a backward jump.

The five names map one-to-one onto the AST tiers from Chapter 4 plus the background-execution case: `execute_list` walked `ZshList`, `execute_pipeline` walked `ZshPipe`, `execute_simple` and `execute_compound` walked the two leaf kinds of `ZshCommand`, and `execute_command_bg` handled `&`. That correspondence is why deleting them is meaningful rather than cosmetic — they were not helpers, they were the entire execution model, one function per level of the grammar. Removing them did not leave a smaller interpreter; it left no interpreter, with the compiler and VM occupying the space they vacated.

What `tree_walker_absent.rs` asserts

The first guard is source-level. `tests/tree_walker_absent.rs` reads `src/vm_helper.rs` as text and asserts that the five function definitions are literally not present. The mechanism is deliberately blunt — it

greps the source for the function signatures and fails if they appear:

```
fn assert_no_fn(src: &str, name: &str) {
    let needles = [format!("fn {}", name), format!("pub fn {}", name)];
    for needle in &needles {
        assert!(
            !src.contains(needle.as_str()),
            "src/exec.rs still defines `{}` - tree-walker dispatch must be gone",
            name
        );
    }
}
```

There is one test per deleted function — `execute_simple_is_deleted`, `execute_pipeline_is_deleted`, `execute_list_is_deleted`, `execute_compound_is_deleted`, `execute_command_bg_is_deleted` — each calling `assert_no_fn`. If anyone reintroduces a function with one of those names, the test fails and the build is red. The doc comment states the intent without hedging:

Their existence in the source would mean an interpreter is still present — even if currently unreachable from `execute_command`. We assert literal absence here so a future refactor can't silently bring them back.

The phrase “even if currently unreachable” is the important one. The test does not care whether the function is called. It cares whether the function *exists*. A dead tree-walker is still a tree-walker waiting to be wired back in, and the test refuses to let one sit in the source as latent temptation.

Pinning the entry point that survives

Deleting the five recursive functions is not enough on its own, because the public entry points that used to *call* them still exist — `execute_command`, `execute_command_capture`. If those were left unconstrained, someone could rebuild the tree-walker inside them: a `match cmd { ShellCommand::Simple(..) => ..., ShellCommand::Compound(..) => ... }` dispatch is a tree-walker even if it does not call functions named `execute_simple`. The test anticipates exactly this and pins the entry points to route through the compiler instead.

`execute_command_dispatches_via_compiler_or_is_absent` allows `execute_command` to exist *only* if its body compiles via the front end and runs on the VM. It reads a window of the function's source and asserts the body contains `parse_init(, ZshCompiler::new()`, and `fusevm::VM::new` — and asserts it does *not* contain the old dispatch pattern:

```
assert!(
    window.contains("parse_init(") && window.contains("ZshCompiler::new()"),
    "execute_command must compile via parse_init+parse+ZshCompiler:\n{}",
    window
);
assert!(
    window.contains("fusevm::VM::new"),
    "execute_command must run the chunk on a fusevm VM:\n{}",
    window
);
assert!(
    !window.contains("ShellCommand::Simple(simple) => self.execute_simple"),
    "execute_command must not match-dispatch to a tree walker"
);
```

A companion test, `no_match_shellcommand_outside_compiler`, asserts the file contains none of the specific AST-variant dispatch patterns — `ShellCommand::Simple(simple) => self.execute_simple`, `ShellCommand::Compound(compound) => self.execute_compound`, `ShellCommand::List(items)`

=> `self.execute_list`. The comment is explicit that the check is structural, not name-based: “Even if a future refactor doesn’t use that exact name, dispatching by AST variant from `vm_helper` is a regression.”

The test is careful about what it does *not* condemn, and this carefulness is what keeps it honest rather than dogmatic. It explicitly keeps three things alive. `execute_command` itself stays — as a compile-and-run entry point. `execute_command_capture` stays — it is what backs `${...}` command substitution — but it is pinned to compile via `ZshCompiler` and capture stdout via `os_pipe::pipe()`, with no hand-rolled `echo/print/pwd` shortcuts (each of those forbidden shortcuts has its own assertion). And `execute_external/execute_external_bg` stay untouched, because they are “the actual fork-exec mechanism — *not* a tree walker. They’re the leaf operation a bytecode `Op::Exec` should ultimately route to.” The distinction the test draws is precise: forking a child process is a leaf action a bytecode op can trigger; recursively interpreting an AST is the thing that must not exist.

The behavioral net: ~160 tests, the “96-test invariant”

Source-text assertions prove the tree-walker is *gone*. They do not prove the bytecode path actually *handles* everything the tree-walker once did. A deletion that left some construct broken would pass `tree_walker_absent.rs` and still be a regression. The second guard closes that gap. `tests/no_tree_walker_dispatch.rs` is a black-box behavioral suite — 160 `#[test]` functions, each running a real `zshrs -f -c <code>` as a subprocess and asserting the exact stdout and exact exit status:

Black-box behavioral proof that the bytecode VM handles every shell construct that the deleted tree-walker dispatch ... used to handle. Every test runs a real `zshrs -f -c <code>` and asserts the EXACT stdout + exit status. Tests are hand-crafted around constructs that historically lived in the tree walker: each assertion fails loudly if the bytecode lowering for that construct regresses.

The harness is small and strict. A helper `run(code)` spawns the binary with a 10-second timeout (panicking with the offending code if it hangs), captures stdout, discards stderr, and returns `(status, stdout)`. The assertion helpers `ok`, `ok_status`, and `ok_serial` then demand exact matches — not “contains,” not “roughly,” but byte-for-byte stdout and an exact integer exit code. Fork-heavy tests (pipelines, command substitution, background `&`) are serialized behind a `FORK_SERIAL` mutex because `zshrs` forks for pipeline stages and post-fork behavior in a multithreaded program is only async-signal-safe in theory; pure-bytecode tests skip the lock and run in parallel.

The doc comment is pointed about why this suite is not redundant with the existing `ztst` test corpus: those “run pre-existing test scripts that may pass for the wrong reasons (output ignored, exit ignored, etc.).” This suite pins a *specific behavior* per test. Each one targets a construct the tree-walker handled — a simple command, a pipeline, an `if`, a `for`, a `case`, a function call, a redirection, a background job — and nails down its exact observable result. If a future change to the compiler’s lowering of, say, `case` quietly breaks `case`, this is where it surfaces, loudly, with the exact `zshrs -f -c` invocation in the failure message.

Together these are the “96-test invariant” — the name predates the current count of 160 `#[test]` functions in `tests/no_tree_walker_dispatch.rs`, but the role is unchanged: the guard against architectural decay. The two halves are complementary and neither alone suffices. `tree_walker_absent.rs` proves the interpreter functions cannot textually return, but it cannot prove the bytecode path actually handles what they handled — a build with the functions deleted and `case` lowering broken passes it. `no_tree_walker_dispatch.rs` proves every construct still produces the exact right output, but it cannot prove the construct is handled *without* a tree-walker — a build that re-added `execute_compound` and routed `case` through it would pass the behavioral suite. Only together do they pin both facts: the old thing is gone (structural) and the new thing is complete (behavioral). Remove either guard and the invariant has a hole a future refactor can walk through.

Siblings: the other invariants the immune system carries

The tree-walker pair is the most visible of a family. The header of `tree_walker_absent.rs` names two siblings that guard adjacent architectural facts, both anchored to a “Phase 5” bucketing in the project’s port plan, and both built the same way — a test that fires if a specific structural property is violated:

- `tests/shared_state_visible.rs` pins that the parameter table (`paramtab`) and shell options stay *shared* across worker threads. The doc comment describes it as the “bucket-2 pin: paramtab / opts must remain shared across worker threads. Fires if a bucket-2 holder is silently demoted to TLS.” This matters because zshrs runs work across a worker pool; if a setting like `$PATH` or a shell option were accidentally moved into thread-local storage, two threads would see different shell state, and the bug would be maddening to track down from behavior alone. The test makes the demotion fail at the structural level.
- `tests/per_evaluator_isolation.rs` pins the opposite property for a different bucket: math evaluation state must remain *per-thread*. The comment calls it the “bucket-1 pin: math TLS state must remain per-thread. Fires if a bucket-1 holder is promoted to shared storage.” Here the danger is the reverse — sharing what should be isolated, so that concurrent arithmetic evaluations stomp each other.

The pattern across all of these is identical and worth naming explicitly: each test takes an architectural decision that is *correct but invisible* — the tree-walker is gone, paramtab is shared, math state is isolated — and gives it a failing build as its defender. None of these properties announce themselves when violated. Demote paramtab to thread-local and most scripts still pass; the breakage only shows up under concurrency, intermittently. Promote math state to shared and single-threaded use looks fine. Reintroduce a tree-walker and every existing behavior keeps working. Each is an absence-or-arrangement that nothing observable protects — until the test does.

This is the project’s general answer to the fragility of architectural commitments. A 30-year-horizon tool cannot rely on every future maintainer remembering why paramtab is shared or why there is no tree-walker. The reasons live in comments, but comments do not fail builds. The tests do. The CLAUDE-level guidance the project gives itself is to *add to* these invariants for every new architectural feature and *never weaken them* — every structural decision that matters gets the same treatment: a behavioral or source-level pin that turns the build red the moment the decision is quietly reversed. The tree-walker tests are simply the most important instance because the tree-walker’s absence is the most important decision.

Why deleting the tree-walker is the whole point

It is worth being blunt about why this much machinery guards an *absence*. The claim that zshrs is a compiled shell is true if and only if there is no interpreter inside. Every Unix shell since 1970 has had a tree-walker at its core; that is what “interpreter” means for a shell, and it is the property that distinguishes zshrs from all of them. If zshrs had a tree-walker too, even a small one for a few awkward constructs, it would be an interpreter with a bytecode fast path bolted on — the same category as every other shell, distinguished only by degree. The entire architectural identity rests on the tree-walker being absent, not merely deprioritized, because the distinction between zshrs and a fast interpreter is categorical, and a single interpreted construct collapses it.

And absences are uniquely fragile. A present feature defends itself — break it and something stops working, a test goes red, a user complains. An absence defends nothing. You can add a tree-walker back without breaking a single existing behavior; everything keeps working, and the architecture is quietly dead. Nothing observable changes the day the first `match cmd { ShellCommand::... }` returns to `vm_helper.rs`. The only thing that changes is that the central claim becomes false, silently, and stays false until someone notices months later — exactly the failure mode the project treats as catastrophic for a 30-year-horizon tool.

So the tests do for the absence what nothing else can. They make the absence *load-bearing* — they give it the self-defending property that present features get for free. Reintroduce a tree-walker function and `tree_walker_absent.rs` fails. Reroute an entry point to AST dispatch and `no_match_shellcommand_outside_compiler` fails. Break the bytecode lowering of any construct and one of the 160 behavioral tests fails. The compiler-and-VM architecture is not held in place by good intentions or by a comment asking people to be

careful. It is held in place by a build that will not go green if anyone, ever, walks the tree again. That is what it means for an absence to be the whole point: the project spends real engineering on guaranteeing that a thing stays gone, because the thing being gone is the thing that makes zshrs what it is.

Chapter 13 — The Cache

Every shell that has ever shipped reads your `.zshrc` the same way: it opens the file, lexes it character by character, parses the token stream into a syntax tree, and walks that tree to execute it. It does this on every single launch. Sixteen synchronized terminal panes, four hundred and seventy-eight thousand commands of muscle memory, a plugin manager juggling a hundred zinit bundles — and underneath all of it, the same cold lex-parse-compile loop runs from scratch each time you press Enter on a new window. The community has spent two decades inventing ways to hide this cost: zinit’s turbo mode defers it, powerlevel10k’s instant prompt paints over it, `zcompile` pre-parses pieces of it into `.zwc` archives. Each of those is a layer of lipstick on the same fact — the shell throws away its work and redoes it.

zshrs throws nothing away. The first time it compiles a script to fusevm bytecode, it keeps the bytecode. The second time you run that script, there is no lex, no parse, no compile — there is an `mmap`, a zero-copy lookup, and execution. This is the cache, and it is the single feature that lets the rest of the design hold its promises. Without it, “first paint equals full functionality” is a slogan. With it, it’s a measurement: 717 milliseconds cold, 7 milliseconds warm.

What it caches, and why that layer

zsh’s `.zwc` files cache a parsed **AST** — the word-code representation of the source. When zsh reads a `.zwc`, it skips the parse but still walks the tree to run it. That’s a real optimization and it’s why `zcompile` exists, but it caches one layer too high. The expensive, repeated work in a modern shell startup isn’t only the parse; it’s the parse *plus* the compile *plus* the tree-walk dispatch on every invocation. bash caches nothing across runs — it re-reads, re-lexes, and re-executes `.bashrc` from source on every interactive launch, and the same is true of dash, fish, and the rest. A `.zwc` archive is the most aggressive caching any shipping shell has done, and it stops at the AST.

zshrs caches the layer below the AST: the fusevm **Chunk**, the bytecode the VM executes. That’s possible because zshrs is a compiled shell rather than a tree-walking interpreter — there is a compiled artifact to persist in the first place, where in a tree-walker there is nothing below the AST to keep. The cache is downstream of the architecture, not a bolt-on. The cost it removes isn’t the parse, which is cheap; it’s the recompilation of every sourced file’s AST into bytecode on every launch, which a `.zwc`-style AST cache leaves on the table by design. Two layers cached versus one is the whole difference between the warm number below and zsh’s.

The shard on disk

The script cache lives in a single file. The module header in `src/extensions/script_cache.rs` describes it bluntly: a “single-file shard at `~/.zshrs/scripts.rkyv`.” Autoload bytecode gets its own shard, built the same way through the sibling `autoload_cache.rs`. The on-disk format is **rkyv** — a zero-copy serialization framework where the archived bytes *are* the in-memory layout. You don’t deserialize an rkyv archive into owned structures to read it; you cast a pointer into the mapped bytes and read fields in place. That property is the entire reason for the choice. A serde-based format — bincode, JSON, MessagePack — has to decode the byte stream into owned **Strings** and **Vecs** before you can index a single entry. rkyv lets the lookup probe the mapped file directly: the archived **HashMap** is a hash table laid out in the file’s bytes, and `get` walks it without allocating.

There is one deliberate exception to the zero-copy rule, documented in the module header: the inner `chunk_`

`blob` is bincode, not rkyv. `fusevm::Chunk` is owned by the upstream fusevm crate and derives only `serde`, not `rkyv::Archive`. Rather than fork fusevm or maintain a mirror archived type — both large refactors the module header rejects as “not needed for the current ‘kill SQLite for bytecode’ goal” — the inner codec stays bincode inside the rkyv outer container. So the outer shard is zero-copy and the bytecode blob inside it is bincode-decoded exactly once, at hand-off to the VM. The zero-copy property is on the index, where lookups are frequent; the one decode is on the payload, which is handed over once per hit.

The structure is three nested types. A `ScriptShard` holds a `ShardHeader` and a `HashMap<String, ScriptEntry>` keyed by canonical script path. Each `ScriptEntry` carries the source file’s mtime (seconds and nanoseconds), the mtime of the zshrs binary at the moment the entry was cached, a cached-at timestamp, and the `chunk_blob` itself — the bytecode. The header carries the load-bearing metadata:

```
pub struct ShardHeader {
    pub magic: u32,
    pub format_version: u32,
    pub zshrs_version: String,
    pub pointer_width: u32,
    pub built_at_secs: u64,
}
```

Those five fields are the contract that keeps a `.zshrc` that worked yesterday working today. We’ll come back to them.

Reading is a memory map, not a read

The hot path — the one that runs on every warm launch — does the least possible work. `MmappedShard::open` opens the file, calls `Mmap::map` to map it into the address space, and then validates it:

```
pub fn open(path: &Path) -> Option<Self> {
    let file = File::open(path).ok()?;
    let mmap = unsafe { Mmap::map(&file).ok()? };
    let archived = rkyv::check_archived_root::<ScriptShard>(&mmap[..]).ok()?;
    let archived_ptr = archived as *const ArchivedScriptShard;
    Some(Self { _mmap: mmap, archived: archived_ptr })
}
```

Three things happen here, and the order is the point. First the file is mapped — no bytes are copied into a buffer; the kernel arranges for the file’s pages to appear in the process’s address space, faulted in on demand as they’re touched. Second, `rkyv::check_archived_root` validates the byte image: it walks the archive’s structure and confirms every offset, every length, every pointer-equivalent lands inside the mapped region. This is the validation tax, and it’s paid exactly once — the `MmappedShard` is kept alive for the whole process so repeat lookups don’t re-validate. Third, a raw pointer to the validated archive root is stashed alongside the live mapping.

The struct that holds both is self-referential: the pointer aliases bytes owned by the `Mmap` field that sits right next to it. The code documents the invariant and asserts `Send + Sync` by hand, because the pointer is into an immutable mapping that lives exactly as long as the struct does:

```
// SAFETY: the pointer aliases an immutable mmap that lives as long as Self.
// rkyv-validated reads are immutable.
unsafe impl Send for MmappedShard {}
unsafe impl Sync for MmappedShard {}
```

From there, a lookup is `self.shard().entries.get(path)` — an `ArchivedHashMap` probe directly against the mapped bytes. There is no decode step to find the entry. The only copy that happens is `entry.chunk_blob.as_slice().to_vec()` at the very end, when the bytecode blob is handed to the VM — and that’s a deliberate boundary, discussed below. Everything up to that point is pointer arithmetic over `mmap`’d pages.

The keying: three things must agree

A cache that returns stale bytecode is worse than no cache. zshrs's invalidation is conservative to the point of paranoia, and it keys on three independent facts. Look at `ScriptCache::get`:

```
pub fn get(&self, path: &str, mtime_secs: i64, mtime_nsecs: i64) -> Option<Vec<u8>> {
    self.ensure_mmap();
    let guard = self.mmap.lock();
    let shard = guard.as_ref()?;
    if !shard.header_ok() {
        return None;
    }
    let entry = shard.lookup(path)?;

    let entry_mtime_s: i64 = entry.mtime_secs.into();
    let entry_mtime_ns: i64 = entry.mtime_nsecs.into();
    if entry_mtime_s != mtime_secs || entry_mtime_ns != mtime_nsecs {
        return None;
    }

    if let Some(bin_mtime) = current_binary_mtime_secs() {
        let cached_bin_mtime: i64 = entry.binary_mtime_at_cache.into();
        if cached_bin_mtime < bin_mtime {
            return None;
        }
    }

    Some(entry.chunk_blob.as_slice().to_vec())
}
```

The first gate is `header_ok()` — magic, format version, zshrs version, pointer width all have to match (more on that next). The second gate is the **source mtime**, both seconds and nanoseconds. The cache key isn't just the path; it's the path plus the exact modification time. Edit the script and save it, and the nanosecond mtime shifts; the cached entry no longer matches; `get` returns `None`; the script is recompiled and re-cached. The lookup in `try_load_bytes` canonicalizes the path first (`path.canonicalize()`) so symlinks and `../` and relative invocations all collapse to one stable key.

The third gate is the **binary mtime**. Every entry records `binary_mtime_at_cache` — the mtime of the running zshrs executable at the moment that bytecode was written. On read, if the current binary is *newer* than the binary that produced the entry, the entry is rejected. This is the gate that catches the subtle, dangerous case: you rebuilt zshrs, the bytecode format or the runtime semantics shifted, but the script file on disk didn't change so its mtime is identical. Source-mtime alone would happily replay yesterday's bytecode through today's VM. The binary-mtime gate makes any rebuild of zshrs silently invalidate every cached entry. `current_binary_mtime_secs` reads `std::env::current_exe()` once and memoizes it in a `OnceLock`, so the check costs nothing after the first call.

Three facts must agree — header, source mtime, binary mtime — or the cache misses and the slow path runs. There is no fourth degree of trust. The cache never guesses.

The versioned header and the silent rebuild

This is the part that earns the right to the design-goal “never break a `.zshrc` that worked yesterday.” `header_ok()` is four equality checks:

```

fn header_ok(&self) -> bool {
    let h = &self.shard().header;
    let magic: u32 = h.magic.into();
    let fv: u32 = h.format_version.into();
    let pw: u32 = h.pointer_width.into();
    magic == SHARD_MAGIC
    && fv == SHARD_FORMAT_VERSION
    && pw as usize == std::mem::size_of::<usize>()
    && h.zshrs_version.as_str() == env!("CARGO_PKG_VERSION")
}

```

`SHARD_MAGIC` is the constant `0x5A525343` — the ASCII “ZRSC” in little-endian. If the file isn’t a zshrs script shard at all, the magic fails fast and the shell falls back to compiling. `SHARD_FORMAT_VERSION` is bumped by hand whenever the rkyv schema changes incompatibly; an old shard with format version 1 read by a zshrs that expects version 2 simply misses every lookup. `pointer_width` guards against a cache built on a 64-bit machine being read on a 32-bit one (or the reverse) — rkyv archives bake in pointer-sized offsets, so a width mismatch means the bytes can’t be trusted. And `zshrs_version` pins the cache to the exact `CARGO_PKG_VERSION` that wrote it.

When any of these fails, nothing dramatic happens. There’s no error printed, no migration prompt, no “your cache is out of date” nag — the global rule against startup chatter holds here too. The shell gets `None` from `header_ok()`, treats it as a miss, recompiles the script, and on the write path lays down a fresh shard with the current header. The corrupt or stale file is overwritten. The user sees a one-time cold start and then warm starts forever after.

The write path enforces the same discipline from the other side. `put` reads the existing shard only if its version, pointer width, and format version all match the running binary; otherwise it starts from `fresh_shard()` and discards the old contents entirely:

```

let mut shard = match read_owned_shard(&self.path) {
    Some(s)
        if s.header.zshrs_version == env!("CARGO_PKG_VERSION")
        && s.header.pointer_width as usize == std::mem::size_of::<usize>()
        && s.header.format_version == SHARD_FORMAT_VERSION =>
    {
        s
    }
    _ => fresh_shard(),
};

```

A `.zshrc` that worked yesterday keeps working because the worst the cache can do is force a recompile. It can never feed wrong bytecode to the VM, and it can never refuse to start the shell. The format is versioned and migration-safe by construction: old formats aren’t migrated, they’re silently rebuilt. That’s the cheaper, more durable choice for a thing that has to survive a thirty-year horizon — there’s no migration code to rot.

Writing safely under sixteen panes

The write path has a harder problem than the read path. Sixteen terminal panes broadcasting keystrokes means sixteen zshrs processes can decide to cache the same script at the same instant. A torn write — one process serializing while another reads half-written bytes — would corrupt the shard and force a cold start for everyone. zshrs handles this with two mechanisms working together: advisory file locking and atomic rename.

Every write acquires an exclusive `flock` on a sidecar lock file before touching the shard:

```
fn acquire_lock(path: &Path) -> Option<nix::fcntl::Flock<File>> {
    let f = File::options()
        .read(true).write(true).create(true).truncate(false)
        .open(path).ok()?;
    nix::fcntl::Flock::lock(f, nix::fcntl::FlockArg::LockExclusive).ok()
}
```

The lock serializes concurrent writers — only one process mutates the shard at a time. But locking alone doesn't protect readers, and zshrs readers don't take the lock (they just `mmap` and go). That's where the atomic rename comes in. `write_shard_atomic` serializes the whole shard with `rkyv::to_bytes`, writes it to a temp file named with the pid and a nanosecond timestamp, `fsyncs` it, and then `renames` it over the real path:

```
let tmp_path = parent.join(format!("{}", tmp_dir, file_name, pid, nanos));
{
    let mut f = File::create(&tmp_path)?;
    f.write_all(&bytes)?;
    f.sync_all()?;
}
std::fs::rename(&tmp_path, path)?;
```

`rename` on a POSIX filesystem is atomic. A reader that `mmaps` the shard sees either the entire old file or the entire new file — never a torn intermediate. After the rename, the writing process calls `invalidate_mmap()` to drop its own stale mapping so its next lookup picks up the new shard. The whole shard is rewritten on every `put`, which sounds wasteful until you remember the access pattern: scripts are compiled once and read thousands of times. The write is the rare event; the read is the common one. Optimizing the read to a pointer cast and paying a full-shard rewrite on the occasional miss is the right trade.

The autoload cache makes that trade explicit. `compinit` doesn't add autoload functions one at a time — it pre-warms sixteen thousand of them at once. Rewriting the whole shard sixteen thousand times would be absurd, so `autoload_cache.rs` exposes three write entry points instead of one. `put_one` is the single-add path for the cold-start case where the interactive shell compiles one autoload at a time. `merge_in` takes a whole `HashMap` of `(name, blob)` pairs, folds them into the existing shard, and commits once. `replace_all` takes a `HashMap` and overwrites the shard wholesale, for the bulk recompile case. The module's header comment spells out the motivation: doing this through per-add shard rewrites — “the SQLite-era pattern” — would “re-serialize 16k entries 160 times.” The bulk paths accumulate every pair in memory and pay the serialize-and-rename cost exactly once, which is what makes a 16k-function fpath survive `compinit` without thrashing the disk. Same trade as the script shard, scaled up: the read is a pointer probe, the write is rare and batched.

The deliberate separation: SQLite is a mirror, never the cache

Here is the design decision most likely to be misread, so it gets stated flatly: **the SQLite databases are not a cache.** `~/.zshrs/catalog.db` and `~/.zshrs/history.db` are read-only mirrors for inspection. They are daemon-hydrated copies of state that exist so you can point `dbview` or an ad-hoc SQL query at them. They are *never* consulted on the cache hit/miss path, and they *never* hold the bytecode that gets executed. The README's overview says it in one breath: `catalog.db` and its SQL views are “daemon-hydrated, **never authoritative for cache hit/miss or execution**.” They are not a second shell cache.”

The separation isn't decoration; it's a division of labor between two storage engines picked for two different access patterns. The rkyv shard answers exactly one question — “give me the bytecode for this canonical path, and only if its mtimes still agree” — and it answers it with a pointer cast. It is the wrong tool for any other question. SQLite answers the other questions: “list every plugin sorted by source time,” “which functions did zinit register,” “how many entries does the catalog hold and how many hooks fire.” Those are relational queries with `ORDER BY` and `JOIN`, and rkyv's archived `HashMap` can't express them — it's a hash probe, not a query engine. So the two stores coexist by role: the authoritative bytecode store is the one optimized for the single

hot lookup; the inspection mirror is the one optimized for ad-hoc SQL. Putting the bytecode in SQLite would force the execution hot path through a SQL prepare-bind-step cycle on every run; putting the catalog in rkyv would make `dbview` impossible. Each store does the one job its data shape suits.

The git history of `plugin_cache.rs` shows this separation being enforced, not just asserted. Script bytecode *used* to live in SQLite, in a `script_bytecode` table behind a `BYTECODE_VERSION` prefix. It was moved out to the rkyv shard, and the migration code in `init_schema` actively destroys the old table on open:

```
-- Migration: legacy script_bytecode table (bytecode now lives in
-- the rkyv shard at ~/.zshrs/scripts.rkyv). Drop on open so
-- existing DBs reclaim the space and don't carry stale bytecode.
DROP INDEX IF EXISTS idx_script_bytecode_path;
DROP TABLE IF EXISTS script_bytecode;
```

There's a test that pins this behavior — `opening_an_existing_db_drops_legacy_script_bytecode_table` hand-builds a pre-migration database with a populated `script_bytecode` table, re-opens it through the production path, and asserts the table is gone. The authoritative store is the rkyv shard, full stop. SQLite holding zero bytecode bytes is a checked invariant, not a hope.

Why keep SQLite at all, then? Because rkyv is a terrible query surface. An `mmap`'d `ArchivedHashMap` is perfect for “give me the bytecode for this exact path” and useless for “show me every plugin sorted by source time” or “which functions did zinit register.” Those are inspection questions, and inspection wants SQL. So the daemon hydrates a mirror — entries, hooks, plugins, stats — into `catalog.db`, and `dbview` reads it. The plugin cache itself, which stores the *side effects* of sourcing a plugin (the params, aliases, options, and functions a plugin sets) rather than its bytecode, does live in SQLite because that data is relational and replayed, not executed as bytecode. But even there, the invalidation logic is the same paranoid three-key scheme: canonical path, source mtime, and binary mtime, with any zshrs rebuild silently invalidating the replay deltas.

The line is clean. Bytecode that gets executed: rkyv shard, the authoritative store, `mmap`'d and validated. Data you query to understand what the shell did: SQLite mirror, read-only, regenerable, never on the execution path. Confusing the two would be the kind of architectural decay the project's invariant tests exist to catch.

Seeing the cache: `dbview` and `zcache`

Because the inspection surface is real SQLite, looking at cache state is a query, not a special command. `dbview` opens the mirror and lets you read it like any database. The shape of a `zcache info` summary — the supervised view of the on-disk cache — reads like this (from the daily-driver cache-architecture section of `docs/AOT_DESIGN.md`):

```
$ zcache
Cache: ~/.zshrs/                                total: 612 MB
index.rkyv                                     1.2 MB      50,234 entries (mmap: hot)
images/                                       247.0 MB      143 shards
system.rkyv                                 12.0 MB       2,341 entries
completions-corpus.rkyv                     82.0 MB      27,387 entries
zpw.rkyv                                    45.0 MB       506 entries (mtime 2m ago)
catalog.db                                  18.0 MB      50,234 entries / 5,492 hooks
history.db                                  340.0 MB     478,231 commands
```

The numbers there come straight off `stats()` and `list_scripts()` on the live caches — entry counts, blob byte totals, cached-at timestamps formatted in local time. None of it is hardcoded; it's all computed from the shards at query time. And resetting the cache needs no special API. The cache directory is just a directory, and `rm` is a valid path: `rm ~/.zshrs/images/plugin-foo.rkyv` forces one shard to rebuild, `rm -rf ~/.zshrs/images/` forces all of them. The `zcache` builtins exist for the cases where the running shell needs to release its `mmap` handles before the unlink and re-hydrate the catalog afterward — but the file is always there to delete by hand. There are no safety prompts, no confirmations, no “are you sure.” The user is root.

The number

```
Cold (cache miss): 717ms - lex + parse + compile + cache write + execute
Warm (cache hit): 7ms - deserialize + execute
```

Seven milliseconds. That's the whole point of the chapter rendered as a measurement. The cold number includes everything: lexing the source, parsing it to an AST, compiling the AST to a fusevm `Chunk`, serializing the chunk into the shard, and running it. The warm number is the `mmap`, the validated `ArchivedHashMap` lookup, the single `to_vec` of the bytecode blob, and execution. A hundredfold gap, and it exists because the cache stores the layer that's expensive to recompute — bytecode — and reads it without copying.

This is what lets zshrs refuse every startup trick the ecosystem accumulated. No instant-prompt fakery, because there's nothing slow to paint over. No turbo-mode deferral, because the work the deferral was hiding doesn't recur. No `zcompile` ritual, because the cache is automatic and keyed correctly. The seven-millisecond warm start *is* the full shell — every plugin, every completion, every autoload, fully loaded. First paint equals full functionality, and the cache is the reason that's a fact and not a slogan.

Torn reads and the ordering discipline

The seven-millisecond number carries a hidden assumption that is worth surfacing, because it is exactly where a naive `mmap` cache goes wrong. A warm client `mmaps` a shard and reads bytecode straight out of it with no copy and no validation loop — that is the whole reason the read is cheap. But a `mmap`ed region is a live window onto a file, not a snapshot: if the daemon is halfway through rewriting that file when the client maps it, the client reads a *torn* image — a header from the new version, a body from the old, or a length field that points past the end of a truncated write. In a format built for zero-copy reads, a torn image is not a parse error you catch; it is a segment of memory you dereference as if it were a valid `ArchivedHashMap`, and the failure mode is a crash or, worse, a silently wrong bytecode chunk. The performance decision — read without copying or checking — is precisely what makes write correctness non-negotiable.

zshrs solves this the way filesystems have always solved it: it never mutates a shard in place. The daemon is the sole writer, and every shard write in `daemon/shard.rs` follows the same three-step ritual — serialize the new `(name → bytecode)` map to rkyv in memory, write those bytes to a temporary file named `{slug}.tmp.{pid}.{tid}`, `fsync` it so the bytes are durable, and only then `rename` the temp file over the real shard path. `rename(2)` within a directory is atomic on every POSIX filesystem: a concurrent reader sees either the entire old file or the entire new file at its old inode, never a blend of the two. The `{pid}.{tid}` in the temp name is not decoration — it guarantees that two daemon threads (or a daemon and a crashed predecessor) writing the same logical shard never collide on the same temp path, so each write stages into a private scratch file before the single atomic swap. A daemon that dies mid-write leaves an orphaned `.tmp.{pid}.{tid}` file that no reader ever consults, because no reader knows its name; the periodic ticker sweeps those orphans on a later pass. The real shard is only ever the fully-written one or the previous fully-written one.

There is a second ordering constraint layered on top, because a shard does not stand alone. The `index.rkyv` file maps top-level names to `(shard_id, generation, byte_offset)`, so a lookup consults the index first and the shard second. If the index were updated before the shard it points into, a client could read a fresh index entry that referenced a shard generation not yet on disk. So the discipline is strict and one-directional: rename the shard into place *first*, then update `index.rkyv`. A reader that races the update either sees the old index pointing at the old shard (coherent) or the new index pointing at the new shard (coherent) — never the new index pointing at a shard that isn't there. The shard header carries its own belt-and-suspenders check: a magic word (`0x5A53_4853`, the ASCII bytes `ZSHS`) and a `SHARD_FORMAT_VERSION` that is bumped on any incompatible rkyv schema change, so a shard written by a future daemon and mapped by an older client fails fast on the magic rather than deserializing garbage. This is the thirty-year format-versioning rule from the endgame chapter made concrete at the byte level: the fast path reads without checking, so the write path checks everything, and the two artifacts — the hot rkyv shard and the queryable `catalog.db` — are each

recoverable from the other if one is ever lost.

Chapter 14 – Compiling Scripts to Native (the AOT Frontier)

The cache from the previous chapter ends one rung short of the obvious conclusion. `zshrs` compiles your script to fusevm bytecode and persists it. The JIT then takes that bytecode and emits native machine code through Cranelift — block JIT for loops and conditionals, linear JIT for straight-line arithmetic, all of it real x86-64 and aarch64 instructions. So the pipeline already runs source through a Cranelift backend at runtime. The question this chapter is about is short: what if you emitted that machine code to an object file instead of to executable memory, and linked it into a standalone binary?

That's ahead-of-time compilation, and it lets `zshrs` ship a script as a native executable. The honest framing has shifted since this chapter was first written, and the shift is worth stating precisely. The design document, [docs/AOT_DESIGN.md](#), still opens with “Status: Draft. No code yet. This doc decides the architectural calls before implementation begins” — the *doc* was never updated. But the *code* has since landed. [src/extensions/aot.rs](#) now exists and runs, exposed through a `zbuild` builtin, and it implements two of the three deployment rungs the design doc laid out: a self-contained source-trailer binary, and — added in the [feat\(zbuild\): --native compiles zsh scripts to native machine code](#) commit — a genuinely AOT-compiled native object linked into a standalone executable. So the chapter has two halves now. The design exposition still describes the vision and the locked architectural calls, because those are what the shipped code is faithful to. The new sections describe what you can actually run today, where the implementation matches the plan and where it took a more pragmatic route. Where the plan and the codebase still diverge — the AOP-weaving and nanosecond-profiling payoff, cross-architecture deployment, the full-suite acceptance gate — the chapter says so plainly, as it always did.

The capability, and where the prior art stops

Every existing shell ships scripts as source. To run a bash script on a server, bash must be installed on that server. Same for zsh, dash, fish, nushell, elvish — the runtime is a prerequisite on the target box. Even the shells with internal JITs parse and compile the source at every invocation, because the artifact they distribute is text. There is a long lineage of “compile a script to a binary” wrappers — [shc](#) for shell, [PAR/pp](#) for Perl, [PyInstaller](#) for Python — but every one of those bundles the *interpreter* plus the *source* into one file and runs the interpreter at launch. They package the runtime; they don't eliminate it. The script still gets lexed, parsed, and interpreted on the target, just from an embedded copy rather than an installed one.

The design doc's claim is that “no shell has ever shipped an AOT-compiled-to-machine-code deployment artifact for scripts” — a deployment binary whose script logic *is* native instructions, not an interpreter replaying embedded text. The second target is performance parity with a Go or Rust binary: zero parse, zero compile, zero interpretation at launch. The doc's workflow sketch is the design intent in three commands:

```
zshrs build script.zsh      # produces ./script, a static native binary
scp script user@server:/usr/local/bin/
ssh user@server script      # no zsh, no zshrs, no runtime on the target
```

No `apt install`, no `brew install`, no version management on the target machine. The binary contains actual machine code for the script's logic and behaves identically to running the source under `zshrs` — same builtins, same expansion semantics, same `eval`.

Why bytecode is one rung from native

The design doc frames three deployment rungs that share the same fusevm `Chunk` and differ only in payload format. Rung one is a source trailer — UTF-8 source appended to a copy of the runtime, paying parse plus compile plus JIT at launch; that’s what the sibling stryke project ships today. Rung two is a bytecode trailer — a bincode-serialized `Chunk` appended instead, paying only JIT or interpretation. Rung three is the native object — a Cranelift-emitted `.o` linked into a static binary, paying zero codegen at launch.

zshrs’s `Chunk` is already serde-ready and already feeds a Cranelift backend. The doc’s reasoning: stryke chose rung one because its `Arc<HeapObject>` runtime values aren’t serde-ready, but zshrs has no such constraint, so rungs one and two are stepping stones it doesn’t need — ship rung three directly. Adding `cranelift-object` is, in the doc’s words, “a parallel output sink on the same IR pass — the same IR that JIT consumes is what AOT consumes.” The runtime JIT already exists; AOT is the same IR pass writing to an object file instead of to executable memory. That equivalence is what makes the doc treat AOT as a near-term increment rather than a new backend — and the increment has since been built. The object-file sink the doc called Phase A is now `fusevm::aot::compile_object`, pulled in by the `aot` feature on the fusevm dependency, and zshrs’s native build calls it directly. The doc predicted “a parallel output sink on the same IR pass”; the shipped `build_native` is exactly that — it compiles the same `Chunk` the JIT would consume, then hands it to `compile_object` to emit a relocatable `.o`.

The pipeline, as designed

```
script.zsh
├── lex module ───────────────────────────────────────────────────────────┐
├── parse.rs free fns ───────────────────────────────────────────────────┐ (existing)
├── ZshCompiler → fusevm::Chunk ─────────────────────────────────────────┐
├── cranelift-object → script.o ─────────────────────────────────────────┐ (NEW: AOT codegen pass)
├── ld script.o + libzshrs_runtime.a ───────────────────────────────────┐ (NEW: link step)
└── script (static native binary)
```

The front half is shared with the daily shell — lex, parse, compile to `Chunk`. The two new components are the **codegen pass** that lowers a `Chunk` to an object file, and the **runtime stub library** `libzshrs_runtime.a` that the object file links against. Cranelift handles target triples natively, so there’s no LLVM dependency and no host C toolchain in the loop; `cranelift-object` is the standard backend for exactly this use.

The runtime stub is where the compat floor is preserved. It’s a `staticlib` crate exposing every zshrs runtime primitive as a C-ABI symbol — the doc estimates roughly 317 `BUILTIN_*` handlers, the `ShellExecutor` state accessors (variables, arrays, options, jobs, traps — the runtime struct itself lives in `src/vm_helper.rs`, not in a file named for execution), the fork/exec dispatcher, the redirect machinery, and the glob and parameter-expansion paths. The compiled `.o` calls into it; the linker resolves the symbols; the result is one self-contained static binary the doc estimates at 5–10 MB, comparable to a Go binary. The size is per-binary fixed overhead — the static runtime is the same regardless of script size, and the doc accepts it on the Go-binary precedent: a hello-world Go binary is the same order of magnitude because it statically links its runtime too.

The stub also embeds a full parser, compiler, and VM. That’s not redundancy with the AOT codegen — it’s the answer to `eval`. A script that calls `eval`, `${(e)...`, or `$cmd arg1` constructs code whose text isn’t known until runtime, and AOT can’t pre-compile what doesn’t exist at build time. The doc weighs an “AOT-strict mode” that would reject such scripts to shrink the binary, and rejects it on the constraint “Compat with his existing world is sacred” — breaking `eval` would break zsh compatibility, and the cold-path embed costs an estimated 2–3 MB that the doc accepts. So AOT mode is a strict superset of interpreted mode: the common case is native code, and the dynamic-dispatch corners fall back to the embedded interpreter at runtime, same as the daily shell.

The source-trailer build: `zbuild`, shipped

The first thing the shipped `zbuild` does is the design doc's rung one — the self-contained source binary — and `src/extensions/aot.rs` is candid that this is not the same thing as native compilation. The module header calls it out: C zsh's closest analog is the `zcompile` builtin (`Src/parse.c::bin_zcompile()`), which writes a parsed-AST `.zwc` cache to a *separate* file beside the script. zshrs's `zbuild` instead bakes the script *into the shell binary itself*, producing a single executable that needs no separate cache and no zsh on the box. There is no C counterpart for that; it is zshrs-original infrastructure.

The invocation is a builtin, not a CLI subcommand, and it takes one or more inputs in order:

```
zbuild --in *.zsh --out app          # glob expansion, sorted by default
zbuild --in lib1.zsh lib2.zsh main.zsh --out app
./app                                # runs all three sequentially under
                                     # one ShellExecutor
```

`--in` is greedy: it consumes every non-flag token that follows, which is what makes `--in *.zsh` work naturally — each path the glob expanded to lands as another input file. zsh has no project concept, so there is no manifest, no entry-point convention, no library-directory walker; order is exactly the order of paths given to `--in`. Files run sequentially in that order under a *single ShellExecutor*, so globals and functions defined by file N are visible to file N+1 — the same as concatenating them and running the whole thing once.

The mechanism is a trailer appended past the end of the binary. The build copies the running `zshrs` executable, strips any trailer it already carries (so nested builds do not stack), then appends a zstd-compressed payload and a fixed 32-byte trailer ending in the eight-byte magic `ZSHRSAOT`:

```
[mach-o / elf bytes of zshrs ...] (unchanged, still runs as `zshrs`)
[zstd-compressed payload ...]
[u64 compressed_len]
[u64 uncompressed_len]
[u32 version]
[u32 reserved (0)]
[8 bytes magic b"ZSHRSAOT"]
```

ELF and Mach-O loaders ignore bytes past the segments their program headers list, so the appended trailer leaves the original `zshrs` fully runnable — the AOT binary is still a working shell, it just also carries an embedded program. The payload is a versioned ordered-file list (format v2: a `u32` file count, then for each file a length-prefixed name and a length-prefixed UTF-8 source). A v1 single-script decoder is kept for backward compatibility so binaries built by the previous `zbuild` still load, but every new build emits v2 even for a single input. At startup, before argument parsing, `try_load_embedded` reads just the last 32 bytes, checks the magic, and bails instantly if it is absent — so a plain `zshrs` invocation pays nothing for the feature's existence. If the magic is present, it seeks back by the recorded compressed length, decompresses, validates that the inflated size matches the header, and runs the embedded files. The decoder is defensive against every corruption it can name — truncated input, a lying count header, an unknown version, an inflated length that does not match — each of which is a separate unit test in the module, alongside round-trip tests that preserve UTF-8 in both names and sources and preserve multi-file order.

This rung pays parse plus compile plus JIT at launch, exactly as the doc said it would — the embedded text is real source, re-lexed and re-compiled by the same front end the daily shell uses. It is the convenient, portable rung: a single file you can `scp` to a box with no zsh, that behaves identically to running the source under `zshrs` because it *is* `zshrs` with a script welded on.

The native build: `zbuild --native`

The `--native` flag is the rung the design doc treated as the destination — script logic as actual machine code, no interpreter dispatch loop at the center. Where the source-trailer build embeds text and re-runs it interpreted, `build_native` compiles the concatenated inputs to a fusevm `Chunk`, lowers that chunk to a native object

through `fusevm::aot::compile_object` (Cranelift's `ObjectModule` emitting a relocatable `.o`), and links the object against the zsh runtime staticlib into a standalone executable:

```
zbuild --native --in script.zsh --out script
./script                                # the script's bytecode runs as
                                         # native code; no parse, no compile,
                                         # no interpreter dispatch at launch
```

The pipeline inside `build_native` is short and matches the design diagram rung-for-rung. It reads and concatenates the inputs (appending a newline between files so the last line of one cannot fuse with the first of the next), runs them through the ordinary `parse_isolated + ZshCompiler::compile` path to get a `Chunk`, and refuses to proceed if that chunk is empty. It then locates the runtime staticlib — `libzsh.a`, found beside the running executable in the dev `target/<profile>/` layout, or overridden by `ZSHRS_AOT_RUNTIME_LIB` — and calls `compile_object` to emit `script.o`. The link step writes a tiny C entry stub whose `main` is nothing but a call into the runtime:

```
extern long fusevm_aot_run_embedded(void);
int main(void){return (int)fusevm_aot_run_embedded();}
```

and invokes `cc` to link the stub, the object, and `libzsh.a` together, pulling `-lcurses` for zsh's terminfo/termcap modules and, on macOS, the four system frameworks the runtime's transitive dependencies need — `CoreFoundation` (chrono/time-zone), `CoreServices` (FSEvents), `Security` and `SystemConfiguration` (TLS, keychain, network config). The object and stub are temporary and cleaned up after the link.

The decisive piece is how the native chunk reaches a live shell. The runtime staticlib exports a `#[no_mangle]` C-ABI hook, `fusevm_aot_register_builtins`, that the fusevm AOT runtime calls at startup of the native binary. That hook stands up a leaked-`'static ShellExecutor`, seeds `$1..` from `argv` after the program name, registers the zsh builtins on the run VM via `fusevm_bridge::register_builtins`, and installs the executor as the thread's current executor — permanently, by `mem::forget`-ing the RAI guard, because an AOT binary runs one program and exits. The comment in the source makes the equivalence explicit: this arranges the native chunk's host calls to reach a live shell “exactly as the interpreter's `run_chunk` arranges.” So the native binary's arithmetic, control flow, and slot operations run as Cranelift-emitted machine code, while command execution, expansion, and command substitution still call into the linked-in `ShellExecutor` — the compat floor is preserved by linking the whole shell runtime in, not by reimplementing it in the codegen. That is the design doc's “strict superset” promise made real: native where it can be, the embedded shell where it must be.

This is where the fusevm version bumps in the latest source matter. `compile_object` can only lower opcodes the AOT backend knows how to emit, and the `0.14.4` and `0.14.5` bumps — “native AOT through heap values” and “wider native AOT op coverage” — are the VM widening that set so more of a real script's chunk compiles to native code rather than falling back. The native path is genuinely shipped, but its coverage is a moving frontier: the more of the opcode set the AOT backend handles, the more of any given script becomes machine code instead of a host call.

What's decided versus what's pending

The design doc is structured around locked decisions and open questions, and the distinction is worth keeping straight because “designed” doesn't mean “vague.”

Decided. The codegen pass goes upstream in fusevm rather than in a zshrs-side wrapper, so the sibling stryke project gets it free from the shared pipeline. The runtime ABI uses a `zshrs_vN_*` symbol-prefix scheme where every version coexists in the runtime `.a` forever — a binary built against `zshrs_v1_*` keeps linking when the runtime ships `zshrs_v2_*`, which the doc calls “the load-bearing decision for the 30-year horizon.” This is the one decision the doc flags as expensive to reverse: once binaries exist in the wild built against a given ABI version, changing the symbol scheme breaks them, so it has to be right before the first `zshrs build` ever runs. The `eval` embed (above) is settled the same way and for the same reason — compat over size. Static linking defaults to musl on Linux with glibc opt-in, which is what makes “no libc on the target” true

rather than aspirational. The memory model is settled hard: no GC, ever, inherited from Rust's ownership model rather than built, because a GC pause inside an AOP intercept on a hot path is the failure the no-GC rule exists to prevent.

Pending. The doc carries a table of open architectural calls with defaults rather than commitments — whether AOP intercepts are file-based or in-script decorators, what the default `--profile` behavior covers, the TLS profile buffer size, the live-endpoint port number. These are defaults the doc proposes to lock before code starts, not decisions code has validated. The whole thing was staged across phases A through F, and the shipped `zbuild` has walked the first several: A adds `cranelift-object` output to fusevm (done — the `aot` feature and `compile_object`); B builds the runtime stub the object links against (done in substance — the staticlib is `libzsh.a`, the whole runtime, rather than the leaner hand-pruned `libzshrs_runtime.a` the doc imagined, which is the one place the implementation chose pragmatism over the plan); C wires up the build CLI for single scripts and D adds multi-script bundles (both done, as the `zbuild --in A --in B` surface); E handles cross-architecture deployment via pre-shipped target stubs, and F is the acceptance gate — build the full 172k-line `zpw` suite as one AOT binary, run it on a clean Linux box with no `zsh` installed, and pass means ship. E and F are the rungs still ahead: the native link today targets the host triple and depends on a local `cc` plus the system terminal library, not the doc's self-contained cross-compiler, and the full-suite acceptance test remains a plan, not a recorded result. The capability runs; the deployment story it was meant to enable is not finished.

The capability that would justify it

The reason this is on the roadmap at all, rather than filed as “faster deployment,” is the stack the doc layers on top of native compilation: AOP weaving and nanosecond profiling baked into the binary at build time. `zshrs` already has runtime AOP — the `intercepts: Vec<Intercept>` field on `ShellExecutor`, advice dispatched at runtime. The AOT design moves that to compile time: at every call site matching an intercept's glob, the codegen pass emits a direct call into the advice chunk instead of dispatching through a runtime vector, and inlines the advice where it's small enough. The profiling story is the same shape — prologue and epilogue emitted around each profiled function reading the cycle counter directly (`rdtscp` on x86, `mrs CNTVCT_EL0` on aarch64), no syscall, sub-microsecond per-function timing, counters in thread-local storage.

The doc's claim for this combination is that no shipping shell supports AOP at all, none does compile-time AOP weaving, and none bakes nanosecond profiling into native code. The runtime AOP half is real and in the tree today — `intercepts` is a live field, advice fires at the prompt. The compile-time weaving and the native profiling prologue/epilogue are the parts that don't exist yet; they're the payoff the phases are ordered to reach. Whether the combination clears the project's bar is a question the phases answer, in order, with tests. Right now the answer is: the runtime AOP exists, AOT compilation to native code now exists too — both the source-trailer and the `--native` object paths run today — but the *build-time weaving* and the *native profiling prologue/epilogue* that were the whole reason to want AOT remain unwritten. The native binary you can build today runs the script's logic as machine code; it does not yet weave advice at the call site or emit an `rdtscp` prologue around a profiled function. That is the part still ahead, and the document marks which is which. So does this chapter.

Chapter 15 — The Discipline of the Port

There are two ways to build a zsh-compatible shell. You can study zsh's behavior, read the manual, run experiments, and write fresh code that reproduces what you observed. Or you can translate zsh's actual C source, function by function, line by line, into Rust. The first is a reimplementation — an approximation that converges on zsh as you find and fix the divergences. The second is a port — a transliteration where the C source is the specification and any divergence is a bug in the translation. zshrs chose the second, and the choice is enforced by tests that fail the build if anyone drifts toward the first.

This isn't an aesthetic preference. It's the only way to hold a compatibility floor that has to survive thirty years and keep running a 172k-line CLI framework, sixteen thousand completion files, and a hundred plugins without regression. An approximation has an error rate that you discover one broken script at a time, in production, at the prompt. A faithful port has the same behavior as the thing it ports because it is the same logic, expressed in a different language.

src/ported/ is strict-port territory

`PORT.md` names three top-level port-rule directories under `src/`. `src/ported/` is the 1:1 port — every file mirrors a `Src/<...>.c` from upstream zsh. `src/extensions/` is the only sanctioned home for features zsh doesn't have: the rkyv caches from Chapter 7, the daemon, the AOT design from Chapter 8, the persistent worker pool. `src/recorder/` holds one feature-gated debug subsystem. Beyond those, `PORT.md` carves out crate-root files as sanctioned non-port code outside the extension directories, on the reasoning that zshrs replaces zsh's tree-walking interpreter with a bytecode VM and there's no 1:1 port of `Src/exec.c`'s `execlist/execpline/execcmd` to be had. `fusevm_bridge.rs` is one such file. The runtime-state struct everything threads through, `ShellExecutor`, lives at the crate root in `src/vm_helper.rs` — which is worth noting because `PORT.md`'s prose still describes that carve-out as `src/exec.rs`, a filename that no longer exists in the tree. The doc's rule (the VM replaces the interpreter, so the executor isn't a port) is current; its filename is stale. That gap is itself a small instance of the maintenance problem the rest of this chapter is about: prose drifts, the tree is the truth.

Inside `src/ported/`, the file layout mirrors zsh's `Src/` tree byte for byte. `Src/builtin.c` becomes `src/ported/builtin.rs`. `Src/subst.c` becomes `src/ported/subst.rs`. `Src/Zle/zle_main.c` becomes `src/ported/zle/zle_main.rs`. `Src/Modules/cap.c` becomes `src/ported/modules/cap.rs`. There are no renames — no `_port` suffix, no `_v2`, no `_impl`, no stripping of any prefix. The Rust file stem is identical to the C file stem. The reason is given in `PORT.md` and it's about review, not taste: a reviewer with `rlimits.c` open in one pane and `rlimits.rs` in the other can scroll both at the same rate and check correspondence by eye. The port is auditable because it's positionally aligned with its source.

`PORT.md` puts the directory under an absolute freeze: the files currently checked in are the closed legal set. No new files, no new directories, ever. If a C function genuinely belongs in a `src/ported/` file that doesn't exist yet, the instruction is to stop and raise it with the maintainer — file creation is a deliberate one-line act, not a side effect of a port. The freeze exists because the failure mode it prevents is documented in detail: bots producing 114 duplicate function names, 148 extra copies, and over 1,600 self-confessed adhoc functions, every drift originating as a new file created to “have somewhere to put” a helper, a new function name invented because no real zsh function matched, or a `*State/*Table` aggregate invented to bundle file-statics into one parameter to thread through. The freeze closes all three vectors.

The rules, stated as the source states them

`PORT.md` is blunt about what a faithful port means, and the rules are worth quoting in their intent rather than paraphrasing them into something softer.

Names must exist in upstream zsh C. A Rust `fn`, `struct`, `enum`, `type`, or `static` under `src/ported/` is legal only if its name appears as the corresponding declaration in `src/zsh/src/**/*.{c,h}`. The function allowlist is `docs/zsh_c_functions.txt`, which on disk holds 2,484 unique C function names. If `grep` doesn't find the name, the name is invented, and invented names are "the drift signature." The document names the exact drift it caught: `shell_quote`, `get_user_home`, `is_directory`, `find_in_path` — plausible Rust helper names, every one of them deleted because no such function exists in zsh. The deletions aren't pedantry. A function named `find_in_path` is a function someone wrote fresh because they couldn't find the zsh function that does the job — which means it's an approximation of a behavior the C source already specifies exactly, and it's the approximation, not the port, that ships the next subtle incompatibility. The name rule catches reinvention at the moment of naming, before the body is even written.

Signatures must be identical to C. The example `PORT.md` gives is `paramtypestr`: C's `paramtypestr(Param pm)` ports to Rust `paramtypestr(pm: ¶m)`, *not* `paramtypestr(ptype: ParamType, flags: &ParamFlags)`. No threading state through as extra parameters, no splitting one C function into several Rust functions, no merging several into one, no reordering or renaming parameters for "Rust idiom." The C name `nam` stays `nam`, not `name`; `argv` stays `argv`, not `args`; `ops` stays `ops`, not `opts`.

Local variables keep the C source's exact names, order, and scope. This rule reaches inside function bodies. C's `int hard`, `limnum`, `lim`; declared at function top ports to `let hard`; `let limnum`; `let lim`; at function top — not `idx`, `count`, `result`, not combined into a tuple, not converted to block-scoped loop variables when C used a function-scoped `int i`; When C iterates `for (hn = ...; hn; hn = hn->next)`, the Rust port keeps `hn` as an `Option<HashNode>` and unwraps inside the loop without renaming it to `hn_opt` or `current`. The C author's choice of identifier is the canonical identifier, full stop.

Bag-of-globals aggregator types are banned. If C declares three separate file-statics, the Rust port has three separate `thread_local!` entries or three `Arc<RwLock<...>>` holders — not one struct bundling them. The document's reference case is `parameter.rs`, where fourteen invented `*Table` and `*State` structs (`CommandsTable`, `FunctionsTable`, `AliasesTable`, `OptionsTable`, `JobsTable`, and the rest) were all deleted on 2026-05-10 in a commit literally titled "port: dissolve bag-of-globals from modules/parameter.rs." None of those fourteen names existed in `Src/Modules/parameter.c`; all fourteen were Rust abstractions invented to make the code feel tidier; all fourteen were drift.

And every statement carries its provenance. The function doc-comment cites `/// Port of <c_fn>() from Src/<file>.c:<line>`, and non-trivial lines carry inline `// c:NNN` tags pointing at the C source line they translate. The C source's own comments port over verbatim, because — as `PORT.md` argues — those comments encode the *why* behind the *what*: which bug a workaround fixes, when a branch is reachable, what the author considered and rejected. Strip them and reconstruct intent later, and you're doing archaeology, not porting.

The enforcement: two tests that are the immune system

Rules in a markdown file are advisory. zshrs makes them load-bearing with two tests that fail CI on violation. They're worth reading because their intent is the whole argument for the port discipline.

`tests/port_purity.rs` enforces three things. First, every Rust file in `src/` (outside the quarantined `extensions/`, `recorder/`, and a tiny exempt set) must have a corresponding C file under `src/zsh/src/` per the strict 1:1 stem map — an orphan Rust file with no C counterpart fails the test. Second, every top-level `fn` must carry the `/// Port of ... from Src/...` citation immediately above its signature; a function without provenance fails. Third, no function may carry the marker `WARNING: THIS IS ADHOC IMPLEMENTATION AND NOT A FAITHFUL PORT`. The test's own header states the fix in one sentence: "The fix for this test is never to suppress it — it is to delete *ad hoc* code or replace it with a faithful port from `src/zsh/src/`." That

sentence is the entire philosophy. The test isn't a gate you route around; it's a detector of work that hasn't been done.

`tests/ported_fn_names_match_c.rs` enforces the name rule with teeth. It walks every free function in `src/ported/**/*.rs` and verifies the name appears in a checked-in C function index — `tests/data/zsh_c_fn_names.txt`, a `<basename>:<fn_name>` snapshot regenerated from the upstream tree by `tests/data/extract_c_fn_names.sh`. The snapshot is checked into git on purpose: the test must pass in a CI runner that has no local zsh source checkout, so it can't grep the C tree live; it checks against the committed list instead. The free-function walk tracks brace depth so it only counts declarations at module level — methods inside `impl/trait` blocks are skipped, because C's struct-of-function-pointers indirection doesn't preserve the function name through the pointer, so an `impl` method has no C name to match. The header comment records why the test exists in concrete terms:

the substitution-bug audit on 2026-05-07 found two helper fns I added (`paramsubst_bridge`, `store_assign`) plus seven pre-existing helpers that drifted from the freeze. The port was claiming "100% port" while running 11 helpers with no C counterpart. This test fails CI on any future drift so the next contributor can't quietly add `helper_to_make_it_work` again.

The test does a second-phase file-mapping check on top of the name check: a function isn't just required to exist in zsh's C source, it's required to live in the Rust file that mirrors the *correct* C source file. A port of `paramsubst` belongs in `subst.rs` because `paramsubst` is defined in `subst.c` — putting it in `params.rs` because that's where it's called from is a violation. The test carries two separate snapshot allowlists for this, and they do different jobs. `tests/data/fake_fn_allowlist.txt` is the name allowlist: the small set of maintainer-approved Rust-only helpers whose names aren't in the C index, each entry an audit-trail line recording a granted exception. `tests/data/ported_fn_file_mapping_allowlist.txt` is the placement allowlist: the snapshot of functions that exist in C but currently live in the wrong Rust file. Both are meant to shrink, never grow; adding to either without recorded maintainer approval is itself a violation, because the allowlist is the audit trail of granted exceptions, not a free pass.

These two tests are the immune system against architectural decay. They don't make the code correct — a faithful port can still have a bug, same as the C it ports from. What they make impossible is the *silent* introduction of code that only looks like a port: same signature, build green, and eighty lines of state-mutating C logic quietly missing. The body shortcut is, as `PORT.md` argues, structurally worse than the file shortcut, because a missing file leaves a clearly-named gap while a stubbed body produces a function that passes every superficial check.

What this discipline buys

The payoff is a real zsh, not an approximation of one. When a `zpw` subcommand exercises some obscure corner of parameter expansion — a flag combination, an edge case in word splitting, a glob qualifier interaction — the behavior it gets is the behavior `src/subst.c` produces, because `src/ported/subst.rs` is that file translated, not a fresh guess at what it does. The compatibility floor isn't a target the project is converging toward by fixing bugs; it's a property the port has by construction, modulo translation errors that the same C source can be diffed against to find.

This is why the project's priority order shifted from "ship the next feature" to "shore up the floor." Real arrays, ZWC autoload, ZLE completion firing, `zstyle`, `zmv`, every parameter flag — the unglamorous bedrock — wins over more parallel primitives and more anti-fork builtins until the daily-driver replacement is proven on a real `.zshrc` and a real `zpw` load. The port discipline is what makes that floor trustworthy. A faster shell that's subtly wrong about expansion semantics is a liability with good benchmarks. Trust-complete beats feature-complete, and trust comes from the C source being the spec.

The instruction the whole chapter reduces to is the one `PORT.md` opens with and the memory of the project repeats: port the C, never reinvent. The C source is forty years of accumulated correctness — every bug fixed, every platform quirk worked around, every edge case discovered by millions of users. An adhoc Rust replacement throws that away and starts the discovery process over. The port keeps it. That's the entire

reason the discipline is worth the friction it imposes.

Chapter 16 – Parameter Expansion and the Glob

Most shells give you variables. Zsh gives you a small expression language that lives inside `${...}` and `*(...)`. It is the part of the shell every nontrivial script leans on — the part that turns a line of text into an array, an array into a filtered subset, a filename pattern into “every file under here larger than ten kilobytes and modified in the last hour.” It is also the part that is hardest to reimplement, because the behavior is defined almost entirely by edge cases. The flag block changes the *type* of the result; the qualifier suffix filters on stat metadata; the slice clamps instead of erroring; the separator can carry quotes. A replacement that gets the common path right but the corners wrong breaks scripts that worked yesterday and passes the demo anyway.

The constraint that follows is mechanical: port each construct and prove it against the binary. Every construct in this chapter was verified by running `zshrs -f -c '...'` and comparing the bytes against zsh’s output rather than by reading the C source and trusting the translation. The audit lives in `docs/GAPS.md` as a list of closed gaps — each one a construct probed, found wanting or already correct, and pinned with a test. This chapter walks the two surfaces that file documents: the parameter-expansion flag system and the glob qualifier grammar. Both are large, both compose, and both have the property that a single off-by-one in the implementation produces output that is *almost* right — which in a shell is worse than output that is obviously wrong, because almost-right survives the smoke test and detonates in production.

The flag soup

The defining feature of zsh parameter expansion is the flag block — a parenthesized set of single letters that prefix the parameter name and transform the result. `${(f)x}` is not `$x` with a modifier tacked on; the `(f)` *changes the type* of the expansion from scalar to array. This is the thing that trips up everyone who comes to zsh from bash, and it is the thing that makes zsh worth using: an expansion can return an array, and the array flows into the next construct without a `for` loop or an `IFS` dance.

The flags also compose, which is what makes them a language rather than a lookup table. `${(@s,: )str}` is three decisions stacked into five characters: keep the result an array even inside double quotes (`@`), split (`s`), on a comma (`:`, `:`). `${(kv)assoc}` is two flags that peek at each other — `k` and `v` interleave keys and values only because each arm checks for its partner. Order matters and is fixed: in `${(@P)var}` the `@` runs before the `P`, so the indirection maps element-wise across the array the `@` produced. The implementation has to honor that ordering, because a flag that runs in the wrong slot silently produces a different shape, and a different shape is a different program. Each composition below was a discrete gap with its own test; none of them fell out of the single-flag code for free.

Start with the splitters. `${(f)str}` splits a scalar on newlines into an array — the workhorse for consuming command output line by line:

```
local lines=("${(f)$(< /etc/hosts)}")
print -l $lines           # one host-file line per array element
```

`${(z)str}` splits using the shell’s own word-parsing rules — quotes, escapes, and all — so `${(z)cmdline}` tokenizes a command line exactly the way the shell would before executing it. `${(w)str}` splits on whitespace into words. All three are array-producing: the handler returns a `Value::Array`, which is why `print -l ${(z)a}` puts each token on its own line rather than collapsing them back into a string. That distinction —

array versus joined scalar — is the single most important invariant in the whole expansion engine, and it is the one most often broken by naive ports.

The set operators come next. `${(u)arr}` returns the array with duplicates removed, first occurrence wins:

```
local a=(x y x z y)
print ${(u)a}      # x y z
```

`${(C)str}` capitalizes the first letter of each word and lowercases the rest — title-case. `${(U)x}` is the brute-force uppercase. `${(o)arr}` and `${(O)arr}` sort ascending and descending. These compose with subscripts and with `[@]`: `${(o)a[@]}`, `${(O)m[@]}` and friends all work, because the flag parser strips a trailing `[@]` or `[*]` before validating the name and lets the array-producing path do its job.

The join and split family covers the rest of the everyday work. `${(F)arr}` joins an array with newlines — the inverse of `(f)`. `${(j: :)arr}` joins on an arbitrary separator, `${(s:,:)str}` splits on one. The separator can carry quotes, and the flags compose: `${(@s:,:)str}` keeps the result as an array even inside double quotes, where a bare split would otherwise be rejoined. That `@-plus-s` composition was a real gap once — inside "...", the `@` ran first and wrapped the scalar in a one-element array, then the `s` arm did nothing because it only knew how to split scalars. The fix was to make `s` and `f` flat-map across every element of an array, so `arr=("a,b" "c,d"); ${(@s:,:)arr}` yields four flat elements, exactly as zsh does.

`${(B)x}` backslash-escapes shell metacharacters; `${(q)x}` and the `:q` modifier quote for re-input. `${(k)assoc}` and `${(v)assoc}` pull keys and values out of an associative array, and `${(kv)assoc}` interleaves them into a flat key-value list:

```
typeset -A m=(one 1 two 2)
print ${(kv)m}      # one 1 two 2 (order assoc-dependent)
```

There is also a flag for asking what a variable *is* rather than what it holds. `${(t)var}` returns zsh's type string — `scalar`, `integer`, `float`, `array`, `association`, with attribute suffixes like `scalar-readonly`, `scalar-export`, `integer-left`. Getting that right meant tracking attributes per variable, not guessing from the runtime value; a `VarAttr` record rides alongside each name and `format_zsh()` emits the canonical `<kind>[-modifier]*` string. All ten baseline shapes match zsh byte for byte.

Quoting, padding, and the gradient flags

Past the everyday splitters and joiners sit the flags that exist to format a value for some downstream consumer that has its own rules — a re-eval, a column, a terminal. The quoting gradient is the sharpest of these. `${(q)x}` backslash-escapes; `${(qq)x}` always single-quotes; `${(qqq)x}` always double-quotes; `${(qqqq)x}` emits the ANSI-C `'...'` form. That is a four-rung ladder, and zshrs had every rung off by one in an early build — `(q)` produced single quotes, `(qq)` double, and so on — which is the kind of bug that looks fine until a script feeds `${(qq)var}` back through `eval` and gets a value that no longer parses. The fix remapped both q-flag handlers and added a distinct level for the `'...'` form; the empty-array case is its own corner, because `(qq)` on an empty array emits `' '` — one quoted empty pair occupying a slot — rather than nothing at all, so an unquoted consumer still sees an argument. Its inverse `${(Q)x}` strips one layer of quoting back off, handling single-quote, double-quote, and bare-backslash escapes the way zsh's parser does.

The padding flags carry their argument inline. `${(l:N:)s}` left-pads to width `N`; `${(r:N:)s}` right-pads. The colon-delimited syntax also takes fill strings: `${(l:N:S1:S2:)s}` uses `S1` as a one-time prefix and `S2` as the repeated fill, so `${(l:5::0:)42}` (empty `S1`, `0` as `S2`) yields `00042` — and getting that right meant reading zsh's `subst.c` to learn that the empty-`S1` form is legal and that `S2` is what fills. Overflow truncates rather than erroring. `${(V)x}` makes a value safe to print: control characters render as `^X`, and `\n\t` stay as their escape spellings, which is what `cat -v` does and what `zle_highlight`'s special category does on screen.

Two more shape-changing flags round out the set. `${(n)arr}` sorts numerically the way a human reads filenames — `file2` before `file10` before `file20` — via a digit-run comparator rather than a float parse, so `file*` strings that all parse to `0.0` no longer stay in input order. `${(P)var}` is the indirection flag: it

treats the value of `var` as the *name* of another parameter and expands that, which is zsh's native answer to bash's `${!var}` (a construct zsh rejects as a bad substitution). `(P)` composes with the rest — `${(@P)arr}` dereferences each element, and `${(P)$(cmd)}` looks up the variable named by the command's output rather than returning the output verbatim — and each of those compositions was a separate fix, because the naive path applies `P` to the wrong operand.

Replacement and filtering on arrays

The pattern operators are where zsh pulls ahead of every other shell, because they apply element-wise to arrays. `${arr/old/new}` replaces the first match of `old` in *each element*; `${arr//old/new}` replaces every match in each element:

```
local files=(a.txt b.txt c.log)
print ${files//.txt/.bak}    # a.bak b.bak c.log
```

The filter operator `${arr:#pattern}` removes every element that matches the glob `pattern`, and `${(M)arr:#pattern}` inverts it — keep only the matches. With `setopt extendedglob`, a leading `^` negates inside the pattern itself, so `${arr:#^*.txt}` keeps only the `.txt` files and `${(M)arr:#^a}` keeps everything that does *not* match `a`. Without `extendedglob` set, the `^` stays literal — the option gate matters, and it is enforced at match time by reading the live option rather than baking a decision at parse time.

Inline pattern flags ride along too. `(#i)` makes a match case-insensitive, `(#l)` makes lowercase pattern characters match either case while uppercase stays exact, and `(#al)` switches to approximate matching — a Wagner-Fischer edit-distance check that tolerates one insertion, deletion, or substitution. These work everywhere a pattern is evaluated, because they all funnel through one matcher: `[[ str = pat ]]`, `case` arms, and the `${arr:#pat}` filter share `glob_match_static`, so a flag fixed in one place is fixed in all of them. That single-matcher discipline is why parity holds across contexts that, in a sloppier port, would each drift independently.

Slicing, with the off-by-ones that matter

Array subscripting in zsh is 1-based and inclusive on both ends, and it clamps rather than panics. `${arr[2,4]}` returns elements two through four. `${arr[-2,-1]}` returns the last two. `${arr[1,-1]}` returns the whole array. An out-of-range bound clamps to the array's edge instead of erroring. Each of those rules is a place where a C-style 0-based port would silently produce wrong answers, so each is pinned with a test, and the slice returns a `Value::Array` so that a downstream `print -l` or `for` consumes it element by element.

Scalars slice the same way but count Unicode codepoints, not bytes: `${str[1]}` is the first character and `${str[2,4]}` is characters two through four, measured in codepoints. This is the deliberate companion to the byte-based `substr/length` pair — two coordinate systems, never to be mixed — and the scalar slicer reaches it by falling through from the array-index path when the name turns out not to be an array.

The subscript itself is an arithmetic context, which is the subtle part. A bare name inside `[...]` resolves as a variable, and a full expression evaluates: `${arr[i]}`, `${arr[i+1]}`, `${arr[len-1]}` all do the arithmetic before indexing. Keys that contain `$` route through a dynamic path that expands the key first, so `${m[$k]}` and `${m[$pre$post]}` look up the computed key. Subscript flags `(r)`, `(R)`, `(i)`, `(I)`, `(e)` — reverse-subscript by value, index-of, exact — combine and compose with the rest.

Quoting, and the empty element that has to survive

There is a corner of array expansion that does not announce itself in any tutorial and bites every reimplementer: what `"$arr"` does to an array that contains an *empty* element. In zsh, `a=(1 "" 3); print "$a"` prints `1 3` — two spaces, because the empty middle element is joined like any other, producing `1`, then the

join separator, then nothing, then the separator, then `3`. Drop the empty and you get `1 3` with one space, which is a different value, and a script that builds a colon-separated `PATH` or a delimited record out of an array with intentionally-empty fields gets silently corrupted. The empty element has to survive the quoted join.

This is harder than it sounds in zshrs's compiler, because of *where* the quoting information lives. A bare `$arr` and a quoted `"$arr"` reach the runtime by different routes. The unquoted form goes through an `EXPAND_TEXT` wrapper that sets a runtime `in_dq_context` flag the variable-read consults to decide whether to word-split and elide empties. But a *simple* quoted read like `"$name"` — just the variable, nothing concatenated — is compiled to a direct `GET_VAR` builtin call with no `EXPAND_TEXT` wrapper at all, so at runtime `in_dq_context` is zero even though the source plainly had double quotes. The plain `GET_VAR` path, seeing the flag clear, word-elides the empty element, and `"$a"` wrongly collapses to `1 3`.

The fix that landed with the array-parity work is a second builtin, `BUILTIN_GET_VAR_DQ`, that shares its entire body with `GET_VAR` through a common `get_var_impl(vm, argc, force_dq)` and differs only in that `force_dq` is `true`. The compiler emits the `_DQ` variant whenever a simple read is quoted — detected the same way the `$_/$*` arms detect it, by either a nonzero double-quote nesting depth or the raw token arriving wrapped in the lexer's double-quote sentinel bytes (`\u{9e}...\u{9e}`), since the brace form `"${name}"` arrives sentinel-wrapped with the depth counter at zero. With `force_dq` set, the array joins via `sepjoin` keeping empties, and a scalar comes back verbatim with no `SH_WORD_SPLIT` and no empty-drop — exactly `Src/subst.c`'s double-quoted behavior. The whole thing is pinned by a `zsh_arrays_parity.rs` suite that runs each construct against real zsh with `-fc` and compares stdout byte for byte: one-indexing, `arr[0]` empty in zsh-native mode, negative indexing and its out-of-range clamp, `[M,N]` slicing including the full `[1,-1]` and middle `[2,-2]` forms, `+=` append, past-end element assignment that extends with gaps, negative-index assignment, and `${#arr}` length — the unglamorous bedrock that a “mostly works” array implementation gets subtly wrong.

The glob qualifier grammar

Now the other surface: the path-matching language. Every shell has wildcards. Zsh has *qualifiers* — a parenthesized suffix on a glob that filters and reorders the matches by file metadata. An experienced zsh user reaches for a one-line glob where a bash user reaches for `find` piped into a loop, and the reason is that the qualifier set covers most of what `find` covers — type, size, time, ownership, link count, sort order — in a syntax that returns a plain word list the rest of the line can consume directly. Like the parameter flags, the qualifiers compose inside one parenthesized block, and like the parameter flags, each one had to be taught to the qualifier scanner before it would parse at all, because an unrecognized qualifier character makes zshrs fall back to treating the whole `(...)` as a literal pattern.

The basic gate is `(D)` — dotglob — which makes a single pattern match dotfiles without flipping a global option:

```
print *(D)           # includes .hidden, .config, ...
```

Crucially, zsh excludes `.` and `..` even when dotglob is on, which the underlying Rust glob crate does not; zshrs filters them out textually so the result matches. And when the last path component itself starts with a literal dot — `print .*` — dotfiles match with no option at all, because the leading `.` is literal in that position. Both of those are corner cases that a naive port gets wrong in opposite directions, and both are pinned.

The size qualifier is `L`, with the full zsh syntax `L[+-]N[k|m|g|p]`. The default unit is bytes — `(L+3)` means “larger than three bytes” — and the suffix letters scale to kilobytes, megabytes, gigabytes, and 512-byte blocks:

```
print *(L0)           # exactly empty files
print *(L+10k)         # larger than 10 KB
print *(L-1m)          # smaller than 1 MB
```

The size check uses `lstat` metadata for symlinks rather than following them, so `*(L+0)` includes symlinks (their own path-string length is nonzero) while `*(L0)` excludes them — a small fidelity point that matters to anyone who globs over a symlink farm.

The time qualifiers are `m` (modification), `a` (access), and `c` (status-change), each taking a unit char, a sign, and

a count: `mh-1` is “modified within the last hour,” `ma-7` keys off access time, `mc-30` off ctime. The unit chars run `s` seconds, `m` minutes, `h` hours, `d` days, `w` weeks, `M` months. The sign reads the way the rest of zsh reads it — `-N` is “more recent than N units ago,” `+N` is “older than”:

```
print *(mh-1)          # changed in the last hour
print **/*(.mh-24)     # plain files touched in the last day, recursively
```

The handler parses the unit, op, and integer, then compares `meta.mtime()/atime()/ctime()` against the computed cutoff. Getting these recognized at all required teaching the qualifier scanner that lowercase `h` and `i` and `g` are valid qualifier characters — they were being rejected at parse time before — and teaching the expander to route plain paths with a trailing `(...)` suffix through globbing, so `/etc/hosts(mh-100)` actually evaluates the qualifier instead of treating it as a literal filename.

Type, sort, and per-match modifiers

The type qualifiers are the ones a `find` user reaches for first. `(.)` keeps only plain regular files, `(/)` only directories, `(@)` only symlinks:

```
print *(.)             # regular files only
print */(/)            # directories only
print **/*(@)          # every symlink under here
```

The subtlety is in `(.)`: zsh excludes symlinks-that-point-to-files, so the filter cannot lean on followed meta-data — a symlink to a regular file passes `is_file()` on the target. The implementation checks `symlink_metadata().file_type().is_symlink()` first and drops links before the regular-file test, which is the difference between `(.)` and `(@)` being two distinct sets and being the same set with a relabel. The link-count qualifier `l[+-]N` rides the same stat: `*(l2)` keeps files with exactly two hard links, `*(l+1)` more than one, read off `MetadataExt::nlink()`.

Qualifiers also reorder, not just filter. `*(om)` sorts the matches by modification time, newest first — the time-sort family defaults to descending, and the uppercase `O` flips it, so `*(Om)` is oldest first; `oa/oc` do the same for access and ctime:

```
print *(om)            # newest mtime first
print */(.om[1])       # the single newest regular file
```

Making this work meant two things that are easy to miss: the qualifier-driven order has to survive the post-glob alphabetical sort that `expand_glob` otherwise applies, and `O` had to join the scanner’s valid-character set or `*(Om)` parsed as a literal pattern with a stray `)`.

The last family is per-match modifiers — the same `:r/:t/:e` history modifiers that work on `$PATH`-style values, applied to each glob match in turn. `*( :r)` strips the extension from every match, `*( :t)` keeps only the basename, `*( :e)` returns the extension. They share one helper with the modifier path elsewhere in the shell, so a fix to `:r` is a fix everywhere `:r` appears.

And the qualifiers OR together, not just AND. A comma at the top level of the block is alternation: `*(.,/)` is “files OR directories,” `*(/,@)` is “directories OR symlinks.” The default chains every qualifier with AND, which makes `*(.,/)` an empty set — nothing is both a regular file and a directory — so the implementation detects a top-level comma (honoring `[...]/(...)` nesting), runs the full AND filter on each clause, and unions the results in original order with dedup.

Ranges and recursion

Two more pieces round out the grammar. The numeric range `<lo-hi>` matches a run of digits whose value falls in an interval, with either bound optional:

```

print file<1-10>      # file1 .. file10
print log<2024->      # log2024, log2025, ...
print v<-5>           # v0 .. v5

```

`<->` matches any number, `<5->` any number at least five, `<-10>` any number at most ten. The matcher translates each occurrence to a digit-capturing group, then verifies the captured value against the remembered bounds after the regex succeeds — a malformed form falls back to a literal `<`.

And the recursive globs. `**/` enumerates directories with the trailing slash preserved; `**/*` matches files *and* directories at every depth. These were both wrong once — `**/` returned the literal pattern and `**/*` matched only files — and the fix had three parts: detect the dirs-only form when the file portion is empty, include directory entries from the walker when descending, and strip the implicit `./` prefix so output matches zsh's relative-path style. Sort order matters too: a non-recursive glob sorts by basename, but `**/*` sorts by full path so the depth-first walk reads coherently rather than interleaving `f`, `g`, and `sub` at random depths. The recursive walk is parallel — it fans out across worker walkers — but the output order is deterministic and matches zsh.

The throughline of both surfaces is the same. Each construct is zsh's, ported one at a time and held in place by a test per gap, with the binary as the oracle. A script written for zsh's `${(f)...}` and `*(.om L+10k mh-1)` runs under zshrs and produces the same bytes. That byte-for-byte equality is the whole specification: a flag that returns a scalar where zsh returns an array, or a qualifier that sorts ascending where zsh sorts descending, is a bug regardless of how reasonable it looks in isolation, because the script downstream was written against zsh's answer and no other.

Chapter 17 – The Line Editor

The line editor is where the shell stops being a program you call and becomes a program you *live inside*. Every keystroke between prompts goes through it. A motion that lands one character off, an undo that loses a chunk of the buffer, a vi text object that grabs the wrong span — these are not abstract bugs. They are felt instantly, on every line, by muscle memory built over years. ZLE is the part of the shell with the least tolerance for “close enough,” because a user who has typed `ciw` ten thousand times will notice the eleventh time it misbehaves before they finish the keystroke.

So ZLE in zshrs is a registry of widgets dispatched through one function-pointer table. zsh’s C keeps the binding tables as pointers into `iwidgets.list`; zshrs keeps the registry as `IWIDGET_NAMES` in `src/ported/zle/zle_bindings.rs` and resolves a name through `iwidget_lookup`, which maps a widget name to its `ZleIntFunc` — the same `int (*)(char **)` shape the C widget signature has, expressed as a Rust fn pointer. The reason to mirror the table rather than invent a dispatch is that the table *is* the editor’s behavior: a keymap is a map from key sequences to widget names, and a widget name is meaningless without a registry that resolves it to exactly the function zsh would have run. Two dispatch mechanisms would be two editors that drift.

193 widgets

`IWIDGET_NAMES` holds 193 base widget names. With the two binding forms zsh tracks per name, that is 386 entries in the thingy table — which is what makes `${#widgets}` report the same count it reports under zsh. The list is walked at startup by `init_thingies()` to populate the table, so `zle -l` can enumerate every widget without each one having to be invoked first through a keymap. That enumeration parity is load-bearing rather than cosmetic: zsh plugins routinely test `(( ${+widgets[some-widget]} ))` to decide whether a feature is available, and a startup framework like the kind a heavy `.zshrc` loads runs dozens of those guards before the first prompt. If the count or the names drift, those guards make the wrong call — a plugin concludes a widget is absent and silently skips a binding, or concludes one is present and binds a key to a no-op. The 193 is therefore a fixed contract, not an implementation detail: it is the number the user’s plugins are written against.

Dispatch is uniform under that contract. Every name resolves through `iwidget_lookup` to a single canonical fn pointer, and a name in the registry that has no dedicated handler yet resolves to the `undefined-key` fallback — which is what zsh does for an unbound name too. So `zle -l` still counts it, a `bindkey` to it still succeeds, and pressing the key still no-ops the same way zsh no-ops. There is one table, one lookup, one pointer per name; there is no second path, no special-cased widget, no shortcut. That uniformity is what lets 193 widgets behave like 193 widgets instead of 193 independent opportunities to diverge.

The widgets fall into categories, and walking them is the fastest way to see the surface. The categories are not arbitrary — they track how a user’s hands move — so the registry’s job is to have the right widget under every key a `bindkey` or a keymap default might land on.

Basic motion and editing is the spine: `forward-char`, `backward-char`, `forward-word`, `backward-word`, `beginning-of-line`, `end-of-line`, `self-insert`, `backward-delete-char`, `delete-char-or-list`, `kill-line`, `kill-whole-line`, `kill-word`, `backward-kill-word`, `transpose-chars`, `transpose-words`, `capitalize-word`, `up-case-word`, `down-case-word`, `quoted-insert`. These are the emacs-default bindings, and they are the ones every interactive line touches.

History navigation layers on: `up-history`, `down-history`, `up-line-or-history`, `down-line-or-history`, `beginning-of-buffer-or-history`, `end-of-buffer-or-history`, `accept-line`, `accept-`

`line-and-down-history`, `accept-and-hold`, `insert-last-word`, `infer-next-history`, plus the `history-beginning-search-backward/-forward` pair that searches on the text before the cursor.

Incremental search is its own cluster, and it has both the literal and the pattern variants: `history-incremental-search-backward` and `-forward` for the classic `^R/^S` incremental search, and `history-incremental-pattern-search-backward/-forward` for the version that matches a glob as you type. The non-incremental `history-search-backward/-forward` round it out. Isearch is a mode, not a single action — it takes over the keymap while active and feeds keystrokes into the search buffer — and the widget set has to expose every entry point so a user's `bindkey` for `^R` lands on the right one.

Undo and redo look simple and are not. `undo`, `redo`, and `split-undo` are the visible widgets; the subtlety is that zsh tracks undo as a change *group*, so a multi-character insertion undoes as a unit rather than character by character, and `split-undo` forces a boundary. A vi user also gets `vi-undo-change`. Get the grouping wrong and undo either does nothing useful or erases too much — both are immediately wrong to the hand.

Yank and the kill ring: `yank`, `yank-pop`, `copy-region-as-kill`, `copy-prev-word`, `copy-prev-shell-word`, `kill-region`, `kill-buffer`. The kill ring is a ring — `yank-pop` cycles through previous kills after a `yank` — and the widgets only make sense as a set.

Shell-aware word motion is the category that separates a shell line editor from a text editor. Plain `forward-word` uses `WORDCHARS`; the shell-aware widgets understand quoting and command structure. `select-a-shell-word`, `select-in-shell-word`, and `copy-prev-shell-word` operate on shell words — a quoted argument is one word even with spaces inside it — which is what you want when editing a command line rather than prose. The emacs-style `emacs-forward-word/emacs-backward-word` pair is here too, alongside the default `forward-word`, because the two families draw word boundaries differently and a user's keymap may want either.

Completion entry points are widgets, not a separate subsystem — the line editor hands control to the completion machinery through them. `complete-word`, `expand-or-complete`, `expand-or-complete-prefix`, `menu-complete`, `reverse-menu-complete`, `menu-expand-or-complete`, `accept-and-menu-complete`, and `list-choices` are the keys a user binds to Tab and its variants. (Chapter 12 follows what happens after `complete-word` fires.) `expand-word`, `expand-history`, `expand-cmd-path`, and `list-expand` cover the in-line expansion forms; `magic-space` is the one that expands history references as the space is typed.

Argument prefixes and meta-commands are the last cluster. `universal-argument`, `digit-argument`, `neg-argument`, and `argument-base` build the numeric prefix that repeats or parameterizes the next widget — `^U 5 ^F` to move five characters in emacs mode. `execute-named-cmd`, `execute-last-named-cmd`, `where-is`, `which-command`, `describe-key-briefly`, and `run-help` are the introspection widgets a user invokes to ask the editor about itself, and `bracketed-paste` is the widget the terminal's paste mode dispatches to so a pasted block is inserted literally rather than executed line by line.

The vi set, in detail

The vi widgets are the largest single category, and that is structural rather than incidental: vi is a grammar, and the editor implements it as a grammar — operators (`vi-change`, `vi-delete`, `vi-yank`) compose with motions (`vi-forward-word`, `vi-find-next-char`) and text objects (`select-in-word`, `select-a-word`) to form verbs the user never typed as a single widget. `ciw` is `vi-change` applied to the span `select-in-word` returns; `d2f.` is `vi-delete` over two repetitions of `vi-find-next-char` for `..`. A vi user notices a wrong widget faster than anyone, because a broken motion breaks every operator that composes with it, and a broken text object breaks every operator-over-object that uses its span. The registry has to carry the full set for the grammar to close.

Mode and entry: `vi-cmd-mode`, `vi-insert`, `vi-insert-bol`, `vi-add-next`, `vi-add-eol`, `vi-open-line-above`, `vi-open-line-below`, `vi-replace`, `vi-replace-chars`, `vi-substitute`. Plus the small-but-load-bearing entry widgets a vi user hits without thinking: `vi-digit-or-beginning-of-line` (the `0`

that means “column zero” unless a count is in progress, in which case it is a digit), `vi-quoted-insert`, `vi-pound-insert`, `vi-backward-delete-char`, and `vi-backward-kill-word`.

Motion: `vi-forward-char`, `vi-backward-char`, `vi-forward-word`, `vi-backward-word`, `vi-forward-blank-word`, `vi-backward-blank-word`, `vi-forward-word-end`, `vi-forward-blank-word-end`, `vi-backward-word-end`, `vi-backward-blank-word-end`, `vi-first-non-blank`, `vi-end-of-line`, `vi-beginning-of-line`, `vi-goto-column`, `vi-match-bracket`.

Find and repeat: `vi-find-next-char`, `vi-find-prev-char`, `vi-find-next-char-skip`, `vi-find-prev-char-skip`, `vi-repeat-find`, `vi-rev-repeat-find`. The `f/F/t/T` family plus `;` and `,` — and `;` repeating the *last* find in the same direction while `,` reverses it is exactly the kind of state a vi user relies on without thinking.

Operators and changes: `vi-change`, `vi-change-eol`, `vi-change-whole-line`, `vi-delete`, `vi-delete-char`, `vi-kill-eol`, `vi-kill-line`, `vi-yank`, `vi-yank-eol`, `vi-yank-whole-line`, `vi-put-after`, `vi-put-before`, `put-replace-selection`, `vi-indent`, `vi-unindent`, `vi-join`, `vi-swap-case`, `vi-oper-swap-case`, `vi-up-case`, `vi-down-case`, `vi-repeat-change`.

Marks: `vi-set-mark`, `vi-goto-mark`, `vi-goto-mark-line`. History and search: `vi-history-search-backward/-forward`, `vi-repeat-search`, `vi-rev-repeat-search`, `vi-fetch-history`, and the line-or-history pair `vi-up-line-or-history/vi-down-line-or-history` that `j/k` bind to. And the register prefix `vi-set-buffer` for “a-style” named registers, plus `vi-undo-change` for the command-mode `u`. The default vi keymap in the binding table wires these to the expected keys — `$` to `vi-end-of-line`, `%` to `vi-match-bracket`, `;` to `vi-repeat-find`, `.` to `vi-repeat-change`, `f/F` to the find pair — so a user dropping into command mode with `^[` finds every key where it should be. The keymap is the second half of the contract: the registry guarantees the widget exists, the keymap guarantees the canonical key reaches it, and a vi user’s muscle memory depends on both holding at once.

Text objects and visual mode

Vi text objects — `iw`, `aw`, the shell-word and blank-word variants — are their own widgets: `select-in-word`, `select-a-word`, `select-in-blank-word`, `select-a-blank-word`, `select-in-shell-word`, `select-a-shell-word`. These define a *span* for an operator to act on, and the span boundaries are exactly where a vi user will catch an error: `diw` that deletes one character too many is wrong in a way that registers before the next keystroke. They live in their own port file, `textobjects.rs`, because the span logic is shared between the operators and the visual selection.

Region and visual mode tie it together. `set-mark-command`, `exchange-point-and-mark`, `deactivate-region`, `visual-mode`, and `visual-line-mode` manage the active region — the highlighted span between mark and cursor. Visual mode is the interactive selection a vi user builds with `v`, then operates on; `visual-line-mode` does the line-wise version with `V`. The region is a first-class piece of editor state, not a side effect of yanking.

Highlighting

What the user sees of all this is governed by `$zle_highlight`. The port is `zle_set_highlight()` in `zle_refresh.rs`, ported from C’s `zle_set_highlight()`, which walks each `category:spec` entry and parses the spec. The categories are zsh’s: `region`, `isearch`, `suffix`, `paste`, `default`, `special`, and `ellipsis`. So `region` colors the visual selection, `isearch` colors the incremental-search match, `paste` colors a bracketed paste, `special` colors otherwise-invisible characters, and `ellipsis` colors the truncation marker when a long line is squeezed. A user who has tuned `zle_highlight=(region:standout isearch:underline)` gets the same on-screen result, because the category names and the parse are zsh’s — and the category set is closed, so a spec naming a category zsh doesn’t have is rejected the same way, rather than silently coloring nothing.

The concrete shape of the table is one match arm per widget: `"undo" => Some(undo)`, `"self-insert" => Some(selfinsert)`, `"vi-cmd-mode"` to its handler, the same way C generates the `widgets[]` array from `iwidgets.list`. The invariant worth stating last is what is *absent*: there is no tree-walker shortcut, no interpreter path that handles “simple” widgets one way and dispatches the rest through the table. Every keystroke, from `self-insert` on the hot path to `vi-oper-swap-case` on the cold one, takes the identical route — name, lookup, pointer, call. That single path is why the editor’s behavior is the registry’s behavior and nothing else, and it is pinned by the same architectural tests that keep the rest of the VM off the tree-walker.

Chapter 18 – Completion

Completion is the hardest thing in the shell to get right and the most valuable, for the same reason: it is enormous, it is mostly *other people's code*, and it is invisible until it breaks. A user's completion experience is the sum of `compinit`, hundreds of autoloaded `_command` functions, the dispatch machinery that decides which one to call, and the matching engine that turns the call into a menu. None of it is one program you can read top to bottom. All of it has to agree, byte for byte, with what zsh does — because the moment Tab produces the wrong list, or no list, the shell has failed at the task the user performs more than any other.

What makes this uniquely hard is the ownership split. The parameter flags and the line-editor widgets are zsh's design that zshrs reimplements; the completion functions are not zsh's at all in the parts users care about — `_git`, `_kubectl`, the framework completions a heavy `.zshrc` autoloads — they are thousands of lines written by other people over decades, against the exact contract of `$compstate`, `compadd`, and the hook timing. zshrs cannot reimplement them; it has to *run* them, unchanged, inside a dispatch chain that presents the same contract those functions were written against. The chain is the port; the functions are the payload.

The path a Tab keypress takes is documented in [docs/COMPLETION_DISPATCH.md](#), verified against both zsh's `Src/Zle/*.c` and the Rust port on a dated audit. It is worth following link by link, because the value of the port is that every link is present and resolves to the same place its C counterpart does.

Two entry points

The dispatch has two shapes depending on whether `compinit` has run. `compinit` is the autoload that sets up the whole completion system — it scans `fpath` for `_command` files, registers them, and, in the loop that matters here, runs `zle -C $widget . $widget _main_complete` for each completion widget. That `zle -C` registers a *Comp widget*: a record carrying a base C function, the widget name, and the completion function name `"_main_complete"`. Before `compinit` runs, Tab does the old thing; after it runs, Tab does the new thing — and the only difference between the two paths is whether that Comp-widget record exists to plant the function name.

Pre-`compinit`, the default Tab binding is `expand-or-complete`. The chain is `expandorcomplete` → `docomplete` → eventually a lookup for a completion shell function that comes back empty, so nothing fires and the path falls through to the legacy `compctl` machinery. In zsh's C this is `getshfunc(NULL)` returning NULL and the if-branch being skipped; in the Rust port it is `callcompfunc` returning early because the function name is empty. Same gate, same outcome: no compsys, fall through.

Post-`compinit`, the key is rebound to `complete-word`, which is registered as a `zle -C` completion widget. Now the chain has a middle: `completeteval` runs first, and its job is to set the global `compfunc` to `"_main_complete"` — the name of the top-level compsys function — before calling the base completion function for the widget. That base function is `completeword`, which calls `docomplete`, which fires the completion hook, which reaches `do_completion`, which calls `callcompfunc(s, compfunc)` — and now `compfunc` is `"_main_complete"`, so the lookup finds the autoloaded function and runs its body.

The detail that makes this real: the literal string `"_main_complete"` appears in *no* `.c` file. It enters the shell-function table only when `compinit` autoloads it, and the C side reaches it purely through the `compfunc` indirection that `completeteval` plants. The Rust port has to reproduce that indirection exactly, because anything else would either hardcode a name zsh never hardcodes or miss the user's own overrides.

The chain, link by link

The port mirrors each C function to a Rust function, and the dispatch document tables them with file and line:

- `expandorcomplete` (zle_tricky.c:299) → `zle_tricky.rs:268` — identical.
- `completecall` (zle_tricky.c:201) → `zle_tricky.rs:98` — identical. This is where `compfunc` gets set from the widget's registered function name.
- `completeword` (zle_tricky.c:216) → `zle_tricky.rs:144` — identical.
- `docomplete` (zle_tricky.c:599) → `zle_tricky.rs:696` — split into two for readability (see below).
- the completion hook fire, C's `runhookdef(COMPLETEHOOK, ...)` → `docompletion` at `zle_tricky.rs:1397` — identical.
- `do_completion` (compcore.c:287) → `compcore.rs:74` — identical.
- `callcompfunc` (compcore.c:544) → `compcore.rs:587` — divergent body, same end-state.
- `getshfunc` (utils.c) → `utils.rs:4723`, `doshfunc` (exec.c) → `exec.rs:5476`, `runhookdef` (module.c) → `module.rs:663` — all identical.

Two of those carry deliberate, documented divergences, and both are the “functionally equivalent end-state” kind rather than the “we did something different and hoped” kind.

The first is `docomplete`. C's single function does six things: a recursion guard, a before-hook, a history-expansion early return, the completion-string extraction, the dispatch on what kind of completion was asked for, and an after-hook. The Rust port splits this into `docomplete` (the guard, the early returns, the extraction, the after-hook, and the dispatch) and `docompletion` (just the hook-fire step — build the payload, look up the registered hook, fire it, and fall through to `do_completion` if no hook is registered, matching C's default-fallback). The same hook fires in the same order with the same payload; the split is for legibility, nothing else.

The second is `callcompfunc`, and it is the more interesting one. In C, `callcompfunc` calls `getshfunc(fn)` to fetch the precompiled function body, then `doshfunc` to run it inside the proper function scope. The Rust port skips the explicit `getshfunc` lookup. Instead it gates on an empty function name (the same gate as C's `getshfunc(NULL)` → skip), builds a synthetic `Shfunc` carrying just the name, and constructs a closure — the `body_runner` — that dispatches the body. The closure tries `compsys::router::try_rust_dispatch(fn_name)` first, falling back to the standard bytecode dispatch through `exec::dispatch_function_call`. Then it calls `doshfunc` with that closure, and `doshfunc` wraps it in the same C-faithful prologue and epilogue: push the parameter scope, save `lastval`, push the function context, handle the return flag.

The end-state is identical — `_main_complete` runs inside a real function scope with `$compstate` and the completion parameters populated. What differs is only *how the body is resolved*: C fetches a compiled wordcode program; Rust hands `doshfunc` a closure that either routes to a native Rust port of a compsys helper or to the bytecode VM. Because `doshfunc`'s scope plumbing is the same on both sides, the observable behavior is the same.

The router, and why it matters

That closure detail is the whole reason zshrs can ship native completions without breaking compatibility. `compsys::router::try_rust_dispatch` resolves a `_NAME` to a Rust function pointer, gated on a per-machine config: `[compsys] backend = "rust"` routes `_NAME` calls to the Rust port tree under `src/compsys/ported/`; `backend = "shell"` bypasses it entirely and lets the standard shell-function autoloader win. The router degrades per name — if no Rust port is registered for a given `_NAME`, it returns `None` and the call falls through to the shell autoloader, so partial coverage is fine. Right now the registered ports are `_main_complete` and `_setup`; everything else falls through to the upstream compsys files unchanged.

This is the architectural divergence stated plainly: C zsh has one completion backend, the shell-function tree autoloader from `fpath`. zshrs ships a *parallel* Rust-port tree reachable through the router, and the user chooses with `backend = "rust"` or `backend = "shell"`. The constraint that keeps the parallel tree honest is that a Rust port must run *inside* `doshfunc`'s scope — as the `body_runner` closure — so the

per-call side effects apply whether the body is native Rust or bytecode. Those side effects are not optional bookkeeping; they are the contract every `_command` function was written against. The local level increments, the `FUNCSTACK` pushes, the `BREAKS` / `CONTF` / `LASTVAL` state saves and restores around the call, the trap state is preserved, and `pipestatus` is deep-copied so the completion function cannot clobber the caller's `$pipestatus`. A `_git` that ran without that scope would see the wrong `$funcstack`, leak a `return` into the caller, or corrupt the exit status the next prompt reads. Skipping `doshfunc` to “just run the Rust function” is a real bug, not a cosmetic one, and the router is structured so there is no path that does it: every body, native or compiled, goes through the same wrapper.

The two practical consequences follow directly. Adding a new built-in completion — a Rust port of `_git`, `_kubectl`, whatever — means registering one line in the router; the chain already routes through `body_runner`, which checks `try_rust_dispatch` first, so the port fires the moment `compfunc` equals its name. And shell-side `_main_complete` keeps working unchanged: anyone autoloading the upstream compsys file gets it executed via the same `body_runner` closure, the fusevm branch, with the same scope semantics and the same parameter population. No behavior shift.

The compctl path, wired to actually complete

The dispatch chain above is the *post-compinit* story. The other half — what a bare `zsh -f` with no completion system loaded does on Tab — runs through the legacy `compctl` machinery, and bringing that path to parity is its own body of work, because for a long time it produced nothing: `l<Tab>` inserted a stray character instead of offering the roughly two hundred and thirty commands stock zsh completes. Making it real meant porting the pieces that feed the default command-position spec — `setup_` seeds the hardwired `cc_compos/cc_default/cc_first` defaults when the default-loaded `zsh/compctl` module registers, before any rc file runs; the `CC_COMPPATH` arm walks each `$path` directory and adds its executables; and `ccmakehookfn` builds the group, runs the global match list, and reports matches-or-none so `do_completion` stops taking the error path. On top of that the word being completed is now classified the way zsh classifies it: a leading `~user` or `=command`, a `$param` or a braced `${PA<Tab>}` parameter reference, the right-hand side of an `x=<Tab>` assignment, or an ordinary command or filename each route to the matching generator, with a leading tilde expanded so `~/path` opens `$HOME` and dotfiles offered only when the word itself starts with a dot.

The listing that results is zsh's listing rather than a raw dump. Matches are sorted by locale collation and de-duplicated — a command found in three `$path` directories appears once — and laid out column-major in a grid sized to the terminal instead of one per line. Long lists no longer flood the screen: `complistmax` is synced from `$LISTMAX`, so a two-hundred-match result prints zsh's *do you wish to see all 230 possibilities* prompt rather than scrolling past. And `MENU_COMPLETE` cycles: `setopt menucomplete` inserts the first match on an ambiguous Tab and steps through the rest on each repeat — `tru<Tab><Tab>` walks `true` → `truncate` → `trust` — keeping the command prefix intact as it goes, rather than re-listing from scratch.

Why completion is the hard part

The reason this chain gets this much care is the ownership split again, viewed from the cost side. Completion is the largest body of code in the zsh ecosystem that zshrs does not own — thousands of lines of `_command` functions written over decades, depending on the exact contents of `$compstate`, the exact timing of the hooks, the exact semantics of `compadd`, the exact behavior of the matching engine. A port that gets the dispatch *almost* right produces completions that work for `ls` and fail for `git`, which is worse than no completion at all because it looks like it works — the user binds Tab, gets a sensible menu for the simple cases, trusts it, and then loses an argument completion three commands into a real session with no error to explain why. The target is narrow: the user autoloads the same compsys files they already use, and Tab produces the same menu. That target is checked at two altitudes — the `Y01completion.ztst` and `Y02compmatch.ztst` harness files exercise the full chain end to end through the test runner, and the unit tests around `callcompfunc` (`callcompfunc_empty_fn_no_panic`, `callcompfunc_sets_compstate_context`) pin the empty-name gate and the `$compstate` setup against C's behavior.

Compatibility is the project's central constraint, and completion is where it is hardest, most invisible, and most expensive to get wrong. The chain carries the cost so the payload — the user's own completion functions, unchanged — can run on top of it without noticing it was ported at all.

Chapter 19 — The Parameter Zoo and the Grammar

A shell is not only a language for running commands. It is also a vast, half-documented surface of *introspection* — ways for a script to ask the shell about itself. What aliases are defined? Is this option set? What's the current epoch time, down to the microsecond? Which inode does this file live on? Zsh accreted answers to all of those questions over thirty years, and most of them are spelled as special parameters, magic associative arrays, or extensions to the test operator and the grammar. Together they form a kind of zoo — a collection of small, individually-tame features that you only notice when one of them is missing. Then your `.zshrc` breaks, your prompt theme goes blank, and a completion function falls over because `${functions[_git]}` came back empty.

This chapter is the tour of that zoo as zshrs reproduces it: the special parameters, the magic assocs that synthesize shell state on the fly, the test operators that go beyond POSIX, the grammar extensions that zsh layered onto the Bourne shell, and `zstyle` — the configuration database that the entire completion system reads from. None of it is glamorous. All of it is load-bearing.

Special parameters

Start with the small ones. Zsh exposes a handful of parameters whose values aren't stored anywhere — they're computed when you read them. `$EPOCHREALTIME` is the canonical example. Read it and you get the current time as a floating-point number of seconds since the epoch, with microsecond resolution. zshrs emits it in the form `SECS.UUUUUU` — seconds, a decimal point, six digits of microseconds — which is the format scripts parse when they want sub-second timing without shelling out to `date`.

```
print $EPOCHREALTIME      # 1750291847.482193
```

The reason that format is pinned to six digits and not five or seven is that it is a *parsed surface*. A prompt theme that wants render-time accounting reads `$EPOCHREALTIME` before and after a segment and subtracts; a profiling harness splits on the decimal point and treats the right half as a fixed-width microsecond field. Emit five digits and the arithmetic that reconstructs the fraction is off by an order of magnitude. The microsecond resolution is not decoration — it is what lets a script do sub-second timing entirely inside the shell, with no `date` fork on the hot path. Every fork avoided on a prompt redraw is latency the user feels directly, so the in-process clock is worth getting byte-exact.

`$argv` is the array alias for the positional parameters. In zsh, `$argv`, `$@`, and `$*` all refer to the same underlying list — `argv` is just the name that reads most clearly inside a function body.

```
set -- a b c
print $argv      # a b c
print ${#argv}   # 3
```

That `${#argv}` returning `3` rather than `5` matters more than it looks. The naive implementation joins the array on `$IFS` and reports the byte length of the joined string — `"a b c"` is five characters, so you get `5`, which is wrong. zshrs special-cases `argv` alongside `@` and `*` in both the length-expansion path and the count builtin, so `${#argv}` reports the *element count*, the way zsh does.

The same alias has to thread through the subscript path, not just the length path. `${argv[2]}`, `$@[N]`, `$@[N,M]`, `$*[N,M]` — these all index the positional list directly, and the bug they replaced is instructive:

every one of those shapes used to fall through to the *scalar*-slice path, which sliced the IFS-joined string rather than the array. The fix recognizes `@`, `*`, and `argv` as special names inside the array-index builtin (1-based, with negative-from-end and slice forms), and teaches both the braced and bare subscript-reference parsers to accept `@/*` as a base even though they aren't alphabetic identifiers. Without all three, `[@1]` returned the first *byte* of `"a b c"`, not the first element. The lesson recurs: a parameter that “aliases” another is only an alias if every reader honors the alias, and the readers in a shell are scattered across length expansion, count, subscript, and slice — four places that each have to special-case the same three names.

Then there's the terminal trio. `$TTY` is the path of the controlling terminal, resolved through libc's `ttyname`. `$TTYIDLE` is how long that terminal has been idle, computed as the delta against the tty device's `atime`. `$TRY_BLOCK_ERROR` is stranger — it only has meaning inside a `try/always` construct, where it carries the error status out of the `try` block so the `always` block can inspect it. `zshrs` sets it via a dedicated `BUILTIN_SET_TRY_BLOCK_ERROR` instruction emitted *between* the two arms, so the `always` block reads the status the `try` block left behind. We'll come back to the construct itself when we reach the grammar. Two more dynamic specials round out the set: `$patchars`, and `$RANDOM_FILE`, which — like mainline `zsh` — stays empty until `zmodload zsh/random` is loaded. That last one is worth a beat: the right behavior for a parameter that depends on an unloaded module is to *match zsh's emptiness*, not to invent a value. Compatibility means reproducing the absence, not just the presence.

There's a discipline question buried here, and `zshrs` answers it carefully: *when is a special parameter “set”*? If you write `${SECONDS-default}`, `zsh` gives you the live value of `SECONDS`, not the string `default`, because `zsh` treats these dynamically-computed parameters as *always-set*. A literal `contains_key` check against the parameter table fails this test — `SECONDS` isn't *in* the table, it's computed on read. `zshrs` whitelists the dynamic specials (`SECONDS`, `EPOCHSECONDS`, `EPOCHREALTIME`, `RANDOM`, `LINENO`, `HISTCMD`, `PPID`, `UID`, `EUID`, `GID`, `EGID`, `SHLVL`) at every place the “is this set?” decision gets made, so the `-`, `:-`, `+`, and `:+` modifiers all behave the way a script author expects.

That whitelist isn't a single check — it's the same list duplicated across three independent decision points: the flag-aware default path, the no-modifier default path, and the parameter-default builtin family. The duplication is the cost of fidelity. A parameter that is “always set” has to be set everywhere the question is asked, and `zsh` asks it in more than one place. The same specials surface a second time in arithmetic context: `$(SECONDS)`, `$(RANDOM)`, `$(PPID)`, `$(EPOCHSECONDS)` all have to resolve to the live value, because the arithmetic evaluator looks names up in a static map that the dynamic getters don't populate by default. `zshrs` injects each one into the evaluator's extras map per evaluation — and `RANDOM` re-resolves on every call, so two arithmetic substitutions in a row return distinct numbers, the way a real PRNG read does. A parameter that reads correctly in `${...}` but returns `0` in `$(...)` would be a silent, maddening divergence, the kind that only surfaces three layers deep in someone else's plugin. The intrinsic read-only specials (`PPID`, `LINENO`, `ZSH_ARGZERO`, `argv0`, `ARGC`, `_`) carry the opposite discipline: assigning to them errors `read-only variable: NAME` exit 1, hard-wired ahead of the user-managed readonly set, because `zsh` refuses the assignment and a script that probes the refusal has to see the same rejection.

The magic associative arrays

This is where introspection turns into something that feels like reflection. `Zsh` exposes a set of associative arrays whose keys and values are the shell's internal tables. They aren't stored as ordinary `assoc`s — they're synthesized at lookup time, every time you index them, so they always reflect the current state.

```
print ${commands[ls]}      # /bin/ls
print ${aliases[ll]}      # ls -l
print ${functions[my_func]} # the full function body
print ${builtins[echo]}    # the builtin's kind
print ${options[interactive]} # on / off
print ${parameters[PATH]}  # scalar-export-special
```

The full set `zshrs` synthesizes is broad: `${commands}` (the command hash), `${aliases}` and `${galiases}` and `${saliases}` (regular, global, and suffix aliases), `${functions}` and `${dis_functions}` (defined

and disabled functions), `${builtins}`, `${reswords}` (reserved words), `${options}`, `${parameters}`, `${modules}`, the job tables (`${jobtexts}`, `${jobdirs}`, `${jobstates}`), and the directory tables (`${nameddirs}`, `${userdirs}` — the latter backed by libc's `getpwnam`). Modules that the daily-driver path actually loads extend the set further: `zmodload zsh/system` makes `${sysparams[pid]}`, `${sysparams[ppid]}`, and `${sysparams[procsbstpid]}` synthesize from the live process state, and `zmodload zsh/mapfile` turns `${mapfile[/path]}` into a verbatim file read — bytes exactly as on disk, trailing newline preserved, so a `"test\n"` file gives `${#mapfile[/path]}` of 5, not 4. All of them route through a single `magic_assoc_lookup` inside the array-index builtin: when a subscript hits one of these special names, instead of reading a stored assoc, `zshrs` synthesizes the answer from the live data structure.

The synthesize-at-lookup design earns its keep when these assocs compose with parameter-expansion flags. `${(@f)mapfile[/path]}` has to split the synthesized bytes on newlines *and* produce the N+1 elements `zsh` produces when the file ends in `\n` — the trailing empty element is part of the contract, not a bug to trim. Making the flag machinery see a freshly-synthesized value as a pre-resolved scalar (rather than re-deriving it) is what lets `${(f)mapfile[...]}`, `${(s:,:)assoc[k]}`, and `${(U)assoc[k]}` all work against magic and ordinary assocs alike. The flags don't know — and shouldn't know — whether the value they're transforming came from a stored table or was conjured at the subscript. That uniformity is the whole point of routing every one through the same lookup.

Why does this matter enough to spend a whole table of names on it? Because the completion system, the prompt themes, and most of the plugins in any serious `.zshrc` are built on these. `Powerlevel10k` checks `${functions}` to decide whether a segment's backing command exists. Plugin managers check `${aliases}` to avoid clobbering user definitions. A completion function that wants to know “is `git` actually installed?” reads `${commands[git]}` rather than forking a `command -v`. There is a measurable reason this is the idiom: `${commands[git]}` is a hash probe against an in-memory table, while `command -v git` in a fork-per-task shell is a `fork`, a `PATH` walk, and a `wait`. A `.zshrc` that runs a few hundred of these on every prompt — and serious ones do — is the difference between an interactive shell that feels instant and one that stutters. The magic assocs exist precisely so introspection costs a table lookup instead of a process. If any one of them returns empty when it shouldn't, the failure surfaces three layers up as a blank prompt or a missing completion, with nothing in the visible output pointing at the cause. Synthesizing them at lookup time — rather than maintaining a parallel copy that can drift out of sync with the real tables — is the only way to keep them honest: there is no second copy to forget to update.

Test operators beyond POSIX

The `[[ ... ]]` conditional is where `zsh` extends the test language well past what POSIX `[` offers. `zshrs` ports the extensions that scripts actually reach for.

`[[ a -ef b ]]` is the same-file test: true when both paths resolve to the same (device, inode) pair. `zshrs` implements it as a dedicated builtin that pulls `fs:metadata` for both operands and compares the device and inode numbers — so a symlink and its target test equal, and two hardlinks to the same data test equal, exactly as `zsh` does.

```
ln file hardlink
[[ file -ef hardlink ]] 66 print same # same
```

`[[ -o option ]]` tests whether a shell option is currently set. The operand gets normalized first — underscores stripped, lowercased — so `[[ -o RC_EXPAND_PARAM ]]` and `[[ -o rc_expand_param ]]` both work, which is the same leniency `zsh` applies. And the `no`-prefix inversion is honored: `[[ -o nounset ]]` and `[[ -o nonounset ]]` return opposite answers. When you name an option that doesn't exist, `zshrs` does what `zsh` does — emits `no such option: NAME` to stderr and returns false, rather than silently swallowing the typo.

The rest of the `zsh` file-test vocabulary is here too — `-nt` and `-ot` (newer-than / older-than by mtime), the permission-bit tests `-k`, `-u`, `-g`, and the ownership tests `-O` (owned by effective uid) and `-G` (owned by effective gid). These are the operators that the conditional-expression corpus, `02cond.ztst`, exercises against real

files: it creates a zero-length file, a modified file, a setgid directory, and runs every operator against them, comparing zshrs's verdict to zsh's.

The error paths in the conditional get the same fidelity as the success paths, because scripts branch on the exact wording. `[ a -ZZ b ]` — an unknown alphabetic operator in the three-argument form — has to error `[:1: unknown condition: -ZZ` and exit 2, not silently fall through to the AND/OR connective splitter and return a misleading status. The bug it fixed is the kind that hides for years: the three-argument path only recognized the *numeric* comparators (`-eq`, `-ne`, and friends), so any unrecognized alphabetic operator slid past into the splitter, which found nothing to split and returned 1 — a plausible-looking false that masks a real typo. Pinning the diagnostic means a script that probes for “is this operator supported?” sees zsh's answer, exit code and message both.

The grammar extensions

Beyond parameters and operators, zsh adds genuine *syntax* — constructs the Bourne grammar never had. Four of them recur often enough in real configurations to walk through.

time as a sublist keyword. In POSIX, `time` times a single pipeline. Zsh lets you write `time { ... }` and time a whole compound block. zshrs compiles the braced sublist as a sub-chunk, runs it, and prints the elapsed wall-clock time using `$TIMEFMT`. The `A08time.ztst` corpus is unforgiving here: it checks that `time x=1` reports a “shell” and “children” line, that `time` over a for-loop with an arithmetic condition produces the right loop result *and* the timing lines, and that `TIMEFMT` format codes — `%E`, `%U`, `%S`, the millisecond-precision `%*E` forms — all render at the documented precision. Getting the *number* right is easy; getting the format string and the multi-line output byte-identical is the work.

It is more unforgiving than that, because the corpus times *failures* with the same rigor as successes. `time builtin nonesuch $(sleep 2)` has to exit 1 with `no such builtin: nonesuch` on stderr — the `time` keyword timing and reporting a command that itself failed. `time /no/such/command` has to exit 127 with `no such file or directory`. `time` under `errexit` is its own row. Each of these is a corner where a shell could plausibly diverge in three different ways at once — wrong exit status, wrong message wording, or timing the wrong thing — and the entry only passes when all three agree with real zsh. The pressure this puts on the implementation is that the timing wrapper and the command's own error reporting have to compose: the elapsed-time block prints *and* the failure surfaces, in the right order, with the right bytes.

```
{
  false
} always {
  print "try block error: $TRY_BLOCK_ERROR"
}
```

for var (a b c) cmd and the braced form. Zsh's short for-loop drops the `in` and the `do/done`, wrapping the word list in parentheses instead. Both the single-command form and the braced-body form work:

```
for f (*.txt) print $f
for f (*.txt) { print $f }
```

There's a lexer quirk underneath this one worth naming, because it's a recurring theme in zshrs: the lexer emits the parenthesized word list as a *single* string token, with the open and close parens encoded as the control bytes `\u{88}` and `\u{8a}` (zsh's `INPAR` and `OUTPAR` tokens). The for-loop parser has to recognize that shape and split it back into words. This is the kind of seam that exists because zshrs ports zsh's tokenization faithfully rather than inventing its own — the token encoding is inherited, so the parser has to speak it. The same encoding shows up in the one-line function-definition path: `foo() echo hello` lexes as a single token whose suffix is `\u{88}\u{8a}`, and the func-def synthesizer has to peel the `()` marker off the name and treat the rest of the line as the body. These aren't independent quirks; they're the same inherited token vocabulary surfacing wherever the grammar reuses parentheses. Reimplementing the lexer to emit cleaner tokens would mean diverging from the C source's behavior at every place that vocabulary leaks into observable output — so zshrs speaks the inherited dialect instead.

`exec {fd}>file` — **named file descriptors**. Instead of hardcoding a descriptor number, you let the shell allocate one and store it in a variable:

```
exec {logfd}>/tmp/log
print -u $logfd "this goes to the log"
exec {logfd}>&-
```

The parser detects the `{NAME}` token immediately followed by a redirection operator and pops the name as a variable id. At runtime, `zshrs` opens the path with the right `libc` flags, dups the result to a descriptor at or above 10 (via `F_DUPFD_CLOEXEC`, so the fd doesn't leak across an `exec`), and stores the allocated number into `$NAME`. Two details there are load-bearing. The descriptor floor of 10 keeps these allocations clear of the standard low fds, matching `zsh`'s behavior so that a script juggling several named fds never collides with 0, 1, or 2 — and never accidentally overwrites a descriptor some other redirection is already using. The `CLOEXEC` flag is the other: a named fd opened for logging shouldn't survive into an `exec`'d child, because the whole point of letting the shell allocate the number is that the script never hard-codes it, and a leaked fd is one the child can't see or close by name. The combination — high number plus close-on-exec — is what makes the feature safe to use in the deep nesting where it actually appears, inside plugin functions that open a side-channel fd and expect it gone the moment they hand off to an external command.

zstyle — the configuration database

Last in the zoo, and the one with the widest real-world blast radius, is `zstyle`. It's a small key-value database keyed by `(context-pattern, style-name)`, and it is the configuration substrate the entire completion system reads from. Every `zstyle ':completion:*' ...` line in a `.zshrc` is registering preferences that `compsys` later queries. If `zstyle` doesn't behave exactly like `zsh`'s, completion behaves subtly wrong everywhere.

`zshrs` implements `zstyle` against an internal `StyleTable`, and the part that demanded the most care is the *listing* output. `zstyle -l` is supposed to emit source-replayable lines — text you could paste back into a `.zshrc` — and bare `zstyle` (no arguments) emits a different, grouped-by-style format. An early bug had the listing path printing the tuple in the wrong order: the table stored `(pattern, style, values)` but the caller printed `(style, pattern, values)`. The fix realigned the semantics so `-l` emits `zsh`'s bare-word form, `zstyle <pattern> <style> <vals>`, quoting a field only when it contains whitespace or is empty — and bare `zstyle` uses `zsh`'s grouped form, the style name on its own line with each pattern indented beneath it.

The diagnostics matter just as much as the data. `Zsh`'s `zstyle` has opinions about malformed invocations, and scripts (and users) depend on the exact wording:

- `zstyle -X` — an unknown flag — errors `invalid option: -X` and exits 1, rather than silently registering `-X` as a pattern.
- `zstyle X` — a single non-flag argument — errors `not enough arguments`, because you need at least a pattern and a style.
- A bare `zstyle` - errors `not enough arguments` (a degenerate flag-only call), *not* `invalid option: -`.
- The get/test flags `-g`, `-s`, `-t`, `-T` error `not enough arguments` when underfed, rather than returning a silent failure.
- `-T` is accepted as “test style, defaulting to true when unset” — the with-default sibling of `-t`.

Each of those is a one-line behavior, and each one is pinned by a test that delegates the expected output to real `zsh`. That's the through-line of this whole chapter: the parameter zoo is dozens of small features, none of which is hard in isolation, all of which fail loudly in aggregate if any one drifts. The only way to keep them honest is to compare every one against the real shell — which is exactly what the next chapter is about.

Chapter 20 — Proving It's Zsh

There's a claim zshrs makes that no amount of prose can support: *this is zsh*. Not “zsh-like,” not “mostly compatible,” not “compatible with the features we tested.” Zsh. A user's `.zshrc` is thousands of lines accreted over years, full of constructs the author no longer remembers writing, leaning on option interactions and parameter-expansion corner cases that aren't written down anywhere except the zsh C source. Powerlevel10k and the plugin ecosystem layer thousands more lines on top. For a daily driver, “compatible” has to mean something stronger than a feature checklist. It has to mean: *run the same code through both shells, and the bytes that come out are identical*.

That is the discipline this chapter describes. It has two halves — a corpus of test files inherited from zsh itself, and a differential-testing harness that pipes code through both shells and compares output byte-for-byte. The result is a single number the build checks. A number the build checks against the real shell is the only compatibility claim that can't be faked: you can write any sentence you like in a README, but you cannot make two shells emit identical bytes by writing a sentence. The bytes either match or they don't.

The `.ztst` corpus

Zsh ships its own test suite — files with a `.ztst` extension, written in a small declarative format that zsh's own `ztst.zsh` driver interprets. Each file is a series of test cases: a fragment of shell code, an expected exit status, and a description, optionally followed by patterns the output must match. Here's the shape, drawn straight from the time-keyword tests:

```
x=0; time for ((i=1; i<=10000; ++i)); do ((x+=i)); done; echo $x
0: `time` for-loop with arithmetic condition
>50005000
*?shell*
*?children*
```

The `0:` declares the expected exit status and a description. The `>` line is exact expected stdout. The `*?` lines are patterns the output must contain (here, the “shell” and “children” lines that `time` prints). zshrs adopts this corpus directly — the same files zsh uses to test itself — organized into series that mirror zsh's own layout:

- **A** — grammar, aliases, quoting, redirection, execution, assignment, control flow, and the `time` keyword (`A01grammar` through `A08time`).
- **B** — builtins: `cd`, `typeset`, `print`, `read`, `eval`, `fc`, `emulate`, `shift`, `hash`, `getopts`, `kill`, `limit`, `whence` (`B01` through `B13`).
- **C** — arithmetic, conditionals, traps, function definitions, the debugger (`C01` through `C05`).
- **D** — prompt expansion, globbing, process substitution, parameters, arrays, subscripting, multibyte handling, command substitution, brace expansion, and the no-fork optimizations (`D01` through `D10`).
- **E** — options, xtrace, POSIX-mode behavior (`E01` through `E03`).
- **V** — modules: `zmodload`, `zregexparse`, `mathfunc`, the styles module, PCRE, `zpty`, `datetime`, `zparseopts`, `zformat`, the system module, GDBM (`V01` through `V14`).
- **X** — ZLE: the line editor, `bindkey`, highlighting, incremental arguments, terminal queries (`X02` through `X06`).
- **Y** — completion: the core engine, match specs, the `_arguments` driver (`Y01` through `Y03`).
- **W** — history, jobs, job parameters (`W01` through `W03`).
- **K, P, Z** — namerefs and parameters (K), privileged mode (P), and the `is-at-least` / math-function / `run-help` utility tests (Z).

This isn't a suite somebody wrote to make zshrs look good. It's zsh's own regression suite — the tests the zsh maintainers run before every release, the tests that encode “here is exactly what this construct is supposed to do.” Passing them means passing the same bar zsh holds itself to. Look back at `C02cond.ztst` and you'll see it goes to the trouble of detecting whether the temp directory is on NFS, creating a setgid directory, chgrp-ing files to the effective gid — all so it can test the file-permission conditionals against *real* filesystem state. That's the level of fidelity the corpus demands.

The breadth is the point as much as the depth. The corpus spans seventy `.ztst` files across the grammar, builtins, arithmetic, prompts, globbing, modules, the line editor, completion, history, and namerefs — every series zsh ships, mirrored into `test_corpus/` under zsh's own filenames. That breadth is what makes the suite a *floor* rather than a sampling. A feature-list approach tests the constructs the author thought of; inheriting zsh's entire suite tests the constructs zsh's maintainers thought of over thirty years, including the ones that only exist because some user filed a bug in 2004. The series that look exotic — `V07pcrc`, `V11db_gdbm`, `X06termquery`, `K01nameref` — are exactly the ones a from-scratch test plan would skip, and exactly the ones a real `.zshrc` trips over. Adopting the upstream corpus wholesale means the test plan isn't curated by the people most likely to share the implementation's blind spots.

How a .ztst test actually fails

The format is denser than it first looks, and the density is the point — it lets a single test case pin behavior across several axes at once. A test entry has three mandatory parts and several optional ones. The leading lines are the code to run, indented by two spaces. The first unindented line is the *result line*: a status, a colon, and a description, like `0: time' for-loop with arithmetic condition`. Then come the output expectations: `>lines are exact stdout,>lines are stdout matched as a glob pattern,?lines are pattern-matched output` (the `time'` tests use these to assert “shell” and “children” lines appear without pinning the exact timing numbers). A test passes only when *all* of those agree: the right exit status, the right exact-match output, and every pattern satisfied.

That conjunction is what makes the corpus unforgiving. Consider the time-keyword case again — `time` over a 10,000-iteration arithmetic loop. It pins three independent things simultaneously: the exit status must be `0`, the loop result on stdout must be exactly `50005000`, and the timing output must contain a “shell” line and a “children” line. A naive implementation might get the loop result right but format the timing block wrong, or produce the timing block but miscount the loop. Either failure trips the entry. There's no partial credit, and there's no way to satisfy the entry without getting every axis right at once.

The corpus also exercises the *error* paths as carefully as the success paths. `A08time.ztst` includes `time builtin nonesuch $(sleep 2)` and asserts exit status 1 with the message `no such builtin: nonesuch` on stderr — testing that the `time` keyword correctly times and reports a *failed* command. It tests `time /no/such/command` and demands exit 127 with `no such file or directory`. It tests `time` under `errexit`. Each of these is a corner where a shell could plausibly diverge — get the status wrong, get the message wording wrong, time the wrong thing — and the corpus closes every one.

Differential testing, byte-exact

The corpus tells you *what* to test. Differential testing is *how* you check the answer. The principle is simple and brutal: take a fragment of shell code, run it through real zsh, run the same fragment through zshrs, and compare the output byte-for-byte. If a single byte differs, zshrs is wrong — full stop. There's no judgment call, no “close enough,” no human deciding whether the difference matters. Real zsh is the oracle; its output is the spec.

This is why so many of the fixes recorded in the gap log (`docs/GAPS.md`) delegate their *expected* value to `/bin/zsh` rather than hardcoding a string. When a fix for `cd/pwd` parity lands, the test doesn't assert a presumed-correct path — it asks real zsh what the path should be and asserts zshrs matches. The same pattern shows up across the test suite: a `zstyle -L` round-trip test feeds styles in and checks the listing

output against zsh's; a prompt-escape test renders `%5/%s` and checks they emit the real terminal capabilities zsh would emit for the live `$TERM`.

The reason delegating the oracle matters — rather than hardcoding what the author believes is correct — is that the author's belief is the thing under test. A hardcoded expected string encodes the implementer's *model* of zsh, and the whole problem is that the model has gaps; the gaps are why a compatibility project exists at all. If the test asserts against a hand-typed string, it passes exactly when the implementation matches the author's misunderstanding, which is worthless. Asking the real binary removes the author from the loop: the spec is whatever `/bin/zsh` does on this machine, today, for the live `$TERM` and the live filesystem. When zsh's behavior is environmental — terminal capabilities differ between `screen` and `xterm`, inode numbers differ between runs — a hardcoded expectation is not just wrong, it's *unwriteable*. Only the live oracle has the answer.

Byte-exactness is harder than it sounds, and the reason is subtle. Zsh works internally with *tokenized* strings — certain characters are replaced with control bytes during lexing (the parens-as-`\u{88}/\u{8a}` encoding from the previous chapter is one example). For the differential harness to compare zshrs's output against zsh's, both sides have to canonicalize through the *same* untokenization path. An early version of the parity decoder used a byte-level untokenizer that was missing several token arms — it didn't handle `Qstring`, `Qtick`, or the process-substitution markers, and it dropped null-bytes. The consequence was that command substitutions, backtick substitutions, process substitutions, double-quote escapes, and `$'...'` ANSI-C strings all *falsely* diverged: the harness reported a mismatch where the shells actually agreed, because the comparison itself was lossy. Fixing it meant routing both sides of the harness through the one faithfully-ported untokenizer, so the comparison measures a real difference rather than an artifact of the decoder.

That's the trap differential testing has to avoid: a harness bug that produces *false* failures is almost worse than no harness, because it trains you to ignore red. The fix is the same fix that keeps the magic assoc's honest — port the real thing, don't reinvent it, and make every comparison route through the same canonical path.

The number, and why it's the only honest claim

So: 43,901 of 43,904 tests passing. Zero failing. Eight ignored. Eighty-two test binaries, none with failures, last run on 2026-06-12 against real zsh on the macOS aarch64 dev box.

The three not-passing tests are the difference between 43,904 and 43,901, and each is *ignored*, not *failing* — documented as flaky for a reason that has nothing to do with zshrs's correctness:

- `read -k` on a non-tty stdin — a timing race in how character-at-a-time reads behave when stdin isn't a terminal.
- The real-world parity run — passes solo, fails only under full-sweep parallel load, which is a resource-contention artifact of the test harness, not a behavior difference.
- `[[ -N /dev/null ]]` — an atime/mtime race on the shared `/dev/null` device node while the suite hammers it in parallel across dozens of threads.

The remaining ignored tests aren't flakes at all — they're environment-gated. Three `binary_parity` cases (`zcompdump_byte_identical_roundtrip`, `zcompdump_synthesize_size_format`, `zstyle_canonical_roundtrip`) exercise the daemon RPC and need `zshrs-daemon` built first, because cargo doesn't hand `CARGO_BIN_EXE_zshrs-daemon` to the root crate's integration tests; build the daemon and they pass 4/4, no code gap. These are honestly accounted for. They're not hidden, not reclassified, not papered over with a special-cased assertion. They sit in the report with a stated cause, and the count above them is real.

There's a temptation, in any project that reports a compatibility number, to make the number go up by adjusting what counts. Reclassify a stub as "ported." Add a phrase to a doc comment that a detector keys off. Loosen an assertion so a failing test passes. zshrs's number resists this for one structural reason: it's checked by the build, against real zsh, byte-for-byte. You cannot make `43,901` go up by editing a comment, because the comparison isn't reading comments — it's running both shells and diffing their bytes. The only way to

move the number is to change zshrs's *behavior* so that more code produces zsh-identical output. The metric is downstream of the work. That's what makes it trustworthy: there's no shortcut from the number to a lie.

This is the structural difference between a differential metric and a self-reported one. A coverage percentage, a “features implemented” count, a stub tally — each is computed by a tool the project controls, and a tool the project controls is a tool the project can quietly teach to lie. A byte-diff against an external binary has no such surface: the oracle is `/bin/zsh`, which the project does not author and cannot edit. To inflate the passing count you would have to change what zsh itself outputs, which means recompiling zsh — at which point you're no longer measuring against zsh. The number is honest because the thing it compares against is outside the project's reach. That property is worth more than any single value the number happens to take, because it means every future read of the number measures the same thing it measured today.

The real-world proof: zinit + p10k

The corpus proves zshrs handles zsh's *constructs*. The real-world parity run proves it handles zsh's *world*. The acid test for any zsh replacement is whether it can load a serious interactive configuration — and the serious configuration is zinit (the plugin manager) loading Powerlevel10k (the prompt theme), with everything those drag in: completion init, the magic assoc the prompt segments query, the `zstyle` database the plugins populate, the option interactions, the parameter-expansion corner cases buried in p10k's instant-prompt machinery.

That run sits at 192 of 192. Every line of the zinit + p10k load path produces output that matches real zsh byte-for-byte. The fixes that got it there are the kind nobody writes from imagination — they fall out of running a real configuration through both shells and chasing every byte that differs. The `fzf-tab` NUL-delimiter case is representative: it hinged on a double-quoted `$'` staying literal inside a `/`-replacement (zsh only triggers ANSI-C decoding on the bare-`'` token, not the double-quoted form), and on the replacement advancing to the *last* spliced node so a later expansion pass doesn't re-scan and double-expand the inserted value. Three separate behaviors had to line up — the ANSI-C trigger, the replacement honoring its double-quote flag, and the splice cursor — for one plugin's tab completion to produce zsh's bytes. The `.zwc` wordcode case is the other shape: the compiled-bytecode dump has to be byte-identical in both directions, so a zshrs-compiled `.zwc` autoloads in real zsh 5.9.1 and a zsh-compiled one autoloads in zshrs. That was cross-verified live, both directions, with the source file deleted so only the dump could satisfy the load. A feature checklist would mark “zwc support: yes” and move on; byte-identity is the bar that catches the off-by-one in the wordcode header that a checklist never would.

What one diverging byte forces

The reason byte-exactness is worth the pain is that a single diverging byte usually isn't a single bug — it's a thread you pull on that unravels a real architectural gap. The gap log is full of these. A handful of `cd` parity tests were failing on a stale inherited `$PWD` surviving the environment import; closing them meant porting zsh's `PWD/OLDPWD` override logic and its `set_pwd_env` path faithfully, after which nine tests flipped green at once. Four prompt-escape tests were failing on `%S/%s`; the fix was a C-exact `init_term` that reads the real `so/se` terminal capabilities for the live `$TERM` — so on screen or tmux, `%S` emits the actual `\e[3m` the terminal uses. Each diverging byte pointed at a subsystem that wasn't yet faithful to the C source, and the fix was always the same shape: port the real behavior, don't approximate it.

There's a cautionary case in the log too — the kind of “fix” the project explicitly refuses. The `[ a -eq b ]` test form was once returning `0` unconditionally, which is catastrophic: it breaks every `if [ ... ]`, every `while [ ... ]`, every `[ ... ] && cmd` in every script. The cause was a compile-time heuristic that saw the `[` in the first word, decided it looked like a dynamic command name, and routed `[` away from the test builtin into the external `/usr/bin/[` — which on macOS is a BSD `test` that happened to return 0 for the malformed argument shapes being passed. The wrong fix would have been to special-case the output. The right fix was to stop misrouting `[` to the external binary. Differential testing caught it because real zsh's `[ a -eq b ]`

returns the *correct* truth value, and zshrs's returned 0 regardless — a byte difference that exposed a routing bug touching the entire conditional-test surface.

That's the discipline working as designed: the number doesn't move until the behavior is genuinely zsh-identical, and the only way to move it is to fix the shell, never the test.

That's the case for the discipline

A feature checklist is a promise. A number the build checks — against the real shell, on real configurations, byte-for-byte — is evidence. The corpus inherits zsh's own definition of correct; the differential harness measures against the real binary; the flaky tests are honestly accounted for rather than hidden; and the one path to a higher number runs through real behavior changes, not metric games. zshrs makes the claim “this is zsh” the only way the claim can be made honestly: by proving it, every build, against the thing itself.

Chapter 21 — The Worker Pool

Every shell since 1970 has had the same answer to the question “how do I do work in the background?” The answer is `fork(2)`. Run a command substitution — fork a child, let it run, read its stdout off a pipe. Run a process substitution — fork a child. Fire off a completion function that has to do real work — fork. Background a job with `&` — fork. The fork-per-task model is so deeply woven into Unix shells that the zsh C source has whole subsystems devoted to managing it: `zfork()` in the execution code, the `forklevel` counter in the init code that tracks how deep you are in nested forks, the job table in the jobs code that tracks every child.

zshrs does something different. Instead of forking a process per background task, it maintains a fixed-size pool of persistent worker threads and dispatches work to them over a channel. This is the kind of choice a compiled shell with a real runtime gets to make and an interpreter that re-reads its own source never could. This chapter is that pool — `src/extensions/worker.rs` — in depth: why threads instead of forks, how the pool is sized, how the channel provides backpressure, and how every piece of background work in the shell rides on it.

Threads, not forks

The module ports no C source, and says so in its header. There is no `Src/worker.c` in zsh, because zsh has no worker pool — its background-work strategy is `fork()`, scattered across `zfork()` and the job-control machinery in `Src/exec.c` and `Src/jobs.c`. zshrs replaces that pattern wholesale, and the rationale is concrete:

- **No fork overhead.** A `fork()` on macOS costs somewhere in the range of 50 to 500 microseconds — copy-on-write page table setup, signal-state duplication, the works. A worker thread is already running; handing it a task is a channel send.
- **No address-space duplication.** `fork()` clones the entire address space (lazily, via copy-on-write, but the bookkeeping is real). A thread shares the address space — there's nothing to clone.
- **Warm stacks.** The pool's threads are spawned once at startup and spend their lives blocked on a channel receive. When work arrives, the stack is already warm, the thread is already scheduled-eligible — no setup latency.
- **Backpressure.** A fork-per-task model has no natural throttle; fire a thousand background tasks and you get a thousand processes fighting for cores. A bounded channel gives the pool a built-in limit, which we'll come to.

None of these is a marginal saving. A `fork()` on macOS runs 50 to 500 microseconds, and a shell that forks on every command substitution, every process substitution, every backgrounded completion is paying that toll continuously — on the prompt-redraw path, where it is latency the user feels keystroke by keystroke. The worker thread has already paid its setup cost once, at startup; dispatching to it is a channel send measured in nanoseconds. The address-space point compounds it: `fork()` clones page tables even under copy-on-write, and the bookkeeping for a process that holds a large environment, a long history, and a populated completion cache is not free. A thread shares all of it — there is nothing to clone, nothing to fault back in on first write. And the backpressure point is the one a fork model structurally cannot reach: a process that forks per task has no throttle, so a runaway loop spawns a runaway number of processes, each fighting the others for the same cores. The pool's bounded queue is a ceiling the fork model never had.

The data structure is small. A `WorkerPool` owns a vector of `Worker` handles, a crossbeam channel sender, the pool size, and three shared atomics — a cancellation flag, a queued-task counter, and a completed-task counter. Each worker is a named OS thread (`zshrs-worker-0`, `zshrs-worker-1`, ...) running a loop that does exactly one thing: receive a task off the channel and run it. The whole replacement for zsh's scattered

fork-and-wait machinery is six fields and a channel — which is itself an argument. The fork-based path in the C source threads through `zfork()`, the `forklevel` depth counter, the job table, and the signal machinery that reaches across the fork boundary; the thread-pool path collapses all of that into one module because a shared address space erases most of the coordination a separate-process model has to do by hand.

Sizing: clamped to [2, 18]

How many workers? The default sizes the pool to the machine's parallelism — `std::thread::available_parallelism()` — clamped to the range `[2, 18]`. The clamp matters at both ends.

The floor of **2** guarantees there's always more than one worker, even on a single-core machine or one where `available_parallelism()` can't determine a count (in which case it falls back to 4 before clamping). A single-worker pool can deadlock itself: if a task running on the only worker submits another task and then waits for it, there's no second worker to pick the new task up. Two is the minimum that keeps the pool from starving itself in the common nested-dispatch case.

The ceiling of **18** is a deliberate cap on how wide the shell goes. There's no point spawning 64 worker threads on a 64-core box for shell work — the tasks are I/O-bound (reading files, spawning external commands, parsing completion functions), not CPU-bound number-crunching, so past a certain width you're just adding scheduler overhead and context-switch churn. The marginal worker on a wide box doesn't find more parallel I/O to do; it finds more contention for the same disk and the same external processes, while costing a stack, a scheduler slot, and the cache pressure of one more runnable thread. Eighteen is the chosen ceiling for the auto-sized default: wide enough to cover the burst of parallel work a `compinit` or a recursive glob throws off, narrow enough that the pool never becomes the bottleneck it was meant to remove.

There's a second sizing path for explicit configuration. The shell's config file can set `[worker_pool] size = N`; when it does, `resolve_pool_size` honors the explicit value clamped to `[1, 64]` rather than the auto range. A `size` of **0** means “auto,” which falls back to the `available_parallelism().clamp(2, 18)` path. The two ranges have different floors and different ceilings, and both differences are intentional. The auto floor is 2 because the auto path can't know whether the workload will nest dispatches, so it has to be deadlock-safe by default; an explicit `size = 1` is allowed because a user who types it is asserting they know their workload is flat. The explicit ceiling is 64 against the auto ceiling of 18 because an explicit number is a deliberate measurement, not a guess `available_parallelism()` made — if someone has profiled their config and wants 32 workers, the config honors it, while the automatic path stays conservative for the machine it can't see. The clamp at 64 still exists even for the explicit path, because an unbounded thread count is never the right answer for a shell; the cap is just higher when a human chose the number than when the runtime did.

The channel and bounded backpressure

The dispatch mechanism is a crossbeam-channel, and the choice of crossbeam over the standard library's `mpsc` is recorded as an audit fix. The naive way to share a receiver across multiple worker threads is `Arc<Mutex<mpsc::Receiver>>` — every worker locks the mutex, pulls a task, unlocks. That mutex is a contention point: with eighteen workers all trying to grab the next task, they serialize on the lock, and the serialization gets worse exactly as the pool gets wider — the more workers you add to go faster, the more they queue behind each other on the one lock guarding the receiver. crossbeam-channel is natively multi-consumer — each worker calls `recv()` directly on its own clone of the receiver, no shared mutex, no contention. The receiver clones are independent; the channel arbitrates internally without a single lock the workers serialize on. This is the kind of bug that doesn't show on a two-core dev box and shows badly on the eighteen-worker machine the pool was sized for, which is why it's logged as an audit fix rather than a design note: it was a latent scaling defect in the obvious implementation.

The channel is **bounded**, with a capacity of four times the pool size. This is the backpressure mechanism, and it's the thing fork-per-task can't do. When you submit a task, the queued counter increments and the task goes into the channel. If the channel is full — four times the pool size worth of tasks already queued — the `send()`

blocks until a worker drains one. That blocking is the feature. It means a runaway producer can't queue an unbounded backlog of work; it gets throttled at the channel boundary, naturally pacing itself to the rate the workers can drain.

The 4× multiplier is a specific tradeoff, not a round number. A capacity of zero (a rendezvous channel) would force every submit to block until a worker was idle, serializing the producer against the slowest worker and erasing the point of having a queue. An unbounded channel would restore the exact failure the pool exists to prevent — a runaway producer queuing a million closures, each holding whatever it captured, until memory runs out. Four times the pool size gives each worker roughly four tasks of runway: enough that a brief stall on one worker doesn't immediately stall the producer, small enough that the backlog can't grow without bound. The producer is allowed to run ahead by a bounded margin and no further. The pool also logs a debug line when the queue depth climbs past twice the pool size — half the channel's capacity — so a building backlog is visible in the trace before it becomes a block, without ever being fatal or surfacing to the user.

Three more properties of the worker loop are worth naming, each an audit fix baked into the design:

- **Panics don't kill workers.** Every task runs inside a `catch_unwind`. If a task panics — a malformed completion function, a bug in a background handler — the panic is caught, logged with the worker id and the panic message, and the worker loops back to receive the next task. A panicking task degrades to a logged error; it never takes a worker thread down with it. There's a test for exactly this: submit a task that panics, then submit ten more, and assert all ten still run.
- **Cancellation is a flag, not a signal.** The pool has a shared `AtomicBool` cancellation flag. Set it, and workers check it before running each task — if it's set, they *drain* the queued tasks without executing them. In the fork model, cancelling background work means a signal storm: `killpg()` across the process group, signals crossing the fork boundary. Here it's a single atomic store, and already-running tasks finish naturally while queued ones get skipped.
- **Shutdown doesn't block on exit.** When the pool drops — at shell exit, on Ctrl-D — it sets the cancellation flag, drops the sender (which closes the channel, so each worker's `recv()` returns an error and the loop exits), and then *detaches* the worker threads rather than joining them. Joining would block: if a worker is mid-parse on a five-hundred-line completion function, `join()` waits for it to finish. On exit that wait is pointless — the OS reclaims the threads when the process dies — so the pool detaches and lets exit be instant. (This Drop path also carries a hard-won lesson about logging: the shutdown announcement was demoted from `info!` to `debug!` after it was found leaking to stdout when a script left a duped fd open, because by the time worker-drop runs the file-backed log writer is closed and tracing's fallback writes to fd 1.)

What rides on the pool

The pool isn't a curiosity off to the side — it's the substrate the shell's in-process background work runs on. Several subsystems submit to the same pool.

Background command execution. When the shell needs to run a batch of commands off the main thread, it ships them all to the pool via `submit_with_result`, collecting a receiver per command. Each task spawns the command and returns its status over a typed channel; the caller gathers the receivers. This is the direct analog of the fork-and-collect pattern, except the collection is a typed Rust channel rather than a pipe you have to parse.

Completion initialization. `compinit` — the completion system's startup, which scans every directory in `$fpath` for completion functions and compiles them — is heavy work, and zshrs ships it to the pool so it doesn't block the prompt. The `compinit-background` path submits the `fpath` scan and the bytecode-blob backfill as pool tasks, logging `compinit: shipping to worker pool` as it goes. This is where the pool pays for itself most visibly: the user gets a prompt immediately while completion compilation happens on worker threads, with no instant-prompt fakery — the work is genuinely running in parallel, not deferred and pretended-away.

Async background jobs. The shell's `async/await`-style background builtin ships its closure to the pool with `submit`, running the command in an `sh -c` subprocess and delivering its `(status, stdout)` back over a

one-shot crossbeam channel. The work that's genuinely thread-shaped — collecting a command's status off the main thread without blocking the prompt — rides one fixed set of warm threads and one bounded queue.

Not everything moves to the pool, and the chapter is precise about where it doesn't. Process substitution — `<( ... )` and `>( ... )` — still forks. `zshrs` ports `getproc` from `Src/exec.c` faithfully: pipe plus fork plus a `/dev/fd/N` path entry, where `N` is the held-open end of the pipe. This is a deliberate non-port of the thread model, and the reason is the observable contract. `<(cmd)` has to produce a real filesystem path that a *separate* program can `open()` — the consumer is often an external tool (`diff <(a) <(b)`), and an external tool can only be handed a path, not a Rust channel. The path has to resolve to a non-seekable pipe the consumer reads the command's stdout from. A worker thread can't manufacture a `/dev/fd/N` entry that an unrelated process opens; only a held-open file descriptor in a forked child does that. So the rule isn't “threads everywhere” — it's “threads for in-process background work, forks where the contract is a file descriptor another process must see.” The pool gives the shell a second concurrency primitive; it doesn't abolish the first.

Two ways to submit

There are two submit paths, and the difference between them is the difference between fire-and-forget and collect-the-result. The plain `submit` takes a closure that returns nothing — you hand the pool a unit of work and walk away. That's what `compinit` uses: the `fpath` scan doesn't return a value to the caller, it writes its results into the completion cache as a side effect, so there's nothing to wait on.

The second path, `submit_with_result`, is the one that replaces fork-and-read-the-pipe. You hand it a closure that *returns* a value, and you get back a typed receiver. Internally it wraps your closure: run it, send the result down a one-shot channel, and the receiver you hold delivers it when it's ready. The module's own comment names the C analog precisely — the closest thing in `zsh` is the pipe-based command-substitution capture, where `getoutput()` reads a child's stdout pipe and the parent has to parse the bytes back into a value. The typed channel sidesteps the marshalling entirely: a Rust value goes in one end and comes out the other, no serialization to text and back. This is the path the background command-execution code uses — ship every command to the pool, collect a `Vec` of receivers, then drain them.

Observability without chatter

The pool carries metrics, and the way it reports them follows the project's hard rule that informational chatter goes to the log, never to the terminal. Three atomics track the pool's state: `queued` (incremented on submit, decremented when a worker pulls a task), `completed` (incremented after each task finishes), and the cancellation flag. The `queue_depth()` and `completed()` accessors expose them for introspection. A `tracing` span fires on submit, and the worker loop logs at debug level — so a developer running with the debug filter can watch tasks flow through, while a normal interactive session sees nothing. When the queue depth climbs past twice the pool size, that's a debug line too: visible if you're looking, silent if you're not.

This is deliberate, and it's the same discipline that runs through the rest of the shell. A worker pool is exactly the kind of subsystem that wants to announce itself — “pool started with 12 workers,” “shutting down, 4,832 tasks completed.” `zshrs` sends all of it to `~/.cache/zshrs/zshrs.log` via `tracing`, never to stdout. The startup line is `info!` (it lands in the log); the shutdown line was demoted to `debug!` precisely because it once leaked to a script's stdout through a duped fd. The pool does heavy work and says nothing about it where the user can see — which is the whole point of a host that does what you typed and nothing more.

The shape of the bet

The worker pool is a bet about what a shell's background work actually is. The bet is that it's mostly I/O — reading files, spawning externals, parsing functions — bounded by syscalls and disk, not by CPU. For that workload, a small pool of persistent threads with a bounded queue beats fork-per-task on every axis: lower per-task latency (a channel send versus a 50-500µs fork), bounded resource use (a capped queue versus

unbounded process sprawl), graceful failure (a caught panic versus a crashed child), and instant shutdown (detach versus join or signal storm).

It's also a concrete illustration of what a compiled runtime buys. An interpreted shell that re-reads and re-evaluates its own source has fork as the only concurrency primitive available to it — there is no persistent runtime to host a thread pool, because the program's model of itself is “read a line, evaluate it.” A compiled shell with a real runtime gets to choose, and for in-process background work zshrs chose a thread pool: sized to the machine, bounded for backpressure, in one small module of six fields and a channel. It doesn't abolish `fork()` — process substitution and external execution still fork, ported faithfully from the C source — but it gives the shell a second primitive that a fork-only interpreter has no place to put. That second primitive is what lets `compinit` run on warm threads while the prompt is already live, with no instant-prompt fakery papering over deferred work: the work is genuinely in flight, on threads that were spawned once and never go away.

Chapter 22 — Parallel and Async Verbs

Every shell ever shipped has known how to do two things in parallel. You background a job with `&`, you collect it with `wait`, and somewhere in the middle you hope the bookkeeping works out. This has been the deal since the Bourne shell, and it has aged about as well as you'd expect. The `&/wait` model is a job-control primitive wearing a parallelism costume — it forks a whole process, hands you a PID, and leaves the orchestration to you. Want ordered output from a hundred parallel compressions? Write the loop, capture the PIDs, sort the results yourself. Want to fire ten thousand conversions and process each one the instant it finishes rather than in submission order? That's not a feature, that's a project.

`zshrs` takes a different position. Parallelism is a first-class surface — a small set of verbs the shell understands natively and runs across a managed pool. The two families divide along the pool they use. `async/await` queue onto a persistent thread pool (`WorkerPool`, sized to `available_parallelism()` clamped to `[2, 18]` — `src/extensions/worker.rs:17`); `pmap/pgrep/peach` fan their items across rayon's work-stealing pool (`src/extensions/ext_builtins.rs:1239`). Either way the verb is the orchestration: you get ordering, filtering, and join semantics as single words instead of PID bookkeeping, temp files, and a sort.

The distinction worth holding onto from the first line is that *the verbs are an orchestration layer, not a different process model*. Each item a parallel verb processes still runs in its own subprocess — the same fork a backgrounded command pays. What the verb buys you is not the elimination of the fork; it's everything *around* the fork: a bounded pool so ten thousand items don't stampede the kernel, a result that comes back in glob order, a filter that keeps only the inputs whose predicate passed, a join that fires once when every task is done. `zsh` has had the raw fork since 1970 and nothing else. `zshrs` wraps that fork in the discipline you'd otherwise hand-roll every time. Keeping that boundary honest — fork underneath, orchestration on top — is the through-line of this chapter, because the alternative framing (fork-free in-VM execution) is something the README once described and the shipped code never did.

This chapter walks through that surface: `async/await`, `pmap`, `pgrep`, `peach`, and `barrier`. Each one is a single word with a clear contract, and they compose with the rest of the shell exactly as you'd hope.

The shape of the thing

Here is the entire concurrent surface, lifted verbatim from the README:

```
# Async/await
id=$(async 'sleep 5; curl https://api.example.com')
result=$(await $id)

# Parallel map - ordered output
pmap 'gzip {}' *.log

# Parallel filter
pgrep 'grep -q TODO {}' **/*.rs

# Parallel for-each - unordered, fire as completed
peach 'convert {} {}.png' *.svg

# Barrier - run all, wait for all
barrier 'npm test' ::: 'cargo test' ::: 'pytest'
```

Five verbs. No flags to memorize, no `xargs -P` invocation to get wrong, no GNU `parallel` to install and learn. The difference between these and the shell scripts you'd write to approximate them is that the verb *is* the orchestration — the pool, the ordering, the join are built in. It's worth being clear about what that comparison is and isn't. `xargs -P` and GNU `parallel` are real, capable tools, and they also fork a process per item; on raw throughput they're in the same class. What they aren't is *part of the shell*. They're external commands with their own argument grammars, their own quoting rules, their own surprises around how `{}` interacts with the surrounding shell. You learn `parallel`'s `:::`, `:::+`, `--colsep`, `--results` dialect, and you keep a tab of its tutorial open. The `zshrs` verbs are builtins that consume a list the glob already expanded and produce a list the next command consumes — they live inside the shell's own evaluation, not beside it as a subprocess you pipe into. That's the trade these five make: not faster than `parallel`, but native, composable, and with a contract small enough to hold in your head. Let's take them one at a time.

async and await — futures for the shell

The classic background-job pattern looks like this:

```
sleep 5; curl https://api.example.com &
# ... do other work ...
wait
```

The problem is that `wait` is a blunt instrument. It waits for a job, or for *all* jobs, identified by PID or job spec, and the result — the actual stdout you wanted — went wherever the job's stdout went, which is to say your terminal, interleaved with everything else. You get the side effects but not a value.

`async` gives you a value. It takes a command, dispatches it to the worker pool, and immediately hands back an `id` — a handle to a future:

```
id=$(async 'sleep 5; curl https://api.example.com')
```

That line returns instantly. The `sleep 5; curl ...` is now running on a pool thread while your prompt comes right back. The `id` is a token you hold onto. When you actually need the answer, you redeem it:

```
result=$(await $id)
```

`await` blocks until that specific future completes and then evaluates to its captured output. The handle is the whole point: it's a *value you can pass around*. You can launch three `async` tasks, hold three `ids`, do some synchronous work, and then await them in whatever order makes sense:

```

a=$(async 'curl https://api.one.com/data')
b=$(async 'curl https://api.two.com/data')
c=$(async 'curl https://api.three.com/data')

# all three HTTP fetches are now in flight on the pool
process_local_stuff

merge "${await $a}" "${await $b}" "${await $c}"

```

Three network round-trips overlap completely, you do useful local work while they're outstanding, and you collect each result by name. The `&wait` equivalent of this — three backgrounded curls writing to three temp files, a `wait`, then three reads — is exactly the kind of bookkeeping `async/await` exists to delete.

What's underneath is concrete and worth stating, because it's the difference between the `pmap` family and this one. `async` doesn't go to rayon; it goes to the `WorkerPool` (`src/extensions/ext_builtins.rs:1134`). The submitted closure runs `Command::new("sh").args(["-c", &code])` with stdout piped and stderr inherited, and ships the captured stdout plus exit code back over a one-slot crossbeam channel; the `id` you hold is the key to that channel, and `await` is a blocking `recv()` on it (`src/extensions/ext_builtins.rs:1135-1136`). Two consequences fall out of that. First, the body of an `async` runs under `/bin/sh`, not under `zshrs` — unlike the `pmap` family, which re-invokes the `zshrs` binary on itself, the `async` path shells out to plain `sh -c`, so `zsh`-specific builtins inside an `async` body don't survive. Second, the pool is the bounded `WorkerPool`, sized to `available_parallelism()` clamped to `[2, 18]` (`src/extensions/worker.rs:17`), with a crossbeam channel sized to four times the pool width for backpressure. The reason that pool exists at all is documented in the module header: C `zsh`'s strategy for every piece of background work is `fork(2)` — a fresh child for each completion run or command substitution, paying 50–500µs of fork overhead and a full address-space duplication each time (`src/extensions/worker.rs:1-18`). The `WorkerPool` replaces that with warm threads that are already stacked and waiting, so dispatch is a channel send rather than a `fork`. Note the nuance, though: the `dispatch` avoids a fork, but the work the worker thread does — running `sh -c` — forks anyway. The pool removes the per-task fork of the *orchestration*, not the fork of the command itself.

This is the future/promise pattern every modern runtime has converged on — JavaScript's `Promise`, Rust's `Future`, Python's `asyncio.Task` — and the shape is identical here: launch returns a handle, the handle redeems to a value, the work happens out of line between the two. The difference from those runtimes is that they multiplex coroutines onto a few OS threads via an event loop, whereas `zshrs` runs each task on a real pool thread that does a blocking subprocess. For shell work — where every task is “go run a command and wait on its I/O” — a thread-per-task pool is the right shape: there's no CPU-bound coroutine to schedule, just a process to wait on, and an OS thread blocked in `recv()` costs almost nothing.

pmap — parallel map with ordered output

`pmap` is the workhorse. It takes a command template containing a `{}` placeholder and a list of inputs, runs the template once per input across the pool, and — crucially — emits the results *in input order*:

```

pmap 'gzip {}' *.log

```

Every `.log` file in the directory gets gzipped, all of them in parallel up to the pool's width, and the output you see comes back in the same order the glob produced. That ordering guarantee is the whole point of `pmap` as distinct from `peach`. When you're building a result that downstream code will index into — or just when you want deterministic, reproducible output — order matters, and `pmap` preserves it even though the actual execution finished in whatever order the scheduler happened to drain the queue.

The `{}` placeholder is the same convention GNU `parallel` and `xargs -I{}` taught everyone, so there's nothing new to learn. What happens underneath is concrete and worth being precise about, because the README's high-level description and the shipped implementation are not the same thing. The implementation in `src/extensions/ext_builtins.rs:1211` substitutes `{}` once — the template `gzip {}` becomes `gzip ${=__ZSHRS_P_ARG__}`, where the leading `=` flag forces word-splitting on `$IFS` so an item like `a`

`b` still expands to two arguments. Then it resolves the running binary with `current_exe()` and fans the items across rayon's work-stealing pool: each item runs `Command::new(exe).args(["--zsh", "-c", ¶metrised]).env("__ZSHRS_P_ARG__", item)`. So every item *does* spawn a subprocess — a fresh `zshrs --zsh -c` invocation with the item passed in via the `__ZSHRS_P_ARG__` environment variable. Running `zshrs` itself rather than `/bin/sh` is deliberate: it preserves `zshrs`-specific builtins (`echo` flags, `print -P`, and the rest) inside the parallel body.

`par_iter().collect()` preserves input order in the result `Vec`, which is how `pmap`'s ordered-output guarantee holds across a parallel run — the items finish in scheduler order but the results are stitched back in glob order. The README also carries an aspirational description of a fork-free, compile-once `VMPool` path (`{f=__zshrs_p_arg__}, fusevm::VMPool, setparam/unsetparam, "no fork"`). That is not what `builtin_pmap` does today: there is no `VMPool` in `src/`, no per-item `setparam`, and each item is a `Command::new(exe)` subprocess. Treat the parallel verbs as a clean, ordered, pool-scheduled wrapper around per-item subprocess execution — not as in-VM bytecode dispatch.

What you still get over a hand-rolled `for f in *.log; do gzip $f & done` loop is real: the ordering, the bounded pool width, the filter/join semantics, and a parallel body that runs through `zshrs` rather than `/bin/sh`. The cost model is a forked subprocess per item, same as a backgrounded loop — the value is in the orchestration the verb does for you.

pgrep — parallel filter

`pgrep` keeps the inputs for which the body succeeds:

```
pgrep 'grep -q TODO {}' **/*.rs
```

The recursive glob `**/*.rs` produces every Rust file in the tree. For each one, the body `grep -q TODO {}` runs on the pool; `grep -q` is silent and just sets an exit status, so what `pgrep` collects is the *list of files where the body exited zero* — every file that contains a `TODO`. It's a parallel *filter*: the predicate is “did this command succeed,” and the output is the surviving inputs.

This is the verb that replaces the `for f in **/*.rs; do grep -q TODO $f && echo $f; done` pattern, and it replaces it with something that fans the predicate across the whole pool at once. On a large source tree this is the difference between a sequential walk and saturating every core you have. The body runs through the same subprocess-per-item path as `pmap` (`builtin_pgrep`, `src/extensions/ext_builtins.rs:1288`): `par_iter()` over the items, each one a `Command::new(exe --zsh -c)` whose exit status decides keep-or-drop, with `.collect()` preserving input order so the surviving files print in glob order.

peach — parallel for-each, unordered

`peach` is for the case where you don't care about order and you'd rather see results the instant they're ready:

```
peach 'convert {} {} .png' *.svg
```

Every `.svg` gets converted to a `.png`, and `peach` fires each result as *it completes* rather than holding everything back to reassemble input order. The README's own framing — “unordered, fire as completed” — is the contract. If one conversion finishes in 3ms and another takes 300ms, you see the fast one's output immediately; `peach` doesn't make the quick jobs wait on the slow ones.

This is genuinely different behavior from `pmap`, not a cosmetic variant. `pmap` buffers each subprocess's captured stdout and replays it in input order; `peach` inherits the parent's stdout/stderr so each subprocess streams straight to the tty as it runs (`builtin_peach`, `src/extensions/ext_builtins.rs:1341`, uses `par_iter().for_each` with no output capture). When you're processing thousands of independent items and you want the throughput of streaming completion — a progress feel, results landing continuously — `peach` is the verb. When you need a deterministic ordered result for something downstream, you reach for `pmap` and accept the buffering. Same rayon pool, same subprocess-per-item model, different output discipline.

Note the double `{}` in the example: `convert {} {}.png`. Both placeholders expand to the same input item, so `frame.svg` becomes `convert frame.svg frame.svg.png`. The placeholder is a substitution, not a positional counter — it stands for “the current item” everywhere it appears in the template.

barrier — run all, wait for all

The first four verbs map *one template over many inputs*. `barrier` is the other axis: *several different commands, all at once, joined at the end*:

```
barrier 'npm test' ::: 'cargo test' ::: 'pytest'
```

The `:::` separator (again, borrowed from GNU `parallel`'s vocabulary so it's familiar) splits the line into distinct commands. `barrier` ships each one to the `WorkerPool` via `submit_with_result`, where the worker runs `Command::new("sh").args(["-c", &cmd])` with stdout and stderr inherited so each suite streams straight to your terminal, and collects a result channel per command (`src/extensions/ext_builtins.rs:1430-1435`). It then blocks on every receiver in turn and returns the *worst* (highest) exit status across all of them — so a single failing suite surfaces as a nonzero `barrier` even if the other two passed. It's a fan-out/fan-in: three different test suites — a Node suite, a Rust suite, a Python suite — kicked off together, waited on as a group, with one aggregate status.

This is the natural primitive for a polyglot repository's CI-on-your-laptop moment. Without `barrier`, you either run the three suites serially (and pay the sum of their runtimes) or you background all three and write the `wait` plumbing by hand. `barrier` is that plumbing as one word: the wall-clock time is the *slowest* suite, not the sum, and you get a single synchronization point where all three have completed before the next line runs. The aggregate-worst-status rule is what makes it usable in a script — `barrier ... && deploy` does the right thing, refusing to proceed if any suite failed, without you inspecting three separate PIDs.

The mental model is a barrier in the concurrency sense — a synchronization point that no thread passes until all threads have arrived. Everything before the `barrier` line can have launched work; the `barrier` itself is the join. Note that `barrier`, like `async`, runs its commands under `sh -c` rather than re-invoking `zshrs` — the `zshrs-in-the-loop` guarantee is specific to the `pmap/pgrep/peach` family, which exists to map one template over a list and so cares about preserving `zshrs` builtins inside the body; `barrier` runs a handful of distinct, fully-formed command lines where that concern doesn't apply.

How they compose

These verbs aren't islands. Because each one is an ordinary shell builtin that consumes a list and (mostly) produces a list, they slot into the rest of the shell — pipes, command substitution, globs, and each other.

The most immediate composition is with globbing, which you've already seen: `*.log`, `**/*.rs`, `*.svg` are all just `zsh` globs, expanded by the shell before the verb ever sees them. Everything `zsh` globbing can do — qualifiers, recursive `**`, exclusion — produces the input list for a parallel verb for free. `pmap 'optimize {}' **/*.png(.Lm+1)` feeds `pmap` exactly the PNG files larger than one megabyte, because the glob qualifier did the filtering and the verb did the parallelism.

Command substitution composes the other direction. Because `pgrep` writes its surviving inputs to stdout, you can capture them:

```
todo_files=$(pgrep 'grep -q TODO {}' **/*.rs)
pmap 'rustfmt {}' $todo_files
```

The first line fans a predicate across the tree and collects the matches; the second line fans a formatter across just those matches. Two parallel passes, the output of one feeding the input of the next, no temp files.

`async/await` compose with the map-style verbs by wrapping them. A `pmap` is itself a unit of work, so you can launch one asynchronously and await it later:

```

job=$(async 'pmap "gzip {}" *.log')
# ... your prompt is free; the whole parallel compression runs out of line ...
await $job

```

Now you have *nested* parallelism: an `async` future whose body is itself a fan-out. Note that the two layers use different pools — `async` queues onto the `[2, 18]`-clamped `WorkerPool` (`src/extensions/worker.rs`), while the inner `pmap` fans across rayon’s work-stealing pool — so the outer future occupies one worker thread that then spawns a rayon fan-out of subprocesses underneath it. It composes, but it isn’t a single shared pool bounding the whole thing.

And `barrier` composes with the rest by being the join for heterogeneous launches. You can `async` three unrelated things, then use the map verbs for the homogeneous bulk, then `barrier` the final independent commands — mixing the “one handle per task” style and the “one template per list” style in the same script.

Why this is a surface, not just `&/wait`

The case for the verbs is about orchestration, not about avoiding the fork. Three properties carry it, and each one is plumbing you’d otherwise write by hand every time.

First, **the pool is bounded and managed for you**. A `for f in *.log; do gzip $f & done` loop forks one background process *per item* with no width cap — ten thousand items means ten thousand simultaneous backgrounded processes fighting the scheduler, thrashing memory, and quite possibly hitting your `ulimit -u`. The parallel verbs run through rayon’s work-stealing pool (`pmap/pgrep/peach`) or the `[2, 18]`-clamped `WorkerPool` (`async/await`), so concurrency is bounded to a sane width and items queue against it rather than stampeding the kernel. Width-capping is the single most common thing people reach for `xargs -P` or `parallel -j` to get; here it’s the default, not a flag.

Second, **ordering and filtering are built in**. `pmap` stitches captured stdout back into glob order via `par_iter().collect()`; `pgrep` collects the pass/fail flags and prints only the survivors in order; `peach` streams unordered as items complete. Getting any of these from `&/wait` by hand means temp files named by index, PID-to-index bookkeeping, and a final sort — the exact plumbing the verbs delete. The fact that `pmap` and `peach` are *different verbs* rather than a `--ordered` flag on one is deliberate: the ordering discipline changes the whole execution model (buffer-and-replay vs stream-as-completed), and making it a verb choice rather than a flag means you pick the model you want at the call site instead of forgetting a flag and getting interleaved garbage.

Third, **the map-family body runs through zshrs, not `/bin/sh`**. Each `pmap/pgrep/peach` item spawns `Command::new(exe).args(["--zsh", "-c", ...])` — the running zshrs binary re-invoked on itself — so zshrs-specific builtins and behavior survive inside the parallel body. A naive parallel loop that shelled out to `sh -c` would lose them. (The `async/barrier` family does run `sh -c`; the zshrs-in-the-loop guarantee is specific to the template-over-list verbs, which is exactly where it matters.)

What the verbs do *not* do is avoid the per-item fork: each item is a subprocess, same as a backgrounded command. There is no `VMPool`, no per-item bytecode dispatch, no compile-once-run-many path in the shipped code — `grep -rn "VMPool" src/` returns nothing, and every parallel verb resolves to a `Command::new(...)`. The value the verbs deliver is the orchestration — bounded pool, ordered/filtered output, zshrs-in-the-loop for the map family, aggregate status for `barrier` — packaged as single words. That’s a smaller claim than “fork-free parallelism,” and it’s the true one.

Chapter 23 — Fish Inside

There is a particular ritual every new zsh user eventually performs. You discover that the shell, out of the box, gives you none of the three things that make a modern command line feel alive — your commands aren't colored as you type, there's no ghost-text suggestion completing your line from history, and there's no way to type `gco` and have it become `git checkout`. So you go find the plugins. You install `zsh-syntax-highlighting`, you install `zsh-autosuggestions`, you install `zsh-abbr`, you wire them into your `.zshrc` in the correct order (highlighting must load last, a fact you learn the hard way), and you accept that every keystroke now runs through three layers of zsh script reimplementing what a shell arguably should have done itself.

fish has done all three natively for years. That was always fish's pitch: the things you bolt onto zsh, fish ships in the box. `zshrs` takes fish at its word — literally. The interactivity layer in `src/extensions/fish_features.rs` is adapted from fish-shell's own Rust codebase, ported to run native inside `zshrs`. The module header is explicit about both the lineage and the reasoning:

```
C zsh has no built-in syntax highlighting, autosuggestion, or abbreviation support; users layer these via plugins (zsh-syntax-highlighting / zsh-autosuggestions / zsh-abbr) which reimplement everything as zsh script. zshrs ships them as native Rust to skip the interpreter overhead.
```

And on why fish specifically:

```
Code adapted from fish-shell's Rust codebase. The fish-shell origin gives us a structurally-validated implementation; the zshrs port keeps the same shape so tests / fuzzers from upstream still apply.
```

That second point is the engineering argument. fish rewrote itself in Rust; that Rust is fuzzed and shipped to a large user base. Porting it shape-for-shape means `zshrs` inherits a validated implementation and can keep tracking upstream's correctness fixes, rather than hand-rolling a third reimplementations of a problem fish already solved. The alternative — writing a fresh highlighter from a blank file — would mean re-discovering every edge case fish already found: nested quoting, escapes inside double quotes, the command-position vs argument-position distinction, the dozen ways a `$` can or can't start a variable. Keeping fish's shape sidesteps that entire bug surface.

There's a second axis to the argument that's specific to the shell-as-host. The zsh plugins that supply these features — `zsh-syntax-highlighting`, `zsh-autosuggestions`, `zsh-abbr` — are bodies of zsh script that re-execute through the interpreter on *every keystroke and every prompt redraw*. The community has spent years optimizing around that cost: async highlighting buffers, debounced redraws, lazy loading via `zinit turbo`, `zcompiled` plugin files. Every one of those optimizations is evidence that the underlying approach — interactivity reimplemented as interpreted script in the keystroke path — is the wrong place to put it. Moving the three features into native Rust inside the binary removes the interpreter from the hot loop entirely, which is the only fix that doesn't need a layer of workarounds stacked on top of it. Let's go through the three features, grounded in what the code actually does.

Syntax highlighting — nineteen roles

The highlighter classifies every token in your command line into one of nineteen roles. The `HighlightRole` enum, adapted from fish's `parse_constants::HighlightRole`, enumerates them: `Normal`, `Command`, `Keyword`, `Statement`, `Param`, `Option`, `Comment`, `Error`, `String`, `Escape`, `Operator`, `Redirection`, `Path`, `PathValid`, `Autosuggestion`, `Selection`, `Search`, `Variable`, and `Quote`. Each maps to an ANSI sequence in `role_to_ansi` — `Command` is bold green, `Keyword` is bold blue, `Error` is bold red, `String` is yellow, `Variable` is bold cyan, `Comment` is gray, and so on.

The entry point is `highlight_shell(line)`, which lexes the line character by character and returns a `Vec<HighlightSpec>` — one spec per byte of the input. The header is clear that this goes beyond what zsh does even with its own machinery:

C zsh's highlighting (`zle_highlight` special parameter, `Src/Zle/zle_refresh.c`) only colors match-positions; full token-classification highlighting is a fish-shell innovation we port here.

The classifier walks a small state machine. It tracks whether it's inside a string and which quote opened it, whether it's inside a comment, where the current word started, whether this is the first word of a command, and whether the previous token was a pipe or semicolon. A `#` outside a string flips comment mode and everything after it colors as `Comment`. A `"` or `'` opens a string; inside a double-quoted string, a `\` escape colors as `Escape`, a `$` colors as `Variable`, and everything else colors as `String`, with the closing quote colored as `Quote`. A bare `$` outside a string starts variable highlighting and consumes the following alphanumerics, underscores, and braces as part of the `Variable` token. The operators `|`, `&`, and `;` color as `Operator` — and, importantly, they reset the “first word” flag, because the token after a pipe is a new command position. Redirections `>` and `<`, including the doubled `>>` and `<<`, color as `Redirection`.

The interesting decision is in `colorize_word`, which decides how a complete word colors based on whether it's in *command position*:

```
let role = if is_command_position {
  if KEYWORDS.contains(&word) {
    HighlightRole::Keyword
  } else if BUILTINS.contains(&word)
    || command_exists(word)
    || (word.contains('/') && std::path::Path::new(word).exists())
  {
    HighlightRole::Command
  } else {
    HighlightRole::Error
  }
} else if word.starts_with('-') {
  HighlightRole::Option
} else if std::path::Path::new(word).exists() {
  HighlightRole::PathValid
} else {
  HighlightRole::Param
};
```

This is the feature that makes the highlighter useful rather than decorative. In command position, a word is checked against the keyword list, then the builtin list, then `command_exists` (`src/extensions/fish_features.rs:358`, which walks `PATH` looking for an executable file), then a path existence check. If none of those match — if you've typed a command that doesn't exist — the word colors `Error`, bold red, *before you hit enter*. The command name turns red the moment you've typed something that won't run, so typos surface at the keystroke instead of as a `command not found` after you commit to it. This is the behavior fish made standard, and it changes the feel of the prompt: the line is a live correctness check, not just decorated text. Out of command position, a `--`-prefixed word is an `Option` (cyan), a word that resolves to an existing path is `PathValid` (underlined green), and everything else is a plain `Param`.

The cost of doing this per-keystroke is the reason it belongs in native code. The check involves a `PATH` walk and a filesystem `exists()` for every word at command position, on every redraw. In an interpreted plugin, that's a `stat` storm filtered through the zsh interpreter's variable lookups and string ops, which is precisely why zsh-syntax-highlighting carries its own caching and why slow prompts are a perennial complaint. In compiled Rust the same walk is a tight loop over `PATH` entries with direct syscalls — fast enough that the highlighter doesn't need a debounce or an async buffer to stay out of the way of your typing.

The keyword and builtin lists are concrete. `KEYWORDS` covers the shell control words — `if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `do`, `done`, `function`, and the zsh-flavored `foreach/end/repeat/always`. `BUILTINS` is a long list of the usual suspects plus the zsh-specific ones

(`autoload`, `zmodload`, `zstyle`, `zle`, `bindkey`, `setopt`, `emulate`, `whence`) and even the project's own `ztest` assertion builtins, listed explicitly so they color as `Command` rather than `Error` at command position. A comment in the source notes exactly why that hardcoded list exists — `highlight_shell` predates the unified `EXT_BUILTIN_NAMES` table that the LSP uses — which is the kind of honest provenance note that tells you this is real ported code with a history, not a sketch.

Finally, `colorize_line(line, colors)` turns the per-byte spec vector into an actual ANSI-colored string, emitting a color escape only when the role changes from the previous character (so you don't get a reset-and-set sequence on every single byte) and a final reset at the end if the line didn't end in `Normal`.

Abbreviations — position-aware expansion

Abbreviations are the feature that lets you type a short key and have it expand, in place, into a longer command — the thing zsh users install `zsh-abbr` for. fish's model, ported here, is richer than a plain alias because expansion is *position-aware*. The `AbbrPosition` enum has two variants: `Command`, meaning the abbreviation only fires when the key is in command position, and `Anywhere`, meaning it fires regardless of where in the line the key appears. The header notes the gap it fills: zsh only has `setopt EXPANDABBREV` via the `zsh-abbr` plugin, not natively.

An `Abbreviation` carries a `name`, a `key` (the token you type), a `replacement` (what it expands to), and a `position`. The `matches` method enforces the position rule directly:

```
pub fn matches(&self, token: &str, is_command_position: bool) -> bool {
    let position_ok = match self.position {
        AbbrPosition::Anywhere => true,
        AbbrPosition::Command => is_command_position,
    };
    position_ok && self.key == token
}
```

So a `Command`-position abbreviation `g` → `git` expands when `g` is the first word, but a bare `g` sitting as an argument later in the line is left alone — which is exactly what you want, because you don't want every stray `g` in your arguments rewritten to `git`. This is the property that makes abbreviations safer than aliases for the short, ambiguous keys people actually want to abbreviate. An alias `g=git` is unconditional: `git log --grep g` would be fine, but the moment a single-letter alias collides with a value you meant literally, you're surprised. Position-awareness scopes the rewrite to the only place the rewrite makes sense.

Abbreviations also differ from aliases in *when* they fire and what you see. An alias is resolved at execution time and stays invisible — you type `g`, hit enter, and `git` runs, but the history records `g` and the line never showed you what ran. An abbreviation expands *in the buffer*, at the keystroke, so the literal `git` text appears on your command line before you run it. That means the history stores the expanded command, the line is auditable, and you can edit the expansion before committing — none of which an alias gives you. This is fish's reason for preferring abbreviations over aliases for command shortcuts, and it's the model ported here.

The abbreviations live in a global `AbbreviationSet`, guarded by a mutex and reached through `with_abbrs` / `with_abbrs_mut`. The set's `add` has a deliberate behavior: if you add an abbreviation whose `name` already exists, it removes the old one first, so re-adding under the same name *replaces* rather than duplicates. Lookup via `find_match` iterates in reverse — “later abbreviations take precedence” — so the most recently defined matching abbreviation wins.

The actual expansion happens in `expand_abbreviation(line, cursor)`. It finds the word ending at the cursor by scanning back to the previous whitespace, determines whether that word is in command position (the text before the word's start is either empty or ends in `|`, `;`, or `&`), looks for a matching abbreviation, and if it finds one, splices the replacement in:

```

with_abbrs(|set| {
  set.find_match(word, is_command_position).map(|abbr| {
    let mut new_line = String::with_capacity(line.len() + abbr.replacement.len());
    new_line.push_str(&line[..word_start]);
    new_line.push_str(&abbr.replacement);
    new_line.push_str(&line[cursor..]);
    let new_cursor = word_start + abbr.replacement.len();
    (new_line, new_cursor)
  })
})

```

It returns the new line *and* the new cursor position, placing the cursor right after the inserted replacement — so the expansion is seamless and you keep typing from exactly where you'd expect. The test in the file confirms the round trip: adding `g` → `git` and expanding `g` at cursor 1 yields `git`. Because the key match is exact (`self.key == token`), abbreviations are precise triggers, not prefix matches — `g` expands, `gx` does not, which keeps expansion predictable.

Autosuggestions — frequency from history

The third feature is the ghost text: as you type, the shell offers a dimmed completion of your line drawn from history, and you accept it with a keystroke. This is `zsh-autosuggestions`, native. `autosuggest_from_history(line, history)` does the lookup, and its strategy is most-recent-first with a case-sensitivity fallback:

```

// Search history in reverse (most recent first)
for entry in history.iter().rev() {
  // Exact prefix match (case-sensitive)
  if entry.starts_with(line) && entry.len() > line.len() {
    return Autosuggestion {
      text: entry[line.len()..].to_string(),
      is_from_history: true,
    };
  }
}

// Case-insensitive prefix match
for entry in history.iter().rev() {
  let entry_lower = entry.to_lowercase();
  if entry_lower.starts_with(&line_lower) && entry.len() > line.len() {
    return Autosuggestion {
      text: entry[line.len()..].to_string(),
      is_from_history: true,
    };
  }
}

```

It scans history in reverse, so the most recently entered matching command wins — which is the frequency-ranked behavior you want, because recency is the dominant signal for “what am I about to type again.” It first tries an exact, case-sensitive prefix match; failing that, it falls back to a case-insensitive prefix match so that `GIT s` still surfaces `git status` from your history. The suggestion text returned is just the *tail* — the part of the history entry beyond what you’ve already typed — which is exactly the ghost text to render after your cursor. The test pins this: with `git status` in history, typing `git s` suggests `tatus`.

Recency-first is the cheap, correct heuristic here, and it’s worth saying why it’s enough. Full frequency — frequency *and* recency, ranked — is what shells like fish layer in over time, weighting a command you run fifty times a day above one you ran once an hour ago. But for prefix-completion of the line you’re currently typing, recency alone captures most of the value: the command you’re about to retype is overwhelmingly likely to be one you typed recently, because work happens in bursts. Scanning history in reverse and returning the first

prefix match is $O(\text{history})$ in the worst case and effectively $O(1)$ in practice, since the match is usually a few entries back. No index, no ranking table, no background recompute — the simplicity is the feature, and it's why the autosuggest path doesn't need its own cache the way the highlighter's `PATH` walk benefits from one.

The companion function `validate_autosuggestion` (`src/extensions/fish_features.rs:611`) exists so the shell can decide whether a suggestion is worth showing — fish dims suggestions that wouldn't actually run. It reconstructs the full line that accepting the suggestion would produce, pulls out the command word, and checks that it exists: in `PATH` via `command_exists`, or in the `BUILTINS` list, or in the `KEYWORDS` list, or as an existing path if it contains a `/`. If none of those hold, the suggestion is invalid and the shell can suppress or de-emphasize it rather than offering a completion to a command that isn't there. This closes a loop with the highlighter: the same existence checks that color a typo red also gate which ghost-text the shell is willing to offer, so the two features agree about what “a runnable command” is rather than disagreeing at the edges.

Three plugins, one binary

Step back and look at what these three functions collectively replace. A typical zsh power-user setup loads `zsh-syntax-highlighting`, `zsh-autosuggestions`, and `zsh-abbr` — three separate plugins, each a body of zsh script that re-runs through the interpreter on every keystroke and every prompt redraw, each with its own load-order constraints (highlighting must load last) and its own performance footprint the community spends real effort optimizing around. In zshrs these are `highlight_shell`, `expand_abbreviation`, and `autosuggest_from_history` — native Rust functions, adapted from a fuzzed upstream, running inside the shell binary with no interpreter in the path and no plugin manager to configure.

The load-order problem is itself instructive. The reason `zsh-syntax-highlighting` has to load last is that it hooks the same ZLE redraw machinery the autosuggestion plugin hooks, and the order in which the hooks register determines which one wins the final say over the displayed line. That fragility is a direct consequence of three independent plugins competing for the same keystroke event through a scripted hook system that was never designed for three of them at once. When the features live in one binary, sharing one classifier and one set of existence checks, there's no registration order to get wrong — they're called in a known sequence by code that owns the redraw, not three scripts racing to patch a global.

The provenance note in the source is the part to hold onto. This isn't zshrs inventing its own highlighter and hoping it matches fish's behavior; it's a port that keeps fish's shape so fish's tests and fuzzers continue to apply. The features your fingers already know from fish are present, native, and first-class — correct for the same reason fish's are.

The fourth absorption — powerlevel10k

The three fish features share one pattern: take a body of interpreted script that the community runs on every keystroke, and move it into the binary. The largest single application of that pattern is not a fish feature at all. It is the prompt theme itself.

Powerlevel10k is roughly thirteen thousand lines of metaprogrammed zsh. Chapter 1 already put it in evidence: its signature feature, instant prompt, exists to hide the cost of pushing those thirteen thousand lines through the interpreter at startup, and it ships a separate C++ daemon — `gitstatusd` — because computing git status in shell script is too slow to do once per prompt. A theme that needs its own daemon and its own startup illusion is not a theme; it is a compatibility layer for a shell that cannot run it.

zshrs absorbs it the same way it absorbed the highlighter: `source powerlevel10k.zsh-theme` is intercepted at the builtin-dispatch layer, the zsh theme never executes, and a native segment engine in `src/extensions/p10k/` — eleven files, a little over twelve thousand lines of Rust — renders the prompt in-process. The interception is surgical. Your `.p10k.zsh` config file still sources normally, because it is not metaprogramming — it is a few hundred plain `typeset -g POWERLEVEL9K_*` assignments that land in the parameter table, and the engine reads them back with p10k's own documented fallback chain. Reconfigure, re-source, and the next prompt reflects it; the engine reads live state, so reload is free. The theme file remains the

spec: eighty-one segment builders — `dir`, `vcs`, `status`, `prompt_char`, `context`, `battery`, `wifi`, `vi_mode`, `kubecontext`, `aws`, `virtualenv`, and the rest of the catalog — are ported from `internal/p10k.zsh` with `// p10k:NNN` line citations, the same discipline `src/ported/` applies to zsh's C. User-defined segments work too: a `prompt_<name>` shell function runs through the real executor, and its `p10k segment -t ... -i ...` calls are collected by the engine exactly as the theme's own protocol specifies. `CONTENT_EXPANSION` overrides `evaluate` through the shell's real `${(e)}` substitution chain, and the transient prompt — the mode that condenses past prompts to a single character — is ported alongside.

Where the original must be clever, the port can be direct. The zsh theme cannot afford to rebuild the prompt string per render, so it compiles each segment into a deferred `${...}`-expansion template, caches the templates, and lets the prompt expander execute them through a small state machine of counter variables. That is impressive engineering against an interpreter's cost model — and it is exactly the machinery a native engine does not need. The Rust port renders eagerly: one linear fold over the segment list per prompt, same case analysis, same emitted escapes, no template cache because there is nothing expensive to cache.

`gitstatusd` is absorbed on the same terms. `git.rs` reads `.git` directly — `HEAD` and its symbolic refs, loose refs and `packed-refs`, the index in its `v2`, `v3`, and `v4` encodings with an `lstat` scan for unstaged and conflicted files, the stash reflog, tag peeling — so branch, action, dirty state, stashes, and ahead/behind counts come from file reads, not from a daemon and not from a fork. One honest seam remains, and the source documents rather than hides it: staged and untracked counts need a diff against the `HEAD` tree, which needs a `zlib` object-store reader the tree doesn't carry yet, so those two fields come from a single gated `git status --porcelain=v2` subprocess per cache miss.

And one feature is missing on purpose. There is no instant prompt, and there never will be: Chapter 9's first-paint-equals-full-functionality rule forbids it. The point of absorbing the theme is not to run the workaround faster. It is to delete the problem the workaround was invented for.

Chapter 24 — Aspect-Oriented Shell

Here is a problem every serious shell user hits eventually. You want to log every `git` command you run, with a timestamp, to a file. Or you want to know how long your slow completion functions take. Or you want to memoize an expensive function so the second call returns instantly. In `zsh`, each of these is a different hack: an alias that wraps `git` and hopes nobody calls the real binary directly, a `preexec/precmd` hook pair with manual timing arithmetic, a hand-rolled cache check stuffed inside the function body. They work, more or less, but they're ad hoc — each one a different shape, each one fragile, none of them composable.

What you actually want is a single, uniform mechanism: a way to say “run this code *before / after / around* any command matching this pattern,” without touching the command itself. That mechanism has a name in software engineering — *aspect-oriented programming* — and `zshrs` brings it to the shell. The header in `src/extensions/intercepts.rs:10` is blunt about the lineage: “`zshrs`-original — no C `zsh` counterpart.” The closest thing in C `zsh` is the function-wrapper hook (`addwrapper()`, used by `zsh/zprof`), but that wraps shell *functions* only and serves a single profiling consumer; per-command before/after/around advice that fires on builtins, externals, and functions alike, driven by user-registered patterns, has no C `zsh` equivalent.

AOP earned its name in the Java world — AspectJ, then Spring AOP — as the answer to *cross-cutting concerns*: logging, timing, caching, transactions, security checks. These are the things that, written inline, end up smeared across every method in a codebase, the same five lines of “start a timer, do the work, stop the timer, log it” copy-pasted a hundred places. AOP's insight was that those concerns are orthogonal to the business logic and should be factored out into *aspects* that the runtime weaves back in at named *join points*. The shell has the exact same problem at a different scale. “Log every `git` command,” “time my slow completion functions,” “memoize this expensive call” are cross-cutting concerns over your command history, and in plain `zsh` each is smeared into a different ad-hoc hack. `intercept` is the shell's aspect weaver.

The three kinds of advice

AOP's vocabulary is *advice* — code that runs at a join point in another piece of code — and `zshrs` implements the three classic kinds in the `AdviceKind` enum: `Before`, `After`, and `Around`. The source documents each precisely:

```
pub enum AdviceKind {
    /// Run code before the command executes.
    Before,
    /// Run code after the command executes. $? and INTERCEPT_MS available.
    After,
    /// Wrap the command. Code must call `intercept_proceed` to run original.
    Around,
}
```

`Before` runs your code, then the command. `After` runs the command, then your code, with the exit status and timing available. `Around` *replaces* the call with your code, and your code decides whether — and how — the original runs, by calling `intercept_proceed`. These three cover the entire space: you can observe before, observe after, or wrap and control. The README shows one of each, verbatim:

```
# Before – log every git command
intercept before git { echo "[$(date)] git $INTERCEPT_ARGS" >> ~/git.log }

# After – timing
intercept after '_*' { echo "$INTERCEPT_NAME took ${INTERCEPT_MS}ms" }

# Around – memoize
intercept around expensive_func {
    local cache=/tmp/cache_${INTERCEPT_ARGS// /_}
    if [[ -f $cache ]]; then cat $cache
    else intercept_proceed | tee $cache; fi
}
```

Three lines of intent, each solving one of the problems from the top of this chapter, and each using the same `intercept <kind> <pattern> { <code> }` form. Let's read them.

Before – observe and log

```
intercept before git { echo "[$(date)] git $INTERCEPT_ARGS" >> ~/git.log }
```

This registers a **Before** intercept against the pattern `git`. Every time you run a `git` command, before `git` itself executes, the advice body runs: it appends a timestamped line to `~/git.log` recording the arguments you passed. `$INTERCEPT_ARGS` is the magic — it's a context variable the intercept machinery sets to the joined arguments of the intercepted command, so your log captures `git status`, `git push origin main`, and so on without you having to parse anything.

The key property is that the original `git` still runs, untouched. **Before** advice is purely additive — it observes and acts, then gets out of the way. This is the audit-log pattern, and it's structurally cleaner than the `zsh` alias-wrapping approach because there's no shadowing. The alias trick — `alias git='_log_it; command git'` or a wrapper function named `git` — works until something calls the real binary by its full path, or until another alias for `git` overwrites yours, or until you forget you defined it and spend an afternoon debugging why `git` behaves strangely in a subshell. A **Before** intercept has none of those failure modes: you haven't redefined `git`, you've registered an aspect that the dispatcher fires alongside it, so there's no name to collide, no path to bypass, and no shadow to forget. In the implementation, `run_intercepts` runs all matching **Before** advice in a loop and then, if there's no **Around** and no **After** advice, returns `None` — the signal to the dispatcher to "let the normal dispatch continue," i.e., run the real `git`. The advice fired; the command proceeds normally.

After – observe results and timing

```
intercept after '_*' { echo "$INTERCEPT_NAME took ${INTERCEPT_MS}ms" }
```

This registers an **After** intercept against the glob `'_*'` — the canonical `zsh` pattern for completion functions, which are conventionally named with a leading underscore (`_files`, `_describe`, `_git`, and so on). After any function matching `_*` runs, the advice fires and prints how long it took.

`$INTERCEPT_NAME` is the command name that matched, and `$INTERCEPT_MS` is the elapsed time in milliseconds. The timing is real, measured around the actual execution. In the source, `run_intercepts` takes a timestamp with `Instant::now()` before running the original command, computes the elapsed time after, and sets both `INTERCEPT_MS` (millis, three decimal places) and `INTERCEPT_US` (micros) before running the **After** advice:

```
let ms = elapsed.as_secs_f64() * 1000.0;
self.set_scalar("INTERCEPT_MS".to_string(), format!("{:.3}", ms));
self.set_scalar("INTERCEPT_US".to_string(), format!("{:.0}", ms * 1000.0));
```

There's a subtle but important consequence in how `After` is handled. When there's no `Around` advice, `run_intercepts` checks whether any `After` advice matched. If not, it can return `None` and let normal dispatch run the command. But if there *is* `After` advice, the machinery can't just return `None` and walk away — it would have no place to hook the timing and the post-run code. So it runs the original command *itself*, via `run_original_command`, brackets it with the timer, and then fires the after advice. This is exactly the right call: to time a command and act on its result, you must be on both sides of its execution, so the intercept layer takes ownership of running it. This is the timing-instrumentation pattern, and it's a clean replacement for the `preexec/precmd` timing dance, which in `zsh` requires storing a start time in one hook and subtracting it in another with `$SECONDS` arithmetic.

Around — wrap and control

```
intercept around expensive_func {
  local cache=/tmp/cache_${INTERCEPT_ARGS// /_}
  if [[ -f $cache ]]; then cat $cache
  else intercept_proceed | tee $cache; fi
}
```

`Around` is the powerful one, because it *replaces* the call. When `expensive_func` is invoked, this advice runs instead of the function. The body builds a cache filename from the arguments (`${INTERCEPT_ARGS// /_}` substitutes spaces for underscores so the args become a safe filename), and then branches: if the cache file exists, it just `cats` it — the original function never runs at all — and if it doesn't, it calls `intercept_proceed`, pipes the output through `tee` to populate the cache, and returns it.

`intercept_proceed` is the join-point call that AOP frameworks universally provide — `proceed()` on AspectJ's `ProceedingJoinPoint`, `call-next-method` in CLOS's `around` methods, `pjp.proceed()` in Spring AOP. The fact that `zshrs` uses the same shape isn't decoration; it's what makes the feature compose the way an experienced AOP user expects. `intercept_proceed` means “run the thing I'm wrapping,” and its presence is what makes `Around` advice a controller rather than just an observer: the advice decides *whether* the original runs (the cache-hit branch never calls it), *how many times* (a retry wrapper could call it in a loop), *with what surrounding context* (a `cd $dir; intercept_proceed` runs the command elsewhere), and *what to do with the result* (pipe it through `tee`, filter it, discard it). The four classic around-advice patterns — caching, retry, timing, and access control — all fall out of those four degrees of freedom. The example here is memoization: run it once, cache the result, serve from cache forever after.

The mechanism in the source is a flag. Before running the around body, `run_intercepts` sets `__intercept_proceed` to `"0"`. The around advice runs; if it calls `intercept_proceed`, that builtin sets the flag to `"1"` and runs the original command inline. Afterward, `run_intercepts` reads the flag back:

```
self.set_scalar("__intercept_proceed".to_string(), "0".to_string());
let advice_result = self.execute_advice(&advice.code);

let proceeded = self
  .scalar("__intercept_proceed")
  .map(|v| v == "1")
  .unwrap_or(false);
```

If `proceeded` is true, the original already ran inside the advice (where `intercept_proceed` invoked it), so the advice's result *is* the command's result. If false — the cache-hit branch, where the advice `cat`'d the file and never called `proceed` — then the command was suppressed entirely and the advice's output stands in for it. Either way, the around advice fully handles the command; `run_intercepts` returns `Some(result)` and the normal dispatcher never sees the call. That `Some` versus `None` return is the whole contract: `Some` means

“handled, don’t dispatch,” `None` means “I only had before-advice, go ahead and dispatch normally.”

The context the advice runs in

Every piece of advice runs with a set of `$INTERCEPT_*` context variables that the machinery populates before firing. From `run_intercepts`, three are set up front for all advice:

```
self.set_scalar("INTERCEPT_NAME".to_string(), cmd_name.to_string());
self.set_scalar("INTERCEPT_ARGS".to_string(), args.join(" "));
self.set_scalar("INTERCEPT_CMD".to_string(), full_cmd.to_string());
```

`$INTERCEPT_NAME` is the command name, `$INTERCEPT_ARGS` is the arguments joined by spaces, and `$INTERCEPT_CMD` is the full command string. For after-advice, `$INTERCEPT_MS` and `$INTERCEPT_US` are added with the measured timing. These are the join-point reflection data — the way your advice knows *what* it’s advising. And the machinery is tidy about lifecycle: after all advice has run, every one of these variables is unset (`INTERCEPT_NAME`, `INTERCEPT_ARGS`, `INTERCEPT_CMD`, `INTERCEPT_MS`, `INTERCEPT_US`, and the internal `__intercept_proceed`), so they don’t leak into the environment of subsequent commands.

Pattern matching is via `intercept_matches`, which supports exact names, `*/all` for everything, and glob patterns with `*` and `?` checked against both the bare command name and the full command line. So `git` matches the command `git`, `git *` matches `git push origin` against the full line, `_*` matches every completion function, and `*` intercepts everything. The matching is the same `glob::Pattern` machinery the rest of the shell uses, so the patterns behave the way you’d expect from globbing elsewhere.

Why zshrs can do this and C zsh structurally cannot

This is the part that matters most, because it’s not a feature anyone forgot to add to zsh — it’s a feature zsh’s architecture cannot host without rewriting the dispatcher. The reason is the *indirection seam*, and it’s worth unpacking what a seam is and why it’s the whole ballgame for AOP.

A seam, in Michael Feathers’ sense, is a place where you can change a program’s behavior without editing the code at that place. AOP lives or dies on having one at the call site of every advisable operation. AspectJ manufactures its seam by rewriting bytecode at load time, inserting a check at every join point. Spring AOP manufactures its seam by wrapping every bean in a dynamic proxy, so the “real” object is never called directly — calls go through the proxy, which is where advice gets a turn. In both cases the framework had to *build* the indirection, because plain Java method calls are direct: `obj.method()` compiles to an `invokevirtual` that goes straight to the implementation with nothing in between. No seam, no AOP — which is why both frameworks spend most of their machinery constructing one.

C zsh is in exactly the no-seam position. When you run a command, the dispatcher calls the builtin’s implementation function more or less directly: `bin_alias` is invoked at its call site with its arguments, and that’s the call. There is no intermediate layer where you could interpose “before you do that, check whether any intercepts match this command name, and if so run their advice first.” To add one, you’d have to thread an interception check through every dispatch path in the C code — the builtin path, the external-command path, the function-call path — which is the cross-cutting change AOP exists to *avoid* having to make by hand. You’d be hand-weaving the aspect into the dispatcher, which is precisely the smearing the whole technique is meant to eliminate.

zshrs already has the seam, and it has it for reasons that have nothing to do with AOP. Command invocation in zshrs is *already* an indirection: dispatch routes through a layer that decides how to execute — straight through the VM, onto the worker pool, or down the JIT — rather than calling a function pointer directly. That routing layer is the natural place to ask “do any intercepts match?” before proceeding. The `run_intercepts` call sits in that path (`src/extensions/intercepts.rs:65`): it gets the command name, the args, and the full command, checks for matches via `intercept_matches`, fires before/around/after as needed, and returns

`Some` (handled, don't dispatch) or `None` (proceed). The very indirection zshrs needed for the worker pool and JIT is the indirection that makes AOP a small, localized addition rather than a dispatcher rewrite.

State the structural claim precisely: AOP is cheap in zshrs not because of cleverness aimed at AOP but because the architecture already routed every command through a layer where interception is a natural hook. zshrs didn't have to build the seam AspectJ rewrites bytecode for and Spring proxies objects for — it already had one, for free, as a side effect of being a compiled VM rather than a tree-walking interpreter. In C zsh the same feature would be expensive and invasive because the architecture routes commands through direct calls with no such layer. The seam is the prerequisite; zshrs has it for independent reasons, and that's the entire reason it can host AOP at the cost of one `run_intercepts` call where C zsh would need a dispatcher rewrite.

One more detail shows how completely the seam is owned: `execute_advice` itself dispatches on an `@` prefix (`src/extensions/intercepts.rs:166`, with the stryke route at `:169`). If your advice code starts with `@`, it's routed to stryke (in the fat binary) via `try_stryke_dispatch`; otherwise it runs as shell. So the advice you attach to a command can be written in either language, going through the same dispatch path the rest of the embedded-stryke surface uses. The aspect layer isn't bolted on top of the shell; it's woven into the same dispatch fabric as everything else — which is, again, the reason it can exist at the cost it does.

Chapter 25 — The Daemon

Every shell before zshrs carried its whole world inside one process. When you opened a terminal, zsh parsed your `.zshrc`, walked `$fpath` to discover completion functions, built the `_comps` dispatch table, replayed `compdef` calls, loaded plugins, and rebuilt the command hash — every time, in every tab. Open a hundred tmux panes and you paid that cost a hundred times over, a hundred independent processes each redoing identical work, each running its own filesystem walks, each holding its own idea of what the shell knows. The optimizations the community piled on top — `zcompile`, `.zcompdump` caching, zinit-turbo deferral, p10k's instant prompt — were all attempts to make that per-process tax cheaper without ever questioning whether it had to be paid at all.

zshrs questions it. The shell is split into two halves that do not look like a shell at all: a fat, stateful daemon and a paper-thin client. The daemon is the brain — it compiles, it caches, it watches the filesystem, it ranks history, it supervises jobs, it brokers messages between every shell on the machine. The client is the body — it reads keystrokes, paints the screen, forks and execs the commands you type, and reads the daemon's precomputed state through memory-mapped files. One daemon serves N clients. The work that zsh did a hundred times now happens once.

This is not a novel idea in systems software. It is the idea. X11 has a display server and dumb client apps; LSP has a language server and thin editor frontends; Postgres has a server that owns the query planner and locks while `psql` just formats results; Docker has `dockerd` and a 3 MB CLI that talks to it; Kubernetes has the API server and `kubect1`. Every one of these put the heavy, shared, stateful work in a long-lived process and made the clients consumers. Shells never made that move. zshrs makes it: the daemon owns state, the clients mmap and render.

The principle underneath is *think once, read many*. The expensive work — parsing tens of thousands of completion files, resolving the command hash, walking `$fpath`, ranking history — is data-parallel across shells in the worst possible way: every shell needs the same answer, and the old model recomputed it independently in each process. A walk that costs 200ms in one shell costs 20 seconds across a hundred panes. Move the walk into one process that does it once and serves the result, and the hundredth pane pays the cost of an mmap, not a re-walk. The asymmetry is the whole argument: configuration changes rarely, shells open constantly, so you pay the compute once per change and amortize it across every shell that reads the result until the next change. `fishd`, fish's universal-variable daemon, gestures at this — it shares variable state across fish sessions over a socket — but it shares only that narrow slice of state and does none of the heavy compilation, completion, or history work daemon-side. `zconvey`, a zsh plugin, brokers cross-shell command dispatch but owns no canonical state and computes nothing; it is a message bus bolted onto N independent full-fat zsh processes. Neither moves the *work* off the client. zshrs moves the work.

The split brain

The division of labor is deliberate and total. Roughly ninety percent of all shell-internal work lives in the daemon, ten percent in the client. The daemon is allowed to be heavy: a full thread pool, hundreds of concurrent requests in flight, long-running jobs, push notifications, cross-shell routing, federation with peer daemons over the network. The client is forbidden from being heavy: tty IO, process bookkeeping, `fork+exec`, and direct mmap reads, and almost nothing else.

The daemon owns:

- **All compilation.** `$fpath` functions, `.zshrc`, plugins, user scripts — all parsed and lowered to bytecode by the daemon, never by a client.

- **All persistence.** rkyv shard writes, `catalog.db` hydration, `history.db` appends. The daemon is the *sole writer* to every file under `~/.zshrs/`. No client ever writes a shard, a database, the pidfile, the socket, or the log.
- **All search and walking.** History full-text search, frequency ranking, fuzzy matching, completion enumeration, tree-sitter parsing.
- **All filesystem enumeration.** `$PATH` and `$FPATH` directory scans, plugin tree walks, completion-file discovery, theme-file reads. Every `readdir` over a shell-internal directory happens once, in the daemon.
- **All starting-state preparation.** `$PATH/$FPATH/$MANPATH` resolution, the command hash table, the autoload function table, the alias table, shell-option state, keybindings, theme templates — all pre-resolved daemon-side and served at boot.
- **All long-running work.** Job supervision, plugin install and update, the background ticker that rotates logs and vacuums the catalog and runs integrity scans.
- **All routing.** Cross-shell publish/subscribe, the shell registry, session tracking.
- **The single fsnotify watcher** for the entire machine.

The client owns:

- **tty IO** — keystroke reads and screen paints.
- **Process attributes** — `$$`, `$!`, `$?`, traps, signal handlers, the fd table, positional parameters, locals.
- **fork+exec** for the user's commands.
- **The IPC client** to the daemon.
- **Direct mmap reads** against the cache — and nothing else against the cache.
- **A tiny in-memory overlay hash** for in-session monkey-patches: an interactive `alias`, a one-off `compdef`, a function you redefined by hand. Those land in the overlay, are checked first on every lookup, and die when you `exec zshrs`.

There is one absolute rule that falls out of this split: **no walking in clients**. Client-side cache access is `hash(key) → mmap_index → value` — exactly one indirection, no probing chains, no iteration over rkyv internals. The shards use perfect hashing generated daemon-side at compile time, so every key has a unique slot and a lookup is a hash, a mask, and a single dereference: about 150 to 200 nanoseconds end to end. When iteration is genuinely required — `{{(k)}_comps}`, say — the daemon either precomputes a sorted, deduplicated flat key array that the client receives as a slice, or serves the iteration over IPC. The client never runs the logic. Even `hash -r` becomes a message to the daemon, not a client-side rebuild of the command table.

The latency budget

The architecture only earns its keep if the client stays fast, so the costs are budgeted explicitly. Connecting to the daemon over the Unix socket: under one millisecond. Memory-mapping an rkyv table — the completion table, the history index, the alias and function tables — about ten microseconds each, zero-copy, because the kernel maps the daemon's already-built pages straight into the client's address space without a single byte of deserialization. There is no parse, no `serde::from_slice`, no tree rebuild: the bytes on disk are the in-memory representation, and a pointer into the mmap is a pointer into a live `ArchivedDefinitionRecord`. Rendering the prompt from cached theme segments: about a millisecond. A keystroke round-trip to the daemon: under a hundred microseconds. Total cold-start for a new shell: **under ten milliseconds**, with no work to do beyond connecting and mmap-ing.

The budget exists because cold-start is the number a user feels a hundred times a day, and every shell that misses it taxes the whole development loop. The two costs that dominate a conventional shell — source parsing and completion-table construction — are both pre-paid daemon-side, so what is left for the client is a connect, a handful of mmaps, and a prompt paint. None of those scale with `$fpath` size, plugin count, or history depth; they are constant-time regardless of how large the configuration grew. A 27,000-file completion corpus and a 200-line `.zshrc` produce the same client cold-start, because the client reads a finished table in both cases and never sees the corpus at all.

Set that against the world it replaces. zsh with `compinit` cold, parsing seventeen thousand completion files,

costs three to five seconds and a full CPU spike on every shell. `zsh` with a warm `.zcompdump` costs three hundred to eight hundred milliseconds — and that cache corrupts every few weeks and demands a manual `rm ~/.zcompdump*`. The daemon path mmaps a pre-built, byte-aligned table and does a single hash lookup per completion query in tens of nanoseconds to a few microseconds. That is not a ten-times speedup on cold-start completion lookup; it is four to six orders of magnitude.

The correctness win matters as much as the speed. The “completion is stale, restart the shell, still stale, `rm ~/.zcompdump`, restart again” loop that every `zsh` user has lived through cannot exist here, because `fsnotify` is the single source of truth. When a file in `$fpath` changes, the daemon rebuilds the affected slice and notifies subscribers — no polling, no cron, no manual deletion. The same property that an IDE’s language server gives you: you never think about cache invalidation again.

One more guarantee underwrites all of this: **source files remain authoritative**. The image cache is an opportunistic accelerator, never a requirement for execution. An image miss, a malformed shard, corruption, or a daemon that is simply not running — any of these and the client falls through to the source-interpretation path silently. Daemon-down does not break your shells. It only makes the first one slower.

The protocol

Clients and daemon speak length-prefixed JSON over `~/.zshrs/daemon.sock`. Each frame is a four-byte big-endian length followed by that many bytes of UTF-8 JSON. A client opens with a `hello` carrying its version, pid, tty, cwd, and `argv0`; the daemon answers with a `welcome` carrying the client id, session id, daemon pid, and uptime. After that, requests carry a monotonic per-connection `id`, an `op`, and an `args` object; responses echo the `id` and carry `ok: true` with a payload or `ok: false` with an error shape like `{"err":{"code":"shard_locked","msg":"..."}}`. Asynchronous events from the daemon — a shard swapped, a pub/sub match, a cross-shell dispatch — carry an `event` field and no `id`, fire-and-forget. The convention is uniform enough to be mechanical: every timestamp field is suffixed `_ns` and holds integer nanoseconds since epoch, every size is suffixed `_bytes`, and an unknown op comes back as `{"err":{"code":"unknown_op","msg":"unsupported by daemon vN"}}` rather than a dropped connection, which keeps version skew between a new client and an old daemon survivable.

JSON is the default for a reason. The protocol is debuggable end to end with `socat` and a terminal — you can hand-type a frame, read the reply, and watch async events stream by, with no special tooling and no schema compiler. That property is worth a constant-factor cost on the cold path, where a request happens once per shell boot. The hot path is the escape hatch: if per-keystroke ops like `highlight` or `suggest` ever show JSON parse cost in a flamegraph, those specific opcodes can migrate to `msgpack` or a fixed binary layout while the rest of the protocol stays human-readable. The default optimizes for the operator reading the wire, not for the microbenchmark, because the cold path is where the wire is actually read by hand.

The same operations are exposed over HTTP, and this is what turns the daemon from an implementation detail of one shell into a service any program can use. The daemon binds an HTTP listener on **port 7733**, configured in `~/.zshrs/daemon.toml`:

```
[http]
listen = "127.0.0.1:7733"
```

The HTTP surface is small and regular: `GET /health` for liveness, `GET /ops` to list every op the daemon accepts, and `POST /op/<NAME>` to invoke an op with a JSON body whose shape matches the IPC `args` payload exactly — same surface, different transport. There is no second protocol to learn and no translation layer between the socket and the port: a `POST /op/job_submit` with a JSON body is byte-for-byte the same `args` object a `zshrs` client sends over the Unix socket, routed to the same handler. The HTTP layer adds exactly three things on top of the IPC surface — the health probe, the op-name list, and the bearer-token gate — and otherwise gets out of the way. Liveness is a one-liner:

```
curl -s http://127.0.0.1:7733/health
# {"ok":true,"version":"0.10.7","uptime_ms":12345}
```

```
curl -s -X POST http://127.0.0.1:7733/op/ping \
  -H 'Content-Type: application/json' \
  -d '{}'
```

```
# {"ok":true,"pong":true,"ts_ns":...,"daemon_uptime_ms":...,"echo":null}
```

The auth floor

The HTTP listener has exactly one security rule, and it is non-negotiable: loopback is open, non-loopback requires a Bearer token. If `listen` is `127.0.0.1`, no token is needed — the surface is reachable only from the local machine, the same trust boundary as the Unix socket. The moment you point `listen` at any non-loopback address, the daemon **refuses to bind** unless tokens are configured. There is no way to expose the daemon to the network without authentication.

```
[http.tokens]
me      = "long-random-secret-1"
ci      = "long-random-secret-2"
vim-lsp = "long-random-secret-3"
```

With tokens configured, every `POST /op/<NAME>` must carry `Authorization: Bearer <token>`; a missing or wrong token returns `401 {"ok":false,"code":"unauthorized","msg":"..."}`.

```
curl -s -X POST http://127.0.0.1:7733/op/info \
  -H 'Authorization: Bearer long-random-secret-1' \
  -H 'Content-Type: application/json' \
  -d '{}' | jq
```

The daemon deliberately does not set CORS headers in v1 — it is built for same-origin tooling, the CLI, editor plugins, and scripts, not for browser pages from the open web. If you need browser access, you put a reverse proxy in front of it.

The op catalog

The daemon accepts roughly a hundred operations — the `OP_NAMES` array in `daemon/ops.rs` defines them, and the `tests/daemon_doc_drift.rs` test fails the build if the documented table and the code drift apart, so the catalog cannot rot silently. The ops are grouped by concern, and the grouping is the point: each family collapses a piece of single-user infrastructure that used to be a separate daemon, a separate file format, or a separate `man` page into the one process that is already running. A tour of what the daemon can do for you:

Meta and diagnostics. `info` for daemon and catalog stats, `ping` for round-trip latency, `doctor` for a full diagnostic report, `verify` for an integrity scan over shards and catalog, `compact` to vacuum and dedup, `metrics` for Prometheus-shaped counters, plus runtime config and log-level controls.

Jobs. The daemon owns a tokio-backed job queue, and the jobs it runs survive client disconnects because the daemon owns the child process, not the shell that submitted it. This is the property a backgrounded `&` job never had: close the terminal that launched it and the job dies with the shell. A daemon job outlives its submitter, because the lifetime is tied to the long-lived process, so a `find /` started in one pane is still running, still capturing output, when that pane is gone. Submit an `argv` array, get a `job_id`, poll for completion, fetch the captured output:

```
# 1. Submit. command must be a JSON array (argv-style).
JOB=$(curl -s -X POST http://127.0.0.1:7733/op/job_submit \
-H 'Content-Type: application/json' \
-d '{"command":["find","/","-type","f","-name","*.zsh"]}' \
| jq -r .job_id)
echo "submitted job_id=$JOB"

# 2. Poll status (the daemon doesn't push; the client loops).
# Terminal states: exited, failed, killed, cancelled.
while ;; do
  ST=$(curl -s -X POST http://127.0.0.1:7733/op/job_status \
-H 'Content-Type: application/json' \
-d '{"id\":"$JOB}" | jq -r .job.state)
  echo "state=$ST"
  case "$ST" in
    exited|failed|killed|cancelled) break ;;
  esac
  sleep 1
done

# 3. Fetch captured stdout. Pass "stderr": true for the err stream.
curl -s -X POST http://127.0.0.1:7733/op/job_output \
-H 'Content-Type: application/json' \
-d '{"id\":"$JOB","stderr\":false}' | jq -r .content > shells.txt
```

The full op family is `job_submit`, `job_status`, `job_output`, `job_list`, `job_kill`, `job_cancel`, and `job_wait`.

Cache. A persistent, namespaced key-value store backed by SQLite at `~/.zshrs/cache.db` — a single-user Redis without the second daemon. Namespaces are arbitrary strings; entries can carry a TTL:

```
curl -s -X POST http://127.0.0.1:7733/op/cache_put \
-H 'Content-Type: application/json' \
-d '{"ns":"ci","key":"target","value":"release","ttl_secs":86400}'
# {"ok":true,"bytes":7,"key":"target","ns":"ci","expires_at":...}

curl -s -X POST http://127.0.0.1:7733/op/cache_get \
-H 'Content-Type: application/json' \
-d '{"ns":"ci","key":"target"}' | jq -r .value
# release
```

The family is `cache_put`, `cache_get`, `cache_del`, `cache_list`, `cache_stats`.

Artifacts. A content-addressed blob cache keyed by sha256 — `sccache` and `ccache` collapsed into the same daemon. Multiple names can point at the same digest and the daemon dedups automatically; blobs ride the wire base64-encoded. The family is `artifact_put`, `artifact_get`, `artifact_get_by_digest`, `artifact_list`, `artifact_gc`.

Locks. Named cross-process mutexes with PID-tied auto-release — `flock(1)` without the lockfile. Pass your pid so the daemon can force-release if you crash mid-section:

```
RESP=$(curl -s -X POST http://127.0.0.1:7733/op/lock_try_acquire \
-H 'Content-Type: application/json' \
-d '{"name\":"deploy\","pid\":"$$"}')
TOK=$(echo "$RESP" | jq -r .token)
```

The family is `lock_acquire`, `lock_try_acquire`, `lock_release`, `lock_list`.

Schedule. A cron-equivalent that persists across daemon restarts. The cron expression is six fields including seconds — `sec min hr dom mon dow` — and the tick fires every second, dispatching `job_submit` for each due row:

```
# Every 15 minutes:
curl -s -X POST http://127.0.0.1:7733/op/schedule_add \
  -H 'Content-Type: application/json' \
  -d '{"cron_expr":"0 */15 * * * *","command":["check-mail.sh']}'
# {"ok":true,"schedule_id":1,"cron_expr":"0 */15 * * * *"}
```

The family is `schedule_add`, `schedule_add_once`, `schedule_remove`, `schedule_list`.

Snapshots. Tag-based captures of canonical state, stored in the same rkyv format as a recorder bundle under `~/.zshrs/snapshots/<tag>.rkyv`. Save before an experiment, diff two tags to see what changed, restore with an atomic swap:

```
curl -s -X POST http://127.0.0.1:7733/op/snapshot_save \
  -H 'Content-Type: application/json' \
  -d '{"tag":"laptop-pre-experiment"}'

curl -s -X POST http://127.0.0.1:7733/op/snapshot_diff \
  -H 'Content-Type: application/json' \
  -d '{"a":"baseline","b":"after-plugin-install"}'
```

Pub/sub and the shell registry. `subscribe`, `unsubscribe`, and `publish` ride a pattern-matched event bus; `list_shells` enumerates every registered session (this powers `zls`); `send` dispatches a command to one shell, a tag group, a user, or everyone. For long-lived HTTP clients there are two Server-Sent Events endpoints that push without polling — `GET /stream/watch?path=DIR` for fsnotify events and `GET /stream/events?channel=PATTERN` for the pub/sub bus:

```
# Listener:
curl -sN 'http://127.0.0.1:7733/stream/events?channel=*.build_done'

# Publisher (separate shell):
curl -s -X POST http://127.0.0.1:7733/op/publish \
  -H 'Content-Type: application/json' \
  -d '{"topic":"build_done","data":{"target":"prod","status":"ok"}}'
```

Completion and history. `complete`, `suggest`, and `highlight` are the per-keystroke hot path — tab-completion enumeration, autosuggestion, syntax highlighting. `history_query` and `history_append` are full-text search and write. `source_resolve` is the bytecode-cache lookup behind `source FILE` and `. FILE`.

The daemon collapses six or seven separate pieces of single-user infrastructure — Redis, sccache, cron, `flock`, watchman, a job runner — into one tokio-backed process. Each of those is, on a single-user machine, a daemon you would otherwise install, configure, supervise, and keep patched: a Redis instance for a key-value cache, a ccache or sccache directory for content-addressed build artifacts, a crontab for scheduled work, a lockfile convention for cross-process mutual exclusion, a watchman or `fswatch` process for filesystem events. The zshrs daemon serves all of them through one socket and one HTTP port, with one lifecycle to manage, because they are all the same shape of problem — durable state and long-running work shared across the processes on one machine — and a single user does not need the multi-tenant, networked, separately-operated versions of any of them.

The strategic consequence is that you do not have to adopt the zshrs shell to use any of this. The daemon is a standalone user-space service with a versioned public API over HTTP on port 7733; a bash user, a Python script, an editor plugin, or a CI runner can submit jobs, take locks, cache artifacts, schedule cron rows, and subscribe to filesystem events without ever launching the zshrs shell. The shell is the daemon's first and richest client, not its only one.

Lifecycle and singleton guarantees

A daemon that owns this much state has to come up cleanly and stay singular. The model is spawn-on-demand: the first zshrs client to launch checks for `daemon.sock`, and if it is absent, fork-spawns the daemon.

Subsequent clients — the second through the hundredth tmux pane — just connect. A `flock` on `daemon.pid` enforces that exactly one daemon exists; a race between two cold-starting clients resolves to one winner holding the lock and the loser connecting to the survivor. There is no separate “start the daemon” step in the user’s path. You open a shell; the substrate is there.

The cache lives in a single directory, `~/.zshrs/`, holding `index.rkyv`, the per-shard images under `images/`, `catalog.db`, `history.db`, `zshrs.log`, `daemon.sock`, and `daemon.pid`. The tradeoff is explicit and accepted: a full `rm -rf ~/.zshrs/` nukes everything, and the next shell rebuilds it. One directory, daemon-owned, queryable, exportable — no more `.zwc` files scattered through plugin trees, no `.zcompdump.zwc` orphan, no `~/.zcompdump*` accumulating across versions.

Shard rebuilds obey a strict ordering rule that exists to prevent torn reads. When the daemon swaps a shard, it atomic-renames the shard file **first**, then rewrites `index.rkyv` — never the reverse, because the reverse would briefly point the index at a shard that hasn’t landed. A generation counter in each shard header lets a client with a stale mmap notice it needs to re-map. Existing client mmaps stay valid across the swap because the kernel pins deleted-but-mapped inodes until the last unmap, on both Linux and macOS, so a client running against the old image keeps reading consistent pages while a new client picks up the new one.

The POSIX off-switch

There is one configuration in which none of this applies, and it is load-bearing for compatibility. POSIX mode — `--posix`, `emulate sh`, or an `argv[0]` basename of `sh`, `dash`, or `bash` — gates the entire daemon layer off. No daemon spawn, no cache directory created, no `z*` builtins available. This is what makes a `/bin/sh → zshrs` symlink safe in containers, cron jobs, and init scripts: a POSIX invocation behaves like a plain POSIX shell, with no companion process, no socket, and no background state. The fat architecture is opt-in by virtue of running `zshrs` as `zshrs`; run it as `sh` and it gets out of the way entirely.

Chapter 26 — The Recorder

Here is a question every zsh user with a real configuration has asked and never had a good answer to: *where did this alias come from?* You type `whence -v gst` and it tells you the value — `git status -sb` — but not the file, not the line, not which of the five places that define `gst` actually won. So you fall back to `grep -rn "alias gst" ~/.zshrc ~/.oh-my-zsh ~/.zinit ~/.zpwrc`, and you get five matches across four files, and now you have a worse problem: which one fired? In what order? Did one get skipped by a conditional guard? Was one defined inside a function that never got called? You trace each by hand.

The reason this is hard is that the configuration that decides what gets loaded is itself part of the input being loaded. `if [[ -o interactive ]]; then source ...; case $TERM in xterm*), [[ -d ~/.rbenv ]] && ...; eval $(starship init zsh); source <(kubectl completion zsh); for f in $ZDOTDIR/*.zsh; do source $f; done`. You cannot know what your shell config does without running it. Static analysis — zinit's `@zinit-report`, antibody's `bundle list`, oh-my-zsh's `omz changelog` — can only ever report what it itself loaded, and none of it can tell you the file and line where a definition lives.

The recorder runs it. There is no prediction; there is only observation.

What it is

The recorder is a one-shot indexing pass over your shell init that captures every state modification — aliases (plain, global, and suffix), function definitions and autoloads, environment exports, typed and untyped parameter assignments, `path/fpath/manpath` edits, named directories (`hash -d`), `zstyle`, `bindkey`, `compdef`, `zle`, `zmodload`, `setopt`, `trap`, `sched`, and `source` — with the exact source file and line number where each one happened, plus the function call-stack at the time, plus a timestamp and a sequence index. It does this by running your real init under instrumented zshrs, with aspect-oriented intercepts woven onto fifteen state-mutating builtin dispatchers. Every time `alias` fires, every time a function is defined, every time `path+=` runs, the aspect captures (`kind`, `name`, `value`, `file`, `line`, `fn_chain`, `ts`) and batches it to the daemon, which stores it in SQLite for querying and an rkyv shard for fast mmap'd reads.

The information comes for free from where the intercept sits. The shell already knows the file and line of the call site — the parser plumbs it through every pipe node — and the function call-stack is the runtime's own funcstack at the moment the dispatcher fires. The aspect is not reconstructing provenance after the fact; it is reading the position the runtime already holds while it executes the mutation. That is why a definition buried five plugin-manager layers deep still resolves to the line that wrote it: the line was never lost, only thrown away, and the aspect catches it before it is discarded.

The result is that `zwhere alias gst` answers instantly and exactly:

```
/Users/wizard/.zpwrc/env/.shell_aliases_functions.sh:1742 (zpwrcLoadAliases ← _zpwrc_init)
  gst → 'git status -sb'
```

— regardless of how `gst` was loaded: zinit, oh-my-zsh, antigen, raw `source`, or inline in `.zshrc`.

Why it sees what static analysis cannot

The mechanism that makes this work is global aspect-oriented programming across builtin dispatchers. The closest facilities other shells offer are all strictly weaker, and they are weaker in the same direction: they hook a fixed handful of named moments, not every dispatcher. bash's `trap ... DEBUG` is a single global hook that

fires before every simple command with no per-builtin granularity — it tells you a command ran, not that `alias` specifically ran with these arguments at this line. zsh's `preexec/precmd/chpwd_functions/periodic_functions` are four or five fixed hook points around the prompt loop, not advice at the dispatcher level. fish's `--on-event` and `--on-variable` are named-event and variable-scope subscriptions — you subscribe to `fish_prompt` or to a specific variable name, not to the union of mutating operations. nushell's plugins run out of process over a Wasm or subprocess boundary and cannot intercept in-runtime dispatch at all; they consume input and produce output, never weaving into the host's execution. None of them can intercept the union of state-mutating dispatchers with structured metadata emission and around-advice that delegates to the original logic. The recorder does, with the same fifteen aspects covering every path by which state enters a session.

The reason this can only exist in zshrs comes down to where the intercept lives. zsh's C codebase calls `bin_alias` directly from the executor with no indirection point that could host advice — retrofitting AOP would mean an indirection layer at every dispatch, a per-dispatcher aspect chain, re-entrancy guards, and cross-aspect composition rules, all bolted onto decade-old C with zero unit tests. zshrs got the indirection for free, because the clean-slate rewrite already needed function-pointer-style dispatch for the worker pool and the JIT. The aspects compose through the same mechanism. AOP costs zshrs almost nothing structurally because the prerequisite was already there for other reasons.

The naive alternative — shadowing each builtin with a shell function that records then calls `builtin alias` `"$@"` — breaks in five different ways. It re-enters itself when oh-my-zsh's helpers call `alias`. It loses argument fidelity across the subtle differences between `builtin alias`, the shadow, and the original. It loses position context, because by the time the shadow runs, `$LINENO` reports the shadow's own line, not the caller's. It has coverage gaps: `VAR=value` is an assignment token, not a builtin call, so a function shadow cannot see it at all. And it runs five to fifty times slower on `compinit`, which makes around twenty thousand builtin calls. The aspect runs inside the already-parsed call site, with full access to the AST node's lineno and filename, the untouched argv, and the runtime's funcstack. No recursion, no quoting hazard, no shadow overhead.

This is also why the recorder is plugin-framework-agnostic by construction. zsh has a finite, stable set of primitives through which state can be installed — there is no other way to put an alias into a session than to call the alias dispatcher. A plugin manager can generate the call inline, defer it behind a `wait` ICE, wrap it in a helper, queue it in an array, code-generate it at install time, fork it across subshells, or emit it from `eval $(...)`. In every case the actual mutation happens at the dispatcher, and the recorder sees the dispatcher fire with the file and line of the call site at the moment it fires. It does not know zinit exists. It catches what zinit-turbo's deferred subshell flushes through the dispatchers at flush time, the same as it catches a raw `alias x=y` in `.zshrc`. Whatever future framework X invents, it will hit a dispatcher to install state, and it will be caught — with no code changes to the recorder.

The same property dissolves shell-mode quirks. Static analysis has to model — or fail to model — whether you are interactive or not, login or not, in a restricted sandbox, in `zsh -c` command mode, under which `$TERM`, in which emulation mode. The recorder evaluates every conditional in its actual environment and records exactly the branches that actually executed:

```
if [[ -o interactive ]] && [[ "$TERM" == xterm* ]] && (( $+commands[fzf] )); then
  bindkey '^R' fzf-history-widget
  alias fzf-friendly-ls='ls --color | fzf'
  fpath+=(/usr/share/fzf/completions)
elif [[ -n "$SSH_CONNECTION" ]]; then
  alias remote-prompt-tweak='print "ssh!"'
fi
```

A static analyzer must guess. The recorder, run in an interactive xterm with fzf installed, produces three records; run over SSH, one; run non-interactively, zero from this snippet. Each run captures the mode it ran in, and `zwhere -m diff interactive non-interactive` answers a question zsh users could not previously answer at all: what does my config do interactively that it doesn't do non-interactively?

Per-definition granularity

Per-definition granularity is what separates the recorder from the prior art. zinit's `@zinit-report` is a before/after state diff at the plugin boundary: it snapshots before sourcing a plugin, lets the plugin do whatever it wants, snapshots after, and reports the delta. The attribution unit is the plugin, so a plugin that defines twelve aliases, eight functions, three bindkeys, four zstyles, and two fpath edits produces one report entry — twenty-nine mutations collapsed into a single “here's what this plugin added.” The recorder produces twenty-nine records, each with its own file:line, fn_chain, and timestamp. Three things the diff loses, the recorder keeps. Temporal order within a plugin — `alias x=A` then a conditional `alias x=B` — is gone in a diff that shows only the final value, but preserved by `order_idx` and the `prev_def_id` override chain. The provenance of an individual mutation is unrecoverable from a namespace diff, because diffing namespaces tells you that `zshz` now exists, not which line of which file defined it; the recorder's attribution unit is the call site itself. And the diff is structurally tied to one plugin manager — it can only report what zinit itself sourced — whereas the recorder watches the dispatchers and is blind to which manager, if any, drove them. The recorder is strictly more granular than zinit-report even on zinit's home turf, and it covers ground the diff cannot reach: non-zinit-loaded code, deferred mutations, `eval`-generated definitions, and mode-conditional branches that a static or snapshot view never sees fire.

Querying with `zwhere`

The query builtin is `zwhere`, and it is always present in the default binary — it is just an IPC client over the daemon's `definitions_query` op, the same tier as `zcache` and `zls`. The simplest form takes a kind and a name; `-l` gives the full lineage, every prior definition in order with the override chain made visible:

```
3 definitions of `gst`:
1. /Users/wizard/.zinit/snippets/OMZ::lib/git.zsh/OMZL::git.zsh:188    gst → 'git status'
   ~ [overridden]
2. /Users/wizard/.zpwrc/env/.shell_aliases_functions.sh:1742          gst → 'git status'
   ~ -sb' [overridden]
3. /Users/wizard/.zshrc:567                                             gst → 'git status'
   ~ -sb --branch' [current]
```

Beyond `where`, `zwhere when` answers *when* — the load order that has bedeviled `$path` and `$fpath` debugging forever:

```
zwhere when path                # every site that mutated $path, in order

1. ~/.zshenv:5                  path=(/usr/local/bin $path)
2. ~/.zshrc:21                  path+=(~/.local/bin)
3. zinit_init:101               path=(/opt/homebrew/bin $path)          [via zinit ice]
4. ~/.zpwrc/env/.zpwrc_env.sh:88 path+=($ZPWR/scripts)
5. /etc/zshrc:12                path+=(/usr/sbin)                     [SYSTEM]
```

The output is tab-separated and grep-friendly by design, so `zwhere -k alias | grep ' git '` does what you'd expect.

Batching and the cost of a run

A recorder run instruments a live shell init, and a real init makes a lot of dispatcher calls — `compinit` alone makes around twenty thousand. Per-event IPC over the daemon socket would tank that, so the recorder buffers. Records accumulate in a `VecDeque` with a soft cap of 512 events and a hard cap of 4096, and a flush fires when the buffer fills, when fifty milliseconds pass with something pending, at a phase boundary like the first `precmd`, or at shell exit. Each flush is a single length-prefixed write of a serialized record vector; the daemon side is a single SQLite transaction with `PRAGMA synchronous=NORMAL`, about ten milliseconds

for four thousand inserts. The whole run — `zpw` plus `zinit` plus `p10k` — costs roughly one and a half times a normal startup, dominated by that SQLite IO. Acceptable for a one-shot indexing pass; not something you'd want on every shell, which is exactly why the recorder is opt-in and lives in its own binary.

What the daemon stores is two-faced by design. The canonical form is a SQLite `definitions` table — one row per state-modifying event, carrying `kind`, `name`, `value`, `file`, `line`, `col`, `fn_chain`, `order_idx`, `ts_ns`, a flags bitmask for dynamic-eval and sourced and plugin-loaded, a `prev_def_id` pointing at the definition this one shadowed, and a `sensitive` flag so a secret-bearing `AWS_SECRET_ACCESS_KEY` export is marked while a plain `PATH` edit is not. That table is queryable and joinable. The fast form is an rkyv shard, `~/.zshrs/recorder-{run_id}.rkyv`, a byte-aligned `Vec<DefinitionRecord>` that the default `zshrs` binary mmap's and binary-searches by `(kind, name)` — the same zero-copy read path the autoload cache uses, with lookup latency from fifty nanoseconds to a few microseconds. The thin client never touches SQLite for a hot `zwhere`; it reads the shard the daemon built.

What this changes for the daily driver

This is not only a power-user feature. Consider the search surface a typical real configuration presents: a `.zshrc` of a few hundred to a couple thousand lines, oh-my-zsh's two hundred plugin directories, `zinit` with five to fifty plugins, a completion corpus of anywhere from seven hundred to twenty-seven thousand files, framework env files in the tens to hundreds of thousands of lines — combined, typically fifty thousand to three hundred thousand lines of zsh code. When you wonder where an alias came from, your options today are `whence -v` and `which -a`, which give you the value, and `grep -In` across four directories, which gives you a list of candidate sites and no way to know which one fired, in what order, or whether a conditional guard skipped it. For functions it is worse, because the function may be autoloaded, defined in a plugin, wrapped by another plugin, and overridden in `.zshrc`, with the real call site buried under several layers of plugin-manager indirection. The recorder turns that manual trace into one query with an exact answer.

Cross-shell federation

One daemon serves every shell on the machine, and the recorder catalog is federated. Each record carries a `shell_id` — `zshrs` by default, but `zsh`, `bash`, `fish`, `nu`, `ksh`, and the rest are reserved, along with vendor-prefixed forms like `editor:vscode`, `ci:github-actions`, and `agent:claude-code`. A bash session can emit records live through a shadowed builtin via `definitions_emit`, a fish recorder can ingest a full bundle via `recorder_ingest`, and `zd defs diff bash fish` shows what differs between them. Rows tagged with one `shell_id` never silently overwrite rows tagged with another; the diff op distinguishes by shell at query time. Any shell that implements the documented recorder protocol writes into the same canonical catalog `zshrs-recorder` writes to — the federation contract is open, and the daemon does not gate-keep it.

Zero bytes in the default binary

The non-functional requirement that ranks above all the others: the default `zshrs` binary contains zero recorder code. Not “zero overhead at runtime” — *zero bytes*. The recorder is a Cargo feature flag, `recorder`, off by default. Every aspect, every IPC type, every dispatcher swap lives behind `#[cfg(feature = "recorder")]` and is deleted by the compiler before linking. A daily-driver `zshrs`, built with a plain `cargo build`, is byte-identical to one built before the recorder feature ever existed. A separate `zshrs-recorder` binary is built with the feature on, from the same source tree, for one-shot indexing runs. The two share the entire codebase except the gated code.

This is verified, not asserted. `nm target/release/zshrs | grep -c recorder` must return `0`. So must the disassembly check and the string check. The binary-size delta from adding the feature must be zero. All four run on every release tag, and a nonzero result is a release blocker. There is no perf regression possible because there is no code to regress; the benchmarks exist only to catch an accidental introduction of

a non-gated branch. The recorder is a power tool you reach for deliberately — at first setup, or after a major config change — and the daily driver pays nothing for its existence.

Chapter 27 — Stryke Inside

The daemon makes zshrs the brain of every shell on the machine. There is a second sense in which zshrs is central, and it shows up in one line of the design goals: **zshrs is the god of all processes**. It is the parent runtime, the process supervisor, the dispatch layer for everything built around it. stryke runs inside it. zpw runs inside it. External tools run as its children or, where the fork cost matters, as its in-process builtins. The shell is not a launcher that hands control to other programs; it is the substrate they live in.

One VM, two languages

strykelang — a bytecode VM with a Cranelift JIT and Rayon parallelism — is not a separate process that zshrs shells out to. It runs inside zshrs through @-prefix dispatch. Type a line that starts with @ and the shell hands it to stryke; everything else is shell. The dispatch decision is a single character check at the front of the line, made before the shell parser is consulted, so the two languages never have to share a grammar — the sigil partitions the input space cleanly and each frontend owns its half. The wiring is small and explicit. The lean zshrs binary ships with no stryke dependency at all, but the fat binary registers stryke as the handler for @ lines through a single hook in `src/lib.rs`:

```
/// Register a handler for @ prefix lines (fat binary sets this to stryke::run).
pub fn set_stryke_handler<F>(f: F)
where
    F: Fn(&str) -> i32 + Send + Sync + 'static,
{
    let _ = STRYKE_HANDLER.set(Box::new(f));
}

/// Try to dispatch a line starting with @ to stryke.
/// Returns Some(exit_code) if handled, None if no handler registered.
pub fn try_stryke_dispatch(code: &str) -> Option<i32> {
    STRYKE_HANDLER.get().map(|f| f(code))
}
```

If no handler is registered — the lean build — `try_stryke_dispatch` returns `None` and the line is handled as ordinary shell. If stryke is present, the @ line runs through it. The dispatch is one `OnceLock` lookup; there is no IPC, no subprocess, no serialization across a boundary. The cost of the check on a non-@ line is a single byte comparison, and the cost on an @ line is the `OnceLock::get` plus a direct call into the registered closure — the same call overhead as any other Rust function, because that is what it is. Nothing crosses a process boundary, so there is no marshalling, no re-parsing in a foreign interpreter, and no second runtime to keep warm.

What makes the two languages genuinely one runtime rather than two co-resident ones is that they share a virtual machine. zshrs's bytecode compiler targets the same `Op` enum that strykelang uses — the `Op` enum that lives in fusevm, the VM both projects share. The lexer and parser in zshrs are direct ports from zsh's C source; only the bytecode compiler is original Rust, and it lowers the zsh 4-tier AST — `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand`, preserved verbatim from zsh for per-construct parity — down to the same opcodes stryke emits. Both frontends share fused superinstructions, extension dispatch, and the Cranelift JIT path. Two languages, two parsers, two grammars; one set of opcodes, one VM, one JIT. A stryke @-line and a zsh pipeline execute through the same machine.

The lean build matters here as a statement of priorities. You can compile zshrs with no stryke dependency

at all — `cargo build --release` from a fresh clone gives you a pure-shell binary, full parallelism intact, `async/await/pmap` all present, and no language runtime bolted in. `stryke` is an opt-in layer the fat build adds, not a hard dependency the shell drags along. The relationship is strictly one-directional: `stryke` embeds inside `zshrs`, and the shell can exist entirely without it, but `stryke`'s `@-dispatch` entry point only exists because the shell offered the hook. That asymmetry is the design goal made literal — when `zshrs` and `stryke` priorities conflict, `zshrs` wins, and the build graph reflects it.

Substrate, not launcher

The architectural default that falls out of “god of all processes” is concrete and measurable: when you design a new feature, you make it live inside `zshrs` as a builtin or a host op rather than as a separate process the shell invokes. A builtin costs zero forks. An external command costs two to five milliseconds each. On a machine running millions of commands a day, every fork avoided is real wall-clock time, and so twenty-three `coreutils` are already in-process builtins rather than `fork+exec` of the system binaries. External tools the maintainer chose not to rewrite — `tmux`, `fzf`, `ripgrep`, `eza`, `atuin`, `bat`, `zoxide` — run as `host.exec` children, but the gravity always pulls toward in-process. The shell is the place computation happens, not a dispatcher that gets out of the way.

This is the same hierarchy the daemon expresses, viewed from the other side. The daemon makes `zshrs` the owner of all shared state across processes; the host runtime makes `zshrs` the owner of execution within a process. Between them, the shell sits underneath everything — it is the parent runtime that compiles your config, supervises your jobs, brokers your messages, hosts your scripting language, and runs your `coreutils` without forking. `stryke` embeds inside `zshrs`, never the other way around, and when the two projects' priorities conflict, `zshrs` wins.

The host reach extends past the languages running inside a single process. The daemon is, deliberately, not an internal helper for the shell — it is a standalone user-space service, and the same `@-dispatched` `stryke` line that runs in-process can drive the daemon's whole op surface, the same surface a bash script or a Python program or a CI runner reaches over HTTP on port 7733. A `stryke` one-liner submitting a job, putting a value in the artifact cache, acquiring a named lock, or publishing a pub/sub event is hitting the identical control plane that `zsync up path` and `zwhere` and a `curl` against `/op/job_submit` all hit. There is one set of operations, exposed three ways — the Unix socket for shell clients, the HTTP port for everything else, and an in-process call path for `stryke` — and all three route to the same handler in the daemon. The shell hosts the scripting language; the daemon hosts the infrastructure; and `stryke`, sitting inside the shell, has the daemon's entire service catalog one builtin away, with no second process to start and no second protocol to learn. The composition stacks cleanly: a `stryke` script can take a lock, submit a job, poll it, cache the result, and publish a completion event, all against the daemon, without forking a single helper process.

The fork economics

It is worth dwelling on why “builtin or host op, not a separate process” is a rule and not a preference, because the numbers are stark at the scale this shell runs. A builtin executes in the shell's own address space and costs zero forks. An external command costs a `fork+exec` pair — two to five milliseconds each, before the program has done any of its own work. That gap is invisible on a single invocation and dominant in aggregate. The maintainer's history holds 478,000 commands; the development workflow runs sixteen synchronized shells broadcasting every keystroke; the daily command count runs into the millions. At that volume, the difference between a `coreutil` that forks and one that runs in-process is real wall-clock time measured in seconds per day, every day, and so twenty-three `coreutils` already live as builtins rather than as forks of the system binaries.

The same logic draws the line at which tools stay external. `tmux`, `fzf`, `ripgrep`, `eza`, `atuin`, `bat`, and `zoxide` run as `host.exec` children, because they are already good enough and a rewrite would duplicate a solved tool without adding anything. The gravity pulls toward in-process, but it is not a mandate to swallow everything — the rule is that a *new* feature defaults to living inside the shell as a builtin or a host op, reaching for “separate

process” only when there is a reason. The shell is the substrate; the question for any new capability is whether it belongs in the substrate or runs on top of it.

One runtime, not a subprocess

There is a temptation to read “shell hosts a scripting language” as a convenience — a fancier `eval`, or a subprocess with a nice prefix. It is neither. A subprocess would mean a fork, a serialization boundary, and two VMs that cannot share a stack frame: every `@`-line would pay a process spawn, every value crossing the boundary would be serialized to text or msgpack and reparsed on the other side, and the two languages could never share a call stack or a heap. `zshrs` and `stryke` share the `Op` enum and the `fusevm` runtime, which means an `@`-line is not marshalled across a process boundary and re-parsed by a foreign interpreter; it is lexed by `stryke`’s frontend, lowered to the same opcodes the shell compiler emits, and run on the same machine the shell pipeline runs on, with the same fused superinstructions and the same Cranelift JIT. Two language frontends converging on one bytecode VM inside one process, dispatched by a single sigil — that works only because both projects were built on the shared VM from the start rather than glued together afterward. A bolt-on integration could match the syntax but not the runtime sharing; the single VM is a consequence of the build graph, not a packaging veneer over two interpreters.

Hosting the stack

The concrete thing all of this exists to host is the maintainer’s own CLI environment — `zpw` and the completion corpus that sits on top of it. `zpw` is `.zshrc`-tier user code, which is to say it lives inside `zshrs` exactly the way your `.zshrc` does: the shell hosts it, the daemon catalogs its aliases and functions and completions, the recorder can attribute every one of its definitions to a file and line. The completion corpus is the stress case that validates the whole architecture — a recorder run absorbs everything from `zpw` plus `zsh-more-completions` plus `zinit` plus fifty plugins, a completion surface that ranges up to **27,387 files**. Under `zsh`, that surface is the three-to-five-second cold `compinit` and the perpetually-stale `.zcompdump`. Under `zshrs`, it is one recorder run, then `mmap`’d lookups in tens of nanoseconds, the daemon as the single source of truth, `fsnotify` keeping it current, and `zwhere` able to point at the exact line that defined any of it.

The shell that hosts a corpus that large, with that latency budget, with per-definition attribution across every plugin framework, is not a shell with a daemon bolted on. It is a runtime that happens to present a shell prompt — the god of all the processes that sit inside it.

Chapter 28 — The Single Directory

Most shells leave a trail. `zsh` scatters its working state across the filesystem the way a long-running campsite accumulates litter: `~/ .zcompdump` here, a `~/ .zcompdump-$HOST-$VERSION` variant there, a `.zcompdump.zwc` next to it, forty `.zwc` files seeded through a plugin tree, a multi-megabyte history file somewhere in a dotted directory, a `.zsh_sessions` folder you forgot you had. None of it is wrong, exactly. It is just unowned. No single process is responsible for any of it, no single directory contains it, and when one piece goes stale — the recurring “completion is stale, restart, still stale, `rm ~/ .zcompdump*`, restart again” ritual every `zsh` user has performed — there is nowhere to point and say *that* is the thing that broke.

`zshrs` takes the opposite position, and states it as a rule rather than a convention. Every file `zshrs` writes lives under one directory: `$ZSHRS_HOME`, defaulting to `~/ .zshrs/`. There is no second location to remember, no XDG split between cache and config and state, no per-host suffix scheme, no version-stamped variant accreting beside its predecessor. One directory holds everything, and `rm -rf ~/ .zshrs/` is the complete reset. The tradeoff is stated in the open: a full `rm -rf ~/ .zshrs/` nukes everything; the user accepts the loss. That is the price of the simplicity, and it is paid deliberately — because the alternative, an XDG-scattered layout with cache in one tree and state in another and config in a third, is the arrangement that produces the stale-`.zcompdump` ritual in the first place. State you have to gather from four directories before you can reason about it is state nobody owns. One directory is the precondition for one owner.

The contrast with what `zsh` leaves behind is not incidental. `zsh`’s litter is not a bug in `zsh`; it is the direct consequence of a design where every interactive shell is its own writer. Each shell that runs `compinit` may write a `.zcompdump`; each `zcompile` invocation drops a `.zwc` next to its source; each plugin manager seeds its own cache tree. No process is responsible for cleaning any of it because no process owns any of it. `zshrs` inverts that at the root: one writer, the daemon, and therefore one place it writes.

What lives there

The locked layout in `docs/DAEMON.md` is short enough to hold in your head:

```
~/ .zshrs/
├── index.rkyv          ← top-level fq_name → (shard_id, generation, byte_offset)
├── images/
│   ├── {hash8}-system.rkyv      ← system / shipped completions
│   ├── {hash8}-completions-corpus.rkyv ← zsh-more-completions
│   ├── {hash8}-zpwrc.rkyv       ← zpwrc functions / completions
│   ├── {hash8}-zshrc.rkyv       ← user .zshrc bytecode (per-user)
│   ├── {hash8}-plugin-{name}.rkyv ← per zinit / oh-my-zsh plugin
│   ├── {hash8}-script-{slug}.rkyv ← per `zshrs FILE` invocation
│   └── {name}.rkyv.lock         ← per-shard advisory flock
├── catalog.db           ← daemon-only writer; queryable mirror
├── history.db           ← daemon-only writer; SQLite FTS
├── zshrs.log            ← tracing output, daemon-rotated, 10 MB cap
├── daemon.sock          ← Unix socket for IPC
└── daemon.pid           ← singleton flock + daemon process ID
```

Nine kinds of thing, and they fall cleanly into three groups: the bytecode cache that the shell actually runs from, the SQLite mirrors that exist only so a human can ask questions, and the daemon’s coordination files.

Each group earns its place for a different reason, and keeping them distinct is what lets the single directory stay coherent instead of becoming a junk drawer.

The rkyv images — the only cache that runs

The shards under `images/` are the one part of `~/.zshrs/` that the running shell reads to do its work. They hold compiled bytecode — your `.zshrc`, the `zsh-more-completions` corpus, each zinit or oh-my-zsh plugin, each script you have run through `zshrs FILE` — serialized with `rkyv` into a byte-aligned format the shell can `mmap` and use without parsing or deserializing first. `zshrs` persists compiled chunks only in `rkyv` shards under `~/.zshrs/images/`. Only there. Nothing else is authoritative for what the shell executes.

The choice of `rkyv` over a more conventional serialization is itself load-bearing. A format like JSON or even bincode requires a deserialize pass that walks the bytes and rebuilds an in-memory structure before the data is usable; that pass scales with the size of the corpus, which for a 27,000-file completion set is exactly the cost `zshrs` exists to eliminate. `rkyv`'s archived representation is laid out so that the mapped bytes *are* the structure — a pointer into the `mmap` is a pointer into a valid value, with no rebuild step. The shell maps the file and reads through it. That is what makes “no parse, no deserialize” literally true rather than a figure of speech, and it is why the cache is `rkyv` and not a database.

`index.rkyv` sits above the shards as the lookup table: a fully-qualified name maps to a `(shard_id, generation, byte_offset)` triple. A completion lookup, an autoload, an alias expansion — each becomes a perfect-hash probe into the index followed by a single dereference into a mapped shard. The cost is ~150-200ns end-to-end, and the constraint that holds it there is an absolute: **no walking in clients**. Cache access is `hash(key) → mmap_index → value`, one indirection, no probing chains, no iteration over `rkyv` internals. The shards use perfect hashing generated daemon-side at compile time, so every key lands in a unique slot and the client never probes a chain on collision. That single rule is why the data plane stays in the nanosecond range instead of the microsecond range, and it is the reason the cache lives as `mmap`-able `rkyv` rather than as something you query. The moment a lookup has to iterate — to walk a chain, scan a list, traverse a tree — it leaves the nanosecond regime, and at 27,000 entries times a hundred shells the difference is the whole performance argument.

The shards carry a generation counter in each header. When the daemon rebuilds a shard it writes the new file and atomic-renames it into place *first*, then updates `index.rkyv` — strict ordering, because the reverse order produces torn reads. An already-running shell that has the old shard mapped keeps reading it safely; the kernel pins deleted-but-mapped inode pages alive until the last unmap, on both Linux and macOS. The next shell you launch sees the new generation. No client polls, no client races, no client ever holds a half-written shard.

The naming inside `images/` is deliberate too. Each shard is `{hash8}-{slug}.rkyv`, and the slug tells you at a glance what compiled corpus it holds: `system` for the shipped completions, `completions-corpus` for `zsh-more-completions`, `zpwir` for the user's own function and completion tree, `zshrc` for the per-user `.zshrc` bytecode, `plugin-{name}` for each zinit or oh-my-zsh plugin, `script-{slug}` for each file run through `zshrs FILE`. The corpus is sharded by *source*, not flattened into one monolith, which is what lets the daemon rebuild exactly the slice a file change affected and leave the rest mapped and untouched. Change one completion in your tree and only the `zpwir` shard rebuilds; the system corpus shard never moves. The `{name}.rkyv.lock` files are per-shard advisory `flocks` that keep two daemon worker threads from rebuilding the same shard at once.

This is the same cache that makes a warm start fast. The pipeline is direct: the first time a script runs, `zshrs` lexes it (a port of `Src/lex.c`), parses it (a port of `Src/parse.c`), compiles it to a fusevm `Chunk`, runs it, and *then* persists the chunk as an `rkyv` shard. Every run after that is `index.rkyv` lookup plus `mmap` plus deserialize plus run — no lex, no parse, no compile. The measured gap is wide: a cold cache-miss run that lexes, parses, compiles, writes the shard, and executes is on the order of 717ms; the warm cache-hit run that only deserializes and executes is about 7ms. The shards are the difference, and they are the only files in `~/.zshrs/` on that hot path.

Worth stating plainly: the shards are an accelerator, not a dependency. The source files stay authoritative. An image miss, a malformed shard, a corrupted index, or a daemon that is simply not running all resolve the same way — the client falls through to the source-interpretation path and runs the script directly, silently. The cache being absent or wrong slows a launch; it never breaks one. That is what lets the daemon own the entire cache without making the daemon a single point of failure for the shell itself.

The SQLite mirrors — for asking, not for running

Beside the shards sit `catalog.db` and `history.db`. They are SQLite, they are queryable, and they are emphatically **not** the cache. These files are read-only mirrors: never read when deciding cache hit/miss, never read when running compiled code. `catalog.db` is a joinable mirror of the catalog metadata, maintained by the daemon for human and tooling reads. `history.db` is command history, orthogonal to bytecode caching — not a cache layer for compiled chunks, just the persistent store behind `dbview history` and the FTS search the daemon serves.

The distinction is load-bearing, not pedantic. If the SQLite files were in the execution path, every Tab and every prompt fire would pay a cold-open or a query cost — that path runs 10-30µs per call, 50-150× slower than the mmap data plane, and putting the daemon or SQLite in the lookup path is wrong by construction. So they are kept out of it entirely. The shards run the shell; the databases let you *inspect* the shell. Two jobs, two file formats, one directory.

The reason both formats exist instead of one is that they answer different questions. rkyv is optimized for a single question asked a hundred million times a day — “what is the value at this key?” — and answers it in one dereference. SQLite is optimized for the questions a human asks rarely and ad-hoc: which functions haven’t been called this week, which completions match a glob, what did I run last Tuesday that touched `cargo`. A perfect-hash mmap cannot answer “all rows where `last_called_at` is older than seven days” without a full scan; a SQL index answers it directly. Forcing one format to do both jobs would either cripple the hot path with query machinery or cripple introspection with a structure you can only point-lookup. zshrs refuses the compromise and ships both, with a hard wall between them.

What you get for that separation is introspection without compromise. The `dbview` builtin browses the mirrors without you ever writing SQL:

```
dbview          # list tables + row counts
dbview autoloads _git    # single function: source, body, bytecode status
dbview comps git        # search completions
dbview history docker    # search history
```

Because `catalog.db` carries the bytecode in BLOB columns alongside the metadata, the mirror is a self-contained record of compiled state, and the two stores can each rebuild from the other. But the rebuild direction matters: rkyv is the hot-path source of truth, SQLite is the readable shadow. You can lose the databases and the shell keeps running off the shards; you query the databases precisely because doing so never touches what the shell is running.

The daemon’s three files

The last group — `daemon.sock`, `daemon.pid`, and `zshrs.log` — is the coordination layer. `daemon.sock` is the Unix socket over which thin shell clients talk to the singleton `zshrs-daemon`: length-prefixed JSON, a `[4-byte u32 BE length][JSON]` frame per message. `daemon.pid` holds the daemon’s process ID and doubles as the singleton guard — the first zshrs client to launch checks for `daemon.sock`, spawns the daemon if it is absent, and an `flock` on `daemon.pid` ensures exactly one daemon exists no matter how many shells race to start it. The user’s normal state is a hundred-plus zshrs panes in tmux; all of them connect to the one daemon that the first of them spawned.

`zshrs.log` is where the shell's voice goes. This is the mechanical consequence of the no-banner rule from the design goals: no init progress to the terminal — indexing, autoload pre-warm, fpath scan, plugin compile all go to `~/.zshrs/zshrs.log` via `tracing`, and the operator never sees them. Everything the shell might otherwise narrate — every compile pass, every shard rebuild, every fsnotify event — is `tracing` output written here, daemon-rotated at a 10 MB cap. The terminal stays silent so the prompt can appear instantly; the log captures the work so it is recoverable when you actually want it. The directory holds the shell's diary so the screen doesn't have to. The fsnotify watcher itself lives only daemon-side, exactly once for the whole machine: a hundred clients each running their own filesystem watcher would be a thundering herd, so the clients run none and the daemon's single watcher feeds them all.

Why one directory is an architecture, not a tidiness preference

It would be easy to read the single-directory rule as housekeeping — a nicer `rm` target, a cleaner `ls ~`. It is more than that. The rule falls directly out of the daemon being the sole writer. No client ever writes to a shard, to `catalog.db`, to `history.db`, to `daemon.pid`, to `daemon.sock`, or to `zshrs.log`. Client mutations land only in process state and a per-client in-memory overlay hash that dies on `exec zshrs`. One writer, one directory, one owner — the daemon — means there is never a question of which process produced a file or whether two of them disagree about it.

That single ownership is also what makes the directory portable. Because the daemon can serialize any of its caches, the whole of `~/.zshrs/` is exportable and re-importable — `zcache export --all` snapshots every target into one archive, `zcache import` restores it, freshness-validated before any merge. Individual targets serialize too: `zcache export path` emits an eval-compatible `path=(...)`, `zcache export catalog --out ~/backup.db` writes the whole catalog, `zcache export shard zpw --format json` dumps one shard as readable JSON. Diagnosing a misbehaving completion, comparing two machines' caches, migrating off `zsh+zinit+p10k`: all of them reduce to operations on the contents of this one directory. There is nothing scattered to gather first, because nothing was scattered.

The directory's permissions are part of the contract, not an afterthought. `~/.zshrs/` is `0700` and the files inside are `0600`, set at daemon startup and checked by `zcache verify`; some sourced files carry secrets — API keys, `export AWS_SECRET_ACCESS_KEY=...` — and the daemon flags those shards sensitive, maps them `MAP_PRIVATE`, and excludes them from `zcache export --all` unless you pass `--include-sensitive`. One directory means one place to lock down, and it is locked down by default.

And it is what finally retires the litter. Once you are on `zshrs`, every `.zwc` and `.zcompdump-*` file is dead disk litter that contributes zero speed. The reason is mechanical: `.zwc`'s entire purpose is to skip the source-parse step on cold load, but `zshrs` clients never parse source on cold load — the daemon parsed your sources once into `rkyv`, clients `mmap` the shards, and there is no source-parse step left for a `.zwc` to short-circuit. Even if a client *did* read a `.zwc`, deserializing it to an AST and re-executing would be more work than mapping indexed bytecode and running it. So the old artifacts are not just redundant, they are strictly slower than the path that replaced them. `zcache clean legacy` sweeps the `.zwc` and `.zcompdump` files out of the directories the daemon already knows about — never a blind `find ~`, only the `fpath`, plugin trees, and source targets it has on record — and `zcache verify`, which runs as part of `zcache info`, flags any that reappear. When the sweep is done, your entire daily-driver disk footprint for shell state is `~/.zshrs/`: one directory, one writer, queryable, exportable, and disposable in a single command. That is the whole point. State you cannot point at is state you cannot trust; `zshrs` gives you exactly one place to point.

Chapter 29 – Using zshrs

For all the architecture underneath it, zshrs is meant to disappear into your hands. You install it, you make it your login shell, and from then on it is just the shell — faster, quieter, but answering to the same `.zshrc`, the same plugins, the same muscle memory you have built over decades. This chapter is the practical one: how to get it on your machine, how to make it your shell, what a session actually looks like, and the handful of CLI verbs worth knowing.

Install

Three paths, pick the one that fits your machine. The Homebrew tap is the maintained route and auto-bumps on every release:

```
# Via Homebrew tap (auto-bumped by each release)
brew tap MenkeTechnologies/menketech
brew install zshrs          # core: zshrs + zd
# OR
brew install zshrs-all     # umbrella: zshrs + zd + zshrs-recorder + zshrs-daemon

# From crates.io
cargo install zshrs

# From source - lean build, pure shell, no stryke dependency
git clone https://github.com/MenkeTechnologies/zshrs
cd zshrs && cargo build --release
# binary: target/release/zshrs
```

The `zshrs` formula gives you the shell and `zd`, the daemon front-end. The `zshrs-all` umbrella adds the recorder and the standalone daemon binary — reach for it if you intend to run the recorder over your full init or run the daemon under `brew services`. The `cargo install zshrs` route is the one-line option when you already live in a Rust toolchain. Building from source produces `target/release/zshrs` and nothing extra; the source build is deliberately lean — pure shell, no stryke dependency pulled in.

The reason there is a lean build at all goes back to the architecture. stryke embeds inside zshrs, but it is the second priority, not a requirement; a shell that had to drag a whole second language runtime in to start would violate the cold-start budget. So the default build is the shell and only the shell — the parallel primitives, the VM, the JIT, the daemon coordination — with stryke as an opt-in, not a baseline cost. What you get from `cargo install zshrs` or a from-source build is the full-featured shell with nothing bolted on that you did not ask for.

If you build from source, note the project convention: `cargo build --release` is the release artifact. For local development you stay on plain `cargo build`. The binary is named `zshrs` in every case, and that single binary is also the language server, the debug adapter, the formatter, and the diagnostic tool — there is no second thing to install for any of those roles.

Set it as your login shell

A login shell has to be registered in `/etc/shells` before `chsh` will accept it. Two commands:

```
sudo sh -c 'echo "$(which zshrs)" >> /etc/shells'
chsh -s "$(which zshrs)"
```

The first appends the resolved path of your `zshrs` binary to the system's list of permitted login shells; the second points your account at it. Open a new terminal and you are in `zshrs`. Nothing else changes — your `~/.zshrc`, `~/.zshenv`, `~/.zprofile`, `~/.zlogin`, `~/.zlogout`, and the system-wide files in `/etc` are read in exactly the order `zsh` reads them, with `$ZDOTDIR` honored the same way. Source compatibility is maintained at the lexer, parser, builtin, parameter, glob, and ZLE layers, so existing `.zshrc`, `zinit/oh-my-zsh` plugins, and the `zsh-more-completions` corpus run unchanged.

This is the part that distinguishes a replacement from a port. A port reproduces `zsh`'s surface and then asks you to rewrite your config to its dialect; `zshrs` reads the config you already have. The startup-file sequence, the `--no-rcs` / `-f` behavior, the `PRIVILEGED` option's effect on which files get sourced — all of it tracks `zsh(1)` literally, because the lexer and parser underneath are faithful ports of `zsh`'s own `Src/lex.c` and `Src/parse.c` rather than a reimplementation that happens to look similar. Your decades-old `.zshrc` is the input, not the migration target. The first time you launch, the recorder teaches the daemon your starting state (covered below); from then on, every shell boots against that captured state.

What a session looks like

The first thing you will notice is what you don't see. There is no banner. No version stripe, no “Type exit to quit,” no first-run wizard, no welcome message, no update prompt, no telemetry notice. The prompt appears immediately on launch, and that is the entire startup experience. You open a terminal and the prompt is simply *there*.

The second thing you won't see is any sign of the work. On a cold start, `zshrs` is compiling and caching and pre-warming — and all of it goes to `~/.zshrs/zshrs.log` via `tracing`, never to your terminal. Indexing, autoload pre-warm, fpath scan, plugin compile: all logged, none printed. The first run does the heavy compilation once; every run after that reads your `.zshrc` from its rkyv image cache and starts in the 10-20ms target window — fully featured, not a faked prompt with work hidden behind it. First paint equals full functionality. There is no instant-prompt trick because the architecture removed the need for one: the prompt you see is backed by a shell that has already finished loading, not one that is still working in the background while showing you a placeholder. That distinction is the whole point of the cold-start number. A shell that paints a prompt in 1ms but isn't ready for 300ms has not started in 1ms; it has lied about starting. `zshrs`'s 10-20ms is the time to *ready*, with the 27,000-completion corpus registered, the full `.zshrc` loaded, `zpw`'s 506-plus subcommands available, and every hook installed.

For context on what that number means: vanilla `zsh` with no plugins starts in 30-50ms; `zsh` with `oh-my-zsh` runs 500ms to two seconds; `zsh` with `zinit-turbo` and `p10k` instant-prompt shows a visible prompt in 100-300ms but takes longer to reach full features. `zshrs` targets 10-20ms fully featured, below the 30ms human-reaction threshold, which is why it needs no instant-prompt layer to *feel* fast — it is fast at the moment it is usable.

What is left is a shell that behaves like a power user's shell and nothing else. No “did you mean” suggestions, no spell correction, no deprecation nags (deprecated builtins like `compctl` silently no-op for compat rather than scolding you), no “are you sure?” before `rm`, no tip-of-the-day, no `--help` hint appended to error messages. Errors arrive on `stderr` in `zsh`'s terse `zshrs: <command>: <reason>` format and stop there. This is not fish, and not `bash` with a friendly wrapper; it is a compiled VM that does what you typed and nothing more. The hostility to hand-holding is a design position, not an oversight — the audience is the operator who has typed these commands a million times and wants the shell out of the way.

The CLI surface

`zshrs` runs scripts and `-c` strings exactly as `zsh` does — `zshrs script.zsh`, `zshrs -c 'echo hi'`, `zshrs -s` to read from `stdin`, `-l/--login`, `-i`, `-f/--no-rcs`, `-v`, `-x`, `-o option`. The single-letter set tracks `zsh(1)`

literally, down to the upper-case `-H/-I/-J/...` family. Anything you would type after `zsh` you type after `zshrs` and it behaves the same. Past that baseline, a handful of `zshrs`-specific verbs are worth knowing.

`--doctor` is the health check. It prints a full diagnostic report — shell health, cache state, daemon presence, worker-pool status, integrity-scan results — and exits. Run it when something feels off, or after a fresh install to confirm the daemon came up and the caches are sound. Because the cache is an accelerator the shell can run without, a problem `--doctor` surfaces is rarely a shell that won't start; it is a shard that went stale or an integrity check that flags a mismatch, the sort of thing that silently degrades you to the slower source path without any visible error. The diagnostic is how you see that degradation, since the shell itself will never print it.

```
zshrs --doctor
```

The same report is available as the `doctor` builtin from inside a session, alongside `profile` for in-process command profiling at nanosecond resolution — both reach into runtime state (`profile`) and the daemon (`doctor`) that an external tool launched after the fact could not see.

`dbview` browses the read-only SQLite mirrors without you writing a line of SQL. It is a builtin, so you run it from inside a session:

```
dbview          # list tables + row counts
dbview autoloads _git # single function: source, body, bytecode status
dbview comps git   # search completions
dbview history docker # search history
```

These read the mirrors, never the `rkyv` cache — `dbview` is for inspecting state, not for changing what the shell runs.

Parity modes turn `zshrs` into a byte-for-byte stand-in for another shell. Each parity flag disables all caches and the daemon, so every `source` re-runs fresh and every `echo` re-fires — exactly what you want when you are running a compatibility suite or symlinking `/bin/sh` → `zshrs` in a container:

```
zshrs --bash      # drop-in for /bin/bash
zshrs --ksh       # drop-in for /bin/ksh (ksh-93)
zshrs --sh        # drop-in for /bin/sh / POSIX (alias of --posix)
zshrs --posix     # POSIX strict: no SQLite, no worker pool, no zsh extensions
zshrs --csh       # drop-in for /bin/csh
zshrs --zsh       # drop-in for /bin/zsh; entry point for compat tests
```

`--posix` is special: it gates the entire daemon/cache layer off completely — no daemon spawn, no cache directory created, no `z*` builtins. That is the mode that makes `/bin/sh` → `zshrs` safe in cron, init, and containers, where a shell that quietly spawned a background daemon would be a liability. The same gate fires on `emulate sh` and on an `argv[0]` basename of `sh`, `dash`, or `bash` — so symlinking the binary into a POSIX slot is enough, no flag required. The `--emulate` flag selects emulation by name (`zsh`, `sh`, `ksh`, `csh`, `bash`) for the cases where you want emulation without committing to the `argv[0]`-basename behavior.

The reason parity modes exist as first-class flags rather than a documentation footnote is the compat-test workload. `zshrs` is validated against real `.ztst` files from upstream `zsh` and against a corpus of every `sh/zsh` construct; running that suite means launching `zshrs` as a byte-exact stand-in for each reference shell, which is precisely what these flags provide. The same property that makes them useful for testing makes them useful in production: a CI image that needs `/bin/bash` semantics gets them from `zshrs --bash` with the cache layer off, so there is no first-run daemon spawn to surprise a sandboxed build.

The `--dump-*` introspection flags expose every pipeline stage to stdout, each accepting `-` for stdin:

```
zshrs --dump-tokens FILE # one TOKNAME<TAB>TOKSTR line per lexer token
zshrs --dump-ast FILE    # parser AST in canonical S-expression form
zshrs --dump-wordcode FILE # wordcode emitter output (EPROG/WORDS/WC[i]/STR5)
zshrs --dump-zwc ZWCFILE [FN] # inspect a compiled .zwc: index, or disassemble FN
zshrs --disasm           # print fusevm opcodes before each VM run (does not suppress
  ↵ execution)
```

`--disasm` is the one you reach for when you want to see the compiler at work — it prints the fusevm bytecode for each compiled unit before it runs, without stopping the run. The rest are forensic: tokens, AST, wordcode, and `.zwc` contents, in that pipeline order.

There is also a family of introspection dumps that feed editor tooling — `--dump-reflection` (JSON of builtins/keywords/options for the IDE reflection window), `--dump-plugins` (JSON of every sourced plugin grouped by manager: zinit, oh-my-zsh, prezto, antidote, antigen, zplug), `--names` (every LSP-known canonical name, one per line), and `--docs <name>` (render the LSP hover card for a name to the terminal, ANSI-colored, with `--color auto|always|never`). These exist mostly to back the editor integration, but `zshrs --docs print` is a direct way to read a hover card without leaving the shell. They are also what the `_zshrs` completion uses: the completer caches `zshrs --names` once per session so Tab-completing a `--docs` argument offers every known name without re-spawning the binary.

The exclusive builtins

Some of what makes a `zshrs` session different shows up only when you reach for it. The parallel primitives are builtins, not external tools, and they run their bodies through the same fusevm VM the shell runs everything else on — zero fork:

```
# Async/await – ship work to the pool, collect later
id=$(async 'sleep 5; curl https://api.example.com')
result=$(await $id)

# Parallel map – ordered output
pmap 'gzip {}' *.log

# Parallel filter
pgrep 'grep -q TODO {}' **/*.rs

# Parallel for-each – unordered, fire as completed
peach 'convert {} {}.png' *.svg

# Barrier – run all in parallel, wait for all
barrier 'npm test' ::: 'cargo test' ::: 'pytest'
```

The `{}` placeholder is substituted into the body, which is parsed and compiled to a fusevm `Chunk` once before iteration begins; each iteration sets the parameter, borrows a VM from a pool that preserves allocation capacity across runs, executes the cached chunk, and returns the VM. No fork, no per-iteration recompile, no per-iteration allocation. They are backed by a persistent worker pool of 2-18 threads, sized in `~/.zshrs/zshrs.toml` under `[worker_pool]`. This is why the parallel builtins are not just `xargs -P` in disguise: `xargs` forks a process per chunk and pays exec cost every time, while these run the compiled body on pooled VMs in-process, so the per-item cost is a VM checkout and a chunk execution rather than a `fork/exec/wait`.

The `intercept` family is the AOP layer — before/after/around advice on any command:

```
intercept before git { echo "[$(date)] git $INTERCEPT_ARGS" >> ~/git.log }
intercept after '*' { echo "$INTERCEPT_NAME took ${INTERCEPT_MS}ms" }
intercept around expensive_func {
    local cache=/tmp/cache_${INTERCEPT_ARGS// /_}
    if [[ -f $cache ]]; then cat $cache
    else intercept_proceed | tee $cache; fi
}
```

And the 23 anti-fork coreutils builtins — `cat`, `head`, `tail`, `wc`, `sort`, `find`, `uniq`, `cut`, `tr`, `seq`, `rev`, `tee`, `date`, `sleep`, `mktemp`, `hostname`, `uname`, `id`, `whoami`, and the rest — are why a session that pipes through these names never pays the 2-5ms fork-and-exec cost per invocation. You do not invoke them differently; they simply resolve in-process instead of forking `/bin/cat`. At the scale this shell targets — hundreds of thousands of commands a day, many of them pipelines that touch `cat` or `wc` or `date` — that fork cost is real wall time,

and eliminating it is the same architectural move as the no-fork worker pool: keep the work in-process on the VM, because a builtin costs zero forks and an external costs two-to-five milliseconds each. The default posture for any common operation is “this is a builtin,” and the coreutils set is that posture applied to the commands a shell pipeline reaches for most.

The daemon, and when you talk to it

Most of the time you never address the daemon directly — it spawned itself when your first shell launched, and every shell after that connected to it. The spawn is automatic and singleton: the first client to start checks for `daemon.sock`, fork-spawns the daemon if it is absent, and an `flock` on `daemon.pid` guarantees exactly one daemon exists no matter how many shells race to launch at once. A hundred tmux panes opening simultaneously produce one daemon and ninety-nine connections, not a hundred daemons.

When you do want to look at it, the `z*` builtin family is the front door. Every one of these is a thin length-prefixed-JSON IPC wrapper around the daemon with zero local logic and zero background threads — the client stays paper-thin. `zcache info` reports shard sizes, entry counts, and in-flight compile jobs; `zls` lists every live shell with its pid, tty, cwd, and tags; `zping` measures round-trip latency to the daemon, and `zping --all` reaches every registered shell. The cross-shell verbs — `zsend` to dispatch a command to another shell (by id, by `--tag`, by `--all` broadcast, or `--wait` for the captured result), `tag` to label this one, `znotify` for status-line messages — are there when you coordinate a hundred tmux panes, and absent from your attention when you don’t. The whole family is namespaced under `z` and build-time-checked against zsh’s own `z*` builtins (`zmv`, `zstyle`, `zcompile`, and the rest) so nothing collides with the compat floor.

One caveat worth internalizing: the entire `z*` layer is gated off in POSIX mode. Under `--posix` there is no daemon, no cache directory, and no `z*` builtins — which is exactly the property that makes `/bin/sh → zshrs` safe to drop into a container or a cron job, where a shell that spawned a background daemon would be a liability.

Running the daemon under brew services

If you installed the `zshrs-all` umbrella and want the daemon up at login rather than spawned lazily by your first shell, the Homebrew formula carries a `service` stanza for exactly that:

```
brew services start zshrs      # daemon starts now + on every login
brew services list             # status
brew services stop zshrs
brew services restart zshrs
```

`brew services` materializes the stanza into a launchd plist on macOS or a systemd user unit on Linux automatically, so you don’t hand-install either. This is optional — the spawn-on-demand path means the daemon comes up the moment it’s needed regardless — but pre-starting it shaves the daemon-spawn cost off your very first shell of the session, so your first pane is as fast as the hundredth. On a workstation that lives in tmux all day the difference is marginal, since the daemon spawned once hours ago and every shell since has just connected; the brew-services route matters most on a machine that opens a single shell, does one thing, and exits, where there is no later shell to amortize the spawn against.

Migrating from zsh: the recorder

The one workflow that is genuinely new is teaching the daemon your starting state. `zshrs` does not parse your `.zshrc` and every plugin in a static analysis pass; it *runs* your init under instrumentation and captures the resulting state. That is the job of the recorder — a one-shot binary (`zshrs-recorder`, in the `zshrs-all` umbrella) that executes your actual init under your actual plugin manager, observes every alias, function, env

var, `compdef`, `bindkey`, `zstyle`, path edit, and `source` it produces, and ships the captured bundle to the daemon.

You run it once, and again whenever your `.zshrc` or plugins change. The reason it beats static analysis is the shape of a real config: a `.zshrc` that switches plugin managers by variable (zinit, antigen, zplug, oh-my-zsh, sheldon, znap, and more, selected at runtime), zinit-turbo deferral that flushes state from background subshells, tens of thousands of completion files. A static analyzer would have to model every one of those plugin managers' syntax and every deferral path to derive the end state, and it would still be wrong wherever the config does something a parser can't predict. The recorder doesn't have to *understand* any of that — it runs your actual init under your actual plugin manager and observes the end state it reached, whatever path it took to get there. Every alias, function, env var, `compdef`, `bindkey`, `zstyle`, path edit, and `source` flows through AOP-instrumented dispatchers, including the ones inside deferred subshells, so the captured end-state matches the steady state after all turbo deferral settles. Once recorded, every subsequent shell boots fast because the canonical state is already sitting in the daemon's rkyv shards — the cold-start work was done once, at record time, not on every launch.

Your legacy history comes across the same way — `zcache import history <path>` ingests it once into `history.db`, and the old `HISTFILE` becomes a no-op. After migration, the `.zwc` and `.zcompdump` files scattered through your old tree are dead weight; `zcache clean legacy` sweeps them out. From that point your entire shell-state footprint is the single `~/.zshrs/` directory from Chapter 22.

Editor integration

The same binary is also a language server and a debug adapter. There are no extra crates to install and no separate process to manage — the LSP and DAP servers are built into `zshrs` itself.

```
zshrs --lsp           # LSP server over stdio
zshrs --dap HOST:PORT # DAP debugger; connect-back to IDE listener
zshrs --fmt [-w] [-t] [-i N] [FILE...] # format zsh source; -w rewrites in place
```

`zshrs --lsp` speaks JSON-RPC over stdio and any LSP client can drive it. The server advertises completion (builtins, keywords, options, special vars, in-file functions), hover cards, goto-definition and references, document highlight, document symbols, folding ranges, rename with `prepareRename`, semantic tokens, syntax-aware formatting, and live diagnostics on open/change/save. Completion fires on the trigger characters `$`, `{`, `-`, and `..`. The LSP and DAP modules are dependency-free additions — no `lsp-server`, `lsp-types`, or `dap-types` crates — with Content-Length framing and JSON-RPC hand-rolled on top of `serde_json` to keep the default build lean. That follows directly from the decade-scale dependency rule: every crate the build pulls in is a crate that has to keep building cleanly in 2035, so the protocol plumbing that a third-party crate would normally supply is written in-house against `serde_json`, which is already a foundational dependency. Wiring it into Helix or Neovim is a few lines:

```
# Helix - ~/.config/helix/languages.toml
[language-server.zshrs]
command = "zshrs"
args = ["--lsp"]
```

```
-- Neovim (nvim-lspconfig style)
require("lspconfig").configs.zshrs = {
  default_config = {
    cmd = { "zshrs", "--lsp" },
    filetypes = { "zsh", "sh" },
    root_dir = function() return vim.fn.getcwd() end,
  },
}
require("lspconfig").zshrs.setup({})
```

For JetBrains IDEs there is a dedicated plugin under `editors/intellij/` that drives both the LSP and the

DAP debugger — line breakpoints from the gutter, step over/into/out, a frames panel, a variables panel that understands scalars and arrays and associative arrays, and a reflection tool window that opens the `zshrs --docs <name>` card on click. It requires a paid JetBrains IDE on 2024.2+ because the platform LSP API is not in the Community editions.

`zshrs --fmt` is the formatter, and it is the same engine the LSP exposes as `formatting`: block-structure reindent plus idiomatic spacing — `a;b` becomes `a; b`, `a&&b` becomes `a && b`, `f ()` becomes `f()` — heredoc-safe and idempotent. With no files it reads stdin and writes stdout; `-w` rewrites in place; `-i N` sets indent width; `-t` indents with tabs.

That is the whole practical surface. Install it, `chsh` to it, and the rest is the shell you already know — only now it compiles.

Chapter 30 – Regular Expressions and the PCRE Module

Chapter 10 covered the `glob` — zsh’s native pattern language, the one that drives filename generation and `case` and the `${name#pat}` family. This chapter covers the other pattern surface, the one that looks like every other language’s regular expressions: the `=~` operator inside `[[ ... ]]`, and the `zsh/pcre` module. zsh historically implemented these as two separate optional modules — `zsh/regex` for `=~` and `zsh/pcre` for the explicit builtins — each binding a different system library (POSIX `regcomp(3)` or `libpcre`). zshrs ports both surfaces, and it unifies them on a single engine: the Rust `regex` crate, compiled in as a hard dependency rather than a feature-gated module you have to `zmodload` before it works.

The `=~` operator

Inside the extended-test brackets, `[[ string =~ pattern ]]` tests whether the regex on the right matches the string on the left. It is the form most scripts reach for, because it carries its captures out in shell variables:

```
if [[ "$version" =~ '([0-9]+)\.([0-9]+)\.([0-9]+)' ]]; then
    major=${match[1]}
    minor=${match[2]}
    patch=${match[3]}
    print "matched $MATCH → $major $minor $patch"
fi
```

On a successful match the operator sets four parameters, and the split between them is the detail to internalize. `$MATCH` holds the whole match — capture group zero. The `$match` array holds the *parenthesized* groups, one through N; the whole match is deliberately not element zero of `$match`, it lives in `$MATCH` on its own. Alongside them, `$mbegin` and `$mend` are parallel arrays giving the start and end offset of each group, with `$mbegin` using zsh’s one-based indexing so `$mbegin[1]` is the column where group one began. A failed match leaves the test false and the arrays cleared; a *malformed* pattern is distinct from a failed match — the operator returns exit status 2, exactly as zsh does, so a script can tell “did not match” apart from “that was not a valid regex.”

By default the pattern is read as a POSIX extended regular expression, matching zsh’s `zsh/regex` behavior. `setopt RE_MATCH_PCRE` flips the flavor to PCRE, which is the same toggle zsh exposes, and which changes how a handful of constructs are interpreted. The port carries zsh’s quirks deliberately rather than inheriting the Rust crate’s defaults: zsh’s regex treats `.` as matching a newline, while the Rust `regex` crate defaults `.` to stop at a newline, so the port forces `dot_matches_new_line` on so that `[[ $'a\nb' =~ 'a.b' ]]` behaves the way a zsh script written against the system library expects. POSIX-ERE bracket semantics get the same treatment — inside `[...]` a backslash is an ordinary class member under `regcomp`, not an escape, and the port translates the bracket contents into the crate’s class syntax so existing patterns keep their meaning. These are not cosmetic; they are pinned regression fixes, because a regex chapter that “mostly works” is a compatibility hole, and the whole point of the port is that a `.zshrc` that matched yesterday matches today.

The `zsh/pcre` module

The second surface is the explicit one: the `zsh/pcre` module, a faithful port of zsh’s `Src/Modules/pcre.c`, providing the `pcre_compile`, `pcre_study`, and `pcre_match` builtins. Where `=~` compiles and matches in

one step, the module separates the phases — you compile a pattern once, optionally `pcre_study` it, and then match it against many subjects:

```
zmodload zsh/pcre
pcre_compile '(\w+)@(\w+\.\w+)'
pcre_match 'user@example.com' && print "user=${match[1]} host=${match[2]}"
```

`pcre_match` reports the result through `$ZPCRE_OP` (the begin/end offset pair of the match) and fills `$match` with the captured groups, the same array `=~` uses, so the two surfaces share a mental model on the output side as well as the engine underneath. The compiled pattern is held per worker thread rather than in a single process-global — zsh's C source keeps one file-static `pcre2_code *pcre_pattern`, and under zshrs's threaded execution that single static becomes a thread-local so each worker's `pcre_compile/pcre_match` pair owns its own compiled regex without stepping on another thread's. The behavior-affecting options zsh's module honors — `RE_MATCH_PCRE`, `CASE_MATCH`, `BASH_REMATCH`, `KSH_ARRAYS`, multibyte — are read the same way, so case-insensitive matching, bash-style `$BASH_REMATCH` arrays, and ksh-style zero-based indexing all toggle as scripts expect.

One engine, and the tradeoff it makes

Both surfaces run on the Rust `regex` crate, and that choice has one consequence worth stating plainly rather than glossing. The crate is from the RE2 lineage: it compiles to a finite automaton and matches in time linear in the length of the input, with no backtracking. That buys a guarantee the C libraries cannot make — there is no catastrophic-backtracking input, no pattern a hostile user can submit that turns a match into a multi-second CPU spin, the ReDoS class of bug simply does not exist on this engine. The price of that guarantee is the two features a backtracking engine needs to support and an automaton cannot: backreferences (`\1` inside the pattern) and lookahead (`(?=...)`, `(?<=...)`). Patterns that lean on those will not compile, and that is a genuine difference from a script that ran against system PCRE. It is a deliberate trade — linear-time safety for two rarely-load-bearing features — and it is the kind of decision the Endgame premise from Chapter 3 forces into the open: a thirty-year shell should not ship an engine whose worst case is an attacker-controlled hang, even at the cost of a feature a handful of patterns use. The common case — character classes, anchors, alternation, quantifiers, capture groups, Unicode-aware matching on UTF-8 — is fully present and fast.

Chapter 31 – The Completion System in Depth

Of all the subsystems zshrs reimplements, the completion engine is the one with the largest surface area and the deepest behavioral debt to upstream zsh. A user who has spent millions of keystrokes pressing Tab has built muscle memory around a contract — what appears, in what order, grouped how, with which descriptions — that is defined nowhere in a spec and everywhere in the C source of `src/Zle/compcore.c`, `complete.c`, `computil.c`, `complist.c`, and the several hundred shell functions under `Completion/`. zshrs's `src/compsys/` module exists to honor that contract while running on a compiled VM instead of an interpreter. This chapter walks the layers from the keypress down to the matches, naming the real files and the real functions that do the work, and is explicit about where zshrs's implementation diverges from the tree it was ported from.

Two engines, one contract

There are, in fact, two distinct completion drivers in the tree, and confusing them is the first mistake to avoid. The first is the **compsys layer** — a faithful Rust port of zsh's shell-function completion system, living under `src/compsys/` with its ported function library under `src/compsys/ported/`. This is what fires at the interactive prompt when you press Tab: `_main_complete` runs, walks the completer chain, dispatches to `_arguments/_files/_describe`, and the matches it accumulates via `compadd` become the menu. The second is the **LSP/tiered flag engine** in `src/extensions/lsp.rs`, which serves completions to an editor over the Language Server Protocol — IntelliJ's tool window, for instance — by classifying the cursor position and pulling flag documentation from baked tables and scraped yodl doc bodies. The two share data but they are separate code paths with separate entry points. The compsys layer is the real shell behavior; the LSP engine is an editor-facing convenience that does not run the shell-function tree at all for the flag case.

The module header in `src/compsys/mod.rs` states the goal plainly: it implements zsh's new completion system with full compatibility for `compadd`, `compset`, `zstyle`, and all completion special parameters. The same file documents two operating modes. In **default mode** the cache is a SQLite-backed `~/.zshrs/compsys.db` populated by a parallel `fpath` scan, with autoload lookups served in roughly 2.7µs; no `.zcompdump` file is written. In **--zsh-compat mode** the traditional sequential scan runs and a `~/.zcompdump` is created, matching upstream behavior for debugging and compatibility. That dual-mode design is itself a divergence point: upstream zsh has exactly one completion backend.

How a Tab keypress is driven

The interactive path is documented at the top of `src/compsys/in_editor.rs` and visible in `complete_at`. When a completion request arrives, the function first snapshots the ZLE line, cursor (`ZLECS`), length (`ZLELL`), and `curcontext`, then repopulates them the way the interactive line editor would before firing Tab. It sets `curcontext` to the floor value `":::"`. It then installs a capture shadow — `COMPADD_CAPTURE_BUFFER` is set so that every subsequent `compadd` call routes its proposed matches into a buffer rather than into the live ZLE match list.

The actual engine invocation is deliberate and well-justified in the source. `complete_at` calls `docomplete(COMP_COMPLETE)` directly rather than the widget-level entry points. The comment explains

why: the interactive Tab default is `expandorcomplete`, which first attempts history/alias/parameter expansion via `doexpansion()` and only falls through to completion on no-match; inside the editor that first phase is wrong because history expansion shouldn't fire (the LSP isn't connected to the user's history stack) and parameter expansion would mutate the buffer in ways the IDE has no way to roll back. `completeword` is likewise rejected because it checks `LASTCHAR == '\t'` against stale ZLE state. `docomplete(COMP_COMPLETE)` is the shared back-half both widgets fall into: parse the line, populate `PREFIX/SUFFIX/IPREFIX/ISUFFIX/CURRENT/compstate`, run the before/after hooks, invoke `_main_complete`. After it returns, the capture buffer is drained, ZLE state is restored, and the matches are returned. This is the choke point: everything below `_main_complete` is the ported shell-function tree.

The router in `src/compsys/router.rs` decides whether a given `_NAME` call is served by a native Rust port or deferred to the autoloaded shell function. `try_rust_dispatch(name)` returns `None` unless `[compsys] backend = "rust"` is set in `~/.config/zshrs/config.toml`, the name begins with `_`, and a port is registered in `rust_compsys_lookup`. The dispatch table there is intentionally small — `_main_complete` and `_setup` — and the module header is candid that this router is zshrs-original with no C source counterpart, because C zsh has only one compsys backend. Critically, the doc comment warns that callers must run the returned function pointer inside `doshfunc`'s scope so the prologue/epilogue side effects (locallevel, FUNCSTACK push, `BREAKS/CONTFLLAG/LASTVAL` save+restore, trap state, pipestats deep-copy) apply identically to a Rust port and a wordcode body; skipping that would leak per-call scope state into the caller — a real bug. This is the philosophy of the whole port: native speed must not change observable scope semantics.

The `_function` convention and `#compdef`

Every completion function in zsh begins with an underscore, and its first line declares what command it completes. zshrs preserves this convention exactly. The `COMPSYS_FN_NAMES` table in `src/compsys/mod.rs` is the canonical inventory of functions with native Rust implementations — roughly fifty entries from `_all_labels` through `_widgets`, including the core dispatcher `_main_complete`, the spec engine `_arguments`, the description renderer `_describe`, and the file completers `_files/_directories/_path_files/_tilde_files`. The accompanying unit tests pin invariants on this list: every entry must start with `_`, the list must be sorted and free of duplicates, and specific functions — `_main_complete`, `_arguments`, the path completers, the per-command completers `_git/_docker/_cargo/_kubectl/_terraform`, and the tag machinery `_tags/_requested/_wanted/_all_labels/_next_label` — must be present or “the completion path is broken.”

The `#compdef` line is parsed in `src/compsys/ported/compinit.rs`. The `CompDef` enum enumerates the variants: `Commands(Vec<String>)` for `#compdef cmd1 cmd2`, `Pattern(String)` for `#compdef -p 'pattern'`, `PostPattern(String)` for `#compdef -P`, `KeyBinding` for `#compdef -k`, and `WidgetKey` for `#compdef -K`. The parser `parse_first_line` handles all of these including the special context entries like `#compdef -default-` and `#compdef -value-,VAR,-default-`. The result of a scan is a `CompInitResult` whose fields mirror the upstream associative arrays exactly: `comps` is the `_comps` hash (command → function), `services` is `_services`, `patcomps` is `_patcomps`, `postpatcomps` is `_postpatcomps`, and `compautos` is `_compautos`. The port even reproduces a subtle upstream quirk — `-P` patterns go directly into `_comps` (not `_patcomps`), because zsh puts these patterns as keys in the `_comps` hash.

`compadd` — the field model

`compadd` is the primitive every completion function ultimately calls to register matches, and zshrs ports it as `bin_compadd` in `src/ported/zle/complete.rs`. The first thing it does is enforce the upstream guard: if `INCOMPFUNC` is not set it emits “can only be called from completion function” and returns 1, mirroring the C check at `complete.c:608`. The field model is the one documented in the reference: each match is broken into seven fields — an ignored prefix from `IPREFIX` plus `-i`, then the visible prefix (`-P`), a hidden prefix (`-p`, considered part of the match but not shown when listing), the body, and the matching suffixes from `-s/-S/-I`. Flags like `-a` (completions are array names), `-k` (associative-array keys), `-d array` (per-completion display strings), `-J/-V` (group into a sorted/unordered group), `-Q` (don't quote), and `-f` (mark as filenames, honoring

`LIST_TYPES`) are all part of the documented contract.

The ported `bin_compadd` also carries shadow hooks that do not exist in upstream and exist purely to serve zshrs's extra surfaces. `COMPADD_PREFIX_INJECTOR` implements the `_approximate` prefix-injection trick faithfully — when set, it prepends a caller-supplied correction prefix to `$PREFIX` for the duration of one call, preserving a leading `~` so tilde expansion still works downstream, then restores `$PREFIX` exactly. `COMPADD_TRACE_ACTIVE` records calls into `_complete_help_funcs` and short-circuits, reproducing the `_complete_help` trace shadow. And `try_capture_compadd_argv` is the in-editor capture path: when the LSP buffer is installed, the call routes its matches into the buffer and mimics a “matches were added” status by returning 0, with parsing of captured matches happening in `crate::compsys::in_editor` so the ported file stays a faithful C port.

`_arguments`, `_describe`, `_files`

`_arguments` is, per its own header, the workhorse spec engine used by hundreds of per-command completion functions, living at `src/compsys/ported/Base/Utility/_arguments.rs`. The port models the spec grammar as a `Spec` enum: `Option { flag, gnu_long, desc, msg, action, optional_arg, repeatable }`, `Positional { idx, msg, action, optional }`, and `Rest { msg, action }`. The header is honest about scope — it covers the common forms `-flag[description]:msg:action`, bare boolean `-flag[description]`, positional `N:msg:action`, and rest `*:msg:action`. The flag parser consumes the standard option section (`-s`, `-S`, `-C`, `-O array`, `-A/-M`) before treating the rest as spec strings, and `parse_spec` handles the `*` repeatable prefix, the `--` GNU-long detection, the `[desc]` brace, and the `::` optional-argument form. The action dispatcher understands three action shapes: a literal `(val1 val2 val3)` list (which it feeds to `compadd -X msg -a`), the `->state` form (which sets `$state` for the caller and returns), and a bare shell-function dispatch like `_files` or `_users`. It reads `words` and `CURRENT` from the shell parameter table to locate the current word, and tracks consumed options including the `--flag=val` GNU form.

`_describe` renders a labeled list of `value:description` pairs and reproduces the upstream body line-by-line: the `getopts "o0t:12JVx"` flag parse, the options-mode `prefix-needed` short-circuit, the `verbose` and `list-separator` style reads, and the `_tags/_next_label` iteration that ends in a `compadd -d` call. It consults the same styles via `lookupstyle/testforstyle`: `prefix-needed`, `verbose`, `list-separator` (default `--`), and `max-matches-width`, building the genuine zsh context strings of the form `:completion:{curcontext}:{type}`. `_files` is the file/directory entry point: it implements the common-case flag parse for `-/` (directories only), `-f` (files only), and `-g <pattern>` (glob filter), defaults the dirs-only glob to `*(#q-/)`, reads the `file-patterns` style and, absent one, falls back to the standard two-pattern default `*(-/):directories` followed by `*:all-files`, then loops each pattern through `_tags/_next_label/_path_files`, leaving glob-qualifier dispatch and the recursive-files fine-tuning to `_path_files` itself.

`zstyle` and the configuration cascade

The whole system is steered by `zstyle`, and zshrs ports the style table from `Src/Modules/zutil.c` into `src/ported/modules/zutil.rs`. The two functions every completer calls are `lookupstyle(ctxt, style)` and `testforstyle(ctxt, style)`. `lookupstyle` locks the global `zstyletab` and returns the matched values, with weight order enforced at insert time so first-match wins — the equivalent of C's `getnode2` + `pattry` pattern walk. `testforstyle` returns 0 on a match and 1 otherwise, faithfully reproducing the C `return !found` convention so `[[]]`-style truth tests work. The cascade is exercised in the table's own tests: a `:completion:*` style and a more specific `:completion:zsh:*` style coexist, and a lookup against `:completion:zsh:complete` resolves to the narrower match while `:completion:bash:complete` falls back to the wildcard. The familiar examples — `zstyle ':completion:*' menu select` and `zstyle ':completion:*' matcher-list 'm:{a-zA-Z}={A-Za-z}'` — flow through exactly this table. Configuration also has a second source: `src/compsys/zpwr_colors.rs` parses `list-colors` and related `zstyles` directly to drive group headers and menu-selection coloring.

compinit, the parallel scan, and the cache

`compinit` itself is where `zshrs`'s architecture most visibly departs from upstream. After installing `_comp_dumpfile`, publishing `_comp_options` and `_comp_setup`, and initializing the pre/post-hook arrays (all pinned to upstream line numbers), it scans `fpath` in parallel using rayon: `fpath.par_iter()` filters out `.` and nonexistent directories, flat-maps `scan_directory` across them, and dedupes function names with a shared `Mutex<HashSet>` where the first one wins. Upstream `compinit` is sequential; the `compinit_bg.rs` extension goes further still, running the scan on a background thread whose results are merged via a non-blocking `drain_compinit_bg` — which is why the reference describes `zshrs`'s `compinit` as `async`: `fpath` is scanned and bytecode-compiled in the background while the prompt drops.

The cache is the other divergence. The `compsys` README and `cache.rs` are emphatic that the authoritative warm-path store is `rkvv-mmmap'd bytecode shards` under `~/.zshrs/*.rkvv`, not SQLite. The README's architecture section stresses that SQLite exists only as read-only mirrors (joinable rows, optional FTS materializations) for humans and `dbview`, and that the mirrors do not define or influence the shell completion cache. The mirror schema creates exactly the tables the README documents — `autoloads`, `comps`, `patcomps`, `services`, `executables`, `zstyles`, `shell_functions` — but bytecode is held separately in the `rkvv` autoload-cache shard. This is the structural answer to the upstream pain of rescanning and reparsing `fpath` on every init: `zshrs` consumes daemon-built images and only compiles on a miss.

Where the LSP tiered engine differs

The editor-facing engine in `src/extensions/lsp.rs` does not run the shell-function tree for flag completion at all. `lsp_completion_context(line, col)` walks the line backward and classifies the cursor into one of many `LspCompletionContext` variants — `BuiltinFlag(name)`, `BuiltinLongFlag(name)`, `ZstyleContext`, `CompdefFn`, `TestOperator`, `MathFunction`, `SubscriptFlag`, `SignalName`, `WidgetName`, and more. For the `BuiltinFlag` case, `extract_builtin_flags` resolves flags through a tiered cascade, each tier cached per-name in a `Mutex<HashMap>`: Tier 0a is a hand table for `zshrs/zsh` itself; Tier 0 is a hand-curated `compsys`-flag table derived from `man zshcompsys`, used because that yodl source uses signature-style headers rather than bullet lists so the scrapers find nothing; Tier 1 is a regex over the builtin's markdown doc body matching bullet-style `- *-X*- desc` entries; Tier 2 is an inline-citation fallback for builtins whose docs describe flags in prose; and Tier 3 merges a hand-curated `BUILTIN_FLAG_DOCS_OVERRIDE` table sourced from `man zshall`, where overrides win on flag-letter collisions and body-scraped entries fill the gaps, pinned by `tests/lsp_man_audit.rs`. The result becomes LSP completion items with an explicit `TextEdit` range covering the typed dash so `print -<tab>` replaces the dash and yields `print -l` rather than `print --l`. This baked-table, scrape-and-merge approach is entirely a `zshrs` invention; it gives the IDE deterministic, sub-keystroke flag completion without spinning up the full `compsys` engine, at the cost of being a separate code path that must be kept in agreement with the real shell behavior.

Chapter 32 – Job Control and Backgrounding

A shell is, at its core, a process orchestrator. Everything else — expansion, globbing, the parameter system — exists to assemble argument vectors that are eventually handed to processes, and once those processes exist the shell has to track them: which are running, which have stopped, which have exited and with what status, and which one currently owns the terminal. POSIX shells call this machinery *job control*, and it is one of the oldest and most intricate parts of any Bourne-family shell. zsh’s implementation lives almost entirely in `Src/jobs.c`, roughly three thousand lines of C built around a single flat array, `jobtab[]`, indexed by job number. zshrs ports that file directly. The port lives at `src/ported/jobs.rs` and it is a *faithful* port, not a reinterpretation: nearly every function carries a `c:NNNN` comment pinning it to the exact line of the C original it reproduces, the status-flag bit values are copied verbatim rather than re-derived, and the control flow of `bin_fg`, `getjob`, `spawnjob`, `update_bg_job` and the rest mirrors the C statement-for-statement. This chapter walks the job-control subsystem as it actually exists in the source, and is careful to distinguish the parts that are genuine ports of zsh’s POSIX behavior from the parts that are zshrs-original — because zshrs is a non-forking, worker-pool shell, and the interaction between classic fork-based job control and that architecture is exactly the place where assumptions go wrong.

The job table and the status flags

The central data structure is `struct job`, ported through `crate::ported::zsh_h::job`, and a parallel `struct process`. A job is a pipeline — `cmd1 | cmd2 | cmd3` is one job containing three process entries — and the job carries a status word, a process-group leader pid (`gleader`), display text, and the list of its constituent process entries in `procs`. The status word is a bitfield, and the port is emphatic about getting the bit values *exactly* right; the header comment records that a previous Rust port used sequential bit shifts (`1<<0`, `1<<1`, ...) which produced different values from C for every flag, so any data ferried between the C-side semantics and the Rust side would misinterpret every stat-flag check. The values are now canonical: `stat::CHANGED = 0x0001`, `stat::STOPPED = 0x0002`, `stat::DONE = 0x0008`, `stat::INUSE = 0x0040`, `stat::SUPERJOB = 0x0080`, `stat::DISOWN = 0x10000`, each annotated with its `Src/zsh.h` line. These flags are the contract between the executor that creates jobs, the signal handler that reaps them, and the `jobs/fg/bg` builtins that report on them. The `process` impl reproduces the `wait(2)` status macros directly: `is_stopped()` is `self.status & 0xff == 0x7f` (C’s `WIFSTOPPED`), `exit_status()` is `(self.status >> 8) & 0xff` (`WEXITSTATUS`), `term_sig()` is `self.status & 0x7f` (`WTERMSIG`). The table itself is `JOBTAB`, a `OnceLock<Mutex<Vec<job>>>`, with the shell’s own slot reserved at index 0 and real jobs allocated from index 1 upward by `initjob`. Three globals track the *named* jobs the `%`-syntax resolves against: `CURJOB` (`%+`), `PREVJOB` (`%-`), and `THISJOB` (the job the shell is currently building, deliberately excluded from listings).

How & backgrounds a job

When the parser sees a list terminated by `&`, it does not emit a special node type; it sets a flag. The `async/sync/disown` discrimination at `parse.c` maps to zshrs’s `ListFlags { async_, disown }` field — the default is `sync`, `&` sets `async_`, and `&!/&|` (zsh’s two spellings of “background *and* immediately disown”) set both. The compiler in `src/extensions/compile_zsh.rs` consumes the `async_` flag: it compiles the

sublist into a self-contained sub-chunk, records that sub-chunk's index, reconstructs the job's display text at compile time, pushes text and sub-chunk index onto the VM stack, and emits `CallBuiltin(BUILTIN_RUN_BG, 2)`.

The `BUILTIN_RUN_BG` handler is in `src/fusevm_bridge.rs`, and this is the load-bearing answer to the question “does zshrs fork for background jobs?” It does. The handler calls `libc::fork()` directly. The child detaches with `setsid()` so a `SIGINT` aimed at the foreground job does not also kill the background one, builds a *fresh* VM over the sub-chunk, re-registers all builtins on it (a forked child cannot safely share the parent's builtin registry), runs the bytecode, and exits with the resulting status. The parent does the bookkeeping that makes the job visible to the rest of the subsystem: it stores the child pid into `lastpid` (the atomic backing `$!`), then enters the canonical jobtab path — `initjob` allocates a slot, `addproc` hangs a process entry carrying the pid and display text off it, the slot is marked `STAT_NOSTTY`, `THISJOB` is set to the new slot, and `spawnjob()` promotes the new job to `CURJOB`. Each of these is annotated with the corresponding `Src/exec.c` line. The parent returns immediately, which is the defining behavior of `&`: the shell does not wait.

There is a second, distinct background path. `execute_external_bg` in `src/vm_helper.rs` spawns externals via `std::process::Command` — `command.spawn()` for the background case (registering the child in an executor-side `JobTable` and printing the `[job] pid` line), or `command.status()` for the foreground case. This is the executor's own safe-Rust process tracker, deliberately segregated from the C-faithful jobtab. The reference notes the consequence honestly: pids spawned via `BUILTIN_RUN_BG` land in the canonical jobtab, while externally launched bg commands via `Command::spawn()` land in the executor-side `JobTable`, and unifying the two is tracked as Phase G6 work. Both paths fork at the OS level; the difference is which in-process registry owns the bookkeeping.

Job specs and the `fg/bg/jobs/wait/disown` dispatcher

The `%`-syntax that `fg`, `bg`, `wait`, `disown` and `kill` accept is resolved by `getjob`, a direct port of `Src/jobs.c:2063`. The parsing is a small state machine over the spec string: anything not beginning with `%` is treated as a job *name* and dispatched to `findjobnam` (which scans for a job whose first process's command text starts with the string); after the leading `%`, `%%`/`%+`/bare `%` resolve to `CURJOB`, `%-` to `PREVJOB`, a digit is validated against `MAXJOB`, and `%{str}` triggers a substring search through every job's process text. Each failure path emits the canonical `zwarnnam` diagnostic (“no current job”, “%N: no such job”, and so on), suppressed when `POSIXBUILTINS` is set.

These five builtins are not five functions — they are one function, `bin_fg`, dispatched on a `func` discriminant (`BIN_FG`, `BIN_BG`, `BIN_JOBS`, `BIN_WAIT`, `BIN_DISOWN`), exactly as zsh routes all of them through a single `bin_fg`. The shared preamble reaps finished children before anything is listed or dispatched, then gates `fg/bg` behind job control being enabled (`!jobbing` emits “no job control in this shell” where `jobbing` is `isset(MONITOR)`). With no arguments, `BIN_JOBS` lists the table skipping the slot equal to `THISJOB` (the port carries a careful note that this must be `thisjob`, not `curjob`, because `spawnjob` promotes freshly backgrounded jobs to `curjob` and skipping that would hide every just-launched `&` job). `fg/bg` send a `SIGCONT` through `killjb`, which delivers to the job's process group. `disown` calls `deletejob(j, true)` where the `true` disowning flag suppresses closing fds and unlinking the job's temp files, so a disowned job's process-substitution fds survive. `wait` is the most involved: rather than hold the `JOBTAB` mutex across a suspend (which would deadlock against the `SIGCHLD` handler's own lock), it loops, finds the next running pid under a short-lived lock, drops the lock, blocks in `waitpid`, and routes the reaped status through `update_bg_job`.

Process groups, terminal control, and what does not fork

Foreground job control means handing the controlling terminal back and forth between the shell and whatever job currently owns it. The port centralizes this in `attachtty`, the canonical wrapper around `tcsetpgrp(2)` and a near-line-for-line port of `Src/utls.c`: it no-ops unless `jobbing()` && `interact()`, reads the saved

shell tty fd `SHTTY`, calls `tcsetpgrp(shtty, pgrp)`, and on failure walks C's exact recovery (reattach the shell's own group if the target is dead; warn, disable `MONITOR`, and latch a flag for anything but `ENOTTY`). `suspend` is the inverse — the shell suspends itself, refusing on a login shell unless `-f` is given, restoring default disposition for `SIGTTIN/SIGTSTP/SIGTTOU` and `killpg(origpggrp, SIGTSTP)` to stop, reacquiring the group on resume. `#!` is read directly from the `lastpid` atomic rather than the parameter table, because `!` is not a valid identifier and never enters `paramtab`; the `BUILTIN_RUN_BG` parent is the single canonical writer of that atomic.

The architectural subtlety is the worker pool. `zshrs` carries a persistent thread pool (`src/extensions/worker.rs`) whose header is explicit that it is *not* a port of any `Src/*.c` file: C `zsh`'s background-work strategy is `fork(2)` — every completion run, process substitution, or command substitution is a child process — and `zshrs` replaces that pattern with a fixed-size thread pool plus crossbeam channel dispatch, sized to `available_parallelism()` clamped to `[2, 18]`. The pool exists to eliminate fork overhead for the shell's *own internal* background work (`compinit_bg.rs` runs on it, for instance); it is *not* what backgrounds a `&` job. Classic job control still forks, because a job-controlled background process must be a real OS process with its own pid and process group that the user can `kill`, `fg`, and `disown` — a thread cannot satisfy that contract. Where the pool surfaces as a job-control-shaped feature is the `zshrs`-original `async/await` pair: `async 'cmd'` submits the command to the worker pool (which runs it in a captured subprocess) and returns an integer job id immediately, while `await ID` blocks on the channel for that id and prints the captured output. The reference states the distinction plainly — unlike `&`, this runs on a thread within the same process, no fork, no `setsid`. So `zshrs` offers two background models side by side: the POSIX one (`&`, forked, jobtab-tracked, signalable, `#!`-addressable) and the pool one (`async/await`, output-capturing, id-addressable, no terminal interaction). They are complements, not substitutes.

Chapter 33 – The Ledger: History as a Database

Every shell keeps history, and for fifty years every shell has kept it the same way: a flat text file, one command per line, appended to on exit and read back in on startup, searched by walking it end to end. zsh's version of this lives in `Src/hist.c` — a doubly-linked ring of `histent` structs in memory, a `$HISTFILE` on disk, and a linear scan for anything you want to find. zshrs ports that file faithfully, because the *interface* to history is decades of muscle memory that cannot break: the `fc` builtin, the `!!` and `!$` event designators, `$HISTFILE` and `$SAVEHIST`, `setopt SHARE_HISTORY`, `HIST_IGNORE_DUPS`, the `zshaddhistory` hook. All of that is preserved verbatim in `src/ported/hist.rs`, function by function, each pinned to its `Src/hist.c` line — `savehistfile` at `c:1192`, `readhistfile` at `c:1528`, the lock discipline around `lockhistfile/unlockhistfile`. But underneath that faithful surface, zshrs replaces the storage engine entirely. The linked list and the flat file become a SQLite database, and the linear scan becomes an indexed query. This chapter is about that substitution — what it keeps, what it changes, and why a shell that compiles to bytecode also stores its past in a real database.

Why a flat file was always the wrong shape

A flat history file is the right data structure for exactly one operation: append. Everything else it does badly. Searching it means reading every line — an $O(N)$ scan that gets slower every day you use the shell, which is why a heavy user's reverse-*i*-search visibly lags once the file crosses a few hundred thousand lines. Deduplicating it means either scanning the whole file on every write (`HIST_IGNORE_ALL_DUPS` in zsh is exactly this cost) or tolerating duplicates. Asking questions a flat file has no columns for — *what did I run in this directory*, *how often do I run this*, *what was the exit code* — is impossible, because the file records only the text and, with `EXTENDED_HISTORY`, a timestamp and duration. The flat file is a log, and a log is not a database; you cannot join it, index it, or query it, only replay it.

zshrs's `src/extensions/history.rs` states the swap in its own header: C zsh keeps history in a flat file plus an in-memory linked list, and zshrs replaces both with SQLite for two reasons — FTS5 full-text search turns fuzzy matching from an $O(N)$ scan into a microsecond-latency index lookup, and frequency, recency, and per-directory tracking can layer onto the same row instead of demanding parallel files. The interactive surface is untouched: `fc`, `$HISTFILE` semantics, `SHARE_HISTORY` all keep the C source's behavior. Only the substrate changes.

The schema

The `HistoryEngine::init_schema` method is the whole design in a dozen lines of SQL. There is one table, `history`, and it carries columns a flat file never could:

```
CREATE TABLE IF NOT EXISTS history (
  id          INTEGER PRIMARY KEY,
  command     TEXT NOT NULL,
  timestamp   INTEGER NOT NULL,
  duration_ms INTEGER,
  exit_code   INTEGER,
  cwd         TEXT,
  frequency   INTEGER DEFAULT 1
);
CREATE INDEX IF NOT EXISTS idx_history_timestamp ON history(timestamp DESC);
CREATE INDEX IF NOT EXISTS idx_history_cwd       ON history(cwd);
CREATE UNIQUE INDEX IF NOT EXISTS idx_history_command ON history(command);
```

Each column is a capability. `duration_ms` and `exit_code` are captured from the `preexec/precmd` hook pair — the shell notes the monotonic clock when a command starts and the wall time and status when it ends — so the ledger records not just what you ran but how long it took and whether it worked. `cwd` records where you ran it, which is what makes per-directory history possible. `frequency` counts how many times a command has been run. And the `UNIQUE INDEX` on `command` is the quiet linchpin: deduplication is not a scan `zshrs` performs, it is a constraint the database enforces. When you re-run a command that already exists, `add` does not insert a second row; it issues an `UPDATE` that bumps the row's `timestamp` to now, increments `frequency`, and refreshes `cwd` — so the single row for `git status` carries a running count of how often you have ever run it and the last time you did. `HIST_IGNORE_ALL_DUPS`, which in `zsh` costs a full-file rescan, is here a free by-product of a unique index.

FTS5 and the trigram tokenizer

The search story is a second table:

```
CREATE VIRTUAL TABLE IF NOT EXISTS history_fts USING fts5(
  command, content='history', content_rowid='id', tokenize='trigram'
);
```

FTS5 is SQLite's full-text search engine, and `history_fts` is an *external-content* index — it stores no data of its own, only an inverted index that points back into the `history` table by rowid, kept in sync by three triggers that fire on insert, delete, and update. The choice that matters is `tokenize='trigram'`. A word tokenizer would only match on whole-word boundaries, which is wrong for shell history, where you want to find `kubect`l by typing `ubect`l or `--all-namespaces` by typing a fragment in the middle. A trigram tokenizer indexes every three-character substring, so any substring query of three or more characters resolves through the index rather than a scan. This is what gives `zshrs` fzf-quality fuzzy matching over half a million commands at microsecond latency — the `search` method escapes the query, wraps it as an FTS5 prefix pattern, joins `history_fts` back to `history`, and orders the results by `frequency DESC`, `timestamp DESC`, so the command you run most and most recently floats to the top. That ordering — frequency first, then recency — is the same ranking `fasd` and `zoxide` made popular for directories, applied to command history and computed by the database rather than a userspace daemon walking a log.

The text mirror, and honesty about two writers

There is a subtlety a database-backed history has to confront that a flat-file shell never did: some tools, and some users, still expect a plain-text `$HISTFILE` they can `grep`, edit, or sync between machines. `zshrs` does not abandon that contract. It keeps a text mirror, and `rehydrate_text_if_stale` handles the migration case — if the SQLite database has rows but the text file is empty (a fresh checkout, or a user who deleted the flat file), it regenerates the entire text history from the database in timestamp order, formatting each line in the `EXTENDED_HISTORY` shape `zsh` uses, so the user-facing file is whole from day one rather than starting empty and slowly refilling. The database is the index of record; the text file is a materialized view of it. This is the

same relationship the daemon's rkyv shards have with `catalog.db` — a fast primary and a human-readable mirror, each reconstructable from the other — applied to history.

None of this changes what `SHARE_HISTORY` means or when a line is written; those semantics come straight from the ported `hist.rs`, with its lock file and its append-on-newline timing intact. What changes is that the append lands in a row with six more columns than a line ever had, indexed three ways, deduplicated by constraint, and searchable by trigram. The shell you type into is the same shell; the memory behind it finally has a shape you can ask questions of.

Chapter 34 – The Prompt

The prompt is the one piece of a shell the user looks at more than any other, and it is also, structurally, one of the most fork-hungry. A modern prompt is not a static string. It is a small program that runs on every keystroke-completed line — resolving the current directory, the git branch and dirty state, the exit status of the last command, the time, the battery, the kubernetes context — and in a traditional shell every one of those dynamic pieces is a command substitution, which is to say a fork. A single redraw of a heavily-themed prompt can fire a dozen `$(...)` subshells, and it fires them again every time you press Enter. This is why prompt frameworks became a byword for shell slowness, and why the community’s answer — Powerlevel10k’s instant prompt — was to paint a fake prompt immediately and fill in the real one behind it. This chapter is about how zshrs renders a prompt that is both faithful to zsh’s escape language and free of the fork tax that made that language slow, and why it refuses the instant-prompt trick on principle.

Prompt expansion, ported whole

zsh’s prompt escape language is large and old: `%~` for the working directory, `%n` for the user, `%m` for the host, `%?` for the last exit status, `%F{red}...%f` for color, `%B...%b` for bold, `%(?.√.X)` for the conditional ternary, `%D{...}` for `strftime` dates, truncation specs like `%20<...<`. All of it is defined in `Src/prompt.c`, and zshrs ports that file directly in `src/ported/prompt.rs`, right down to the central recursive routine `putpromptchar` (a port of `Src/prompt.c:615`) that walks the format string one escape at a time, and `promptexpand` (`Src/prompt.c:182`) that drives it. The escape handling is line-for-line: `%` followed by `(` enters the ternary parser, a leading `-` after `%` toggles the truncation direction, `%c/%./%-/%C` all route through `promptpath` with the right home-directory-stripping flag. Because the parser is the ported one, `PROMPT_SUBST`, `PROMPT_PERCENT`, and `PROMPT_BANG` behave exactly as their zsh options dictate — `PROMPTPERCENT` gates whether `%` escapes are interpreted at all, and the port checks `isset(PROMPTPERCENT)` at the top of the loop exactly where C does.

One porting detail is worth surfacing because it shows how carefully the boundary is drawn. C’s `prompt.c` reads a scatter of file-scope globals during expansion — the logical `$PWD`, the last exit status, `curhist`, the job table, the function stack — because in C those are simply ambient. Rust has no ambient globals, so `src/ported/prompt.rs` introduces a `prompt_tls` thread-local block that mirrors each of those values, with a `sync_from_globals` call that refreshes them from the canonical parameter table and job table immediately before an expansion walk. The prompt code reads its inputs from that snapshot, so a single redraw sees a coherent view of the shell’s state rather than values that might shift under it mid-expansion. The escape semantics are zsh’s; the plumbing that feeds them is zshrs’s, and the seam between the two is explicit in the source.

The dynamic pieces run without forking

The escapes above are the cheap part — string formatting over shell state the prompt code already has in hand. The expensive part of a real prompt is the `$(...)` substitutions a theme wraps around them: `$(git rev-parse ...)`, `$(kubectl config current-context)`, `$(date +%H:%M)`. In every traditional shell each of those forks a child, and a prompt that fires a dozen of them per redraw pays a dozen fork/exec chains — two to five milliseconds each — before the cursor is ready. This is the single largest reason a themed zsh feels laggy, and it is the reason the whole architecture of Chapter 2 matters most acutely right here.

zshrs’s answer is the `dup2` command-substitution capture from the no-fork chapter, applied where it is felt

most. A `$(date)` in a prompt does not fork: `date` is one of the 23 in-process coreutils, so the substitution rewires the shell's own stdout to a buffer, runs the builtin, and reads the result back — four syscalls, no process. `$(basename ...)`, `$(whoami)`, `$(hostname)` in a prompt are the same story. The substitutions a prompt fires most often are exactly the small, frequent commands zshrs made into builtins, so the prompt is exactly where the 2000-to-5000x per-call speedup compounds hardest. A prompt that would cost a zsh user twelve forks per Enter costs a zshrs user, for the builtin-backed pieces, zero. The genuinely external pieces — a `git` invocation, a `kubectl` call — still fork, because `git` is not a builtin, but even those can be moved off the critical path: an async prompt segment ships the work to the worker pool via `async` and fills the segment in when the result arrives, without the fork blocking the first paint.

No instant-prompt, because there is nothing to fake

Which brings the chapter to the principle. Powerlevel10k's instant prompt exists because zsh cannot render a working prompt fast enough, so it renders a *placeholder* — a prompt drawn from a cached snapshot of last session's state — and swaps in the real one once initialization finishes. The cost is a specific dishonesty: the prompt you see first is not connected to a shell that can do anything yet, and a command typed into it may stall while deferred initialization catches up. zshrs rejects this outright, and the rejection is not aesthetic — it is the same first-paint-equals-full-functionality rule that governs the whole project. Because the `.zshrc` is mmap'd bytecode rather than re-parsed source, and because the prompt's dynamic pieces run in-process rather than forking, the real prompt is ready in the ten-to-twenty-millisecond window that instant-prompt was invented to paper over. There is no gap to hide, so there is no placeholder. A transient prompt — collapsing a previous line's rich prompt to a minimal one after you submit it — is offered, because that is a genuine display choice the user makes, not a latency mask. But the first prompt zshrs paints is the whole prompt, wired to a whole shell. The prompt is the most visible place the architecture's honesty shows, precisely because it is the place every other shell chose to fake.

Powerlevel10k, absorbed into the binary

The sharpest illustration of everything this chapter argues is what zshrs does with Powerlevel10k itself. p10k is roughly thirteen thousand lines of metaprogrammed zsh, and it is the most-installed prompt theme in the world — a compatibility layer whose very existence is a monument to how much work it takes to make zsh's prompt fast and rich at once. Its headline feature, instant prompt, exists solely to hide the cost of running those thirteen thousand lines through an interpreter on every startup, and it ships a separate C++ daemon, `gitstatusd`, because computing git status in shell script is too slow to do per prompt. zshrs takes the theme not as something to load but as a specification to port.

`src/extensions/p10k/` is an in-process Rust segment engine — 12,211 lines across eleven files — that renders the prompt the theme describes without the theme ever executing. Sourcing `powerlevel10k.zsh-theme` is intercepted at builtin dispatch, so the zsh theme code never runs; the user's `.p10k.zsh` still sources normally, its `POWERLEVEL9K_*` typesets landing in the parameter table where the engine reads them back through p10k's own fallback chain, so an unchanged config drives the native engine as a drop-in. Eighty-one segment builders are ported line-cited from the theme source (`// p10k:NNN`) — `directory`, `vcs`, `prompt_char`, `context`, `battery`, `kubecontext`, `aws`, `virtualenv`, and the rest — and the user-defined `prompt_<name>` / `p10k segment` custom-segment protocol runs real shell functions where a user has written them, so custom segments keep working. `CONTENT_EXPANSION` templates evaluate through the shell's real `${(e)}` substitution chain, and the transient prompt is ported.

`gitstatusd` is absorbed too: rather than shell out to a C++ sidecar, a native `.git` reader parses HEAD, refs, packed-refs, the index (v2/v3/v4), the stash reflog, and tags in-process, so the git segment computes without a daemon and without a fork per prompt in the cached case. Where the zsh theme compiles deferred `${...}` templates and re-evaluates them through the prompt expander's state machine on every redraw, the port renders eagerly — one linear fold per prompt. The one honest seam: staged and untracked counts still shell out to a single gated `git status --porcelain=v2` per cache miss, because a full HEAD-tree diff needs a

zlib object store the reader does not yet carry — documented, not hidden. And instant prompt is deliberately not ported, for the reason the previous section gave: it is the one p10k feature that is a latency mask rather than a rendering choice, and the whole point of absorbing the engine into the binary is that there is no longer any latency to mask.

Chapter 35 – Arithmetic and the Two-Tier JIT

Shell arithmetic looks like a small corner of the language — `$(( i + 1 ))`, a `for (( i=0; i<n; i++ ))` loop, a `(( count++ ))` in a condition — but it is the corner where zshrs's compiler most visibly stops being a metaphor and becomes a code generator that emits native machine instructions. Arithmetic is the one part of shell evaluation that is pure, typed, and side-effect-light: integers and floats, the full set of C operators, no filesystem, no forks. That purity is exactly what a JIT wants. This chapter follows an arithmetic expression from the `$((` that opens it to the x86-64 or aarch64 instructions Cranelift emits for it, through the two compilation tiers zshrs runs it against.

From `matheval` to a compiler

zsh evaluates arithmetic the way it evaluates everything: in one pass, at run time. `src/math.c`'s `matheval` tokenizes the expression and walks it on the math evaluation stack via `mathevall`, re-tokenizing and re-evaluating on every execution — the same interpret-every-time pattern the whole book has been about, applied to numbers. zshrs keeps the *semantics* of that file exactly: `src/ported/math.rs` is a direct port supporting integer and floating-point arithmetic, every C operator, zsh's `**` power operator, the ternary, the assignment operators, pre- and post-increment, base conversion (`16#FF`, `2#1010`, `[16]FF`), `Inf/NaN`, and variable references. The behavior a `.zshrc` depends on — operator precedence, integer overflow semantics, the way `10#` forces base ten — comes across faithfully, because arithmetic edge cases are exactly the kind of decades-old dependency the compat floor forbids breaking.

But zshrs adds a layer C never had. `src/extensions/arith_compiler.rs` is an `ArithCompiler` that *lowers* an arithmetic expression to fusevm bytecode instead of evaluating it inline. Its tokenizer matches the shape of `math.c`'s `zlex/mathlex`, so it splits the expression exactly as zsh would, but where the C code pushes onto an eval stack, the compiler emits `fusevm::Op` instructions — `Add`, `Sub`, `Mul`, `GetSlot`, `SetSlot`, `NumLt`, and so on — into a `ChunkBuilder`, mapping each variable name to a numbered slot. The header states the split precisely: C zsh has no arithmetic compiler; zshrs separates compilation from evaluation so the resulting op stream can be specialized by the JIT. Once an expression is bytecode, it is subject to everything the rest of the VM enjoys — it can be cached, and it can be compiled to native code.

Two tiers: linear and block

fusevm's JIT, in `jit.rs`, has two tiers, and arithmetic is what distinguishes them. The **linear JIT** compiles straight-line sequences — no branches — in a single Cranelift basic block, with the invariant that the stack ends holding exactly one value. This is the tier for an expression like `$(( a*b + c ))`: a flat run of loads and arithmetic ops that maps directly onto Cranelift IR with no control flow. The **block JIT** handles bytecode with branches — loops, conditionals, the short-circuit `&&` and `||` — by building a basic-block control-flow graph and reconciling the abstract operand stack at each block join. This is the tier for a `for (( i=0; i<n; i++ ))` counting loop or a `(( x > 0 && y < 10 ))` condition. Both tiers draw from the same pool of JIT-eligible universal ops — the integer and float arithmetic, the comparisons and spaceship, the bitwise operators, the slot loads and stores — and the block tier additionally handles the jump family (`JumpIfTrue`, `JumpIfFalse`, and their keep-variants) that the linear tier by definition cannot.

The most telling detail in the eligible-op list is the fused superinstructions built specifically for arithmetic loops: `SlotLtIntJumpIfFalse`, `SlotIncLtIntJumpBack`, `AccumSumLoop`, `AddAssignSlotVoid`, `PreIncSlotVoid`. These are not general operations; they are the recognizable shapes of a counting loop collapsed into single ops — “load a slot, compare it to an integer, jump if the comparison fails” fused into one instruction, or “increment a slot, compare, jump back to the top of the loop” fused into another. A `for ((i=0; i<1000; i++ ))` loop, lowered through these superinstructions and handed to the block JIT, becomes a tight native loop with the loop variable in a register and the bound as an immediate — the same code a C compiler would emit for the equivalent `for` loop. This is the arithmetic case reaching the floor of the tier table from Chapter 1: not interpreted bytecode, not even generic JIT, but fused superinstructions compiled to native machine code.

When the JIT fires

Compiling to native code is not free — Cranelift has to run — so `zshrs` does not JIT everything on first sight. The `JitThresholds` in `jit.rs` govern the decision, and their defaults encode a policy. `trace_threshold` defaults to 50: a hot path must be seen 50 times before the tracing tier invests in it. `block_threshold` defaults to 1, meaning the block JIT compiles a chunk on its second whole-chunk invocation — aggressive, because a block-eligible chunk (a loop, a conditional) is almost always worth compiling if it runs more than once, and the compile cost is amortized immediately by the loop’s own iterations. The thresholds are surfaced as tunables precisely because the right answer depends on workload: a long-running interactive session with hot loops wants low thresholds to compile early, while a script that runs once and exits wants high thresholds so it never pays for a compilation it cannot amortize. The linear tier’s requirement that the stack end with exactly one value, and the block tier’s abstract-stack merge at joins, are the correctness guards that let this happen safely — if a chunk does not fit the tier’s shape, the JIT declines and the interpreter runs it, so JIT eligibility is always a performance question and never a correctness one. A block that overflows, or hits a case the native code cannot represent, discards its result and re-runs in the interpreter; the block-eligible ops are pure, so discarding and retrying is always sound.

The payoff lands where arithmetic actually lives: loops. A shell script that does real numeric work — summing a column, iterating a counter tens of thousands of times, computing offsets — runs that loop as native code in `zshrs`, once the block JIT has claimed it, instead of stepping an interpreter through the loop body on every iteration. It is the clearest possible demonstration of the book’s thesis at the level of a single language construct: the same `for (( ))` loop that every other shell walks and re-walks, `zshrs` compiles once and runs at the speed of the metal underneath.

Chapter 36 – The Federation: Many Shells, One Catalog

Everything to this point has described a single shell talking to a single daemon. But the daemon is a machine-wide singleton, and a developer does not run one shell — they run a dozen: split panes in `tmux`, a login shell and a script shell, a `bash` invoked by a `Makefile`, a `fish` in another terminal, editor-embedded terminals. A traditional shell treats each of these as an island. Its aliases, its functions, its history, its notion of “the current job” are private to its own process and die with it. `zshrs`, because a daemon already sits above all of them owning the compiled state, can do something no island shell can: it can make the shells a *federation* — a set of sessions that know about each other, share a queryable catalog of their definitions, and can send each other messages. This chapter is about that layer, which turns “the daemon serves my shell” into “the daemon coordinates all my shells.”

`shell_id`: the federation is not `zshrs`-only

The keystone is the `shell_id`, and the important thing about it is that it is not specific to `zshrs`. The daemon’s session registry, in `daemon/state.rs`, tracks every registered session with a stable `shell_id` — `bash`, `zsh`, `zshrs`, `fish` — and a per-process `bundle_id` the daemon uses to scope that session’s state. The `zid` builtin prints the current shell’s pair. Because the identity is a shell-agnostic string rather than a `zshrs` internal, a `bash` or `fish` session running the appropriate recorder can register with the same daemon under its own identity, write into the same catalog, and be seen by every other session. This is what `docs/SHELL_IDS.md` formalizes and what makes the word “federation” literal: the catalog is not `zshrs`’s private store that other shells are locked out of, it is a shared table that any participating shell writes to as a first-class member.

The definitions catalog and `zwhere`

What the federation shares first is *definitions*. Every alias, function, environment variable, keybinding, `compdef`, `setopt`, and typed parameter a shell defines during its init is captured by the recorder — via the AOP intercepts of Chapter 24, so it is captured by observing the shell define them, not by parsing source — and shipped to the daemon’s canonical engine as records tagged by `kind` and `shell_id`. The `daemon/definitions.rs` ops expose that store as a read-only query API: `definitions_query` takes an optional `kind`, `name`, and `prefix` and returns matching records; `definitions_kinds` lists every populated kind. The client-side surface is the `zwhere` builtin — a thin IPC wrapper over `definitions_query` that answers “where is this alias/function/binding defined, across every shell.” `zwhere KIND NAME` prints every row for that name across all shells, `zwhere KIND --prefix STR` does prefix search, `zwhere --shell-id ID ...` restricts to one shell, and the output is deliberately narrow — one row per line, `kind`, `name`, `value`, `shell_id`, `file:line`, tab-separated — so a script can `awk` it cleanly rather than fighting a pretty-printed table. The companion `zls` lists catalog entries filtered by kind and shell (`zls --kind alias --shell-id bash`), the cross-shell mirror of the local `alias/functions/typeset` builtins.

The capability this unlocks is `definitions_diff`: because every shell’s definitions live in one catalog keyed by `shell_id`, you can ask what your `zsh` config defines that your `bash` config does not, or what drifted between two sessions. A question that is meaningless for island shells — “how do my two shells differ?” — becomes a database diff.

Coordination: pub/sub, zsend, and zask

Beyond querying each other's definitions, federated shells can talk. The daemon runs a pub/sub bus, documented in `daemon/pubsub.rs`, with a scope-and-topic subscription syntax: scopes are `shell:<id>` for a single shell, `tag:<name>` for every shell carrying a tag, `user:<name>` for cross-user (root only), and `*` for all; topics are the lifecycle events — `commands`, `chpwd`, `prompt`, `precmd`, `preexec`, `exit`, `signal`, `error`. A shell subscribes to a scope-and-topic pattern and the daemon delivers every matching event to it: subscribe to `*.chpwd` and you see every directory change in every shell; subscribe to `tag:prod.commands` and you see every command run in any shell tagged `prod`. The tagging is user-driven — `ztag prod-deploy` labels the current session, and other shells filter on it via `zls --tag` or a `tag:` subscription — so the federation is not a flat pool but a set the user partitions by intent.

On top of the bus sit the directed builtins. `zsend` sends a one-shot message to a specific shell by `shell_id` or `bundle_id` — the point-to-point counterpart to broadcasting on the bus. `znotify` routes a notification to the user's desktop, dispatching to `osascript` on macOS, `notify-send` on Linux, or an in-shell UI when no platform notifier exists — so a long build finishing in one pane can raise a system notification without that pane needing to know how notifications work on this OS. And `zask` is the inverse channel, the daemon asking *you* a question. It is deliberately pull-mode and never interferes with the prompt: the daemon never auto-renders a dialog over your work, it pushes an `ask:pending` event that increments a status-line counter and optionally fires an OSC-9, and the request sits in an inbox until you pull it with a keybinding or `zask take`. Progress updates are the lone exception allowed to touch the status line in place, because the status line is reserved passive space. The design ethic is the same one that governs the prompt: the coordination layer is powerful, but it is not permitted to seize the terminal from under the user. A federated shell can be reached, queried, and messaged — and it still never paints anything you did not ask to see.

Chapter 37 — The Endgame

Every chapter to this point has described a mechanism: the no-fork worker pool, the bytecode VM, the rkyv image cache, the daemon, the single directory. This last chapter is about the constraint that produced all of them, and that will outlive every one of them. `zshrs` is not a faster `zsh` that the maintainer happens to like. It is the endgame shell for his lifetime — and *endgame* is not rhetoric here. It is an engineering constraint with a number attached: thirty years.

Thirty years is the spec

Plan in decades; a 30+ year horizon; every architectural decision must survive that. That single requirement reaches back through the entire codebase and pins down choices that would otherwise be matters of taste. Bytecode formats are versioned, so an image written today still loads after the format evolves rather than forcing a silent rebuild that could break a `.zshrc` that worked yesterday. SQLite schemas are migration-safe, so the catalog upgrades cleanly instead of demanding a wipe. Dependencies are chosen to be vendorable, audited, and durable — the foundational crates (`serde`, `rusqlite`, `regex`, the rkyv stack) that will still build cleanly in 2035, not the fashionable ones that won't. `Cargo.lock` is committed. The MSRV is pinned. And the design names the actual person the code is being written for: the future maintainer at 65 has to navigate this. Readable over clever, every time, because the person who has to debug a torn-shard race in 2050 is the same person writing the atomic-rename ordering today.

The choices compound. Versioned formats plus committed lockfile plus pinned MSRV plus durable deps is not four separate hygiene items; it is one stance applied four ways, and the stance is that `cargo build` on a fresh machine should work in 2030 with whatever toolchain exists then. No curl-pipe-bash install step that bit-rots, no build script that depends on a network fetch, no dependency that will be abandoned and stop compiling on a future Rust. The whole supply chain has to survive the same thirty years the format does, because a shell you cannot rebuild is a shell you cannot trust to still be yours in a decade.

This is why there will be no further shell migrations. The maintainer is not going to switch to the next thing, and is not going to ship a “`zshrs2`” that abandons the format. The decisions have to be right the first time *because there is no second tool to escape into*. That is what an endgame constraint does: it removes the option of starting over, and in removing it, forces every decision to carry its own thirty-year warranty. It also reframes what counts as a bug. In a project you expect to replace in three years, a format you will have to migrate is a future chore; in a project you intend to use for thirty, it is a defect you fix now, because there is no replacement version coming to absorb the cost later.

The compat floor is sacred

The hardest constraint is also the most invisible one. `zshrs` hosts a single-author CLI environment of unusual scale — `zpw` at 172k lines and 506+ subcommands, the `zsh-more-completions` corpus at tens of thousands of files, a `.zshrc` that has accreted across decades. None of that can break. Compat with the existing world is sacred, and compat-floor regressions are catastrophic; ship none.

So the entire `src/ported/` directory is a strict 1:1 port of `zsh`'s C source — every Rust function maps to a real `Src/<x>.c` function, every top-level `fn` carries a `/// Port of` citation to the upstream line, and the file set is frozen at 107 files with a test, `tests/port_purity.rs`, that fails CI if anyone invents a helper or adds a file. The point of that severity is not purity for its own sake. It is that the `zsh` C source is the specification

for what compatible *means*, and the only way to guarantee a `.zshrc` that worked yesterday works tomorrow is to port the behavior faithfully rather than re-derive it. Re-derivation is how subtle incompatibilities creep in: a reimplemented parameter-expansion flag that handles 95% of cases and silently mangles the rest is worse than no reimplementation at all, because the 5% is exactly the decades-old config that depends on an obscure corner. Porting from the C source means the corners come across with the common cases. The 4-tier `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand` AST is preserved verbatim from zsh for the same reason — keep the construct boundaries identical and per-construct behavior stays in parity even as the execution model underneath changes completely.

The novel work — the worker pool, the VM, the daemon, the AOP — lives in a separate `src/extensions/` directory that is forbidden from shadowing the ports, with `port_purity` enforcing the wall. Innovation and compatibility are kept in different rooms on purpose. The floor never moves; the ceiling has no limit.

The behavioral discipline backing that promise is just as strict. zshrs deleted its own tree-walker — the AST-walking dispatch every prior shell relies on, roughly 1,275 lines of `execute_simple/pipeline/list/compound` dispatch — and a test, `tests/tree_walker_absent.rs`, fails CI if anyone tries to reintroduce it. Every `ShellCommand` variant now compiles to a fusevm chunk; there is no second execution path for behavior to drift into. And every change ships with behavioral tests that pin exact stdout and exit status — no invariant, no merge. When a bug is found, the fix lands with a test that fails before it and passes after. The test corpus is the concrete form of the promise: hundreds of construct tests, real `.ztst` files lifted from upstream zsh, per-construct bytecode smoke tests, AST-shape probes. This is what lets a thirty-year compatibility promise be more than a hope: it is enforced by a body of tests that would catch the regression the day it was introduced rather than the day a user hit it.

There is one more posture the endgame imposes, and it is the opposite of a clone's. zshrs is meant to be the synthesis of every good shell idea, not just zsh-in-Rust. Anything fish, nushell, elvish, oil, fzf, or atuin innovates gets evaluated for absorption — fish-style autosuggestions, abbreviations, and syntax highlighting are already ported in. Endgame means convergence into one tool the maintainer never has to leave. That is the difference between a replacement and a port: a port reproduces one tradition, while zshrs is built to be the place every shell tradition ends up. A clone of zsh would stop at zsh's feature set; a synthesis keeps absorbing, which is the only way "I will never need another shell" can be true thirty years out, when shells that don't exist yet have shipped ideas worth taking.

No fakery — first paint is full functionality

The performance target — 10-20ms cold start, fully featured, with a hard ceiling below the 30ms human-reaction threshold — is the one most likely to be misread as just another speed number. It is not. It is a *correctness* constraint about what "started" means, and it is the cleanest expression of the maintainer's whole argument for why zshrs had to exist.

The case against zsh is not that zsh is slow; it is that zsh's slowness has been papered over by an ever-growing stack of optimization layers — zinit turbo, p10k instant prompt, `zcompile`, `BG_NICE`, lazy autoload, `.zcompdump` — each one evidence that the underlying problem is unfixable in userspace. When a tool accumulates N separate well-known tricks for the same startup problem, each trick is a confession that the problem cannot be fixed at its root; communities do not stack workarounds on top of problems that have a real fix. The instant-prompt trick in particular paints a prompt immediately while the real work happens behind it, so "started" is a visual lie: the prompt is there but the shell isn't ready, and the first command you type may stall while deferred init catches up. zshrs rejects that outright. Never instant-prompt fakery; first paint equals full functionality, period. Ten of the architecture's hard constraints exist to make that honesty affordable — cache everything at install/update time not startup time, mmap not cold-open, lazy worker-pool init, zero synchronous external commands at startup, no DNS ever at startup, no directory scans at startup, single-threaded fast path, minimal hot-path allocation. The acceptance test for any future performance work is a discriminator: does this *remove* the need for an optimization trick, or does it implement one? A change that adds a deferral layer fails the test even if it improves a benchmark, because it is the lipstick pattern by another name. A shell that needs its own instant-prompt has, by this standard, already failed — and the standard is the point, because it is what

keeps the project from drifting back into the very pattern it was built to escape.

The next rung: AOT, now partly built

There is a discipline behind that severity worth naming, because it is what keeps the project from drifting. The design goals run every proposal through a ten-gate gauntlet, and the first two gates are the whole filter: does this enable a capability that doesn't exist anywhere in software, and will it be top-tier fast in its category? Both legs must pass; either alone is insufficient. Most proposals die at gate one. A faster `awk` is dead — `awk` exists. A faster `lsof` is dead — `lsof` exists. The maintainer authored record-tier implementations of both and cut them from his stack anyway, which is the strongest evidence the rule is real: if he relegates his own record-holders to maintenance mode for clearing only one leg, nothing gets a pass. That is also why this book has no chapter on a fast `cat` — the in-process `cat` builtin earns its place not by being fast but by eliminating the fork, a capability the architecture has and an interpreter's cannot. Speed alone was never the point. A genuinely new capability paired with a top-tier implementation is the only thing that survives the gauntlet, which is why the surviving feature list is short and every item on it does something no other shell does.

There is one capability past the ones this book has described, and the right way to present it is the way the project itself does: with the scope marked precisely. The `AOT_DESIGN.md` document lays out ahead-of-time compilation — taking a shell script all the way to a native machine-code executable through the same fusevm IR that already feeds the Cranelift JIT, with AOP advice-weaving and nanosecond profiling baked into the resulting binary. The claim it makes is specific and falsifiable: no shell has ever shipped an AOT-compiled-to-machine-code deployment artifact for scripts.

What made it reachable rather than aspirational is that it is one rung up a ladder the project had already climbed — and since this book was first written, the project has climbed it. The bytecode VM exists. The Cranelift JIT exists. AOT adds `cranelift-object` as a parallel output sink on the same IR pass the JIT already consumes — no LLVM, no heavyweight compiler backend that would bit-rot, target triples handled natively by Cranelift — and that sink is now shipped as fusevm's `aot` feature, driven by a `zbuild --native` builtin that compiles a script to a `.o` and links it against the zsh runtime staticlib into a standalone executable. The discipline held through the implementation: the `eval` and dynamic-command cases stay cold paths served by the linked-in shell runtime, so AOT remained a strict superset of the interpreted path with no compat-floor regression, and the source-trailer format got the same versioned header every other artifact gets. What is *not* yet built is the part that justified wanting AOT in the first place — the compile-time AOP weaving and the native nanosecond-profiling prologue/epilogue — and a `--native` binary today targets only the host architecture and leans on a local `cc`, not the doc's self-contained cross-compiler. So the capability runs and the deployment-and-instrumentation story it was meant to enable is partway there. Saying exactly that — built here, unbuilt there — rather than rounding it up to “shipped,” is the same engineering ethic that refuses to fake a prompt. A book that claimed the whole AOT stack as finished would be doing exactly what an instant-prompt does: presenting a placeholder as the finished thing.

The shell that did not have to stay an interpreter

The single sentence under which the whole project sits is the one on the README's first screen: since the Bourne shell at Bell Labs in 1970, every Unix shell has been an interpreter, and zshrs is the first to be a compiler. Fifty-plus years of shells walked an AST at runtime — re-parsing, re-walking, forking for the smallest builtin — and the entire field treated that as simply *what a shell is*. It wasn't. It was an assumption nobody had spent the years to overturn, and it is worth being precise about why overturning it took this long: a shell that compiles has to first reproduce the entire behavior of a shell that interprets, because nobody will adopt a fast shell that breaks their config. The compiler half is the novel part, but the port half — the 107 files mirroring `Src/<x>.c`, the verbatim AST, the `.ztest` corpus — is the expensive part, and it is the part that makes the compiler usable rather than a research toy.

The mechanism that makes the claim true is worth stating once more, plainly, because it is the hinge the whole project turns on. Every command — interactive or scripted — flows `lex` → `parse` → `compile` →

`fusevm bytecode` → VM. The lexer and parser are faithful ports of zsh's `Src/lex.c` and `Src/parse.c`; only the bytecode compiler is original. The four-tier `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand` AST is preserved verbatim from zsh, so per-construct behavior stays in parity while the execution model changes underneath it. And the VM does not merely interpret the bytecode: hot loops and branches go through Cranelift to native machine code via a block JIT, straight-line arithmetic through a linear JIT, and only the genuinely interpretive work — builtins, I/O, string ops — falls through to a jump-table dispatcher with fused superinstructions. That is the compiler the field spent fifty years assuming a shell could not be.

zshrs overturns it, and everything in this book is downstream of that one decision. The no-fork worker pool is possible because builtins are bytecode the VM runs in-process. The 10-20ms cold start is possible because the `.zshrc` is mmap'd bytecode, not source re-parsed on every launch. The daemon, the rkyv shards, the single directory, the AOP intercepts, the AOT rung now under your hands and the build-time weaving still to come — none of them are reachable from an interpreter's architecture at any speed. They are reachable from a compiler's. That is the difference the project was built around, and it is the only kind of difference the bar admits: a genuinely new capability and a top-tier implementation, both at once, with even record-tier work cut when it clears only one leg. zshrs clears both. It is the first compiled Unix shell, and it is built to still be the maintainer's shell in thirty years — not because nothing better could come along, but because the design absorbs anything better into this one rather than leaving for it. The interpreter era of the Unix shell is over. This is what comes after it.

This book has walked the whole of it: the no-fork worker pool that runs builtins in-process, the fusevm bytecode and its Cranelift JIT, the rkyv image cache that makes a warm start a memory map instead of a re-parse, the singleton daemon that owns all of it for a hundred shells at once, the single directory that holds the result, and the AOT rung that now compiles a script to a native binary with its build-time instrumentation still to come. Each was a mechanism in service of one constraint — a shell its maintainer can trust completely, instantly, for the rest of his working life. The pieces are not separable from that goal; they are what the goal looks like once it is built. A shell that compiles, that does not fork, that starts in the blink of an eye fully loaded, that hides its work in a log and shows you nothing you did not ask for, that keeps faith with a `.zshrc` written decades ago and will keep it decades from now. That is the endgame. The rest is using it.

Appendix A – The Command-Line Surface

A shell's command line is its smallest public API — the part you type before you've typed anything else. zshrs keeps zsh's single-letter options unchanged, then layers a band of zshrs-only flags on top for parity emulation, pipeline introspection, VM debugging, and editor integration. This appendix catalogs the whole surface, grouped by what each flag is *for*. The man page ([man/man1/zshrs.1](#)) is the source for the shell-mode options, parser dumpers, parity modes, and environment; the documentation and editor-integration flags are defined in the completion file ([completions/_zshrs](#)). The split matters in practice: `man zshrs` shows you the day-to-day shell flags, while a fresh `_zshrs` checkout is where the tooling flags live, because those drive the docs build and the IntelliJ plugin rather than interactive use.

A.1 – Special options

These three terminate immediately. None of them start a shell. They are the flags you can run in a script or CI step without worrying about side effects on shell state.

Flag	What it does
<code>--help</code>	Print the option summary and exit. The complete single-letter option table is printed here, so it is the fastest way to see every flag the binary accepts at the current version.
<code>--version</code>	Print the zshrs version and exit. Useful in scripts that gate on a minimum version, and the same string <code>--doctor</code> and <code>/health</code> report.
<code>--doctor</code>	Full diagnostic report of shell health, cache state, daemon presence, worker pool, and integrity-scan results.

`--doctor` is the one to reach for when something feels off but nothing has obviously broken — it is the single command that tells you whether the daemon is up, whether the rkyv caches are coherent, and whether the worker pool came up clean. Run it before assuming a problem is in your `.zshrc`: a stale shard or a daemon that failed to start will surface here rather than as a confusing shell-level symptom.

A.2 – Core shell-mode options

These are the zsh single-letter options, carrying their zsh meanings unchanged; the long-option names follow each in the man page, and the completion file carries the long forms.

Flag	Long form	Meaning
<code>-c</code>	—	Take the first argument as a command and execute it. The primary entry point for one-shot invocations from scripts, makefiles, and other shells.
<code>-s</code>	<code>--shinstdin</code>	Read commands from standard input. Combine with other flags (e.g. <code>-fis</code>) when feeding a shell a pipeline of commands rather than a file.
<code>-b</code>	—	End option processing (like <code>--</code>). Use it when the next argument might begin with <code>-</code> but should be treated as data.
<code>-f</code>	<code>--no-rcs</code>	Skip startup files. The fast path for a clean shell with no <code>.zshrc</code> /plugin load — what you want for reproducible scripts and benchmarks.
<code>-i</code>	—	Force interactive mode even when stdin is not a tty, so prompts, line editing, and interactive-only startup files apply.
<code>-l</code>	<code>--login</code>	Force login shell, which adds <code>/etc/zprofile</code> , <code>.zprofile</code> , <code>.zlogin</code> , and the login-shutdown files to the startup sequence.
<code>-o option</code>	—	Set a named option; see <code>zshoptions(1)</code> . Lets you flip any setopt before the script runs without editing it.
<code>+o option</code>	—	Unset a named option (the <code>+</code> form inverts <code>-o</code>).
<code>-v</code>	<code>--verbose</code>	Echo each command line after history substitution — the input-echo half of debugging.
<code>-x</code>	<code>--xtrace</code>	Echo each command and its arguments as they execute — the execution-trace half, and the flag you set when a script does something unexpected.

The man page additionally tracks the full zsh single-letter set literally: `-a -d -e -g -h -k -m -n -p -r -t -u -w -y` and the upper-case `-H -I -J -K -L -M -N -O -P -Q -R -S -T -U -V -W -X -Y -Z` family. These are the standard zsh option letters and carry their zsh meanings unchanged — `zshrs --help` prints the complete table, so there is no need to memorize which letter maps to which setopt.

```
zshrs -c 'print -l $path'      # run one command, exit
zshrs -fis                    # interactive, no rc files, read stdin
zshrs -o extendedglob script.zsh # set an option by name before running
```

A.3 – Parity modes

Each parity mode is a behavioral promise: match the named reference shell byte-for-byte. The man page states the cost directly — each parity mode disables all caches and the daemon and matches the named reference shell byte-for-byte; every `source` re-runs the file fresh, every `echo` re-fires. This is the mode you reach for when you are running a compatibility test suite and need zshrs to behave as if it were the original interpreter, with none of the caching that makes full mode fast. It is also how you symlink a single `zshrs` binary into the role of several reference shells at once for a test harness.

Flag	Drop-in for	Notes
<code>--zsh</code>	<code>/bin/zsh</code>	The reference-zsh entry point for compatibility tests — when you want to confirm zshrs and stock zsh agree on a script's output.
<code>--zsh-compat</code>	<code>/bin/zsh</code>	Legacy alias for <code>--zsh</code> ; the completion file describes it as enabling <code>.zcompdump</code> use and fpath scanning, the path that mirrors a conventional zsh completion setup.
<code>--bash</code>	<code>/bin/bash</code>	Caches off, daemon off; run a bash script under zshrs's bash semantics without bash installed.
<code>--ksh</code>	<code>/bin/ksh</code>	ksh-93 semantics, for scripts written against the Korn shell dialect.
<code>--posix</code>	<code>/bin/sh</code>	Bourne / dash. The strictest mode: the completion file marks it “no SQLite, no worker pool, no zsh extensions” — the closest zshrs gets to a plain POSIX <code>sh</code> .
<code>--sh</code>	<code>/bin/sh</code>	Alias of <code>--posix</code> , for invocation under the <code>sh</code> name.
<code>--csh</code>	<code>/bin/csh</code>	csh parity, for the C-shell dialect (also why <code>where</code> honors csh-style bare-path output).
<code>--emulate mode</code>	one of <code>zsh sh ksh csh bash</code>	Specify the emulation mode explicitly rather than via the per-shell flag — useful when the mode name is itself a variable.

The reason parity modes disable the caches is correctness, not speed. The whole value of a parity mode is that zshrs becomes indistinguishable from the reference shell, and the daemon's snapshot-and-overlay model — where a `source` of an unchanged file returns cached bytecode instead of re-running it — is exactly the kind of behavior a strict compatibility test must not see. So parity mode strips it all out: every `source` re-runs fresh, every `echo` re-fires, no shard is consulted, the daemon is not contacted. You pay full interpreter-equivalent cost and get byte-exact reference behavior in return. Full mode is the inverse trade and the everyday default.

With no parity flag, zshrs runs in full mode — the man page describes the default as “rkyv `script_cache` + `plugin_cache` + daemon enabled.”

```
ln -s "$(command -v zshrs)" /usr/local/bin/zsh-test
zshrs --zsh -c 'setopt | head'      # behave exactly like /bin/zsh, no caches
```

A.4 – Startup-file control

`--no-rcs` (`-f`) and the `PRIVILEGED` option modulate which startup files are sourced, identically to `zsh`. `zshrs` reads the same startup files in the same order as `zsh(1)`: `/etc/zshenv`, `$ZDOTDIR/.zshenv`, `/etc/zprofile` + `$ZDOTDIR/.zprofile` (login), `/etc/zshrc` + `$ZDOTDIR/.zshrc` (interactive), `/etc/zlogin` + `$ZDOTDIR/.zlogin` (login), and `$ZDOTDIR/.zlogout` + `/etc/zlogout` on login shutdown. When `$ZDOTDIR` is unset, `$HOME` substitutes.

A.5 – Parser-pipeline dumpers

Each dumper writes to stdout and accepts `-` for stdin. These expose the stages of `zshrs`'s compile pipeline — lexer, parser, wordcode emitter — so you can see exactly what the front end produced before the VM ever ran. They are the tools you reach for when a script behaves differently under `zshrs` than under `zsh`: dump the tokens to confirm the lexer split the line the same way, dump the AST to confirm the parser built the same tree, dump the wordcode to confirm the emitter produced equivalent `EPROG`. Because each accepts `-`, you can pipe a one-liner straight in without writing a file.

Flag	Output
<code>--dump-tokens FILE</code>	One <code>TOKNAME</code> → <code>TOKSTR</code> line per lexer token — the first stage, for confirming the lexer split a line the way <code>zsh</code> would.
<code>--dump-ast FILE</code>	Parser AST in canonical S-expression form — the second stage, for confirming the parser built the same tree.
<code>--dump-wordcode FILE</code>	Wordcode emitter output (<code>EPROG</code> , <code>WORDS</code> , <code>WC[i]</code> , <code>STRS</code>) — the third stage, the compiled representation the AST lowers to.
<code>--dump-zwc ZWCFILE [FN]</code>	Inspect a compiled <code>.zwc</code> cache: list its function index, or with <code>FN</code> , disassemble a single entry. The one dumper that reads an existing on-disk artifact rather than compiling fresh input, useful for auditing a legacy autoload cache before importing it.

A.6 – VM debugging

Flag	What it does
<code>--disasm</code>	Print <code>fusevm</code> opcodes for each compiled unit before VM execution. Does not suppress execution.

`--disasm` is the bridge between “what the shell parsed” and “what the VM runs” — it prints the bytecode and then runs it, so you see the disassembly and its effects in one pass.

```
zshrs --dump-ast - <<<'for i in 1 2 3; do print $i; done'
zshrs --disasm -c 'x=$((1+2)); print $x'
```

A.7 – Documentation and reflection emitters

These flags drive the docs build and the IntelliJ tooling, and are defined in `completions/_zshrs`. They are the mechanism by which the shell binary is also the authority for its own documentation — the reference

HTML, the hover cards an editor shows, and the name lists a completer needs all come out of the same `zshrs` invocation, so they cannot drift from what the binary actually implements.

Flag	What it does
<code>--names</code>	Print every LSP-known canonical name, one per line (builtins / keywords / options / specials / compsys / extensions / operators). The flat name universe the completer and the docs build both index against.
<code>--docs NAME</code>	Render the LSP hover card for <code>NAME</code> — the same card an editor shows on hover, available from the command line for a quick lookup.
<code>--color mode</code>	ANSI color in <code>--docs</code> output: <code>auto</code> , <code>always</code> , or <code>never</code> . Set <code>never</code> when piping <code>--docs</code> into a file or another tool.
<code>--dump-reflection</code>	Emit the JSON consumed by the IntelliJ reflection tool window — the structured form of the same name/doc data, for the plugin to render.
<code>--dump-reference-html</code>	Emit the HTML chapter sections for <code>docs/reference.html</code> , so the published reference is generated from the binary rather than hand-maintained.
<code>--gen-docs path</code>	Walk <code>PATH</code> for shell sources and emit Markdown module docs — point it at a plugin or function library to generate per-module documentation.
<code>--out dir</code>	Output directory for <code>--gen-docs</code> (default <code>docs/</code>).

`--names` is also the dynamic fallback that the `_zshrs` completer uses on a fresh checkout: when the committed `lsp_completion_words.txt` snapshot is missing, the completion spawns `zshrs --names` once per session (~30 ms cold) to populate its name cache.

A.8 – Editor integration

Flag	What it does
<code>--lsp</code>	Run the in-process LSP server over stdio — the Language Server Protocol endpoint an editor speaks to for completion, hover, and diagnostics.
<code>--dap</code>	Run the in-process DAP debugger (<code>host:port</code>) — the Debug Adapter Protocol endpoint for stepping through a script under a debugger UI.

These two are why `zshrs` doubles as its own language server: `--lsp` is the same binary an editor talks to for completion, hover, and diagnostics, and `--dap` is the debug-adapter endpoint. Because both run in-process, the language intelligence an editor sees is exactly the shell's own parser and name tables — there is no separate analyzer to fall out of sync with the runtime.

A.9 – Environment and exit status

Beyond the standard `zsh`-recognized variables (`PATH`, `FPATH`, `HOME`, `USER`, `LOGNAME`, `UID`, `EUID`, `SHELL`, `TERM`, `COLUMNS`, `LINES`, `ZDOTDIR`, `TMPDIR`, `MANPATH`, `PS1`, `PROMPT`, `RCPROMPT`), `zshrs` reads three of its own:

Variable	Meaning
<code>ZSHRS_CACHE</code>	Directory for rkyv bytecode images (default <code>\$HOME/.cache/zshrs</code>). Point it elsewhere to isolate caches per-project or per-test-run, or onto faster storage.
<code>ZSHRS_LOG</code>	<code>tracing</code> -style filter for the structured log at <code>\$HOME/.cache/zshrs/zshrs.log</code> (default <code>info</code>). Raise it to <code>debug/trace</code> to capture daemon and compile-pipeline detail when diagnosing a problem; the log is where all of zshrs's diagnostic output goes rather than the terminal.
<code>ZSHRS_LONG_CMD_PRE_THRESHOLD</code>	Seconds before a long-running-command warning fires — the cutoff that decides when a still-running command is reported as slow.

zshrs exits with the status of the last command, or with `$?` at the time of an `exit` builtin call. Non-existent commands exit 127; permission errors 126 — matching zsh.

Appendix B – The Daemon Operation Catalog

The zshrs daemon is the fat half of a thin-client / fat-server split: roughly 90% of all shell-internal work lives in the daemon, 10% in the client ([docs/DAEMON.md](#) §“The 90/10 work split”). Clients connect, mmap the daemon’s rkyv shards, and run against that snapshot; the daemon owns all compilation, all persistence, all filesystem enumeration, all routing, and the single machine-wide fsnotify watcher.

Everything the daemon does is reachable through a flat operation table. The authoritative source is [daemon/ops.rs:OP_NAMES](#). The catalog below groups those ops by area, with one-line descriptions grounded in [docs/DAEMON.md](#)’s operation table. A few structural facts make the catalog easier to read. First, the same op dispatches identically whether it arrived over the Unix socket or over HTTP — both transports funnel through one [ops::dispatch](#) function, so an op is an op regardless of how a client reached it. Second, the wire format is length-prefixed JSON: a 4-byte big-endian length followed by that many bytes of UTF-8 JSON, debuggable with [socat](#) or [nc](#). Third, ops carry a monotonic per-connection [id](#) that the response echoes, while async events (Appendix B.13) carry no [id](#) because nothing is waiting on them. With those three facts in hand, the operation table reads as a flat menu: pick an area, pick an op, send `{"id":N,"op":"...","args":{"..."}}`, read back `{"id":N,"ok":true,...}`.

B.1 – Meta operations

The daemon’s own housekeeping and introspection surface.

Op	Purpose
info	Daemon stats, catalog stats, uptime, paths.
ping	Round-trip latency + pong.
daemon	Status / stop / restart control verb.
register	Session register (tag / cwd / argv0); implicit on connect.
doctor	Full diagnostic report.
verify	Integrity scan on shards + catalog.
compact	Vacuum catalog.db , dedup shards.
clean	Unlink shards / index / cache by target .
metrics	Prometheus-shaped metrics as JSON.
log_level / log_rotate / log_stats	tracing controls.
config_get / config_set / config_list	Runtime config knobs.
stats_flush	Merge batched per-call stats into entry_stats .

Op	Purpose
<code>replay_log</code>	Per-shell replay-log read-back.

The meta block is where you look first when something is wrong. `info` and `doctor` answer “what is the daemon doing”; `verify` and `compact` answer “is the daemon’s storage sound”; the `log_*` and `config_*` ops let you adjust verbosity and runtime knobs without a restart. `clean` takes a `target` and unlinks shards, the index, or whole cache regions — the reset hammer when a shard set is suspected stale.

B.2 – Recorder and canonical state

State attribution flows through `zshrs-recorder`, a one-shot binary that runs the user’s init under runtime AOP and ships the captured bundle to the daemon. The design pivot worth knowing: the daemon does **not** parse `.zshrc` or plugin syntax to derive starting state. It is a passive canonical store. The recorder observes the user’s real init — under their real plugin manager, whether that is zinit, antigen, zplug, oh-my-zsh, or a dozen others — and the daemon folds whatever it captures into the catalog. When `.zshrc` or a plugin changes, the user re-runs the recorder.

Area	Op	Purpose
recorder	<code>recorder_ingest</code>	Ingest a <code>RecorderBundle</code> ; replaces all rows for the bundle’s <code>shell_id</code> ; broadcasts <code>recorder_ingested</code> .
canon	<code>canonical_hydrate_view</code>	Rebuild the SQLite mirror of canonical state.
canon	<code>push_canonical</code> / <code>pull_canonical</code> / <code>diff_canonical</code>	Per-shell overlay promote / fetch / diff (the <code>zsync</code> family).

B.3 – Definitions (the shell-state catalog)

The recorder-fed, `shell_id`-federated catalog of every alias, function, parameter, binding, and completion. This is the heart of what makes the daemon more than a cache: it is a queryable, cross-shell database of *what is defined right now and where it came from*. Because every record carries a `shell_id`, a bash recorder and a zsh recorder can both write into it with their own identity, and `definitions_diff` will tell you exactly what one shell defines that another does not. The `KNOWN_KINDS` table covers aliases (plain, global, suffix), functions and autoloads, env vars and typed parameters, bindkey / compdef / zle declarations, zstyle / zmodload / setopt, path / fpath / manpath edits, named directories, traps, scheduled events, and completion files.

Op	Purpose
<code>definitions_query</code>	Federated catalog query (filter by kind / name / prefix / <code>shell_id</code>).
<code>definitions_kinds</code>	List every populated kind.
<code>definitions_emit</code>	Single-record write (third-party shell recorders).
<code>definitions_diff</code>	Cross-shell diff between two <code>shell_ids</code> .
<code>definitions_subscribe</code> / <code>definitions_unsubscribe</code>	Opt in / out of <code>recorder_ingested</code> events.

B.4 – Watch (fsnotify)

The daemon runs a single machine-wide fsnotify watcher and fans its events out to subscribers, so a hundred shells watching the same directory cost one OS-level watch rather than a hundred. Subscriptions are refcounted: the watch is dropped only when the last subscriber leaves.

Op	Purpose
<code>watch_subscribe</code> / <code>watch_unsubscribe</code> / <code>watch_list</code>	Refcounted fsnotify subscriptions — subscribe a path, drop it, or list current watches.
<code>watcher_stats</code>	fsnotify watcher counts + state, for seeing how many paths the single watcher is tracking.
<code>fpath_changed</code>	Client tells the daemon a new fpath dir was added, so the daemon can begin watching it for completion-function changes.

B.5 – Events (pub/sub)

A general publish/subscribe bus layered on the same connection clients already hold. Subscriptions are pattern-based, so one subscribe call can match a family of channels; matches arrive as the async `match` event (Appendix B.13). This is the primitive shell scripts use to coordinate without polling — one shell publishes, others react.

Op	Purpose
<code>subscribe</code> / <code>unsubscribe</code>	Pub/sub pattern subscription — register or drop a channel pattern.
<code>subscription_set_paused</code>	Pause / resume a subscription without tearing it down, for muting a noisy channel temporarily.
<code>publish</code>	Publish a pub/sub event to a channel for any matching subscriber.
<code>subscribe_shard</code>	Per-shard update push — get notified specifically when a named shard is rebuilt, narrower than the general bus.

B.6 – Shell registry and dispatch

Cross-shell coordination — the machinery behind `zls`, `zsend`, and `znotify`.

Op	Purpose
<code>list_shells</code>	Every registered session (powers <code>zls</code>).
<code>tag</code> / <code>untag</code>	Self-tag for routing.
<code>send</code>	<code>zsend</code> dispatch (single / broadcast / by tag / by user).
<code>cmd_started</code> / <code>cmd_result</code>	Long-command tracking + result event.
<code>notify</code>	<code>znotify</code> OSC-9 / status-line message.

B.7 – Ask (pull-mode UI queue)

A queue for questions aimed at a human. When the daemon — or another shell, or a scheduled job — needs the user to decide something, it enqueues an ask; the user's interactive shell drains the queue at a convenient

moment and posts a response back. The name is deliberate: `zask` (not `zui`), because the daemon **asks** the user, and `zui` was already taken by an existing zsh TUI plugin.

Op	Purpose
<code>ask_ask</code> / <code>ask_pending</code> / <code>ask_take</code> / <code>ask_dismiss</code> / <code>ask_response</code>	Pull-mode UI queue — the daemon (or another shell) asks the user something.

B.8 – Cache and artifacts

These two areas turn the daemon into a general-purpose key-value and content-addressed store for shell scripts and external tooling — a place to stash build results, memoized computations, or deduplicated blobs that outlive any single shell. The cache is namespaced and TTL-aware; the artifact store keys by sha256 so identical content is stored once.

Area	Op	Purpose
cache	<code>cache_put</code> / <code>cache_get</code> / <code>cache_del</code> / <code>cache_list</code> / <code>cache_stats</code>	Persistent KV store (sqlite-backed, TTL-aware, namespaced).
artifact	<code>artifact_put</code> / <code>artifact_get</code> / <code>artifact_get_by_digest</code> / <code>artifact_list</code> / <code>artifact_gc</code>	sha256 content-addressed cache.

B.9 – Jobs, schedule, locks

This trio is the daemon's process-orchestration surface. Jobs are detached tokio tasks the daemon supervises with per-job stdout/stderr files and terminal states (`exited` / `failed` / `killed` / `cancelled`). The scheduler is a cron-equivalent that fires those jobs on a 1 Hz tick. Locks are named cross-process mutexes, tied to a PID so they auto-release if the holder dies — the primitive you reach for when two shells must not run the same critical section at once.

Area	Op	Purpose
job	<code>job_submit</code> / <code>job_status</code> / <code>job_output</code> / <code>job_list</code> / <code>job_kill</code> / <code>job_cancel</code> / <code>job_wait</code>	tokio task queue. <code>job_wait</code> blocks a client until a job reaches a terminal state; <code>job_output</code> reads the per-job stdout/stderr files the daemon captures.
job (pty)	<code>job_input</code> / <code>job_resize</code>	Drive a pty-mode job: <code>job_input</code> writes base64-decoded bytes to the job's pty stdin; <code>job_resize</code> sets its window rows/cols via <code>ioctl</code> . Both error <code>not_pty</code> unless the job was submitted with <code>pty=true</code> .
schedule	<code>schedule_add</code> / <code>schedule_add_once</code> / <code>schedule_remove</code> / <code>schedule_list</code>	cron-equivalent (1 Hz tick fires <code>job_submit</code> with tag <code>scheduled</code>). <code>schedule_add_once</code> is the fire-at-a-time variant for one-shot future work.

Area	Op	Purpose
lock	<code>lock_acquire</code> / <code>lock_try_acquire</code> / <code>lock_release</code> / <code>lock_list</code>	Named cross-process mutex (PID-tied auto-release). <code>lock_try_acquire</code> is the non-blocking form that returns immediately rather than waiting for the holder to release.

B.10 – Snapshots

A tag is a named capture of the daemon's whole canonical state — every alias, function, path, binding, and option as of that moment. Snapshots let you save a known-good configuration, diff it against the present, and roll back to it. The use case is bisecting a configuration regression: snapshot before a risky plugin change, and `snapshot_diff` tells you exactly what the change altered.

Op	Purpose
<code>snapshot_save</code> / <code>snapshot_list</code> / <code>snapshot_load</code> / <code>snapshot_diff</code>	Tag-based canonical-state captures — save under a tag, enumerate tags, restore one, or diff two.

(`snapshot_bisect` / `publish` / `sign` / `verify` are named for forward compatibility but deferred.)

B.11 – Source and artifact ingest

The `source` and `.` builtins are daemon-aware: each call routes through `source_resolve`, which keys a compiled-bytecode cache on (`path`, `mtime`, `inode`) and returns a hit-from-cache or compiles-and-stores on a miss. The same pattern unifies `zshrs FILE` (via `load_script`), `source ~/init.sh`, and `.zshrc` cold-start. The `import_*` and `export` families handle migration in and out: legacy `.zwc` and `.zcompdump` files, history files, and full catalog or shard backups can be ingested on explicit user request (always freshness-validated, never auto-imported), and any cache target can be dumped in a dozen formats.

Op	Purpose
<code>source_resolve</code>	Bytecode-cache lookup for <code>source FILE</code> / <code>. FILE</code> .
<code>load_script</code>	Cold-load <code>zshrs FILE</code> .
<code>import_zwc</code> / <code>import_zcompdump</code> / <code>import_history</code> / <code>import_catalog</code> / <code>import_shard</code> / <code>import_all</code>	User-explicit legacy / backup ingest.
<code>export</code> / <code>view</code> / <code>export_all</code> / <code>export_catalog</code> / <code>export_shard</code> / <code>export_zcompdump</code>	Catalog dump / human view.

B.12 – History and compsys

The per-keystroke hot path lives here — these are the ops a client fires while the user is typing, so they must be the fastest in the catalog. They route to the SQLite FTS mirror and the perfect-hash shards rather than to any compile step.

Area	Op	Purpose
history	<code>history_query</code> / <code>history_append</code>	FTS search + write — full-text history lookup on every up-arrow or <code>Ctrl-R</code> , and the append on every accepted line.
compsys	<code>complete</code>	Tab-completion enumeration — the candidate set behind every <code>Tab</code> .
compsys	<code>suggest</code>	Autosuggest, frequency-ranked from history — the grey inline prediction as you type.
compsys	<code>highlight</code>	Syntax highlight — the per-keystroke coloring of the command line.
compsys	<code>keys</code>	Iterate keys for a daemon-served special parameter, so <code>\${(k)assoc}</code> over a daemon-backed assoc resolves without pulling the whole map.

B.13 — Async events (daemon → client)

Everything above is request/response: a client sends an op, the daemon answers. The events below run the other direction. The daemon pushes them without a request `id`; they are fire-and-forget, and a client only sees them after subscribing. They are how a long-lived shell learns that the world changed underneath it — a shard was rebuilt, another shell promoted a path change to canonical, a `zsend` arrived — without polling for any of it.

Event	Trigger
<code>shard_updated</code>	Daemon swapped a shard for a newer generation.
<code>rebuild_complete</code>	Async compile job finished.
<code>canonical_changed</code>	A subsystem's canonical state (path / fpath / alias / named_dir / zstyle / bindkey / setopt / zmodload) was promoted by some client.
<code>match</code>	Pub/sub pattern matched.
<code>cmd:execute</code>	<code>zsend</code> dispatch arrived.
<code>notify</code>	<code>znotify</code> arrived.
<code>daemon_shutdown</code>	Daemon going down (graceful, with grace period).

B.14 — HTTP transport

Every op is also reachable over HTTP at `POST /op/<NAME>` — the daemon dispatches HTTP and unix-socket calls through the same `ops::dispatch` function, so the wire format is the only difference (`docs/DAEMON_AS_SERVICE.md`). The listener binds `127.0.0.1:7733` and is **disabled by default**: it refuses to bind anywhere except loopback unless at least one bearer token is configured (`daemon/http.rs:95`). For solo use, loopback HTTP needs no auth.

Endpoint	Returns
GET /health	{ ok, version, uptime_ms }
GET /metrics	Prometheus 0.0.4 text (daemon_uptime_seconds, daemon_op_total{op}, ...)
POST /op/<NAME>	The op's JSON payload, or { "err": { ... } }
GET /stream/watch?path=DIR&recursive=BOOL	SSE event: fs
GET /stream/events?channel=PATTERN	SSE event: pub
GET /stream/definitions	SSE event: defs

Auth floor. Bearer tokens live in `~/.zshrs/zshrs-daemon.toml`. A flat-string token grants full access (backward compat); the scoped inline-table form (`{ token = "...", scopes = ["defs.read", "cache.*"] }`) restricts a token to `<area>.<verb>` scopes enforced against the op → scope table in `daemon/auth.rs:op_scope`. Scope patterns are `*`, `<area>.*`, `*.<verb>`, or exact `<area>.<verb>` — no nested globs, no regex. A mismatch returns 403 with code `scope_denied`.

```
# Liveness – no auth needed on loopback
curl -s http://127.0.0.1:7733/health
# {"ok":true,"version":"0.10.7","uptime_ms":12345}

# Invoke an op (POST /op/<NAME> with a JSON body)
curl -s -X POST http://127.0.0.1:7733/op/cache_put \
  -H 'Content-Type: application/json' \
  -d '{"ns":"ci","key":"target","value":"release","ttl_secs":86400}'
# {"ok":true,"bytes":7,"key":"target","ns":"ci","expires_at":...}

# Prometheus scrape
curl -s http://127.0.0.1:7733/metrics
```

Appendix C – The ZLE Widget Catalog

zshrs ships **193 base widget names** in its internal widget registry — the Rust analog of C zsh's `Src/Zle/iwidgets.list`, restricted to the names that have a Rust port. The registry is `IWIDGET_NAMES` in `src/ported/zle/zle_bindings.rs`; it is walked at startup by `init_thingies()` to populate the thingy table so `zle -l` can enumerate widgets without each name having to be invoked first via the keymap. Each base name registers in two forms (`name` and the `.name` builtin-protected alias), so `${#widgets}` reports $193 \times 2 = 386$ entries, matching zsh's count.

The names below are the registry's, grouped here by function for reading. The registry itself stores them alphabetically within two blocks: the dispatch-ported set, which has a real Rust function pointer in `iwidget_lookup` and runs its own implementation, and the `undefinedkey`-fallback set, which registers the name but resolves to the no-op fallback until a port lands. The grouping is editorial; the names are canonical. For the authoritative flat list at any version, run `zle -l` in a running shell or read `IWIDGET_NAMES` directly — the catalog here is a snapshot for reading, the source is the source of truth. The functional grouping below is the useful lens in practice: when you are binding keys or writing a ZLE plugin, you reach for widgets by what they do (motion, kill, history search), not by where they fall alphabetically.

C.1 – Basic motion and editing

`backward-char` · `forward-char` · `backward-word` · `forward-word` · `beginning-of-line` · `end-of-line` · `beginning-of-buffer-or-history` · `end-of-buffer-or-history` · `self-insert` · `self-insert-unmeta` · `delete-char` · `delete-char-or-list` · `backward-delete-char` · `transpose-chars` · `transpose-words` · `capitalize-word` · `up-case-word` · `down-case-word` · `quoted-insert` · `quote-line` · `quote-region` · `overwrite-mode` · `vi-quoted-insert` · `magic-space` · `bracketed-paste` · `emacs-backward-word` · `emacs-forward-word` · `up-line` · `down-line` · `gosmacs-transpose-chars`

C.2 – Line acceptance and screen

`accept-line` · `accept-line-and-down-history` · `accept-and-hold` · `accept-and-infer-next-history` · `accept-and-menu-complete` · `push-line` · `push-input` · `push-line-or-edit` · `get-line` · `clear-screen` · `redisplay` · `reset-prompt` · `recursive-edit` · `read-command` · `send-break` · `beep` · `pound-insert` · `vi-pound-insert`

C.3 – History

`up-history` · `down-history` · `up-line-or-history` · `down-line-or-history` · `up-line-or-search` · `down-line-or-search` · `beginning-of-history` · `end-of-history` · `beginning-of-line-hist` · `end-of-line-hist` · `insert-last-word` · `copy-prev-word` · `copy-prev-shell-word` · `expand-history` · `infer-next-history` · `set-local-history` · `vi-fetch-history` · `vi-up-line-or-history` · `vi-down-line-or-history`

C.4 – Search and incremental search

history-search-backward · history-search-forward · history-incremental-search-backward · history-incremental-search-forward · history-incremental-pattern-search-backward · history-incremental-pattern-search-forward · history-beginning-search-backward · history-beginning-search-forward · accept-search · vi-history-search-backward · vi-history-search-forward

C.5 – Undo / redo

undo · redo · split-undo · vi-undo-change

C.6 – Kill and yank

kill-line · kill-whole-line · kill-word · kill-buffer · kill-region · backward-kill-word · backward-kill-line · backward-delete-word · delete-word · copy-region-as-kill · yank · yank-pop · put-replace-selection

C.7 – Completion and menu

complete-word · expand-or-complete · expand-or-complete-prefix · expand-cmd-path · expand-word · menu-complete · menu-expand-or-complete · reverse-menu-complete · list-choices · list-expand · end-of-list · auto-suffix-remove · auto-suffix-retain

C.8 – Arguments and named commands

digit-argument · neg-argument · universal-argument · argument-base · execute-named-cmd · execute-last-named-cmd

C.9 – Region / mark / visual

set-mark-command · exchange-point-and-mark · deactivate-region · visual-mode · visual-line-mode · vi-set-mark · vi-goto-mark · vi-goto-mark-line

C.10 – Text objects (selection)

select-a-word · select-in-word · select-a-blank-word · select-in-blank-word · select-a-shell-word · select-in-shell-word

C.11 – vi-mode command and editing

vi-cmd-mode · vi-insert · vi-insert-bol · vi-add-next · vi-add-eol · vi-replace · vi-replace-chars · vi-substitute · vi-change · vi-change-eol · vi-change-whole-line · vi-delete · vi-delete-char · vi-backward-delete-char · vi-backward-kill-word · vi-kill-line · vi-kill-eol · vi-yank · vi-yank-eol · vi-yank-whole-line · vi-put-before · vi-put-after · vi-indent · vi-unindent · vi-join · vi-open-line-above · vi-open-line-below · vi-swap-case · vi-oper-swap-case · vi-up-case · vi-down-case · vi-repeat-change · vi-set-buffer · vi-caps-lock-panic

C.12 – vi-mode motion

`vi-backward-char` · `vi-forward-char` · `vi-backward-word` · `vi-forward-word` · `vi-forward-word-end` · `vi-backward-word-end` · `vi-backward-blank-word` · `vi-forward-blank-word` · `vi-forward-blank-word-end` · `vi-backward-blank-word-end` · `vi-beginning-of-line` · `vi-end-of-line` · `vi-first-non-blank` · `vi-digit-or-beginning-of-line` · `vi-goto-column` · `vi-match-bracket`

C.13 – vi-mode find and repeat-search

`vi-find-next-char` · `vi-find-next-char-skip` · `vi-find-prev-char` · `vi-find-prev-char-skip` · `vi-repeat-find` · `vi-rev-repeat-find` · `vi-repeat-search` · `vi-rev-repeat-search`

C.14 – Help and introspection

`run-help` · `which-command` · `where-is` · `what-cursor-position` · `describe-key-briefly` · `spell-word`

C.15 – Fallback

`undefined-key` is the catch-all widget bound to any key sequence with no other binding; pressing such a key invokes it, which by default beeps. It is also the dispatch target the registry uses to keep its count complete: names present in C zsh's `iwidgets.list` that do not yet have a dedicated Rust dispatch port register with the `undefinedkey` fallback, so `${#widgets}` stays at zsh's 193 base names rather than dropping to only the ported subset. Every name zsh exposes, zshrs exposes, even where the body is still the fallback. In day-to-day use this is invisible — the widgets you actually bind are the ported ones; the fallback entries exist so that a lookup never misses, not because you would deliberately bind a key to a no-op.

This matters for plugin compatibility. A plugin that does `bindkey '^X^E' edit-command-line` or queries `${widgets[name]}` expects every standard widget name to resolve. If zshrs only registered the widgets it had fully ported, those lookups would miss and the plugin would break. Registering the full 193 with a fallback body means the namespace is complete from day one; ports fill in real behavior over time without ever changing what names exist.

Appendix D – Glob Qualifiers and Parameter Flags

zshrs ports zsh's two most idiosyncratic expansion features — glob qualifiers (the `(...)` suffix that filters a glob by file metadata) and parameter-expansion flags (the `(...)` prefix that transforms a `${...}` result). Both are where zsh diverges hardest from POSIX sh, and both are where compatibility is hardest to fake: a plugin or a `.zshrc` one-liner that uses `*(.om[1,10])` or `${(@f)$(cat file)}` will silently produce wrong output if the qualifier or flag is half-implemented. Each entry below has a one-line meaning and a minimal example. This is a snapshot of a moving target — the gap audit is ongoing, and the canonical list is whatever the current binary passes — so treat the catalog as the set known to work today, not a frozen contract. The companion lesson is that these features fail quietly when incomplete: a half-ported qualifier returns a wrong file set rather than an error, which is why the gap audit pins each one with a regression test rather than trusting it by inspection.

D.1 – Glob qualifiers

A glob qualifier is a parenthesized suffix on a pattern that restricts the match by file attributes. zshrs's `looks_like_glob_qualifiers / filter_by_qualifiers` path implements the following.

Qualifier	Meaning	Example
<code>(D)</code>	Per-pattern dotglob — include dotfiles for this pattern only.	<code>print *(D)</code>
<code>(L0)</code>	Files of exactly size 0 (the <code>L</code> size qualifier).	<code>*(L0)</code>
<code>(L+10k)</code>	Files larger than 10 KB; suffix <code>k/m/g/p</code> selects the unit.	<code>*(L+10k)</code>
<code>(L-1m)</code>	Files smaller than 1 MB. Default unit is bytes.	<code>*(L-1m)</code>
<code>(mh-N)</code>	Modified within the last N hours (<code>m</code> = mtime).	<code>/etc/hosts(mh-100)</code>
<code>(mm-N)</code>	Modified within the last N minutes.	<code>*(mm-30)</code>
<code>(mw-N)</code>	Modified within the last N weeks.	<code>*(mw-2)</code>
<code>(a...)</code>	Same time syntax as <code>m</code> , on access time (atime).	<code>*(ah-1)</code>
<code>(c...)</code>	Same time syntax, on inode-change time (ctime).	<code>*(cd-1)</code>
<code>(.)</code>	Plain qualifier — restrict to a class (used in the time-qualifier port's tests).	<code>*(.)</code>

The time-qualifier syntax parses a unit char (`s/m/h/d/w/M`), an op (`+/-`), and an integer N, then filters via `mtime()/atime()/ctime()` against the cutoff. The size suffix `L[+-]N[k|m|g|p]` maps to KB/MB/GB/bytes.

D.2 – Range and recursive globbing

These are pattern-level features that compose with qualifiers.

Form	Meaning	Example
<code><lo-hi></code>	Numeric range glob; <code>lo</code> and/or <code>hi</code> optional.	<code>file<1-10>, <->, <5->, <-10></code>
<code>**/</code>	Recursive directory walk, dirs only, trailing slash preserved.	<code>print **/</code>
<code>**/*</code>	Recursive walk matching both files and directories.	<code>print **/*.rs</code>

`<lo-hi>` translates each occurrence to a `(\d+)` capture and verifies the parsed number against the bounds; `**/` enumerates directories (dirs-only form) while `**/*` includes files, matching zsh's relative-path output style.

D.3 – Parameter-expansion flags

A parameter-expansion flag is a parenthesized prefix inside `${...}` that transforms the result. zshrs implements these in `BUILTIN_PARAM_FLAG` and friends.

Flag	Meaning	Example
<code>(f)</code>	Split on newlines into an array.	<code>\${(f)text}</code>
<code>(z)</code>	Split as the shell would tokenize a command line (array-producing).	<code>print -l \${(z)cmd}</code>
<code>(w)</code>	Word-split into an array.	<code>\${(w)str}</code>
<code>(s:sep:)</code>	Split on an explicit separator string.	<code>\${(s:,:)csv}</code>
<code>(u)</code>	Unique — preserve first occurrence, drop duplicates.	<code>\${(u)arr}</code>
<code>(C)</code>	Capitalize first letter of each word, lowercase the rest.	<code>\${(C)name}</code>
<code>(U)</code>	Uppercase (verified for the literal form).	<code>\${(U)str}</code>
<code>(B)</code>	Backslash-escape shell metacharacters.	<code>\${(B)x}</code>
<code>(k)</code>	Keys of an associative array.	<code>\${(k)assoc}</code>
<code>(v)</code>	Values of an associative array.	<code>\${(v)assoc}</code>
<code>(kv) / (vk)</code>	Interleaved key/value (or value/key) pairs.	<code>\${(kv)assoc}</code>
<code>(t)</code>	Type + attribute introspection (<code>integer</code> , <code>float</code> , <code>scalar-readonly</code> , ...). The flag a script uses to branch on whether a name is an array, an integer, readonly, exported, and so on.	<code>\${(t)var}</code>

Flag	Meaning	Example
<code>(@)</code>	Preserve array elements as distinct words inside double quotes — the flag that stops an array from being joined into one string when quoted.	<code>"\${(@f)str}"</code>
<code>(o) / (O)</code>	Sort ascending / descending.	<code>\${(o)arr}, \${(O)arr}</code>
<code>(F)</code>	Join an array with newlines (the inverse of <code>(f)</code> splitting). Useful for printing one element per line.	<code>print \${(F)arr}</code>
<code>(q) / (qq) / (Q)</code>	Quote / dequote: <code>(q)</code> backslash-escapes shell-special chars, <code>(qq)</code> wraps in single quotes, <code>(Q)</code> strips one level of quoting. The pair you use to make a value safe to re-eval and to undo it.	<code>\${(q)x}, \${(Q)x}</code>

Flags compose. `${(@s:,)str}` applies `@` (array context) then splits on `,`; `${(kv)a[@]}` combines a flag prefix with an `[@]` subscript; `${(@f)mapfile[/path]}` reads a file via the `zsh/mapfile` magic assoc and line-splits it, correctly producing N+1 elements when the file ends in a newline.

0.4 – Subscript flags

A different `(...)` again — this one lives inside the `[...]` of an array subscript and turns a subscript from “give me element N” into “give me the element (or index) that matches this pattern.” They are combinable: `(re)` means “last matching value, exact match”; `(ie)` means “first matching index, exact match.” For associative arrays the value/index split has a twist — `r/R` search the values, while `i/I` return the matching key.

Flag	Meaning
<code>(r)pat</code>	First matching value (for assoc, searches values).
<code>(R)pat</code>	Last matching value (reverse).
<code>(i)pat</code>	First matching index, 1-based (<code>len+1</code> if no match; for assoc, returns the matching key).
<code>(I)pat</code>	Last matching index (<code>0</code> if no match).
<code>(e)str</code>	Exact (literal) match instead of glob.

0.5 – Extended-glob pattern flags

Inline `(#...)` flags inside a pattern, applied by `glob_match_static` before regex translation. These flip matching behavior mid-pattern rather than for the whole expression, which is what makes them composable — `(#i)foo(#I)BAR` matches `foo` case-insensitively but `BAR` case-sensitively. The approximate-match flag `(#a<n>)` is the most unusual: it short-circuits to a Wagner-Fischer Levenshtein-distance check, so `(#a2)hello` matches any string within two edits of `hello`. All paths that route through `glob_match_static` — `[[ str = pat ]]`, `case` arms, the `${arr:#pat}` filter — pick these up automatically.

Flag	Meaning	Example
<code>(#i)</code>	Case-insensitive from here on.	<code>[[\$x = (#i)foo]]</code>
<code>(#I)</code>	Cancel <code>(#i)</code> (case-sensitive again).	<code>(#i)f(#I)00</code>
<code>(#l)</code>	Lowercase pattern chars match either case (uppercase chars stay exact).	<code>(#l)foo</code>
<code>(#a<n>)</code>	Approximate match, Levenshtein distance $\leq n$ via Wagner-Fischer (default $n=1$). The fuzzy-match flag — <code>(#a2)hello</code> matches any string within two edits.	<code>(#a2)hello</code>
<code>(#cN) / (#cN,M)</code>	Repetition count: match the preceding atom exactly N times, or between N and M. Compiles to a regex <code>{N} / {N,M}</code> quantifier.	<code>a(#c2,3)</code>
<code>^pat</code>	Negation in <code>\${arr:#pat}</code> and <code>[[str = pat]]</code> when <code>extendedglob</code> is set; without the option, <code>^</code> stays literal.	<code>\${arr:#^*.txt}</code>

0.6 – ksh-style extended-glob patterns

A separate pattern dialect, gated on `setopt kshglob` so it stays off by default and matches zsh. These are the ksh/bash `extglob` operators, translated by `ksh_extglob_body_to_regex` to a `(?:body){suffix}` regex. They are what lets a ported bash or ksh script's pattern matching work under zshrs once the option is set; without `kshglob`, the parentheses keep their plain-glob meaning.

Form	Meaning
<code>?(pat)</code>	Match the body zero or one time.
<code>*(pat)</code>	Match the body zero or more times.
<code>+(pat)</code>	Match the body one or more times.
<code>@(pat)</code>	Match the body exactly once (an alternation group).
<code>!(pat)</code>	Match anything that is <i>not</i> the body. Only the standalone <code>!(...)</code> form is supported; <code>prefix!(pat)suffix</code> needs negative lookahead and stays literal.

Appendix E — A Glossary of zshrs Terms

A short reference for the vocabulary this book uses. zshrs introduces a fair amount of its own terminology — partly because it compiles to bytecode and needs words for things a traditional interpreting shell had no need to name (the VM, the fused superinstruction, the rkyv image), partly because its daemon architecture borrows the thin-client / fat-server vocabulary of display servers and language servers. The definitions below are grounded in the source and docs cited throughout the preceding chapters; where a term names a concrete file or op, the citation points to it. Read this section as a map of the architecture as much as a word list — the terms cluster into three groups (the VM and compile layer, the daemon and its caches, and the cross-shell coordination builtins), and seeing which group a word belongs to is half of understanding it.

Anti-fork — A class of builtin or design decision that avoids the `fork(2)` overhead a traditional shell pays for every subshell or external helper. Each `fork` copies the process and tears it down again; in a loop or a hot prompt that cost dominates. zshrs’s persistent worker pool exists specifically to eliminate most fork overhead for builtins, running them on already-live threads instead of spawning a process per call.

AOP intercept — Aspect-oriented programming hook. zshrs adds AOP intercepts on every command; the recorder uses runtime AOP on the alias dispatcher, function-define path, assign/typeset path, and each relevant builtin to capture shell state without parsing source.

AOT (ahead-of-time compilation) — The compile mode that bundles `fusevm` together with bytecode into a single executable, as opposed to compiling lazily at run time. The payoff is startup: an AOT build skips the parse-and-compile step for the code it bundled, so the cold path is load-and-run rather than read-source-and-compile.

catalog.db — The daemon-only-writer SQLite database that mirrors all compiled state (plugins, entries, hooks, stats, `compiled_files`). It is the queryable, joinable counterpart to the hot-path rkyv shards, and the two can reconstruct each other.

Compat floor — The non-negotiable baseline of zsh behavior zshrs must reproduce: lexer, parser, builtins, parameters, globs, and ZLE. Existing `.zshrc`, `.zprofile`, zinit/oh-my-zsh plugins, and the zsh-more-completions corpus must run unchanged.

Daemon (zshrs-daemon) — The optional singleton coordinator process. It owns all compilation, persistence, filesystem enumeration, routing, and the single machine-wide fsnotify watcher — roughly 90% of shell-internal work. See `zd(1)` and Appendix B.

Definitions catalog — The recorder-fed, `shell_id`-federated store of every alias, function, parameter, binding, completion, and option captured during init. Queried via `definitions_query`; diffed across shells via `definitions_diff`.

fused superinstruction — A single VM opcode that performs work which would otherwise take several separate opcodes, fused at compile time. Fusing common opcode sequences into one dispatch reduces per-instruction interpreter overhead — fewer dispatch-loop iterations, better branch prediction, less stack traffic between the fused steps. The “fuse” in `fusevm` refers to this fusion of bytecode operations; it is a technique that depends on having a bytecode stream to fuse, which a tree-walking interpreter does not have.

fusevm — The purpose-built bytecode virtual machine zshrs compiles to and runs on. Rather than walk the parsed AST at run time, zshrs lowers it to fusevm bytecode and executes that, which is what the tree-walker-absent invariant pins. Shared with stryke, so the same VM runs both the shell and the embedded

language.

hash -d (named directory) — A directory abbreviation registered with `hash -d name=path`. The daemon serves the named-directory table; the recorder captures `hash -d` entries via AOP.

Overlay — The per-client in-memory hash that holds a shell's interactive mutations (`compdef`, `alias`, `function`, monkey-patched globals, autoloaded bodies). It is checked before the mmap'd rkyv image and dies on `exec zshrs` unless promoted to canonical with `zsync`.

Parity mode — A command-line mode (`--zsh`, `--bash`, `--ksh`, `--posix`, `--csh`) that disables all caches and the daemon to match a named reference shell byte-for-byte. See Appendix A.3.

Perfect hashing (PHF) — The hashing scheme used inside rkyv shards so client lookup is one indirection: compute hash, mask to slot, single mmap dereference (~150–200 ns). Generated daemon-side at compile time; it is what enforces the “no walking in clients” rule.

The port / extension split — zshrs's two-layer architecture: the *port* layer reproduces zsh faithfully (the compat floor); the *extension* layer adds the daemon, worker pool, AOP, caches, and the `z*` builtin family on top. Code under `src/ported/` is the former.

Recorder (zshrs-recorder) — The one-shot binary (`bins/zshrs-recorder.rs`, `--features recorder`) that runs the user's init under runtime AOP and ships the captured `RecorderBundle` to the daemon via `recorder_ingest`. Re-run when `.zshrc` or plugins change.

rkyv image / shard — A zero-copy serialized cache file (`{hash8}-*.rkyv`) that clients mmap and read without deserializing. Shards are perfect-hash-indexed and atomically renamed; the daemon is the sole writer.

shell_id — The federation identifier that lets bash, fish, ksh, and other shells' recorders write to the same catalog with their own identity, enabling cross-shell `definitions_diff`. See `docs/SHELL_IDS.md`.

source_resolve — The daemon op behind the `source` / `.` builtins: client sends `(path, mtime, inode)`, daemon returns cached bytecode on a hit or compiles-and-caches on a miss. On-demand only — no transitive auto-walk of the closure reachable from `.zshrc`.

Thin client — The lightweight half of the client/daemon split. It owns tty IO, process attributes (`$$`, `#!`, `$?`, traps, fds, locals), `fork+exec` for user commands, the IPC connection, and direct mmap reads — nothing more. Total cold-start under 10 ms.

Tree-walker-absent invariant — The architectural rule, pinned by tests (`tree_walker_absent.rs`, `no_tree_walker_dispatch.rs`), that zshrs never executes by walking an AST at run time; everything goes through bytecode on `fusevm`. The tests fail if any execution path dispatches off the parse tree directly, which keeps later changes from quietly reintroducing an interpreter as a shortcut. It is the behavioral pin that makes “compiles to bytecode” a checked property rather than an aspiration.

Worker pool — The persistent pool of threads the daemon (and full-mode client path) uses so most builtins never pay `fork(2)` cost. Disabled in `--posix` parity mode.

zask — The pull-mode UI queue (ops `ask_ask` / `ask_pending` / `ask_take` / `ask_dismiss` / `ask_response`) by which the daemon or another shell asks the user a question. Named `zask` because `zui` was already taken by an existing zsh plugin.

zcache — The umbrella `z*` builtin for cache management and the universal `view` / `export` / `import` surface over the daemon's caches. Every named target (path, fpath, aliases, `_comps`, `zstyle`, history, ...) is a subcommand; `eval $(zcache export <target>)` resets the overlay back to canonical.

.zwc autoload — zsh's pre-compiled wordcode cache for autoloaded functions. zshrs treats `.zwc` files as invisible to the recorder and source resolver — they are dead disk litter once on zshrs, since the cold path is mmap + indexed bytecode + execute, with no source-parse step for `.zwc` to short-circuit. Importable on demand via `zcache import zwc`.

ZLE widget — A named editor action bound to a key or invoked via `zle`, the unit a line-editor keymap maps keys to. zshrs ships 193 base widget names in `IWIDGET_NAMES` (Appendix C), the Rust analog of C zsh's

`iwidgets.list`; registering the full set means plugin lookups like `${widgets[name]}` never miss even where the body is still the fallback.

`zls` / `zsend` / `znotify` — The cross-shell coordination builtins. `zls` lists every registered shell session; `zsend` dispatches a command to one shell, a tag, a user, or all; `znotify` delivers an OSC-9 / status-line message. All are thin JSON-IPC wrappers over the daemon.

`zsync` — The builtin family that promotes a client's overlay state to daemon canonical (`zsync up path`, `zsync up alias --all`, `zsync diff`, `zsync pull`), so a session change takes effect for future shells. Backed by `push_canonical` / `pull_canonical` / `diff_canonical`.

`@-dispatch` — The prefix mechanism by which stryke embeds inside zshrs: an `@`-prefixed token routes to the embedded stryke language rather than the shell, so a single command line can hand part of its work to stryke without leaving the shell. stryke embeds inside zshrs, not the reverse — the shell is the host.

`zwhere` — The `where` builtin's output contract: zshrs emits zsh's bare path form (`/bin/ls`) rather than the verbose `ls is /bin/ls`, honoring csh-style output for aliases, functions, and not-found cases. See [docs/GAPS.md](#) §“`where` builtin output format”.